



SAP LeanIX

Generated on: 2026-06-15 03:57:24 GMT+0000

SAP LeanIX | CLOUD

Public

Original content: <https://help.sap.com/docs/LEANIX/72d375467c1e4dcb872dfa2998b6328d?locale=en-US&state=PRODUCTION&version=CLOUD>

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

Developer Guide

Resources for developers working with SAP LeanIX.

As a developer, you can retrieve data programmatically and build custom integrations with SAP LeanIX. Explore the topics in this section to find the information you need:

- [Basics](#): Learn how to choose a technology for your integration and authenticate to SAP LeanIX services.
- [Guides and Tutorials](#): Explore practical examples of using our technologies.
- [SAP LeanIX APIs](#): Get an overview of our APIs and learn when to use each.
- [Reporting Library and Custom Report Tools](#): Build custom reports tailored to your needs.
- [MCP Server](#): Learn how the MCP server for SAP LeanIX works and how to connect to it.

For additional information, refer to the following resources:

- [Getting Started](#): Get an overview of SAP LeanIX and explore its key concepts and features.
- [SAP LeanIX Community](#) 🗨️ : Connect with our community to share experiences, exchange ideas, and seek advice. Join the [SAP LeanIX Developers](#) 🗨️ group to discuss technical topics with developers and industry experts.

Basics

Explore the essentials to start building your integrations.

- [Choosing a Technology for Your Integration](#)
- [Authentication to SAP LeanIX Services](#)
- [Base URL](#)

Choosing a Technology for Your Integration

Factors that you need to consider when choosing a technology for your integration.

Overview

This article outlines the key factors that you need to consider when choosing a technology for your integration. With integrations, you can seamlessly import and export data from external systems and automate workflows for your organization.

To cater to a diverse set of requirements that you may have, we offer several integration solutions, including the following:

- [Out-of-the-box integrations](#)
- [GraphQL API](#)
- [REST APIs](#)
- [Webhooks](#)

- [Integration API](#)

Before you initiate the integration process, collect the requirements for your integration and evaluate the setup of the external system. Then, based on your requirements, select a technology considering the advantages and disadvantages of each solution.

Use Out-of-the-Box Integrations

To explore pre-built solutions designed for integrating with third-party systems, see [Discovery and Integrations](#).

Advantages:

- SAP LeanIX handles the development, operation, and maintenance of the integration.
- A workspace administrator can configure the integration directly within the user interface of SAP LeanIX.
- For assistance in creating a suitable configuration for your specific use case, you can refer to our extensive documentation or seek guidance from our Customer Success Team.
- You don't need any middleware or engineering resources for the integration setup.

Disadvantages:

- Our out-of-the-box integrations are designed for specific use cases and might not fully align with your requirements.
- To configure some out-of-the-box integrations, you may need to purchase additional SAP LeanIX products.
- The synchronization timing, whether real-time or based on a daily schedule, depends on the capabilities of specific integrations.

Integrate Using the GraphQL API, REST APIs, and Webhooks

We provide an extensive range of APIs to accommodate your use cases and requirements. Explore our [GraphQL API](#), [REST APIs](#), and [webhooks](#).

Advantages:

- SAP LeanIX handles the development, operation, and maintenance of APIs, ensuring their stability and reliability.
- You can tailor the integration to your unique use cases or requirements as long as you can implement the integration using our APIs.
- There is a wide range of technologies and tools to develop, operate, and maintain your custom integration. You can choose any solution according to your engineering guidelines and policies.
- You can adjust the synchronization timing for the integration according to your requirements. You can schedule full synchronizations at a specific time or set up a webhook event to trigger real-time synchronizations.

Disadvantages:

- Developing, operating, and maintaining your custom integration is your responsibility, which requires dedicated engineering capabilities and resources on your end.

Build a Custom Integration Using the Integration API

We offer the Integration API that enables you to build a custom integration according to your use case and requirements while investing less effort and resources compared to other alternative solutions. For more information, see [Integration API](#).

Advantages:

- The Integration API allows you to significantly reduce the amount of resources and requirements that you need on your end. Instead of implementing an entire integration from scratch, you need to configure the external system to provide or consume relevant data using our custom [LeanIX Data Interchange Format \(LDIF\)](#) format.
- Anyone with sufficient technical knowledge can operate and maintain the actual mapping of your custom integration. You don't need a dedicated engineering team for that.
- To assist you with setting up an appropriate configuration for your use case, we provide comprehensive documentation and offer guidance through our Customer Success Team.

Disadvantages:

- You need to use a middleware to consume or provide your data in the LDIF format.
- While you don't need a professional engineering background, investing time in learning the language of the processor configurations within the Integration API may be necessary.
- Developing and maintaining the processor configuration does not provide the same developer experience that you may be accustomed to with other implementation solutions. This relates to possibilities such as auto-completion, debugging, or automated testing.
- The Integration API does not support real-time synchronization of data.

Authentication to SAP LeanIX Services

Learn how to authenticate to SAP LeanIX services.

In this guide, you will learn how authentication to SAP LeanIX services works.

To authenticate your requests, follow these key steps:

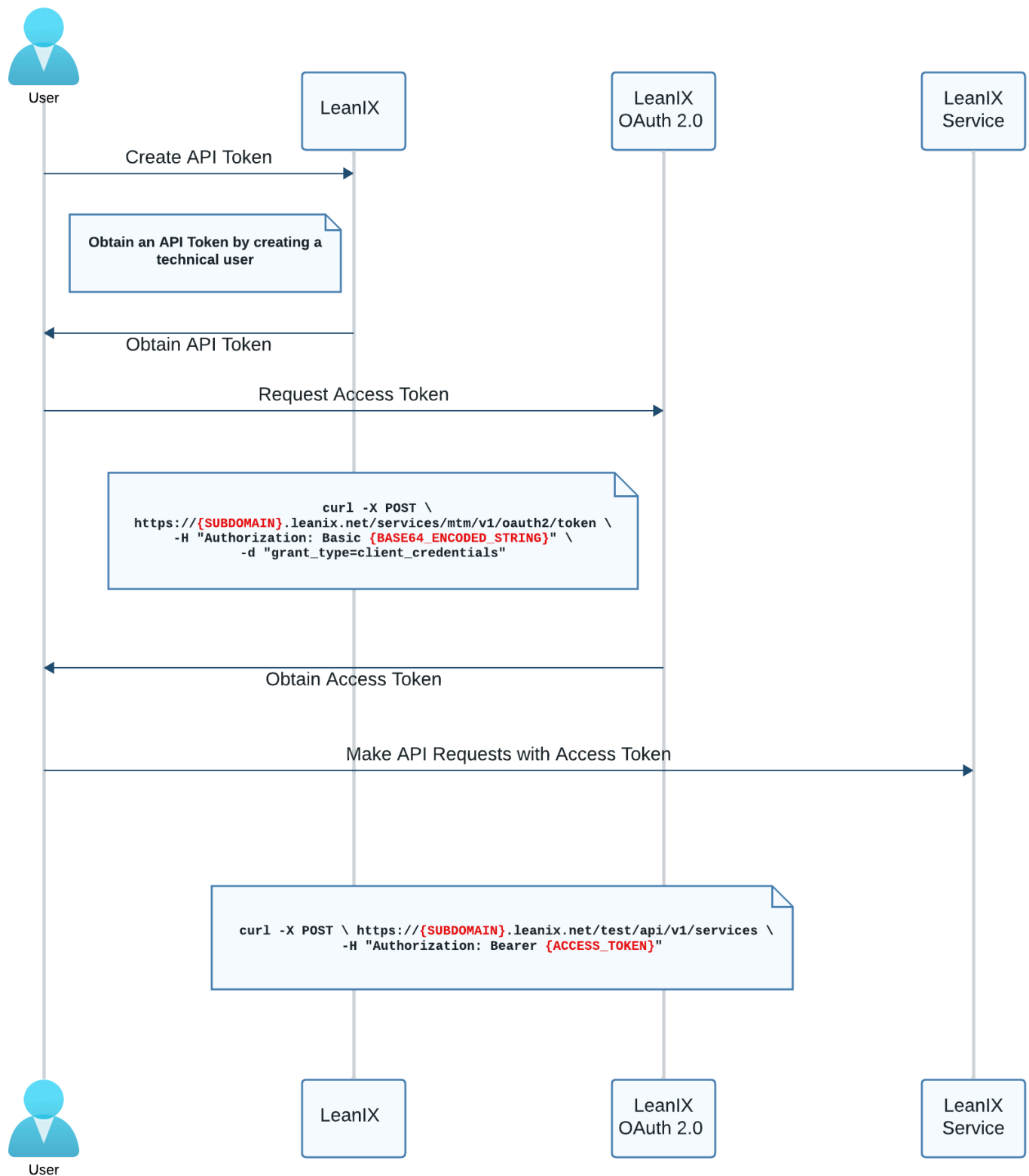
1. Get an API token by creating a Technical User.
2. Get an access token.
3. Make authenticated requests to services by including the access token in the Authorization header.

i Note

Before you begin, make sure that you know your SAP LeanIX subdomain. You can copy your subdomain value from the Workspace URL. To learn more, see [Base URL](#).

Authentication Flow

SAP LeanIX employs the [OAuth 2.0](#)  protocol for user authentication across all available APIs. In the image below, you can see how the authentication process works for an example API request.



SAP LeanIX OAuth 2.0 Flow

Step 1: Get an API Token

To request an access token, you need to get an API token first. As an admin, you can generate an API token by creating a technical user. To learn how, see [Technical Users](#).

⚠ Caution

The API token is shown only once. Make sure that you store it securely.

Step 2: Get an Access Token

Now that you have an API token, you can obtain an access token.

Request an Access Token

To request an access token, make a POST request to the following endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token
```

This API uses the Basic HTTP authentication scheme. For more information, see [The 'Basic' HTTP Authentication Scheme](#)  section of the Internet Standards Track document issued by the Internet Engineering Task Force (IETF).

For credentials, use `apitoken` as the username and the obtained API token as the password. Set the `grant_type` to `client_credentials`.

The following examples show how to make a request to obtain an access token.

JAANYFMYQcNtZ6ettSH9pF88RLdVBGfuY6bDFThw is an example API token that we use for illustrative purposes.

Example request:

cURL

```
curl -u apitoken:JAANYFMYQcNtZ6ettSH9pF88RLdVBGfuY6bDFThw \
  --request POST https://{SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token \
  --data grant_type=client_credentials
```

JavaScript

```
const axios = require('axios');
const qs = require('qs');
const username = "apitoken"
const password = "JAANYFMYQcNtZ6ettSH9pF88RLdVBGfuY6bDFThw"
const data = { 'grant_type': 'client_credentials' };
const instance = "https://{SUBDOMAIN}.leanix.net";
const tokenEndpoint = "https://{SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token"
```

```
// Step 2: Make a POST request to obtain the access token
const getToken = async () => {
  try {
    const response = await axios.post(
      tokenEndpoint,
      data: qs.stringify(data),
      {
        auth: {
          username: username,
          password: password,
        },
        headers: {
          'Content-Type': 'application/x-www-form-urlencoded',
        },
      },
    );

    // Step 3: Receive and use the access token
    const accessToken = response.data.access_token;
  } catch (error) {
    console.error('Error obtaining access token:', error.message);
  }
};
```

Java

```
OkHttpClient client = new OkHttpClient();

MediaType mediaType = MediaType.parse("application/x-www-form-urlencoded");
RequestBody body = RequestBody.create(mediaType, "grant_type=client_credentials");
Request request = new Request.Builder()
    .url("https://{SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token")
    .post(body)
    .addHeader("Authorization", Credentials.basic("apitoken", "JAANYFMYQcNtZ6ettSH9pF88RLdVBGfuY6bDFThw"))
    .build();

Response response = client.newCall(request).execute();
```

Python

```
import requests

# Set a timeout to prevent hanging requests
TIMEOUT = 10
OAUTH2_URL = "https://{SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token"

# Step 2: Make a POST request to obtain the access token, for obtain the access token we
# use the Basic Authorization scheme, you can implement this by using the `auth` parameter
# of the requests library.
response = requests.post(
    OAUTH2_URL,
    auth=("apitoken", "JAANYFMYQcNtZ6ettSH9pF88RLdVBGfuY6bDFThw"),
    data={"grant_type": "client_credentials"},
    timeout=TIMEOUT,
)
# Check if the request was successful
response.raise_for_status()
# The access token is available under the `access_token` response field.
response_payload = response.json()
access_token = response_payload["access_token"]
```

Review the Response

The response from the `https://{SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token` endpoint includes your access token. Handle the response according to your programming language or tool.

Example response:

```
{
  "scope": "",
  "expired": false,
  "access_token": "eyJhbGciOiJSUzI1NiJ9.eyJz [ ... ] ssqaPSA",
  "token_type": "bearer",
  "expires_in": 3599
}
```

The `access_token` is returned in JWT (JSON Web Token) format. JWTs are encoded tokens that carry information in a compact and self-contained manner. In the context of OAuth 2.0, access tokens are often JWTs.

Tokens are signed with a private key, enabling our APIs to validate their authenticity. The signing process ensures that the token has not been tampered with and comes from a trusted source.

You can decode your access token using tools such as [JWT.IO](#) . Once you have decoded your token, you can view the token details, such as:

- **Permissions:** Information about the user's permissions or scope, indicating what actions or resources the token holder is authorized to access.

- **Expiration time:** The validity duration of the access token in seconds is specified in `expires_in`. The access token is valid for 3600 seconds, which is equivalent to one hour.

i Note

OAuth 2.0 access tokens often have a limited lifespan for security purposes. By setting an expiration time, the system can automatically revoke access after a certain period, minimizing the risk of unauthorized access if the token is compromised. This practice is part of the OAuth 2.0 security model to ensure that tokens are regularly refreshed, requiring the client to reauthenticate to obtain a new valid token.

Step 3: Make Authenticated Requests to Services

After obtaining an access token, you can use it to make authenticated requests to SAP LeanIX services.

To authorize your request, insert your access token into the `Authorization` header. Be sure to include the `Bearer` prefix to specify the token type.

As an example, let's fetch a list of Applications, internally referred to as services, from the IT Inventory of a Workspace.

i Note

Reuse the access token in your API requests within the token lifetime. Do not request new access tokens while the current token is still valid.

Repeatedly requesting new tokens without necessity may lead to security vulnerabilities and unnecessary load on the authorization server. Adhere to the token expiration time and only request a new token when necessary.

In the following example request, the access token (`eyJhbGciOiJSUzI1NiJ9.eyJz [...] ssqaPSA`) is shortened. When implementing this in your applications, ensure that you use a complete and unaltered access token.

Example request:

cURL

```
curl -H "Authorization: Bearer {ACCESS_TOKEN}" \
https://{SUBDOMAIN}.leanix.net/services/poll/v2/pollRuns
```

JavaScript

```
const axios = require('axios');

// Replace {SUBDOMAIN} with your actual subdomain
const url = 'https://{SUBDOMAIN}.leanix.net/services/poll/v2/pollRuns';
const accessToken = 'eyJhbGciOiJSUzI1NiJ9.eyJz [...] ssqaPSA'; // Replace with your actual access token

// Step 2: Make a GET request to the /v1/services API endpoint
axios.get(url, {
  headers: {
    'Authorization': `Bearer ${accessToken}`,
  },
})
.then(response => {
  // Process the response as needed
  console.log(response.data);
})
.catch(error => {
  console.error('Error:', error.message);
});
```


Java

```
OkHttpClient client = new OkHttpClient();

Request request = new Request.Builder()
    .url("https://{SUBDOMAIN}.leanix.net/services/poll/v2/pollRuns")
    .get()
    .addHeader("Authorization", "Bearer eyJhbGciOiJSUzI1NiJ9.eyJz [...] ssqaPSA")
    .build();

Response response = client.newCall(request).execute();
```

Python

```
import requests

timeout = 20
url = 'https://{SUBDOMAIN}.leanix.net/services/poll/v2/pollRuns'
headers = {'Authorization': 'Bearer eyJhbGciOiJSUzI1NiJ9.eyJz [...] ssqaPSA'}

response = requests.get(url, headers=headers, timeout=timeout)

# Process the response as needed
print(response.text)
```

Technical Users

To get an API token, create a technical user. Manage technical users collaboratively with other administrators.

As an administrator, you can get an API token by creating a technical user. With an API token, you can request an access token to authenticate to SAP LeanIX services. For more information, see [Authentication to SAP LeanIX Services](#).

With technical users, you get the following benefits:

- Integrations are no longer tied to a specific person, which allows you to avoid problems such as the person being on vacation or otherwise unavailable.
- Users who don't have access to a workspace can still manage integrations and access SAP LeanIX APIs.
- Technical users can be subscribed to fact sheets for better integration building.
- Technical users can be subscribed to notifications with a group email address, which allows you to efficiently manage notifications in your organization.

Creating a Technical User

Follow these steps:

1. In the user profile menu, select **Administration**, and then go to **Technical Users**.
2. Click **New Technical User**.
3. Enter the details for a technical user:
 - **Username**: Enter a username for the technical user.
 - (Optional) **Description**: Enter a description for the technical user.
 - **Permission Role**: Select a permission role: Admin, Viewer, or Member.

- (Optional) **Customer Roles**: Roles that you can define and configure in services that support them, such as the [Pathfinder](#) . In contrast, standard roles are an integral part of the SAP LeanIX authorization scheme and are recognized and supported by all services. You can configure custom roles only if your organization manages user roles within a single-sign-on (SSO) identity provider. For more information, see [Managing User Roles with SSO](#).
- (Optional) **Access Control Entities**: Use this parameter to map a technical user to Access Control Entities (ACEs) and Access Control Lists (ACLs) created with virtual workspaces. To learn more, see [Virtual Workspaces](#).
- **Expiry Date**: Set the expiration date for the API token associated with this user.

4. Click **Save**.

← New Technical User

Username *

Demo Technical User

Description

Permission Role *

ADMIN

Customer Roles

Access Control Entities

Expiry Date *

01.01.2025

Cancel

Save

Creating a Technical User

A technical user is created, and an overlay with an API token is displayed.

Save the API token. It is shown only once.

Technical Users

New Technical User

Name

Demo Technical User

INTEGRATION_API

Test User

Access token for: Demo Technical User

UMS

/Wjlr

PLEASE COPY IT IMMEDIATELY!

When this dialog closes, you won't be able to see it again.

OK

Expiry Date

2025-01-01

2123-02-03

2100-01-01

An API token is generated once you have created a technical user

Managing Technical Users

Multiple administrators can collaboratively manage technical users. As an administrator, you can:

- **Create technical users**
- **Replace the API token of a technical user:** To do this, click **Replace token** on the technical user you need, then confirm your action. When you replace an API token, the previous token is disabled. Any integrations or scripts using the previous token will stop working.
- **Update a technical user:** To update the details associated with a technical user, select a user on the **Technical Users** page, and update the information you need. You can change the expiration date of the associated API token, but this action doesn't replace or delete the token.
- **View the changelog of associated actions:** To view the history of updates for a technical user, click **History** on the technical user you need. You can see the full changelog of actions associated with the user.
- **Delete a technical user:** To do this, click the trash bin icon on the technical user you need, then confirm your action. When you delete a technical user, the API token associated with the user is also deleted. Any integrations or scripts using the token will stop working.

Best Practices

Here are some best practices to securely use API tokens. This list is non-exhaustive.

- Treat an API token like a password or other sensitive credentials.
- Don't share API tokens with anyone who shouldn't be using them.
- Delete unnecessary API tokens by deleting technical users.
- Replace API tokens periodically.
- Store API tokens securely, for example, by using secrets management tools.
- Securely use API tokens in your code.
- Transfer API tokens securely.
- Set up monitoring and auditing to detect and respond to any unauthorized access.

Configuring Authentication to SAP LeanIX APIs in Postman

Configure OAuth 2.0 authentication to SAP LeanIX APIs for a collection in Postman. The configuration applies to both REST and GraphQL protocols.

[Postman](#)  is an API platform for building and using APIs. For more information, refer to the Postman documentation.

Prerequisites

- Learn how authentication to SAP LeanIX APIs works. For details, see [Authentication to SAP LeanIX Services](#).
- Get an API token by creating a technical user. To learn how, see [Technical Users](#).
- Install Postman and ensure that you know how to use it.

Procedure

1. Select the collection for which you want to configure authentication. If needed, create a new collection.
2. On the collection page, navigate to the **Authorization** tab.
3. In the **Type** list, select **OAuth 2.0**.
4. Under **Configure New Token**, click **Edit token configuration** and specify the following details:
 - **Token Name**: Enter a name for the token.
 - **Grant Type**: Select **Client Credentials**.
 - **Access Token URL**: Enter the following URL of the authentication endpoint:
`https://{SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token`. Replace `{SUBDOMAIN}` with your SAP LeanIX subdomain. You can copy the subdomain value from the URL of your workspace.
 - **Client ID**: Enter `apitoken`.
 - **Client Secret**: Enter the API token that you obtained by creating a Technical User.
 - **Client Authentication**: Select **Send as Basic Auth header**.

LeanIX

Overview **Authorization** ● Pre-request Script Tests Variables Runs

This authorization method will be used for every request in this collection. You can override this by specifying one in the request.

Configure New Token

Token Name	LeanIX Access Token
Grant Type	Client Credentials ▼
Access Token URL ⓘ	https://{SUBDOMAIN}.leanix.net/ser'...
Client ID ⓘ	apitoken ⚠
Client Secret ⓘ	{API_TOKEN} ⚠
Scope ⓘ	e.g. read:org
Client Authentication ⓘ	Send as Basic Auth header ▼

Configuring a New Token for OAuth 2.0 Authentication in Postman

5. To save the changes that you applied to the collection, click **Save**.

The authorization method that you configured will be used for every request in the collection.

To request a new access token, click **Get New Access Token**.

Base URL

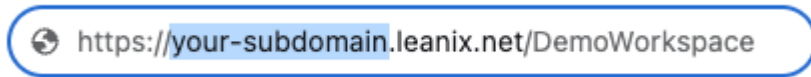
Find your base URL to use in API requests or configurations.

The SAP LeanIX base URL varies among customers. In our documentation, the base URL appears as follows:

`https://{SUBDOMAIN}.leanix.net`

→ Tip

Copy the subdomain value from your workspace URL. If you need help accessing the workspace, contact your workspace administrator.



Example Subdomain in a Workspace URL

Guides and Tutorials

The guides and tutorials in this section offer practical examples and detailed instructions to enhance your development processes. Use these resources to learn the essentials and find inspiration for your integrations.

- [Custom Reports](#)
- [Code-Based Automations](#)
- [Importing and Exporting Data](#)
- [Bookmarks](#)
- [Updating User Permissions](#)
- [Retrieving Survey Results](#)
- [Sending Alerts to Slack and Microsoft Teams](#)

Custom Reports

Learn how to build custom reports tailored to your organization's needs.

SAP LeanIX offers a wide range of standard, ready-to-use reports that enable you to visualize and analyze data. To learn more about using reports, see [Reports](#).

While standard reports cater to a broad range of requirements, there may be specific use cases that benefit from a more customized approach. For example, your organization may need reports that are tailored to your particular domain or industry.

As a developer, you can build custom reports using the [Reporting Library and Custom Report Tools](#). This toolkit enables you to connect to your SAP LeanIX workspace, retrieve data, build reports, and then upload them to your workspace or publish them on the SAP LeanIX extension hub.

In this section, you'll learn how to:

- Build custom reports in various formats
- Query data for reports

- Visualize report data
- Upload reports to your workspace or publish them on the extension hub

Set Up Your Custom Reports Project

Before you start developing custom reports, configure your project with the scaffolding tool.

Overview

The SAP LeanIX custom report tools accelerates the creation of custom reports using the SAP LeanIX reporting library. It generates a project with all the necessary files and dependencies.

In this tutorial, you'll learn how to create and configure a custom reports project using the [@sap/create-leanix-custom-report](#) npm package.

By the end of this tutorial, you'll be able to create your custom report project.

Prerequisites

Before you start:

- Install [NodeJS LTS](#) .
- Get admin access to your SAP LeanIX workspace.
- Obtain an API token by creating a technical user. For more information, see [Technical Users](#).

This tutorial assumes you have working knowledge of:

- [Reports](#) in SAP LeanIX
- [SAP LeanIX Reporting Library](#)
- React and TypeScript (see <https://react.dev/learn/typescript>)

Step 1: Create Your Project

Run the scaffolding tool using a terminal and your [Node.js](#) installation:

```
npm create @sap/leanix-custom-report@latest
```

```
[+] Workspace npm create @sap/leanix-custom-report@latest  

> npx  

> "create-leanix-custom-report"  

SAP LeanIX Custom Report Scaffolding Tool v2.4.2  

[✓] Project name: ... my-custom-report  

[✓] Unique id for this report in Java package notation (e.g. net.leanix.barcharts) ... custom.report.demo  

[✓] Who is the author of this report (e.g. LeanIX GmbH) ... SAP LeanIX  

[✓] A title to be shown in LeanIX when report is installed ... Custom Report Demo  

[✓] Description of your project ... My Demo Custom Report  

[✓] Which host do you want to work with? ... demo-eu.leanix.net  

[✓] API-Token for Authentication (see: https://dev.leanix.net/docs/authentication#section-generate-api-tokens)  

[✓] ⚠ Security advice: API token will be persisted in the report config file ... LXT_XXXXXXXXXXXXXXXXXXXXX  

[✓] Are you behind a proxy? ... No / Yes  

✓ Successfully authenticated with LeanIX  

✓ Set up local MCP servers for AI development?  

  - Chrome DevTools MCP (requires Chrome browser)  

  - LeanIX MCP Server (workspace data access)  

  ⚠ Security advice: API token will be persisted in the MCP server config files  

  Config files are gitignored and take precedence over global settings. ... No / Yes  

🔧 Scaffolding project in /Users/I529256/Workspace/my-custom-report...  

Using React + TypeScript template  

🔥 Done. Now run:  

cd my-custom-report  

npm install  

npm run dev  

✓ MCP servers configured (.vscode/mcp.json, .mcp.json)  

Supports: GitHub Copilot (VS Code) and Claude Code  

  - Chrome DevTools MCP (AI report verification)  

  - LeanIX MCP Server (workspace data access)
```

Once you've answered the prompts, a new project subdirectory will be created with the following structure:

```
my-custom-report/
├── README.md
├── lxr.json
├── package.json
├── vite.config.js
├── tsconfig.json
├── index.html
├── src/
│   ├── App.tsx
│   ├── App.css
│   └── main.tsx
```

Code Walkthrough

The project is a fully-working template report and includes the following key files:

- `lxr.json`: Contains your SAP LeanIXworkspace credentials used by the development server.
- `package.json`: Contains project metadata, dependencies, and scripts for developing, building, and uploading your report.
- `vite.config.js`: Vite configuration with the SAP LeanIXX plugin pre-configured.
- `src/App.tsx`: The main application component and your starting point for building the report.

Step 2: Optional - Configure Workspace Credentials

The `lxr.json` file contains your SAP LeanIX workspace credentials used during development. This file will not be part of the package that is eventually installed to a workspace.

You typically only need to modify this file in specific situations:

- Switching to a different workspace, for example, uploading the report in another workspace
- Your API token has expired and needs to be replaced
- The API token was entered incorrectly during project creation

```
{
  "host": "${YOUR_INSTANCE}.leanix.net",
  "apitoken": "${YOUR_API_TOKEN}"
}
```

Replace:

- `${YOUR_INSTANCE}` is the subdomain of your SAP LeanIX workspace host, for example, `demo-eu-1` in `demo-eu-1.leanix.net`
- `${YOUR_API_TOKEN}` is your SAP LeanIX technical user API token see [Technical Users](#)

If you're working behind a proxy, add the `proxyURL` property:

```
{
  ...,
  "proxyURL": "http://your-proxy:8080"
}
```

Step 3: Configure Report Metadata

The `package.json` file configures report metadata for the SAP LeanIX reporting library via the `leanixReport` field.

package.json

The following table contains configurable fields in the `package.json` file.

Property	Description	Required
name	The name of the project. Used as folder and npm package name. Must follow npm package naming scheme, e.g.: my-custom-report	Required
version	The version of the custom report as displayed in the workspace.	Required
author	The author of the custom report.	Optional
description	The description of the custom report.	Optional
leanixReport	The object that contains additional information related to the report.	Required

The `leanixReport` object contains fields listed in the following table.

Property	Description	Required
id	The unique ID of the report.	Required

Property	Description	Required
	<code>id</code> adheres to the Java package naming convention. Ensure the ID follows these specifications: - Alphanumeric strings <code>a - z , 0 - 9</code> - Underscores <code>_</code> - Dots <code>.</code> - must not end with a dot or comma The part of the <code>id</code> following the last dot is the unique identifier displayed in the SAP LeanIX reporting menu. Ensure it is unique to your workspace avoid overriding existing reports.	
<code>title</code>	The title of the report that is displayed in the user interface.	Optional
<code>aiAssisted</code>	Indicates whether the report was built with AI assistance. Set to <code>true</code> when using AI to build custom reports. Defaults to <code>false</code> .	Optional
<code>defaultConfig</code>	The default configuration object.	Optional

Summary

In this tutorial, you learned how to create and configure a custom reports project using the SAP LeanIX custom report tools. A React/TypeScript project skeleton was created, equipped with all the necessary files and dependencies for building custom reports.

Next Steps

Once you've installed all the dependencies and configured your environment, you're ready to start developing your first custom report. Go to [Your First Custom Report](#).

Related Information

- [Reporting Library and Custom Report Tools](#)
- [Your First Custom Report](#)
- [Reports](#)
- [Technical Users](#)
- React and TypeScript (see <https://react.dev/learn/typescript>)
- [Node.js](#) 

Your First Custom Report

Learn how to create custom reports that visualize applications by business criticality.

Overview

Custom reports offer a powerful way to extract valuable insights from your SAP LeanIX data. By creating tailored reports, you can focus on specific aspects of your organization's IT landscape and track key performance indicators (KPIs) that align with your strategic goals.

When you create a new project using the SAP LeanIX custom report tools, it includes a fully functional example report out of the box. This report groups applications by business criticality level and displays the distribution across the levels as an interactive bar chart.

In this tutorial, you'll create a report and explore how it queries data from SAP LeanIX and renders a visualization. By the end of this tutorial, you'll have a running custom report based on the data in your SAP LeanIX workspace.

Prerequisites

Before you start, create a project using the SAP LeanIX custom report tools. To learn more, see [Set Up Your Custom Reports Project](#).

- Get admin access to your SAP LeanIX workspace.

Step 1: Navigate to Your Project Directory

Switch to the created directory:

```
cd my-custom-report
```

Step 2: Explore the Template Report

The created project already includes a working example report. The report code lives in `src/App.tsx`. Run `npm run dev` and open the link to view it in your SAP LeanIX workspace.

The implementation demonstrates the core building blocks of the SAP LeanIX reporting library:

- `src/main.tsx` is the application entry point. It mounts the React app into the `#report` element, which is the container expected by the SAP LeanIX reporting environment.
- `src/App.tsx` is the main report component. It initializes the SAP LeanIX reporting library, queries the configured workspace for application fact sheets, and renders a bar chart grouped by business criticality.

Code Walkthrough

Explore the key building block of a custom report:

- **Library:** `lx.init()` connects the report to the SAP LeanIX workspace and returns a `setup` object containing the workspace data model and settings.
- **Data model:** The business criticality field metadata, including value order, labels, and colors, is read from `setup.settings.dataModel`. This ensures the chart matches the workspace configuration.
- **Facet registration:** `lx.ready()` registers a facet that requests `Application` fact sheets with their `id`, `displayName`, and `businessCriticality` attributes. SAP LeanIX calls the facet `callback` whenever the data changes, for example, when the user applies filters.

- **Chart:** The BarChart component uses [Chart.js](#) to render the distribution of applications across criticality levels. It uses the colors and labels defined in the workspace data model.

Step 3: Install Dependencies

Switch to your project directory and install dependencies:

```
npm install
```

Step 4: View the Custom Report

Switch to your project directory and install dependencies:

```
npm run dev
```

```
> my-custom-report@0.0.0 dev
> vite
Using workspace: workspaceName
VITE vX.Y.Z ready in 312 ms
Your LeanIX Custom Report is running at:
http://localhost:62551/workspaceName/reporting/dev?url=http%3A%2F%2Flocalhost%3A5173&
```

Open the SAP LeanIX URL displayed in your terminal. The report opens under **Reports** in your SAP LeanIX workspace. If the report does not open automatically, hold **Ctrl** (**Cmd** on macOS) and click the URL. Alternatively, copy and paste the URL into your browser.

The report displays a bar chart showing the distribution of your applications across business criticality levels using the colors configured in your workspace data model. The different levels of business criticality span from administrative to mission critical.



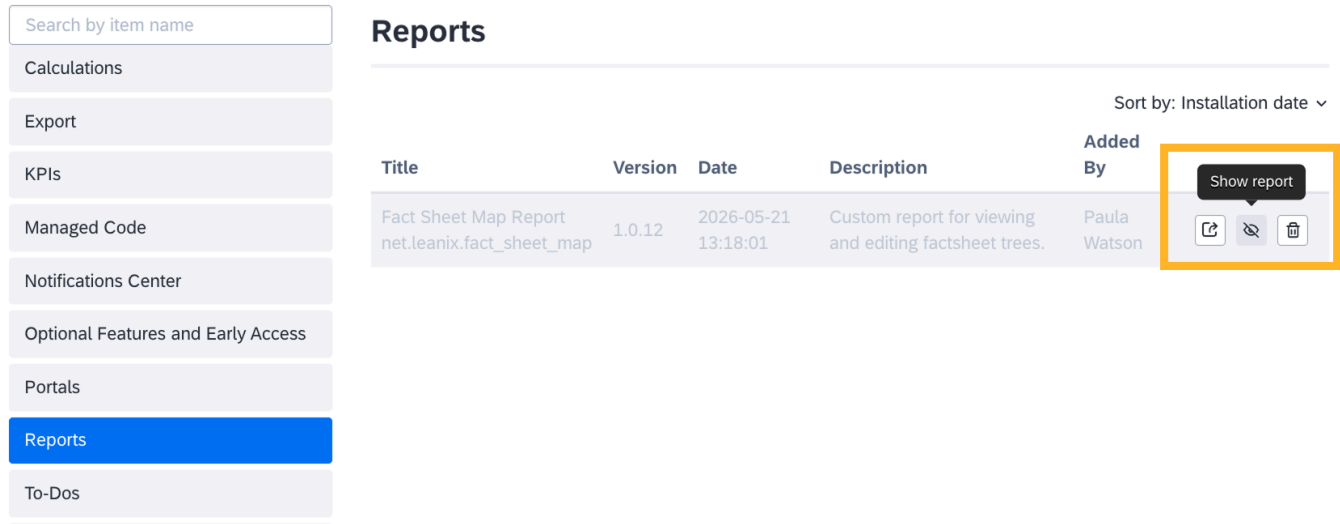
Step 5: Upload the Custom Report to Your Workspace

Upload the custom report to your SAP LeanIX workspace:

```
npm run upload
```

This builds the report and publishes it to the workspace configured in your `lxr.json` file. To make it available to other users change its visibility.

- 1. Navigate to **Administration > Reports**.
- 2. The uploaded report is displayed greyed out. Click **Show report**.



Summary

In this tutorial, you created a new custom report project, went through the basic development steps, and finally you published the report to your workspace.

This tutorial covered the following key concepts of the SAP LeanIX reporting library:

- Initializing the library with `lx.init()` and receiving the workspace `setup` object
- Reading field metadata (labels, colors, value order) from the workspace data model
- Registering a facet with `lx.ready()` to query fact sheet data and respond to filter changes to receive live data updates
- Rendering workspace data using React and Chart.js

Next Steps

Now that you have a running report, you can customize it to fit your organization's needs. Change the fact sheet type, query different fields, or replace the chart type. Move on to [Facet Filtering in Custom Reports](#)

Facet Filtering in Custom Reports

Learn how to use facets to filter data when building custom reports.

Overview

Fact sheets form the backbone of SAP LeanIX. They serve as the primary source of information about your IT landscape. Generating reports across different fact sheet types gives your organization deeper insights into your IT environment.

Custom reports in SAP LeanIX have built-in filtering capabilities. This allows for a more thorough and detailed overview of your IT landscape.

Facets control which fact sheets are retrieved. They support additional callbacks, serving custom business logic when filter changes occur. Using facets, you refine the data you're retrieving. Facets also dynamically respond to filter changes, making your reports more flexible and robust.

In this tutorial, you'll learn how to use the facet filtering feature of the SAP LeanIX reporting library. You'll calculate the count of fact sheets and the average completion ratio for the fact sheets in your workspace. By the end of this tutorial, you'll be able to use facets to filter data when creating custom reports.

For more details, see Facet filter tool [ReportFacetsConfig | @leanix/reporting - v0.4.177](#) .

Prerequisites

Before you start, create a project using the SAP LeanIX custom report tools. To learn more, see [Set Up Your Custom Reports Project](#)

Step 1: Update the Report Component

Open `src/App.tsx` and replace its contents with the following sample code.

This configures the facets to request `type` and `completion` { `completion` } for all fact sheets. It groups the fact sheets by type, and uses the existing `BarChart` component to render the average completion percentage per fact sheet type. Each bar is colored using the workspace-configured color for that fact sheet type, and labels are translated to match the user's language settings.

```
import type { lxr } from '@leanix/reporting';
import { lx } from '@leanix/reporting';
import Chart from 'chart.js/auto';
import { useEffect, useMemo, useRef, useState } from 'react';
import './App.css';
interface BarChartProps {
  data: {
    labels: string[];
    values: number[];
    colors: string[];
  };
}
function BarChart({ data }: BarChartProps) {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const chartRef = useRef<Chart | null>(null);
  useEffect(() => {
    if (!canvasRef.current || data.labels.length === 0) return;
    if (chartRef.current) chartRef.current.destroy();
    const ctx = canvasRef.current.getContext('2d');
    if (!ctx) return;
    chartRef.current = new Chart(ctx, {
      type: 'bar',
      data: {
        labels: data.labels,
        datasets: [
          {
            data: data.values,
            backgroundColor: data.colors,
            borderColor: data.colors,
            borderWidth: 1
          }
        ]
      }
    });
  }, [data]);
}
```

```

    },
    options: {
      maintainAspectRatio: false,
      plugins: {
        legend: { display: false },
        tooltip: {
          callbacks: { label: (item) => `${item.formattedValue} %` }
        }
      },
      scales: {
        y: {
          min: 0,
          max: 100,
          ticks: { callback: (v) => `${v}%` }
        }
      }
    }
  });
  return () => {
    chartRef.current?.destroy();
  };
}, [data]);
return (
  <div style={{ width: '100%', height: '100%' }}>
    <canvas ref={canvasRef} />
  </div>
);
}
function App() {
  const [factSheets, setFactSheets] = useState<lxr.FactSheet[]>([]);
  useEffect(() => {
    const initReport = async () => {
      await lx.init();
      lx.ready({
        facets: [
          {
            key: 'main',
            attributes: ['type', 'completion { completion }'],
            callback: (data) => {
              setFactSheets(data || []);
            }
          }
        ]
      });
    };
    initReport();
  }, []);
  const chartData = useMemo(() => {
    if (!lx.currentSetup?.settings?.viewModel)
      return { labels: [], values: [], colors: [] };
    const byType: Record<string, number[]> = {};
    for (const fs of factSheets) {
      const type = fs.type as string;
      const completion =
        (fs.completion as { completion: number } | undefined)?.completion ?? 0;
      if (!byType[type]) byType[type] = [];
      byType[type].push(completion);
    }
    const viewModelFactSheets = lx.currentSetup.settings.viewModel.factSheets;
    const fsTypes = viewModelFactSheets
      .map((f) => f.type)
      .filter((type) => byType[type]);
    const labels = fsTypes.map((type) => lx.translateFactSheetType(type));
    const values = fsTypes.map((type) => {
      const completions = byType[type];
      const avg =
        completions.reduce((sum, c) => sum + c, 0) / completions.length;
      return parseFloat((avg * 100).toFixed(2));
    });
    const colors = fsTypes.map((type) => {
      return (
        viewModelFactSheets.find((f) => f.type === type)?.bgColor ?? '#888888'
      );
    });
  });
}

```

```

    return { labels, values, colors };
  }, [factSheets]));
  return (
    <div className="app">
      <div className="chart-container">
        <BarChart data={chartData} />
      </div>
    </div>
  );
}
export default App;

```

Code Walkthrough

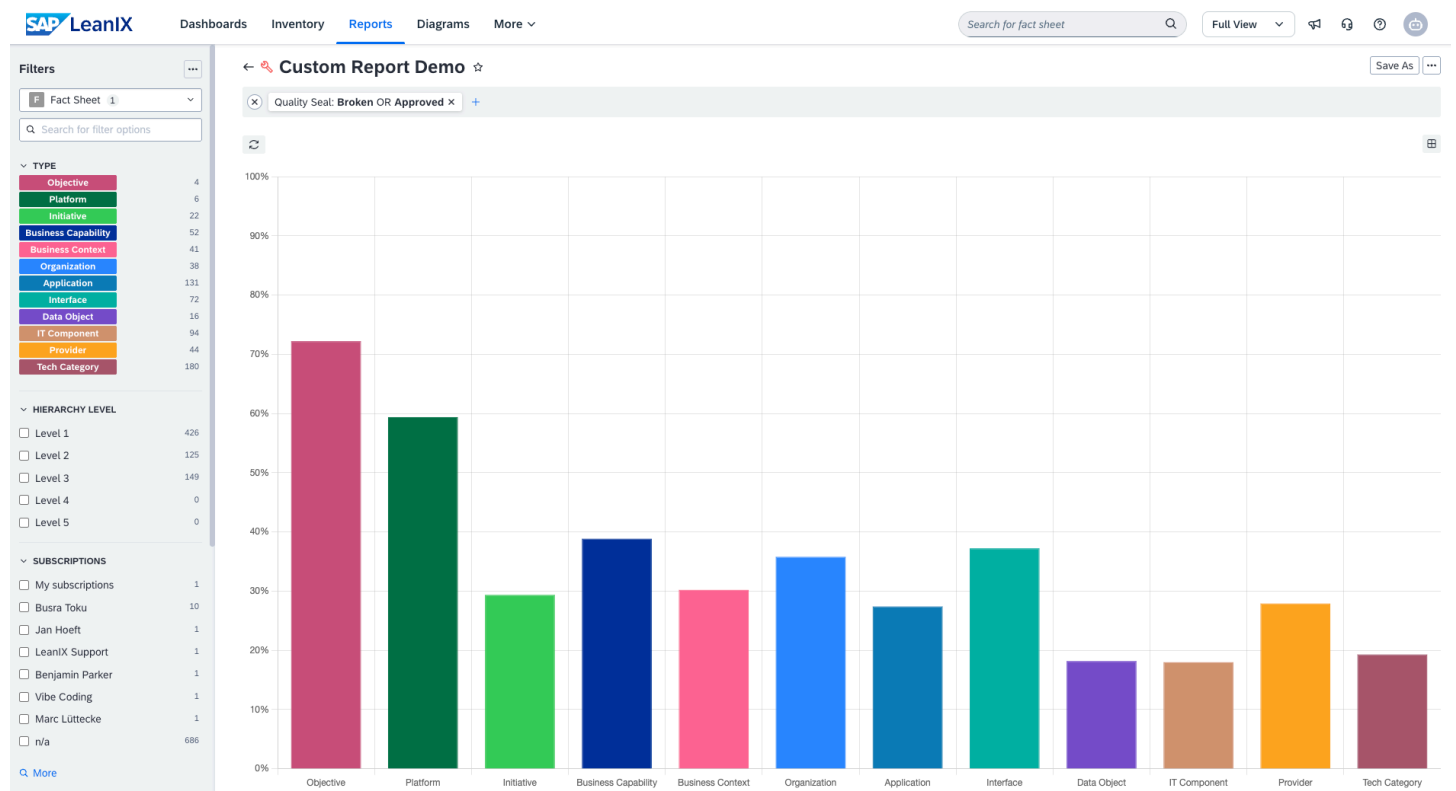
- The facet attributes request type and completion { completion }, so the SAP LeanIX reporting library fetches completion data for all fact sheet types.
- The chartData memo groups fact sheets by type and computes the average completion percentage per type.
- The BarChart component and the .app / .chart - container layout the report, they are provided by the scaffolded template and do not require changes.
- The callback(lx.ready > facets > callback) is invoked automatically by the SAP LeanIX reporting library both on initialization and whenever the user changes filters in the filter sidebar without additional wiring.

Step 2: View the Custom Report

Start the development server:

```
npm run dev
```

Open the SAP LeanIX URL displayed in the terminal. The report opens in your SAP LeanIX workspace and displays the total fact sheet count and average completion score for all fact sheets in your workspace.



Step 3: Use Filters in the Filter Sidebar

Filters in the filter sidebar under **Reports** in SAP LeanIX let you customize the data shown in your custom report. Because the `facetCallback` is invoked automatically on every filter change, the fact sheet count and average completion score update in real time as you apply or remove filters.

Experiment with the filters to see how they impact the data shown in your custom report.

Summary

In this tutorial, you learned how to use facets in the SAP LeanIX reporting library to filter data when creating a custom report:

- Use the `attributes` field within a facet to request the `completion` attribute from fact sheets
- Calculate the average completion score reactively using `useMemo`, so the report updates automatically when the facet callback delivers new data
- Use filters in the filter sidebar under **Reports** in SAP LeanIX to dynamically control which fact sheets are included in the report

Next Steps

To explore more advanced filtering capabilities, see the next tutorial [GraphQL Filtering in Custom Reports](#).

GraphQL Filtering in Custom Reports

Learn how to use GraphQL queries when creating custom reports.

Overview

GraphQL provides a high degree of flexibility, enabling you to customize queries to meet your exact requirements and retrieve specific data without over-fetching or under-fetching. When integrated with the SAP LeanIX reporting library, it facilitates the creation of custom reports that are both efficient and tailored to your unique needs.

In this tutorial, you'll learn how to create a custom report using [GraphQL queries](#) with the SAP LeanIX reporting library. Specifically, you'll use the `lx.executeGraphQL` method, which allows you to run custom GraphQL queries directly against your SAP LeanIX workspace. This report calculates the count of fact sheets and the average completion ratio for the fact sheets in your workspace.

Prerequisites

Before you start, create a project using the SAP LeanIX custom report tools. To learn more, see [Set Up Your Custom Reports Project](#).

Step 1: Update the Report Component

Open `src/App.tsx` and replace its contents with the following sample code.

In the `App` component, the facet callback is changed to `facetFiltersChangedCallback`. The new callback fetches data using `lx.executeGraphQL` and displays a legend through `lx.showLegend()`.

```
import type { lxr } from '@leanix/reporting';
import { lx } from '@leanix/reporting';
import Chart from 'chart.js/auto';
```



```

import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import './App.css';
const QUERY = `
  query($filter: FilterInput!) {
    allFactSheets(filter: $filter) {
      edges {
        node {
          type
          completion { completion }
        }
      }
    }
  }
`;
interface FactSheetEdge {
  node: {
    type: string;
    completion: { completion: number };
  };
}
interface ChartData {
  labels: string[];
  values: number[];
  colors: string[];
}
function PolarAreaChart({ data }: { data: ChartData }) {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const chartRef = useRef<Chart | null>(null);
  useEffect(() => {
    if (!canvasRef.current || data.labels.length === 0) return;
    if (chartRef.current) chartRef.current.destroy();
    const ctx = canvasRef.current.getContext('2d');
    if (!ctx) return;
    chartRef.current = new Chart(ctx, {
      type: 'polarArea',
      data: {
        labels: data.labels,
        datasets: [
          {
            data: data.values,
            backgroundColor: data.colors.map((c) => `${c}cc`),
            borderColor: data.colors,
            borderWidth: 1
          }
        ]
      },
      options: {
        maintainAspectRatio: false,
        plugins: {
          legend: { display: false },
          tooltip: {
            callbacks: { label: (item) => `${item.formattedValue} %` }
          }
        },
        scales: {
          r: {
            min: 0,
            max: 100,
            ticks: { callback: (v) => `${v}%` }
          }
        }
      }
    });
  });
  return () => {
    chartRef.current?.destroy();
  };
}, [data]);
return (
  <div style={{ width: '100%', height: '100%' }}>
    <canvas ref={canvasRef} />
  </div>
);
}
function App() {

```

```

const [edges, setEdges] = useState<FactSheetEdge[]>([]);
const fetchData = useCallback(
  async (filters: Partial<lxr.ReportFacetsSelection> = {}) => {
    const {
      facets: facetFilters = [],
      fullTextSearchTerm: fullTextSearch = '',
      directHits = []
    } = filters;
    const qlFilter = {
      facetFilters: facetFilters.length > 0 ? facetFilters : undefined,
      fullTextSearch: fullTextSearch?.trim() ? fullTextSearch : undefined,
      ids: directHits.length > 0 ? directHits.map(({ id }) => id) : undefined
    };
    const response = await lx.executeGraphQL(
      QUERY,
      JSON.stringify({ filter: qlFilter })
    );
    setEdges(response?.allFactSheets?.edges ?? []);
  },
  []
);
useEffect(() => {
  const initReport = async () => {
    await lx.init();
    lx.ready({
      facets: [
        {
          key: 'main',
          facetFiltersChangedCallback: (filters) => {
            fetchData(filters);
          }
        }
      ]
    });
    fetchData();
  };
  initReport();
}, [fetchData]);
const chartData = useMemo(): ChartData => {
  if (!lx.currentSetup?.settings?.viewModel)
    return { labels: [], values: [], colors: [] };
  const byType: Record<string, number[]> = {};
  for (const { node } of edges) {
    if (!byType[node.type]) byType[node.type] = [];
    byType[node.type].push(node.completion.completion);
  }
  const viewModelFactSheets = lx.currentSetup.settings.viewModel.factSheets;
  const fsTypes = viewModelFactSheets
    .map((f) => f.type)
    .filter((type) => byType[type]);
  const labels = fsTypes.map((type) => lx.translateFactSheetType(type));
  const values = fsTypes.map((type) => {
    const completions = byType[type];
    const avg =
      completions.reduce((sum, c) => sum + c, 0) / completions.length;
    return parseFloat((avg * 100).toFixed(2));
  });
  const colors = fsTypes.map(
    (type) =>
      viewModelFactSheets.find((f) => f.type === type)?.bgColor ?? '#888888'
  );
  return { labels, values, colors };
}, [edges]);
useEffect(() => {
  lx.showLegend(
    chartData.labels.map((label, i) => ({ label, bgColor: chartData.colors[i] }))
  );
}, [chartData]);
return (
  <div className="app">
    <div className="chart-container">
      <PolarAreaChart data={chartData} />
    </div>
  </div>

```

```
);
}
export default App;
```

Code Walkthrough

- The facet attributes requests type and completion { completion }, so the SAP LeanIX reporting library fetches completion data across all fact sheet types.
- The chart is a `PolarAreaChart` where each segment represents a fact sheet type and its radius reflects the average completion percentage.
- `lx.showLegend()` is called whenever `chartData` changes to keep the legend in sync with the visible fact sheet types and their workspace-configured colors.
- The facet callback is invoked automatically by the SAP LeanIX reporting library both on initialization and whenever the user changes filters in the filter sidebar without additional wiring.

Automatic vs. Manual Data Fetching

By default, the SAP LeanIX reporting library manages data fetching automatically: when a user changes a filter, the library calls the facet callback with pre-filtered fact sheet data, so your report code only needs to handle the result. See a code sample for automatic data fetching in the tutorial [Facet Filtering in Custom Reports](#).

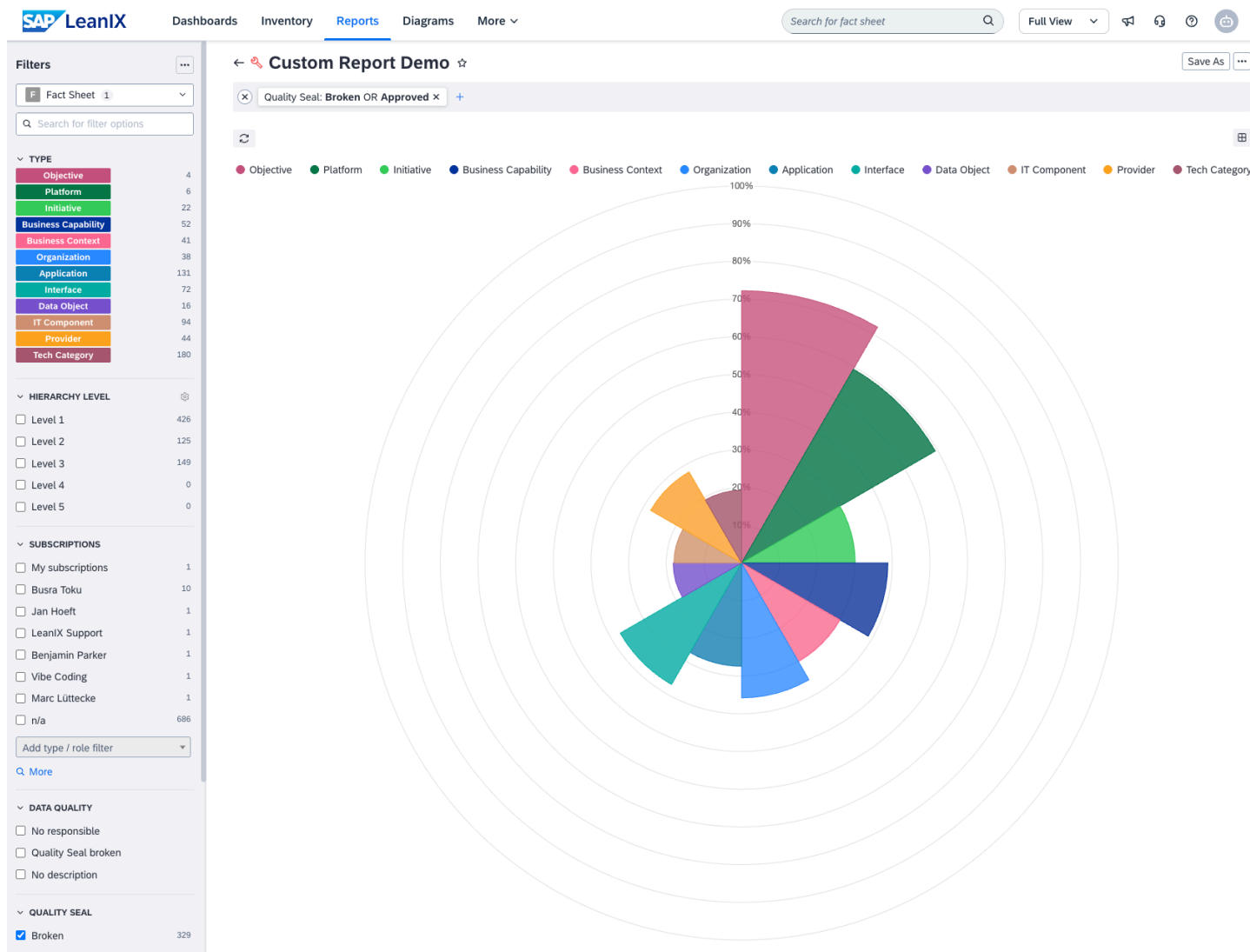
With `facetFiltersChangedCallback` and `lx.executeGraphQL`, you take direct control. You write the GraphQL query and variables yourself, and your callback is responsible for both the initial data fetch and responding to every subsequent filter change. See the code sample in Step 1 of this tutorial.

Step 2: View the Custom Report

Start the development server:

```
npm run dev
```

Open the SAP LeanIX URL displayed in the terminal. The report opens in your SAP LeanIX workspace and displays total count of fact sheets and the average completion ratio. Use the filter sidebar to see both values update in real time as filters are applied or removed.



Summary

In this tutorial, you learned how to use GraphQL queries when creating a custom report with the SAP LeanIX reporting library. You learned how to:

- Use `lx.executeGraphQL` to run a custom GraphQL query against your SAP LeanIX workspace and retrieve fact sheet completion data
- Use `facetFiltersChangedCallback` to respond to filter sidebar changes and re-execute the query with updated filter variables
- Manually trigger the initial data fetch after `lx.ready()`, which is required when using `facetFiltersChangedCallback` rather than the `facet callback`
- Display a synchronized legend using `lx.showLegend()` that updates whenever the visible fact sheet types change

Next Steps

If you want to explore more options for custom reports, try our next tutorial

Configure Your Custom Report

Learn how to add a button that allows configuration of your custom report and opens a modal to let users select which data to display.

Overview

SAP LeanIX custom reports can expose native toolbar controls that let users configure what data is displayed without leaving the workspace. This tutorial introduces two complementary UI controls:

- **View Selection:** Switch between available workspace views, such as lifecycle or business criticality with a **View** dropdown
- **Report Configuration:** Open a modal to select the fact sheet type with a **Configure** button

Next to adding the UI controls you'll extend the template report. Display your fact sheet selection as a half-doughnut chart, grouped by the view selected in the **View** dropdown.

By the end of this tutorial, you'll be able to add a native **View** dropdown and a configurable modal to any custom report using the reporting library APIs including `reportViewFactSheetType`, `reportViewCallback`, `menuActions`, `configureCallback`, and `lx.openFormModal`.

Prerequisites

Before you start, create a project using the SAP LeanIX custom report tools. To learn more, see [Set Up Your Custom Reports Project](#).

Step 1: Update the Report Component

Open `src/App.tsx` and replace its contents with the following sample code.

This extends the template report with three new concepts:

- `reportViewFactSheetType` and `reportViewCallback` to wire the native **View** dropdown.
- `menuActions` and `lx.openFormModal` to add the **Settings** button and modal.
- `lx.updateConfiguration()` to switch the fact sheet type at runtime. The `DoughnutChart` component follows the same pattern as the `BarChart` in the template report.

```
import type { Lxr } from '@leanix/reporting';
import { lx } from '@leanix/reporting';
import Chart from 'chart.js/auto';
import { useEffect, useMemo, useRef, useState } from 'react';
import './App.css';
interface DoughnutChartProps {
  data: {
    labels: string[];
    values: number[];
    colors: string[];
  };
}
function DoughnutChart({ data }: DoughnutChartProps) {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const chartRef = useRef<Chart | null>(null);
  useEffect(() => {
    if (!canvasRef.current || data.values.length === 0) return;
    if (chartRef.current) chartRef.current.destroy();
    const ctx = canvasRef.current.getContext('2d');
    if (!ctx) return;
    chartRef.current = new Chart(ctx, {
      type: 'doughnut',
      data: {
```

```

    labels: data.labels,
    datasets: [
      {
        data: data.values,
        backgroundColor: data.colors,
        borderColor: '#d4d4d4',
        borderWidth: 2
      }
    ]
  },
  options: {
    maintainAspectRatio: false,
    circumference: 180,
    rotation: -90,
    plugins: { legend: { display: false } }
  }
});
return () => {
  chartRef.current?.destroy();
};
}, [data]);
return (
  <div style={{ width: '100%', height: '100%' }}>
    <canvas ref={canvasRef} />
  </div>
);
}
function App() {
  const [factSheets, setFactSheets] = useState<lxr.FactSheet[]>([]);
  const [viewResult, setViewResult] = useState<lxr.ViewResult | null>(null);
  const selectedTypeRef = useRef<string>('');
  useEffect(() => {
    const initReport = async () => {
      const setup = await lx.init();
      const types = Object.keys(setup.settings.dataModel.factSheets);
      const defaultType = types.includes('Application')
        ? 'Application'
        : (types[0] ?? '');
      selectedTypeRef.current = defaultType;
      const makeFacet = (type: string): lxr.ReportFacetsConfig => ({
        key: type,
        fixedFactSheetType: type,
        attributes: ['id'],
        callback: (data) => setFactSheets(data || [])
      });
      const makeConfig = (type: string): lxr.ReportConfiguration => ({
        facets: [makeFacet(type)],
        reportViewFactSheetType: type,
        reportViewCallback: (view) => setViewResult(view),
        menuActions: {
          showConfigure: true,
          configureCallback: async () => {
            const fields: lxr.FormModalFields = {
              factSheetType: {
                type: 'SingleSelect',
                label: 'Fact Sheet Type',
                options: types.map((t) => ({
                  value: t,
                  label: lx.translateFactSheetType(t)
                }))
            }
          }
        };
      });
      const values = await lx.openFormModal(fields, {
        factSheetType: selectedTypeRef.current
      });
      if (values) {
        const newType = values.factSheetType;
        if (
          typeof newType === 'string' &&
          newType !== selectedTypeRef.current
        ) {
          selectedTypeRef.current = newType;
          lx.updateConfiguration(makeConfig(newType));
        }
      }
    };
  });
}

```

```

    }
  }
}
});
lx.ready(makeConfig(defaultType));
};
initReport();
}, []);
const chartData = useMemo(() => {
  if (!viewResult || factSheets.length === 0)
    return { labels: [], values: [], colors: [] };
  // Map fact sheet ID → legend item ID
  const fsToLegendId = new Map(
    viewResult.mapping.map((m) => [m.fsId, m.legendId])
  );
  // Count filtered fact sheets per legend item
  const counts = new Map<number, number>();
  for (const fs of factSheets) {
    const legendId = fsToLegendId.get(fs.id as string);
    if (legendId !== undefined) {
      counts.set(legendId, (counts.get(legendId) ?? 0) + 1);
    }
  }
  // Only include legend items that have data
  const items = viewResult.legendItems.filter(
    (item) => item.inLegend && (counts.get(item.id) ?? 0) > 0
  );
  return {
    labels: items.map((item) => item.label ?? item.value),
    values: items.map((item) => counts.get(item.id) ?? 0),
    colors: items.map((item) => item.bgColor)
  };
}, [factSheets, viewResult]);
// Sync legend with the active view
useEffect(() => {
  if (!viewResult) return;
  lx.showLegend(
    viewResult.legendItems
      .filter((item) => item.inLegend)
      .map((item) => ({
        label: item.label ?? item.value,
        bgColor: item.bgColor
      }))
  );
}, [viewResult]);
return (
  <div className="app">
    <div className="chart-container">
      <DoughnutChart data={chartData} />
    </div>
  </div>
);
}
export default App;

```

Code Walkthrough

- The **View** dropdown uses `reportViewFactSheetType` and `reportViewCallback`. The **Configure** button uses `menuActions` and `lx.openFormModal`.
- `reportViewFactSheetType` specifies which fact sheet types' views to offer in the native **View** dropdown. `reportViewCallback` is invoked on initialization, and each time the user switches views. It provides a `ViewResult` with `legendItems` (colors and labels) and a `mapping` that associates each fact sheet ID with a legend item.
- `menuActions` is passed to `lx.ready()` with `showConfigure: true` to add a **Settings** button to the report toolbar. The `configureCallback` is invoked each time the user clicks it.
- `lx.openFormModal` opens a native SAP LeanIX modal with the fields you define. It returns a promise that resolves with the submitted values, or `false` if the user dismisses the modal. Here, it renders a `SingleSelect` dropdown populated

with all fact sheet types available in the workspace data model, translated via `lx.translateFactSheetType()`.

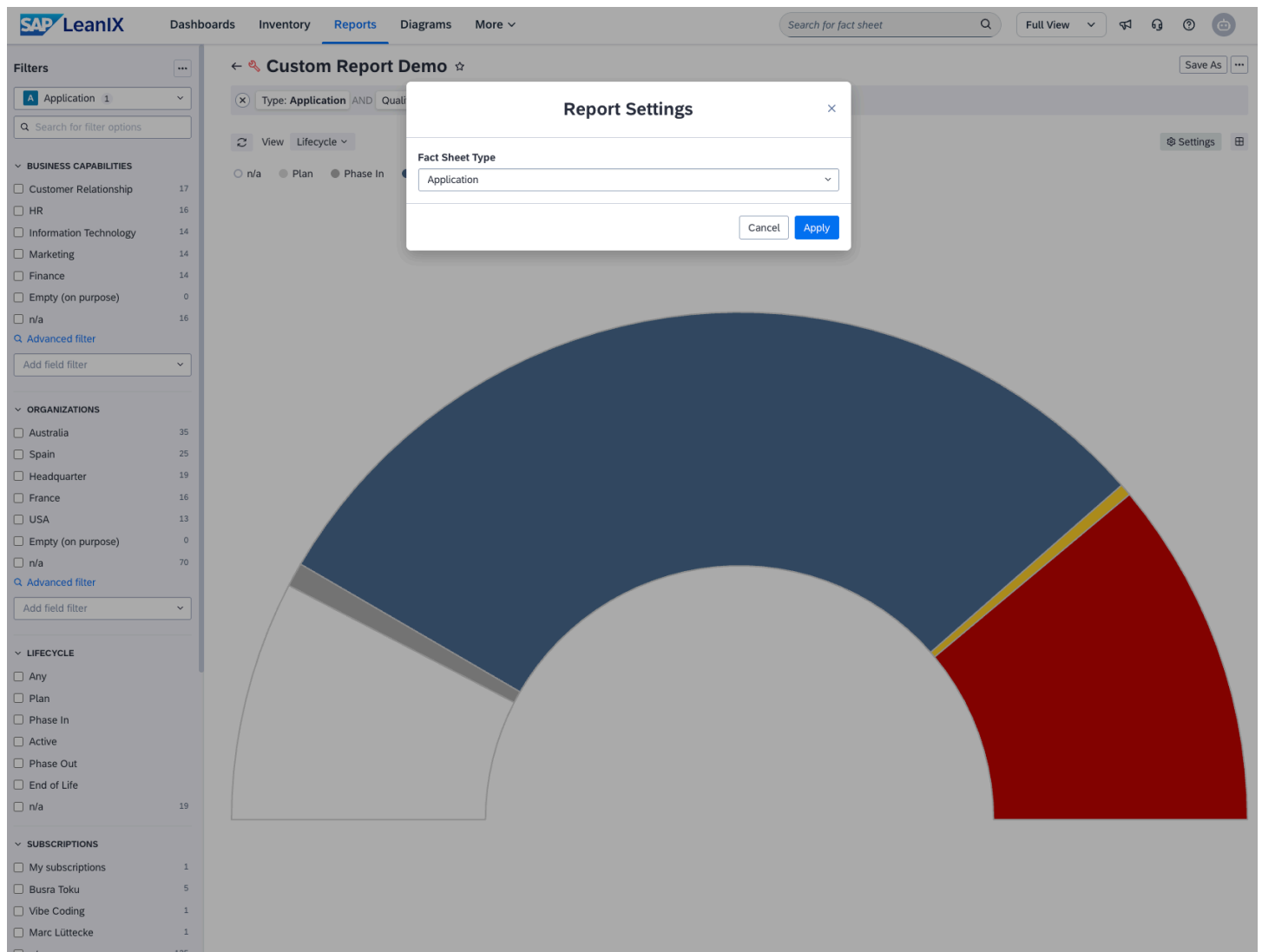
- `selectedTypeRef` holds the currently selected fact sheet type. This ensures that `configureCallback`, registered once in `lx.ready()`, always reads the latest value when the modal opens.
- `lx.updateConfiguration()` is called when the user confirms a new fact sheet type. It updates both the facet's `fixedFactSheetType` and `reportViewFactSheetType` in a single call, so the view dropdown options stay relevant for the newly selected type.
- `lx.showLegend()` is called whenever the view changes to keep the legend panel in sync with the active view's colors and labels.
- The chart data is computed reactively in `useMemo`: for each filtered fact sheet, its ID is matched to its legend item. The counts per legend item are used to build the doughnut segments.

Step 2: View the Custom Report

Start the development server:

```
npm run dev
```

Open the SAP LeanIX displayed in the terminal. The report opens in your SAP LeanIX workspace as half-doughnut chart. It displays applications distributed across the default view's values. Segment colors match your workspace configuration.



Step 3: Configure the Custom Report Using the Modal

Click the **View** dropdown on the left of the toolbar to switch between available views for the selected fact sheet type, for example, **Lifecycle** or **Business Criticality**. The chart and legend update immediately to reflect the new view.

Click the **Settings** button on the right of the toolbar to open the fact sheet type modal. Select a different type and confirm. The report updates both the view options and the chart data for the newly selected type.

Summary

In this tutorial, you learned how to add a native **View** dropdown and a configurable modal to a custom report using the SAP LeanIX reporting library. You learned how to:

- Use `reportViewFactSheetType` and `reportViewCallback` in `lx.ready()` to wire the native **View** dropdown and receive per-fact-sheet view mappings whenever the user switches views
- Use `menuActions` with `showConfigure: true` and `configureCallback` to add a **Settings** button to the report toolbar
- Use `lx.openFormModal` to open a native SAP LeanIX modal and collect user input, using the resolved values to update the report's fact sheet type
- Use `lx.updateConfiguration()` to switch the active fact sheet type at runtime without re-initializing the report
- Use `lx.showLegend()` to keep the legend panel synchronized with the active view's colors and labels

Publishing Custom Reports to the SAP LeanIX Extension Hub

Learn how the SAP LeanIX extension hub works and how to publish your custom reports.

Getting Started with the Extension Hub

The [extension hub](#)  provides access to custom reports developed by SAP LeanIX and external providers. As an admin user, you can download custom reports to your workspace and make them available to all workspace users. To learn more, see [Downloading Reports from the Extension Hub](#).

The extension hub includes free and paid reports. SAP LeanIX reserves the right to release a paid option and other asset types, such as survey templates, configurations, and snapshots, in the future.

The extension hub is designed to:

- Further grow the SAP LeanIX ecosystem by driving continuous product innovation in addition to the core product
- Provide partners with a platform to share their knowledge and offer their expertise
- Offer customers quick access to best practice insights, such as domain or industry-specific, to generate value in no time

Review Criteria

Before you publish your custom report on the extension hub, we review it. To learn about the key steps in this process and the review criteria, refer to [Review Criteria for Custom Reports](#).

Step 1: Register as a Publisher

Follow these steps:

1. Create a [GitLab](#)  account using the same email address that is associated with your SAP LeanIX user account. We use GitLab as a source code repository for both public and private custom reports.

i Note

Each report must be approved by SAP LeanIX before you can publish it to an SAP LeanIX workspace.

If you're new to GitLab, refer to the [GitLab documentation](#)  to learn how to get started. Ensure to set up your environment according to best practices, such as by adding an SSH key.

2. Sign in to the extension hub from your workspace. To do that, choose **More > Extension Hub**.
3. After signing in to the extension hub, click **Publish**, scroll down to the bottom of the page, then click **Send Request**.
4. In the dialog that opens, enter your GitLab username, then click **Submit Request**.
 - o If you're an **SAP LeanIX partner**, you need to sign the Content Provider Agreement to get your publisher request approved. This agreement covers topics such as report ownership and intellectual property. For any questions, please contact your SAP LeanIX Partner Manager.
 - o If you're an **SAP LeanIX customer**, you can create custom reports for your own workspaces without needing a formal agreement. However, if you want to share reports with other customers, you'll need to sign the Content Provider Agreement. For any questions, please contact your Customer Success Manager.

You'll receive an email with further information regarding your request.

Step 2: Create a Report

Follow these steps:

1. In your extension hub dashboard, navigate to the **Reports** section, then click the plus button next to **Reports Management**.
2. In the dialog that appears, enter a name for your report, then click **Create**. This will create a link to a GitLab project within your repository, using the GitLab account that you specified when submitting a publisher request.
3. Develop your custom report in your GitLab project. The report appears on the list of reports in your dashboard.

You can maintain the source code of your report in GitLab. The link to the GitLab repository appears in the report's detail view. You can also change report details such as the description, logo, title, and more.

The following publishing options are available:

- **Public:** Any SAP LeanIX customer can install the report and view its source code.
- **Preview:** Users can view a sample of the report but can't download it to their workspace.
- **Private:** You can select which workspaces the report is available to. It doesn't necessarily have to be your organization's workspaces. Specify the IDs of workspaces where the report should be uploaded. The workspace ID is available in the **API Tokens** section of the administration area.

Step 3: Initiate the Review Process

After you finished developing your report, save your work and initiate the review process by clicking **Submit for Review** in the report detail view.

i Note

Before submitting your report for review, ensure that the final working code is in the `develop` branch.

The SAP LeanIX Customer Success Team will review the report's code, content, technical implementation, functional suitability, and licenses used. For more information, visit [Review Criteria for Custom Reports](#).

You will receive an email notification regarding the approval status of your report or if there are any issues that need addressing. Upon approval, the report is published and made available in the extension hub based on the publishing mode that you selected.

You can check the status of developed reports in your publisher dashboard.

Step 4: Create New Report Versions

Once the report is published, you can create new report versions. To do that, in the **Reports** section of the publisher dashboard, navigate to **Published Reports**, then click **Create New Version**.

You can submit updated versions of your reports featuring improvements, added functionalities, or bug fixes. These new versions will undergo the same review process as a new report.

Please ensure to increment the version number in your GitLab project's `package.json` file when submitting a new version, otherwise uploading the report to a workspace may result in an error.

Review Criteria for Custom Reports

Before you publish your custom report on the SAP LeanIX extension hub, we review it. Learn about the key steps in this process and the review criteria.

This topic outlines the criteria that we're using during the custom report review process. The following steps illustrate our approach to reviewing a report before its publication.

As a general rule, the entire review process should not take longer than 10 business days. However, the duration may occasionally extend due to the complexity and volume of ongoing review requests. If the review involves minor changes to an existing report, please inform us to facilitate a faster process.

Report Metadata Review

This section outlines items required for the reporting metadata.

Screenshots

- Screenshots should be exactly 1280px x 800px and should not exceed a size of 1 MB.
- Multiple screenshots should show a clear progression throughout the report.
- Screenshots should always display the entire browser window without any cropping.

Thumbnails

Thumbnails should be exactly 1280px x 800px and should not exceed a size of 400KB.

Description

- The report should include a concise summary detailing its use cases, functionalities, used fact sheet types, and tags.
- The description should specify if the report modifies existing data (for example, by using a mutation).

Requirements

The requirement of the following for the report:

- Configured data fields
- Specific tags
- Data required for the report to work

Technical Review

This section outlines criteria of the technical review.

Dependencies

Dependencies are audited using [npm-audit](#) 🐼. Vulnerable dependencies are rejected. Please provide details about the cases where changing the used library is not feasible.

Code Readability and Error Handling

- The code should be extensively commented.
- Asynchronous code should handle errors explicitly, using `.catch()` for Promise chains or `try/catch` blocks for `async/await`.
- Traversing JSON response objects should include a null-checker.

API Calls

- API calls, especially mutations, need to be clearly visible and must not obfuscate the usage of data.
- Calls to an external API, other than an SAP LeanIX API, are not allowed. Sending data to external systems is strictly forbidden. Please contact SAP LeanIX Support if your use case necessitates this functionality.

Readme

The code should have a readme file that, at a minimum, lists all the steps required for installation. If dependencies with special installation requirements are used, they should also be listed.

Recreating Existing Functionality

Framework functionality should be used as much as possible and not be recreated.

Commercial Dependencies

- The licence status of all used dependencies is checked.
- Commercial dependencies should be clearly marked in the documentation, and confirmation of eligibility will be requested.

Functional Review

This section outlines criteria of the functional review.

Stability

The report must run out of the box without generating any unhandled errors.

Look and Feel

Reports should be using the same look and feel as default reports, including the following:

- Filtering
- Configuration
- Design

Use Cases

Reports need to have a clear use case.

Code-Based Automations

Streamline workflows for your organization by setting up code-based automations.

To eliminate manual tasks and processes, you can set up event-driven triggers that initiate automated actions. Automations ensure consistent and efficient execution of workflows, which allows you to free up your organization's resources and enable your team to focus on strategic initiatives.

Explore the tutorials in this section to learn how to set up code-based automations.

Calculating Risk Scores and Updating a Fact Sheet Based on Survey Responses

Learn how to set up an event-triggered automation that calculates the aggregated risk score for a fact sheet based on survey responses and updates this field on a fact sheet.

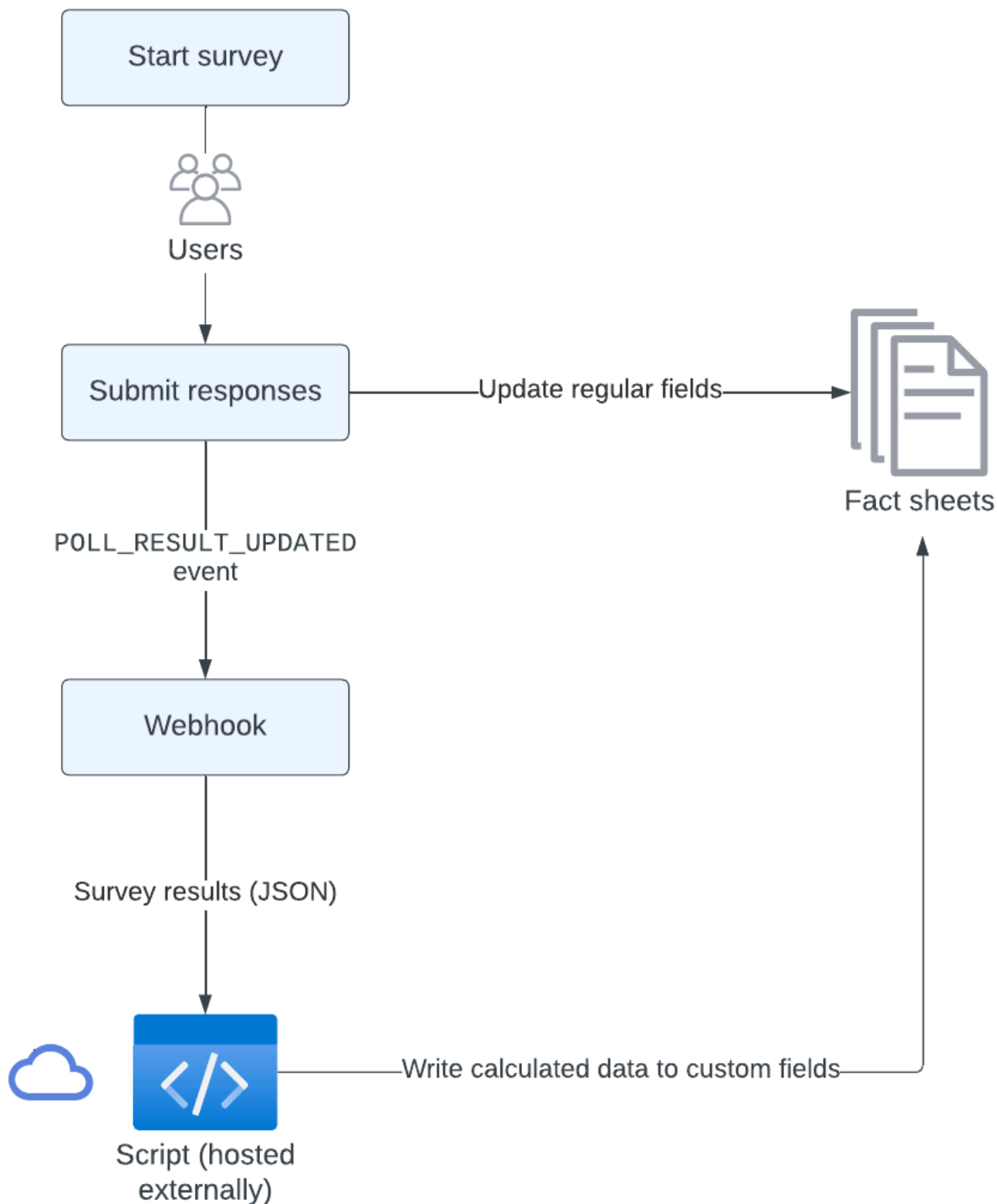
Overview

Risk management is a critical aspect of any organization's IT operations. Understanding the potential risks associated with different areas of your IT landscape allows you to make informed decisions and implement appropriate measures to mitigate these risks. One efficient way to assess risk is through surveys, where stakeholders can provide their insights on various risk factors. To learn more about surveys in SAP LeanIX, see [Surveys](#).

In this tutorial, you'll learn how to set up a trigger-based automation that assigns an aggregated risk score to a fact sheet based on survey responses gathered from stakeholders. The total risk score is computed within a calculated field in the survey. Once a survey run is completed for a fact sheet, a Python script initiates the following actions:

- Maps the total risk score value from the survey to readable values: Low risk, Medium risk, or High risk
- Updates custom fields associated with risk assessment on the target fact sheet

The process flow below illustrates how survey results are written to fact sheets when both regular and calculated fields are included in a survey. Regular fields are updated directly on fact sheets in SAP LeanIX. For calculated fields, an externally hosted script writes the data to custom fields.



Process Flow: Updating Fact Sheets from Survey Responses

Prerequisites

Before you start, do the following:

- Get admin access to your SAP LeanIX workspace.

- Obtain an API token by creating a technical user with the admin role. For more information, see [Technical Users](#).

This tutorial assumes you have basic knowledge of:

- JavaScript
- GraphQL
- REST APIs
- Webhooks
- Python

Step 1: Create a Survey for Risk Assessment

To gather information from stakeholders, create a survey for risk assessment with answer options for each question. All questions in the survey are mandatory. Each answer is associated with a score, which is then used to calculate the total risk score. To learn how to create surveys, see [Creating a Survey](#).

For the purpose of this tutorial, we use a survey with basic questions and answer options. You can use this survey as a template and adjust it to your organization's needs.

Question Category	Survey Question	Low Risk (1)	Medium Risk (2)	High Risk (3)
Security compliance	Assess the compliance of this technology with our organization's security standards.	Fully compliant	Partially compliant	Not compliant
Vendor support	Rate the level of support provided by the vendor for this technology.	Excellent support	Adequate support	Poor support
Data sensitivity	Assess the sensitivity of the data handled by this technology.	Low sensitivity	Medium sensitivity	High sensitivity
Operational impact	Assess the potential impact on operations if this technology were to fail.	Low impact	Medium impact	High impact

The aggregated risk score is computed in a calculated field within the survey, which uses a JavaScript code. This code employs a weighted scoring system to determine the final risk score. In our scenario, each question is assigned an equal weight of 0.25, meaning all questions contribute equally to the final score. To learn how to configure calculated fields, see [Calculated Fields in Surveys](#).

Configuration of the Aggregated Risk Score calculated field:

```
var score1 = 0;
var score2 = 0;
var score3 = 0;
var score4 = 0;

// Distributed weight for questions 1 to 4. The total weight equals to 1.
const weight = 0.25;

score1 += answers[0] === 'Fully compliant' ? 1 : 0;
```

```

score1 += answers[0] === 'Partially compliant' ? 2 : 0;
score1 += answers[0] === 'Not compliant' ? 3 : 0;

score2 += answers[1] === 'Excellent support' ? 1 : 0;
score2 += answers[1] === 'Adequate support' ? 2 : 0;
score2 += answers[1] === 'Poor support' ? 3 : 0;

score3 += answers[2] === 'Low sensitivity' ? 1 : 0;
score3 += answers[2] === 'Medium sensitivity' ? 2 : 0;
score3 += answers[2] === 'High sensitivity' ? 3 : 0;

score4 += answers[3] === 'Low impact' ? 1 : 0;
score4 += answers[3] === 'Medium impact' ? 2 : 0;
score4 += answers[3] === 'High impact' ? 3 : 0;

// Total and average score values
// totalValue = score1 + score2 + score3 + score4;
// avgValues = totalValue/4;

var scoreTotal = 0;
// Each score is multiplied by the weight
scoreTotal = score1 * weight + score2 * weight + score3 * weight + score4 * weight;
return scoreTotal.toFixed(2);

```

The following code snippet represents the survey for risk assessment in JSON format. You can import this survey to your workspace. To learn how to import surveys, see [Importing and Exporting a Survey](#).

Survey for risk assessment in JSON format:

```

{
  "title": "Technology Risk Assessment",
  "questions": [
    {
      "id": "f01ec0ea-5137-75e6-73b0-53d8fbf02e27",
      "label": "Assess the compliance of this technology with our organization's security standard",
      "type": "radio",
      "options": [
        {
          "id": "c224120d-a771-ffb8-cb00-1bd0c96cfaa3",
          "label": "Fully compliant"
        },
        {
          "id": "57d39808-566d-9397-4165-208e118152e4",
          "label": "Partially compliant"
        },
        {
          "id": "04e78da6-161a-2162-1d9e-527deb5987de",
          "label": "Not compliant"
        }
      ],
      "powerfeature": false,
      "settings": {
        "version": 1,
        "isMandatory": true
      }
    },
    {
      "id": "b10cb0ee-4137-85e6-73b0-53d8fbf02e28",
      "label": "Rate the level of support provided by the vendor for this technology.",
      "type": "radio",

```



```

"options": [
  {
    "id": "a124120d-a771-ffb8-cb00-1bd0c96cfaa4",
    "label": "Excellent support"
  },
  {
    "id": "67d39808-566d-9397-4165-208e118152e5",
    "label": "Adequate support"
  },
  {
    "id": "14e78da6-161a-2162-1d9e-527deb5987df",
    "label": "Poor support"
  }
],
"powerfeature": false,
"settings": {
  "version": 1,
  "isMandatory": true
}
},
{
  "id": "c20db0ef-6137-85e6-73b0-53d8fbf02e29",
  "label": "Assess the sensitivity of the data handled by this technology.",
  "type": "radio",
  "options": [
    {
      "id": "b214120d-a771-ffb8-cb00-1bd0c96cfaa5",
      "label": "Low sensitivity"
    },
    {
      "id": "77d39808-566d-9397-4165-208e118152e6",
      "label": "Medium sensitivity"
    },
    {
      "id": "24e78da6-161a-2162-1d9e-527deb5987dg",
      "label": "High sensitivity"
    }
  ],
  "powerfeature": false,
  "settings": {
    "version": 1,
    "isMandatory": true
  }
},
{
  "id": "d30ec0ef-7137-85e6-73b0-53d8fbf02e30",
  "label": "Assess the potential impact on operations if this technology were to fail.",
  "type": "radio",
  "options": [
    {
      "id": "c214120d-a771-ffb8-cb00-1bd0c96cfaa6",
      "label": "Low impact"
    },
  ],
  {

```

```

        "id": "87d39808-566d-9397-4165-208e118152e7",
        "label": "Medium impact"
    },
    {
        "id": "34e78da6-161a-2162-1d9e-527deb5987dh",
        "label": "High impact"
    }
],
"powerfeature": false,
"settings": {
    "version": 1,
    "isMandatory": true
}
},
{
    "id": "f206fcd0-d174-e9d3-bfcf-9841650e6b7c",
    "label": "Aggregated Risk Score",
    "type": "calc",
    "powerfeature": true,
    "settings": {
        "version": 1,
        "formula": "var score1 = 0;\nvar score2 = 0;\nvar score3 = 0;\nvar score4 = 0;\n\n// Distribute scores among categories"
    }
}
]
}

```

Step 2: Get the Survey ID

The automation is initiated only for a specific survey. To get the survey ID, navigate to the survey page in your workspace and copy the `id` from the URL.

Alternatively, you can get the survey `id` through the API. To do that, make a GET request to the following endpoint and save the `id` from the response. To learn how to get an access token, see [Authentication to SAP LeanIX Services](#).

`https://{SUBDOMAIN}.leanix.net/services/poll/v2/polls`

Example request:

```

curl -X GET
--header 'Accept: application/json'
--header 'Authorization: Bearer {ACCESS_TOKEN}'
'https://{SUBDOMAIN}.leanix.net/services/poll/v2/polls?q=Technology%Risk%20Assessment'

```

Example response:

```

{
    "status": "OK",
    "type": "Poll",
    "errors": [],
    "total": 1,
    "data": [
        {

```

```

    "id": "411f278a-eb38-4b45-84b3-5abf648d0e49",
    "legacyId": null,
    "questionnaire": {
      "id": "f01ec0ea-5137-75e6-73b0-53d8fbf02e27",
      "questions": [...]
    },
    "title": "Technology Risk Assessment",
    ...
  }
]
}

```

Step 3: Create Custom Fields on Fact Sheets

Create custom fields for risk assessment on the fact sheet types for which you plan to assess risks, such as applications, IT components, data objects, or providers.

To create custom fields on a fact sheet, follow these steps:

1. In the administration area, navigate to the [Meta Model Configuration](#) section.
2. Select a fact sheet type for which you plan to run risk assessments. You land on the fact sheet configuration page.
3. Create a dedicated subsection for risk assessment. To do that, select any section, click [Add subsection](#), then create a name and label for the subsection in the right-side panel. Save the changes.
4. To create custom fields within the subsection, select it, click [Add field](#), then specify the field details in the right-side panel. Use the following table for reference.

Key	Field Type	Displayed As	Label	Values and Labels
riskAssessmentScore	String	Text	Aggregated Risk Score	N/a
technicalRiskLevel	Single Select	Status	Risk Level	- lowRisk: Low risk - mediumRisk: Medium risk - highRisk: High risk

5. Save the changes.

Custom fields are added to the fact sheet. If needed, add the same custom fields to more fact sheet types.

Step 4: Create and Deploy a Function

Create and deploy a function that assigns risk scores to fact sheets based on survey responses. You can deploy the function using your preferred method, such as through a Function as a Service (FaaS) provider. For instructions, refer to the documentation of your FaaS provider.

The script provided in this tutorial performs the following tasks:

- Authenticates to SAP LeanIX services
- Parses the webhook payload to retrieve the results of a completed survey run
- Maps the aggregated risk score calculated within the survey to readable values, as shown in the following table:

Risk Level	Aggregated Risk Score Range	Description
Low risk	1-1.6	The technology is generally compliant, has strong vendor support, is not complex, and handles low sensitivity data.
Medium risk	1.7-2.3	The technology may have some areas of non-compliance, weaker vendor support, moderate complexity, and handles medium sensitivity data.
High risk	2.4-3	The technology is not compliant, has poor vendor support, is highly complex, and handles high sensitivity data.

- Updates the following custom fields on the target fact sheet using the `updateFactSheet` GraphQL mutation:
 - `riskAssessmentScore`
 - `technicalRiskLevel`

Sample code:

```
import json
import logging
import requests
import os

logging.basicConfig(level=logging.DEBUG)

# Request timeout
TIMEOUT = 20

# Poll ID is used to retrieve the Poll

# API token and Subdomain are set as env variables.
# It is advised not to hard code sensitive information in your code.
LEANIX_API_TOKEN = os.getenv('LEANIX_API_TOKEN')
LEANIX_SUBDOMAIN = os.getenv('LEANIX_SUBDOMAIN')
LEANIX_FQDN = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services'

# OAuth2 URL to request the access token.
LEANIX_OAUTH2_URL = f'{LEANIX_FQDN}/mtm/v1/oauth2/token'

# GraphQL Endpoint
LEANIX_GRAPHQL_URL = f'{LEANIX_FQDN}/pathfinder/v1/graphql'

# Pass the desired Poll ID and Question ID as env variables, to be able to
# collect answers from different polls and runs.
POLL_ID = os.getenv('POLL_ID')
QUESTION_ID = os.getenv('QUESTION_ID')

# We want to trigger the survey based only for the `POLL_RESULT_FINALIZED` event.
EVENT_TYPE = 'POLL_RESULT_FINALIZED'

def obtain_access_token() -> str:
    """Obtains a LeanIX Access token using the Technical User generated
    API secret.
```

```

Returns:
    str: The LeanIX Access Token
"""
if not LEANIX_API_TOKEN:
    raise Exception('A valid token is required')
response = requests.post(
    LEANIX_OAUTH2_URL,
    auth=('apitoken', LEANIX_API_TOKEN),
    data={'grant_type': 'client_credentials'},
    timeout=TIMEOUT
)
response.raise_for_status()
return response.json().get('access_token')

def _request_headers() -> dict:
    """Generates the necessary headers for interacting with the LeanIX GraphQL API
    Returns:
        dict: A dictionary with the headers required for making the request
    """
    access_token = obtain_access_token()
    auth_header = f'Bearer {access_token}'
    # Provide the headers
    headers = {
        'Authorization': auth_header,
    }
    return headers

def risk_level_from_score(score: float) -> str|None:
    """Function to determine risk level based on the provided score.
    Args:
        score (float): The score based on which risk level is to be determined.
    Returns:
        str: Returns the string 'lowRisk' if the score is less than or equal to 1.6, 'mediumRisk' :
        None: This is returned if the provided score does not meet any of the conditions for risk 1
    """
    if score <= 1.6:
        return 'lowRisk'
    elif score >1.6 and score <= 2.3:
        return 'mediumRisk'
    elif score >2.3:
        return 'highRisk'

def execute_mutation(mutation: str, variables: dict) -> dict:
    """This function executes a mutation request on a GraphQL endpoint.
    Args:
        mutation (str): The GraphQL mutation to be performed.
        variables (dict): The variables to be used in the mutation.
    Returns:
        dict: The JSON response from the GraphQL endpoint as a dictionary.
    Raises:
        HTTPError: If the request to the GraphQL endpoint results in an HTTP error.
    """
    headers = _request_headers()
    # In practice with the GraphQL API, mutations are technically not placed under
    # a 'mutation' field. Instead, they are placed under the `query` attribute.
    response = requests.post(
        LEANIX_GRAPHQL_URL,
        headers=headers,
        json={'query': mutation, 'variables': variables},
        timeout=TIMEOUT
    )
    response.raise_for_status()
    return response.json()

def update_fact_sheet(
    fact_sheet_id: str, risk_assesment_score: float, risk_level: str
) -> dict:
    """ This function updates the fact sheet with the given risk assessment score and risk level.
    Args:
        fact_sheet_id (str): The ID of the fact sheet to be updated.
        risk_assesment_score (float): The risk assessment score used to update the fact sheet.
        risk_level (str): The risk level used to update the fact sheet.
    Returns:
        dict: The updated fact sheet as a dictionary.

```

```

"""
mutation= """
  mutation RiskAssessment($id: ID!, $patches: [Patch]!, $validateOnly: Boolean) {
    updateFactSheet(id: $id, patches: $patches, validateOnly: $validateOnly) {
      factSheet {
        ... on Application {
          type
          id
          name
          riskAssessmentScore
          technicalRiskLevel
        }
      }
    }
  }
"""
variables = {
  'id': fact_sheet_id,
  'patches': [
    {
      'op': 'replace',
      'path': '/riskAssessmentScore',
      'value': f'{risk_assesment_score}'
    },
    {
      'op': 'replace',
      'path': '/technicalRiskLevel',
      'value': f'{risk_level}'
    }
  ],
  'validateOnly': False
}
logging.debug(f'Executing mutation: {mutation} with variables: {variables}')
response = execute_mutation(mutation, variables)
logging.debug(f'Response from GraphQL request was: {response}')
return response

def parse_webhook(req: dict) -> requests.Response|None:
    """Function to parse a webhook request. It checks if the poll id matches the expected POLL_ID.
    If it matches, it searches for a fact sheet id and the relative poll answers.
    If these are found, a risk-level is computed, and the fact sheet is updated with the risk level
    Args:
        req (dict): The webhook request payload.
    Returns:
        requests.Response: The response from the `update_fact_sheet` function call.
        None: If poll id does not match the `POLL_ID`, or if `fact_sheet_id` is not provided
        or answers are not provided for the `QUESTION_ID`.
    """
    webhook_poll_id = req.get('pollRun', {}).get('poll', {}).get('id')
    if webhook_poll_id == POLL_ID:
        fact_sheet_id = req.get('pollResult', {}).get('factSheet', {}).get('id')
        if not fact_sheet_id:
            # Don't trigger if there is no fact sheet id in the payload.
            logging.error('No fact sheet id was provided, aborting run')
            return
        poll_answers = req.get('pollResult', {}).get('answers', [])
        for poll_answer in poll_answers:
            if poll_answer.get('questionId') == QUESTION_ID:
                if not poll_answer.get('answer', [])[0]:
                    logging.info(f'No answers were provided for poll: {POLL_ID}')
                    return
                else:
                    score = float(poll_answer.get('answer', [])[0])
                    risk_level = risk_level_from_score(score)
                    return update_fact_sheet(fact_sheet_id, score, risk_level)

def event_handler(req: dict) -> bool:
    """Handles webhook requests. Logs that a webhook request has been received.
    Checks if both `POLL_ID` and `QUESTION_ID` are non-empty.
    If one or both are empty, an error is logged and an exception is thrown.
    If both are present, it checks the request's poll result event type.
    If the event type is the one expected, it calls the parse_webhook function to handle the request.
    """

```

```

Args:
    req (dict): The webhook request as a dictionary to be handled.
Returns:
    bool: Returns True if successfully executed, False otherwise.
Raises:
    Exception: If no POLL_ID or QUESTION_ID was provided.
"""
logging.info('Received webhook request')
if not QUESTION_ID or not POLL_ID:
    logging.error('No POLL_ID or QUESTION_ID provided')
    raise Exception('No POLL_ID or QUESTION_ID was provided, aborting run')
# Trigger the script only if the survey has been finalized
event_type = req.get('pollResultEventType')
if event_type == EVENT_TYPE:
    parse_webhook(req)

```

Environment Variables

For the script to work correctly, configure the environment variables listed in the following table.

These variables are crucial for handling sensitive data such as access tokens and IDs, as well as for controlling the behavioral aspects of the script. By configuring these variables, you make it adaptable for various run scenarios.

Environment Variable	Data Type	Description
LEANIX_API_TOKEN	String	The authentication token used for accessing the API. To learn how to get an access token, see Authentication to SAP LeanIX Services .
LEANIX_SUBDOMAIN	String	The subdomain of your SAP LeanIX workspace. You can copy the subdomain value from the workspace URL.
POLL_ID	String	The ID of the survey to be run.
QUESTION_ID	String	The ID of a specific question within the survey.
EVENT_TYPE	String	The event type within a survey run. As the survey progresses, multiple events occur that reflect the state changes of each result. This variable ensures that the automation is only initiated when the webhook event matches the POLL_RESULT_FINALIZED event type, preventing triggers for other event types.

Step 5: Create a Webhook for Survey Responses

Create a PUSH webhook for the POLL_RESULT_UPDATED event. In the **Target URL** field, enter the function URL. For instructions, see [Creating a Webhook](#).

Step 6: Start a Survey Run

To send the survey to stakeholders and start collecting responses, initiate a survey run. Run the survey only for the fact sheet types on which you've created custom fields. To learn how to start a survey run, see [Sending Out a Survey](#).

The automation is triggered once a survey run is completed for a specific fact sheet.

i **Note**

Because each new survey response overwrites the previous responses, you may want to send the survey only to a user with the Accountable subscription type who carries overall accountability for the fact sheet.

You can run the survey as often as you need. The fact sheet fields are updated accordingly on each completed survey run.

Summary

In this tutorial, you learned how to implement a trigger-based automation to calculate an aggregated risk score for fact sheets based on survey responses.

You can use this tutorial as a starting point and modify the survey and script to your needs. For example, you can calculate the business criticality of applications, the technical fit of IT components, and more.

Assigning Tags to Fact Sheets Based on Field Value Ranges

Learn how to set up an event-triggered automation to assign tags to IT component fact sheets based on the calculated energy consumption value.

Overview

IT components represent the technology (software and hardware) or services that an organization's applications depend on. To operate, IT components consume energy. Understanding the energy consumption levels of IT components allows organizations to effectively manage associated costs and plan their sustainability initiatives. For more information, visit [IT Component Modeling Guidelines](#).

In this tutorial, you will learn how to set up an automation to assign tags to IT component fact sheets based on their energy consumption values. Tags represent T-shirt sizes for each determined energy consumption range, such as XS, S, M, L, and XL.

The automation triggers every time the fact sheet field associated with energy consumption is updated. Based on this value, the corresponding tag representing a T-shirt size is assigned to the IT component fact sheet.

Prerequisites

Before you start, do the following:

- Get admin access to your workspace.
- Obtain an API token by creating a technical user with admin permissions. For more information, see [Technical Users](#).
- Prepare a mapping matrix of T-shirt sizes to energy consumption ranges for IT components in your organization's infrastructure. The energy consumption of IT components can vary significantly depending on whether the component is a piece of software, server hardware, network equipment, or other forms of technology. See an example matrix in the following table.

Example mapping of T-shirt sizes to energy consumption ranges for server hardware:

T-Shirt Size	Energy Consumption Range
XS	100-300 kWh/year

T-Shirt Size	Energy Consumption Range
S	300-700 kWh/year
M	700-1,200 kWh/year
L	1,200-2,500 kWh/year
XL	2,500 kWh/year and higher

This tutorial assumes you have basic knowledge of:

- Python
- GraphQL
- Webhooks

Step 1: Create a Custom Attribute for Energy Consumption on IT Component Fact Sheets

Follow these steps:

1. On the user profile, select **Administration**, and then select **Meta Model Configuration** in the sidebar.
2. On the **Meta Model Configuration** page, select the IT component fact sheet. You land on the fact sheet configuration page.
3. In the subsection where you want to place the custom attribute, click **Add field**. If needed, create a new section with a subsection.
4. In the sidebar, specify the attribute details:
 - In the **Key** field, enter a unique key that serves as the attribute identifier, such as `EnergyConsumptionLevel`.
 - In the **Type** list, select **Integer**.
 - Specify other attribute details and click **Create**.
 - On the tab indicated by the globe icon, enter translations for the attribute label to the target languages.
5. In the sidebar, click **Show changes**, and then click **Apply**.

The custom attribute for energy consumption values is added to IT component fact sheets.

Step 2: Create a Tag Group for T-Shirt Sizes

Create a tag group with tags representing T-shirt sizes. You can do it in the application user interface (UI) or through the GraphQL API. To learn how to manage tag groups in the application UI, see [Tagging](#).

When creating a tag group, do the following:

- Restrict the tag group to IT component fact sheets.
- Set the group mode to **Single**.

Step 3: Create a Webhook for Energy Consumption Updates

Create a webhook that triggers upon updates to the `EnergyConsumptionLevel` field on IT component fact sheets.

Follow these steps:

1. On the user profile, select **Administration**, and then select **Webhooks** in the sidebar.
2. On the **Webhooks** page, click **New Webhook**.
3. Specify the webhook details:
 - **Triggering Events**: Select **FACT_SHEET_UPDATED**.
 - **Managing User**: Select a technical user with admin permissions.
 - **Type**: Select **PUSH**.
 - **Target URL**: Enter the URL of the endpoint to send webhook notifications to. Before you complete other steps in this tutorial, enter a test target URL.
4. Optional: In the **Callback** field, enter JavaScript code to manipulate the webhook payload.

Sample code:

```
var payload = delivery.payload;
delivery.active = false;
var tempDict = {};
tempDict['fields'] = [];
var attributeList = ['EnergyConsumptionLevel'];

if (payload.factSheet.type === 'ITComponent') {
  tempDict['id'] = payload.factSheet.id;
  tempDict['type'] = payload.factSheet.type;
  tempDict['rev'] = payload.factSheet.rev;

  for (var i = 0; i < payload.factSheet.fields.length; ++i) {
    if (attributeList.indexOf(payload.factSheet.fields[i].name) >= 0) {
      tempDict.fields.push({ key: payload.factSheet.fields[i].name, value: payload.fact
    }
  }

  if (tempDict['fields'].length > 0) {
    delivery.payload = tempDict;
    delivery.active = true;
  }
}
```

With the provided callback, the webhook payload appears as follows.

Example webhook payload:

```
{
  "fields": [
    {
      "key": "EnergyConsumptionLevel",
      "value": {
        "value": 820,
        "type": "IntegerValue"
      }
    }
  ],
  "id": "cb943942-39fd-4b45-8909-916b99c1286a",
  "type": "ITComponent",
  "rev": 15
}
```

5. Optional: Specify other optional webhook details.

6. Click **Create**.

A webhook with a test target URL is created. To test the webhook, update the energy consumption value on an IT component fact sheet and verify the payload sent to the target URL.

Step 4: Create a Function to Assign Tags

Create a function to assign tags representing T-shirt sizes based on energy consumption ranges to IT component fact sheets. You can use your preferred Function as a Service (FaaS) provider.

The function should do the following:

- Perform authentication to SAP LeanIX services. For more information, see [Authentication to SAP LeanIX Services](#).
- Parse the webhook payload that contains the following:
 - The updated value of the EnergyConsumptionLevel field
 - The ID of the associated IT component fact sheet
- Identify the tag to be assigned to the fact sheet based on the energy consumption value.
- Retrieve the IDs of tags from the created tag group using a GraphQL query.

Example query:

```
{
  allTags(filter: {tagGroupName: "Energy Consumption"}) {
    edges {
      node {
        id
        name
      }
    }
  }
}
```

Example response:

```
{
  "data": {
    "allTags": {
      "edges": [
        {
          "node": {
            "id": "db9fbe30-8c75-4d3b-b498-e30b55da0872",
            "name": "XS"
          }
        },
        {
          "node": {
            "id": "6977e724-d2ec-4692-b59b-fe47252ce28a",
            "name": "S"
          }
        },
        {
          "node": {
```

```

        "id": "6b1e4df0-fea5-403f-90d5-755f79a5af9c",
        "name": "M"
    },
    {
        "node": {
            "id": "f84775e5-d57b-436c-8e37-342320e1aa21",
            "name": "L"
        }
    },
    {
        "node": {
            "id": "0b16ee28-7c9d-43a9-b735-14d18a95a999",
            "name": "XL"
        }
    }
]
}
}
}

```

- Assign the appropriate tag to the fact sheet using a GraphQL mutation. For more information, see [Adding Tags to a Fact Sheet](#).

Sample code:

```

import json
import logging
import os
import requests

logging.basicConfig(level=logging.INFO)

# Add a timeout to prevent the request hanging
TIMEOUT = 10

# API token and Subdomain are set as env variables.
# FaaS services support environment variables, it is advised not to hard code
# sensitive information in your code.

LEANIX_API_TOKEN = os.getenv('LEANIX_API_TOKEN')
LEANIX_SUBDOMAIN = os.getenv('LEANIX_SUBDOMAIN')
LEANIX_GRAPHQL_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/pathfinder/v1/graphql'
LEANIX_OAUTH2_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token'

# `T_SHIRT_SIZES` is a mapping of t-shirt sizes to their IDs, based on the output
# of a GraphQL query. Alternatively, you can programmatically call the query and
# parse the response, but this introduces overhead due to additional network
# requests.

T_SHIRT_SIZES = {
    "XS": "db9fbe30-8c75-4d3b-b498-e30b55da0872",
    "S": "6977e724-d2ec-4692-b59b-fe47252ce28a",
    "M": "6b1e4df0-fea5-403f-90d5-755f79a5af9c",
    "L": "f84775e5-d57b-436c-8e37-342320e1aa21",
    "XL": "0b16ee28-7c9d-43a9-b735-14d18a95a999",
}

def _obtain_access_token():
    """Obtains a LeanIX Access token using the Technical User generated
    API secret.

    Returns:

```

```

    """
    str: The LeanIX Access Token
    """
    if not LEANIX_API_TOKEN:
        raise Exception('A valid token is required')
    response = requests.post(
        LEANIX_OAUTH2_URL,
        auth=("apitoken", LEANIX_API_TOKEN),
        data={"grant_type": "client_credentials"},
    )
    response.raise_for_status()
    return response.json().get('access_token')

def _determine_t_shirt_size(payload):
    """Determines the appropriate T-shirt size tag based on the extracted energy
    consumption value from the parsed payload.

    Args:
        payload (dict): The webhook payload

    Returns:
        Optional[str]: The t-shirt size of the energy consumption value.
    """
    energy_consumption_level = None
    t_shirt_size = None

    # Loop through the payload fields
    for field in payload["fields"]:
        # Extract the `EnergyConsumptionLevel` value from the payload, convert it to
        # an integer, and store it in the `energy_consumption_level` variable.
        # Stop iterating the loop once the value has been found.
        if field.get("key") == "EnergyConsumptionLevel":
            energy_consumption_level = field.get("value", {}).get("value")
            break

    # Check the Energy Consumption t-shirt size
    if energy_consumption_level is not None:
        if 100 <= energy_consumption_level <= 300:
            t_shirt_size = "XS"
        elif 300 <= energy_consumption_level <= 700:
            t_shirt_size = "S"
        elif 700 <= energy_consumption_level <= 1200:
            t_shirt_size = "M"
        elif 1200 <= energy_consumption_level <= 2500:
            t_shirt_size = "L"
    return t_shirt_size

def update_fact_sheet(fact_sheet_id, t_shirt_size):
    """Tags the specified FactSheet with the appropriate t-shirt size
    Energy Consumption tag.

    Args:
        fact_sheet_id (str): The FactSheet UUID to be updated.
        t_shirt_size (str): The t-shirt size (XS, S, M, L, XL).
    """
    # Fetch the `id` of the t-shirt size
    t_shirt_size_id = T_SHIRT_SIZES.get(t_shirt_size)
    # If there is no match `id` bail out
    if not t_shirt_size_id:
        logging.warning(f"Could not retrieve `id` for t-shirt size: {t_shirt_size}")
        return

    # The GraphQL mutation to `replace` the tag of the FactSheet
    # `value` is a stringified list of the JSON object.
    # NOTE: Use `\\` to escape the `value` fields
    graphql_mutation = """
    mutation {
      updateFactSheet(
        id: "%s",
        patches:[
          {
            op: replace,
            path: "/tags",
            value: "[{\\\"tagId\\\": \\\"%s\\\"}]"
          }
        ]
      ) {
        factSheet {
          tags {
            tagId
            value
          }
        }
      }
    }
    """

```

```

    ]
  ){
    factSheet {
      id
      name
      tags {
        id
        name
      }
    }
  }
}

""" % (fact_sheet_id, t_shirt_size_id)

# Fetch the access token and set the Authorization Header
access_token = _obtain_access_token()
auth_header = f'Bearer {access_token}'
# Provide the headers
headers = {
    "Authorization": auth_header,
}

response = requests.post(
    LEANIX_GRAPHQL_URL,
    data=json.dumps({"query": graphql_mutation}),
    timeout=TIMEOUT,
    headers=headers,
)
response.raise_for_status()
response_data = response.json()
# GraphQL always returns a 200 response even if errors are included
# as such we check if `errors` is not empty.
errors=response_data.get("errors", [])
if len(errors):
    logging.error(f"Request was not successfull: {errors}")
    return
logging.info(f"GraphQL mutation success: {response_data}")

def event_handler(payload):
    """Receives the webhook payload, extracts relevant information,
    and initiates a GraphQL mutation to add the corresponding tag group.

    Args:
        payload (dict): The webhook payload
    """
    if payload:
        fact_sheet_id = payload.get("id")
        t_shirt_size = _determine_t_shirt_size(payload)
        if fact_sheet_id and t_shirt_size:
            update_fact_sheet(fact_sheet_id, t_shirt_size)
        else:
            logging.error("Could not update FactSheet")

```

Before proceeding, test your function locally. When ready, deploy the function. For instructions, refer to the documentation of your FaaS provider.

Step 5: Update the Webhook with the Function URL

Once you've successfully tested the function, update the webhook with the function URL.

Follow these steps:

1. On the user profile, select **Administration**, and then select **Webhooks** in the sidebar.
2. On the **Webhooks** page, select the webhook that you created.
3. In the **Target URL** field, enter the function URL.
4. Click **Save**.

The automation is set up. To ensure that it works as expected, update the energy consumption value on a test fact sheet and verify that the corresponding tag is assigned.

Summary

In this tutorial, you learned how to calculate the total energy consumption value for IT component fact sheets and assign the corresponding tags representing T-shirt sizes using a Python script.

Managing Fact Sheet Tags Dynamically Based on Relation Changes

Learn how to set up an event-triggered automation that assigns or removes fact sheet tags on creating or removing relations to a specific fact sheet type.

Overview

To indicate the relationships between fact sheets more clearly, you can assign corresponding tags to them once a relation is established or delete tags once the relation is removed.

In this tutorial, you'll learn how to automate the process of assigning and removing tags on creating or removing relations using a Python script that is triggered by a webhook.

As an example scenario, we assume that an organization categorizes IT components as either **Server** or **Database**, tagging them accordingly. When an application fact sheet is linked to an IT component, the script automatically assigns a tag reflecting the IT component type: `Linked_to_Server` or `Linked_to_Database`. If the relation is deleted, the tag is automatically removed.

This tagging system allows stakeholders to swiftly comprehend the landscape of dependencies within your IT architecture. It's especially beneficial when planning system maintenance, scaling operations, or conducting security audits. Knowing whether an application is linked to a server type or a database type IT component can affect decisions and strategies.

Prerequisites

Before you start, do the following:

- Get admin access to your SAP LeanIX workspace.
- Obtain an API token by creating a technical user with the admin role. For more information, see [Technical Users](#).

This tutorial assumes you have basic knowledge of:

- GraphQL
- Webhooks
- Python

Step 1: Create Tag Groups

Create the necessary tag groups with tags according to your organization's requirements. To learn how to manage tag groups in SAP LeanIX, see [Tagging](#).

In our example scenario, we need to create two tag groups, as shown in the following table. Restrict each group to a specific fact sheet type and set the group mode to **Single**.

Tag Group	Tags
IT component tag	• Server • Database
Application tag	• Linked_to_Server • Linked_to_Database

i Note

In this tutorial, we assume that tags are already assigned to IT component fact sheets.

Step 2: Create a Webhook for Fact Sheet Relations

Create a PUSH webhook for the RELATION_CREATED and RELATION_UPDATED events. For instructions, see [Creating a Webhook](#).

To test the automation, use a test target URL. If the automation works as expected, replace the test URL with the function URL.

Step 3: Create and Deploy a Function

Create a function to assign tags to applications once they are linked to IT components or remove tags once the relation is removed. You can deploy the function using your preferred method, such as through a Function as a Service (FaaS) provider. For instructions, refer to the documentation of your FaaS provider.

The script provided in this tutorial performs the following tasks:

- Authenticates to SAP LeanIX services
- Parses the webhook payload for the IDs of related fact sheets
- Retrieves the IDs of tags using a GraphQL query
- Identifies tags assigned to IT components using a GraphQL query
- Assigns or removes the appropriate tag on application fact sheets using a GraphQL mutation. For an example mutation, see [Adding Tags to a Fact Sheet](#).

Sample code:

```
import logging
import os

import requests

logging.basicConfig(level=logging.DEBUG)

# Request timeout
TIMEOUT = 20

# API token and Subdomain are set as env variables.
```



```

# It is advised not to hard code sensitive information in your code.
LEANIX_API_TOKEN = os.getenv('LEANIX_API_TOKEN')
LEANIX_SUBDOMAIN = os.getenv('LEANIX_SUBDOMAIN')
LEANIX_FQDN = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services'

# OAuth2 URL to request the access token.
LEANIX_OAUTH2_URL = f'{LEANIX_FQDN}/mtm/v1/oauth2/token'

# GraphQL Endpoint
LEANIX_GRAPHQL_URL = f'{LEANIX_FQDN}/pathfinder/v1/graphql'

# The IDs of the tags that will be assigned to the application factsheet
LINKED_TO_SERVER_ID = os.getenv('LINKED_TO_SERVER_ID')
LINKED_TO_DATABASE_ID = os.getenv('LINKED_TO_DATABASE_ID')

# Names of the tags of the IT Component
ITC_SERVER_TAG = 'Server'
ITC_DATABASE_TAG = 'Database'

def obtain_access_token() -> str:
    """Obtains a LeanIX Access token using the Technical User generated
    API secret.
    Returns:
        str: The LeanIX Access Token
    """
    if not LEANIX_API_TOKEN:
        raise Exception('A valid token is required')
    response = requests.post(
        LEANIX_OAUTH2_URL,
        auth=('apitoken', LEANIX_API_TOKEN),
        data={'grant_type': 'client_credentials'},
        timeout=TIMEOUT
    )
    response.raise_for_status()
    return response.json().get('access_token')

def _request_headers() -> dict:
    """Generates the necessary headers for interacting with the LeanIX GraphQL API
    Returns:
        dict: A dictionary with the headers required for making the request
    """
    access_token = obtain_access_token()
    auth_header = f'Bearer {access_token}'
    # Provide the headers
    headers = {
        'Authorization': auth_header,
    }
    return headers

def execute_mutation(mutation: str, variables: dict) -> dict:
    """This function executes a mutation request on a GraphQL endpoint.
    Args:
        mutation (str): The GraphQL mutation to be performed.
        variables (dict): The variables to be used in the mutation.
    Returns:
        dict: The JSON response from the GraphQL endpoint as a dictionary.
    Raises:
        HTTPError: If the request to the GraphQL endpoint results in an HTTP error.
    """
    headers = _request_headers()
    # In practice with the GraphQL API, mutations are technically not placed under
    # a 'mutation' field. Instead, they are placed under the 'query' attribute.
    response = requests.post(
        LEANIX_GRAPHQL_URL,
        headers=headers,
        json={'query': mutation, 'variables': variables},
        timeout=TIMEOUT
    )
    response.raise_for_status()
    return response.json()

def get_tags(id: str) -> dict:
    """This function queries the tags of the given IT Component Fact Sheet.

```

```

Args:
    id (str): The id of the IT Component Fact Sheet that is to be read.

Returns:
    dict: Result of the query as a dict.
"""
query = """
{
    factSheet(id: "%s") {
        id
        rev
        ... on ITComponent {
            tags {
                name
            }
        }
        tags {
            tagId: id
        }
    }
}
""" % (id)
return execute_mutation(query, {})

def parse_tags(tags: dict) -> str:
    """This function reads the tag dictionary of a given query response and determines if a certain

Args:
    tags (dict): The tag dictionary of a IT Component Fact Sheet.

Returns:
    str: ID of the tag the it component has set.
"""
    tag_list = tags.get("data").get("factSheet").get("tags")
    for tag in tag_list:
        if tag.get("name") == ITC_SERVER_TAG:
            return LINKED_TO_SERVER_ID
        elif tag.get("name") == ITC_DATABASE_TAG:
            return LINKED_TO_DATABASE_ID
    return ''

def remove_tag(id: str) -> dict:
    """This function cleans up the tags of the desired tag group before any further updates.

Args:
    id (str): The id of the Fact Sheet that will have its tags removed.

Returns:
    dict: The updated Fact Sheet as a dictionary.
"""
    mutation = """
    mutation DeleteTags($id: ID!, $patches: [Patch]!, $validateOnly: Boolean) {
        updateFactSheet(id: $id, patches: $patches, validateOnly: $validateOnly) {
            factSheet {
                name
            }
        }
    }
    """
    variables = {
        'id': id,
        'patches': [
            {
                'op': 'remove',
                'path': '/tags',
                'value': ' [{\\"tagId\\":\\"' + LINKED_TO_SERVER_ID + '\\"}, {\\"tagId\\":\\"' + LINKED_TO_
            }
        ],
        'validateOnly': False
    }
    return execute_mutation(mutation, variables)

def set_tag(id: str, itc_tag_id: str) -> dict:
    """This function sets the tag of the Application Fact Sheet based on the previously determined

```

```

Args:
    id (str): The id of the Fact Sheet that is to be updated.
    itc_tag_id (str): The id of the tag that will be assigned to the Fact Sheet.

Raises:
    e: Exception: If no correct tag was read from the IT Component an Exception is raised.

Returns:
    dict: The updated Fact Sheet as a dictionary.
"""
mutation = """
    mutation UpdateTags($id: ID!, $patches: [Patch]!, $validateOnly: Boolean) {
      updateFactSheet(id: $id, patches: $patches, validateOnly: $validateOnly) {
        factSheet {
          name
        }
      }
    }
"""
variables = {
    'id': id,
    'patches': [
        {
            'op': 'add',
            'path': '/tags',
            'value': ' [{\\"tagId\\":\\"'+ itc_tag_id +'\\"}]'
        }
    ],
    'validateOnly': False
}
try:
    return execute_mutation(mutation, variables)
except Exception as e:
    logging.debug(f'Error during the Fact Sheet update: {e}')
    raise e

```

```

def event_handler(req: dict) -> requests.Response|None:
    """Handles the request sent by the webhook.
    It distinguishes between the relevant events and proceeds according to the provided logic.

```

```

Args:
    req (dict): The webhook request as a dictionary to be handled.

Returns:
    requests.Response|None: Response of the mutations.
"""
logging.info('Received webhook request')
application_id = req.get('fromDetails').get('factSheetInfos').get('idAndRev').get('id')
itc_id = req.get('toDetails').get('factSheetInfos').get('idAndRev').get('id')
if req.get('type') == 'RelationCreatedEvent' or req.get('type') == 'RelationUpdatedEvent':
    remove_tag(application_id)
    return set_tag(application_id, parse_tags(get_tags(itc_id)))
elif req.get('type') == 'RelationDeletedEvent':
    return remove_tag(application_id)
else:
    return

```

Environment Variables

For the script to work correctly, configure the environment variables listed in the following table.

These variables are crucial for handling sensitive data such as access tokens and IDs, as well as for controlling the behavioral aspects of the script. By configuring these variables, you make it adaptable for various run scenarios.

Environment Variable	Data Type	Description
LEANIX_API_TOKEN	String	The authentication token used for accessing the API. To learn how to get an

Environment Variable	Data Type	Description
		access token, see Authentication to SAP LeanIX Services .
LEANIX_SUBDOMAIN	String	The subdomain of your SAP LeanIX workspace. You can copy the subdomain value from the workspace URL.
LINKED_TO_SERVER_ID	String	The ID of the Linked_to_Server tag.
LINKED_TO_DATABASE_ID	String	The ID of the Linked_to_Database tag.

Summary

In this tutorial, you learned how to automate the process of assigning and removing fact sheet tags on creating or removing relations using a Python script that is triggered by a webhook. You can modify the logic of the automation based on your requirements, using the provided code as an example.

Categorizing Applications Using the TIME Framework with Tags

In this guide, you will learn how to update tags on applications using Azure Functions. Tags reflect the elements of the Gartner® TIME framework, which allows you to map and measure the business value of your applications.

The Gartner® TIME framework represents a 2x2 quadrant and is an acronym for Tolerate, Invest, Migrate, Eliminate. You can use the setup provided and add your own logic on top in order to implement the specific case where a Tag on Fact Sheet will be updated according to "Functional Fit" and "Technical Fit". For more information, see [Gartner® TIME Framework](#).

Getting Started with Azure Functions

As defined by [Microsoft](#) , Azure Functions is a serverless solution that allows you to write less code, maintain less infrastructure, and save on costs. Instead of worrying about deploying and maintaining servers, the cloud infrastructure provides all the up-to-date resources needed to keep your applications running.

Azure Functions are triggered by specific events that you define.

Create an Azure Function

You can use one of the below options to create an Azure Function:

- [Visual Studio Code](#) 
- [Visual Studio](#) 
- [Command line](#) 
- [Maven \(Java\)](#) 

Upon creation of the Azure Function you will receive a unique endpoint which can be used to invoke the function and will be used in the Webhooks configuration at a later point. You can also use the link highlighted below to get your Function URL.

Editing functions in the portal is not supported for Python Function Apps. Please use the CLI or VS Code for development. [Learn more](#)

TATDemo - HttpTrigger

Function Apps

Search: "TATDemo"

All subscriptions

Function Apps

TATDemo

Functions (Read Only)

HttpTrigger

Integrate

Manage

Monitor

Proxies (Read Only)

__init__.py

Save Run </> Get function URL

```

1 import logging
2 import json
3 import os
4 import requests
5 import base64
6
7 import azure.functions as func
8
9 base_url = "https://app.leanix.net"
10
11 def handleRequest(reqBody):
12     fsId = reqBody.get('id')
13     targetCriticality = ''
14     financialLoss = reqBody.get('financialLoss')
15     if(financialLoss == 'high'):
16         targetCriticality = "missionCritical"
17     elif (financialLoss == "low"):

```

View files Test

Get Function URL

Connect SAP LeanIX to Azure Function

- Once you have created an Azure Function from the above section you can create a webhook in SAP LeanIX.
- Updated the Target URL in the above as per the one available from Azure Function created above.
- You can use a callback to determine what you want to send to Azure Function.

Step 1: Prepare Your Environment

Follow these steps:

1. Clone SAP LeanIX public scripts.
 - a. Clone the SAP LeanIX [repository with scripts](#) .
 - b. Go to the [timeModelSync folder](#) .
2. Create an Azure Function app. You can use one of the following options to create an app on your Azure Environment:
 - [Command Line](#)
 - [Azure Portal](#)
3. Deploy the project to your environment.

```
func azure functionapp publish timeModelSync
```

Copy the Invoke URL that is posted in your console after the successful deployment.

Step 2: Create a Tag Group

Create a tag group with the following tags: invest, eliminate, tolerate, and migrate. To learn how to create tag groups, see [Tagging](#).

← Edit Tag Group: Time Model

Tag Group

Tags

+ Add Tag

Name	Description	Color	# Fact Sheets	Status
Eliminate	Eliminate useless Applications (possible reasons: no business value, not used, low utility, based on obsolete software).		29	ACTIVE
Invest	Modernize the application because it has a high business value (e.g. application with high usage, but supported by outdated technology).		36	ACTIVE
Migrate	Discard the application, migrate the data and users on an existing Application (e.g. redundant applications). Unify multiple applications to a common version/technology platform. Merge applications (either physical, logical or both). Replace the application with a standard commercial solution.		24	ACTIVE
Tolerate	Keep the application and consider investing further in it, if usage stays high (e.g. high utility in good technical condition). Tolerate the application as it serves its purpose (e.g. a certain degree of utility in good technical condition) or because there is no adequate alternative.		22	ACTIVE

Tag Group with Tags That Represent the Elements of the Gartner® TIME Framework

Step 3: Create a Webhook

To learn more about webhooks, see [Webhooks](#).

To create a webhook, follow these steps:

1. Create a PUSH webhook for the following events: FACT_SHEET_CREATED and FACT_SHEET_UPDATED.
2. In the **Target URL** field, enter the invoke URL of your Azure function.
3. In the **Callback** field, enter the callback function from the callback.js file.
4. Select the **Ignore Errors** checkbox.
5. Click **Save**.

Step 4: Set Azure Environment Variables

1. Create an environment variable in your Azure portal called "HOST " and set it to your host (e.g. app.leanix.net)
2. Paste the following JSON into an environment variable called "TAG_MAPPING" with the correct ids for your workspace.

```
{
  "tolerate": "<tag id>",
  "invest": "<tag id>",
  "migrate": "<tag id>",
  "eliminate": "<tag id>"
}
```

→ Tip

To save your API token for a live environment SAP LeanIX advises you to use the [Microsoft KeyVault](#) . Create a link to the Key Vault from an environment variable called: "DEMO_TOKEN"

Now you are ready to use the Azure Functions and run some tests by changing values of the Functional Fit and Technical Fit. The corresponding tags are assigned to fact sheets.

Importing and Exporting Data

Learn how to import and export data between your organization's system and SAP LeanIX.

Importing and exporting data is an essential step to ensure consistency of information between your organization's system and SAP LeanIX.

This is custom documentation. For more information, please visit [SAP Help Portal](#).

To import and export your organization's data, you can leverage the following integrations:

- [Out-of-the-box integrations](#) with third-party systems, such as SAP Signavio or ServiceNow.
- Integrations through commonly used file formats such as XLS. To learn more, refer to:
 - [Importing Fact Sheet Data Through Excel File](#)
 - [Exporting Fact Sheet Data as Excel File](#)
- Custom integrations leveraging SAP LeanIX [REST APIs](#) and [GraphQL APIs](#).
- Custom integrations that you can set up on top of our [Integration API](#).

As a developer, you can streamline the process of importing and exporting data for your organization. The technology that you should choose depends on the data type.

Here are some examples of technologies for specific data types:

- Fact Sheets and related data: [GraphQL API](#) or [Integration API](#)
- Users: [MTM REST API](#) ➡
- Metrics: [Metrics REST API](#) ➡
- To-dos: [To-do REST API](#) ➡
- Surveys: [Poll REST API](#) ➡

Importing and Updating the Total Annual Costs of Applications

Import and update total annual costs using a Python script.

Overview

In this tutorial, you will learn how to import and update the total annual costs of applications using a Python script. Use this tutorial when completing an initial setup of your workspace or updating values in bulk.

Analyzing total annual costs can help identify inefficiencies in your organization's IT spending, enabling optimizations that can significantly reduce expenses. It also provides a clear view of IT investments, facilitating informed strategic decisions about resource allocation and budget planning.

i Note

In the standard meta model configuration, the `costTotalAnnual` attribute is stored in the relation between an application and an IT component fact sheet.

Prerequisites

Before you start, do the following:

- Get an API token by creating a technical user. For more information, see [Technical Users](#).
- Prepare the total annual cost data to be imported.

This tutorial assumes you have basic knowledge of:

- Python
- GraphQL
- Fact sheets in SAP LeanIX

Importing Total Annual Costs

When you complete an initial setup of your workspace, you can import fact sheets and related cost data using a Python script.

Before you start, prepare the data to be imported. The following table contains example data that we use in this tutorial.

Application Name	IT Component Name	Total Annual Costs
HR Management App	Storage System	125
HR Management App	Web Server	250
Finance Management App	Storage System	500
Finance Management App	Web Server	1000

Example CSV file:

```
application_name,it_component,total_annual_cost
HR Management App,Storage System,125
HR Management App,Web Server,250
Finance Management App,Storage System,500
Finance Management App,Web Server,1000
```

Once you have your input data ready, run the following Python script. The script completes the following tasks:

- Perform authentication to SAP LeanIX services
- Create application and IT component fact sheets using data from the input file
- Create relations between application and IT component fact sheets and update the `costTotalAnnual` attribute with values from the input file
- Export the CSV data including the newly created fact sheet IDs as a JSON file

Example script:

```
import csv
import json
import logging
import os

import requests

logging.basicConfig(level=logging.INFO)

CSV_FILE = os.getenv('CSV_FILE')

LEANIX_API_TOKEN = os.getenv('LEANIX_API_TOKEN')
LEANIX_SUBDOMAIN = os.getenv('LEANIX_SUBDOMAIN')
LEANIX_GRAPHQL_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/pathfinder/v1/graphql'
LEANIX_OAUTH2_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token'
TIMEOUT = 20
IT_COMPONENT_MUTATION = """
mutation ITComponentMutation($input: BaseFactSheetInput!) {
  createFactSheet(input: $input) {
```



```

        factSheet {
            id
            name
            type
        }
    }
}
"""
APPLICATION_MUTATION = """
mutation ApplicationMutation($input: BaseFactSheetInput!, $patches: [Patch!]) {
    createFactSheet(input: $input, patches: $patches) {
        factSheet {
            id
            name
            type
        }
    }
}
"""

def _obtain_access_token():
    """Obtain a LeanIX Access token using the Technical User generated
    API secret.

    Returns
    -----
    Optional(str): The LeanIX OAuth2 Access Token

    """
    if not LEANIX_API_TOKEN:
        raise Exception('A valid token is required')
    response = requests.post(
        LEANIX_OAUTH2_URL,
        auth=('apitoken', LEANIX_API_TOKEN),
        data={'grant_type': 'client_credentials'},
        timeout=TIMEOUT
    )
    response.raise_for_status()
    return response.json().get('access_token')

def make_request(payload: dict):
    """Perform a GraphQL request to the LeanIX GraphQL API endpoint.

    Args:
    ----
        payload (dict): The query or the mutation to perform against the API endpoint.

    Returns:
    -----
        Optional(dict): The GraphQL response.

    """
    # Fetch the access token and set the Authorization Header
    access_token = _obtain_access_token()
    auth_header = f'Bearer {access_token}'
    # Provide the headers
    headers = {
        'Authorization': auth_header,
    }

    response = requests.post(
        LEANIX_GRAPHQL_URL,
        json=payload,
        headers=headers,
        timeout=TIMEOUT
    )
    response.raise_for_status()
    json_response = response.json()
    # GraphQL always returns a 200 response even if errors are included
    # as such we check if `errors` is not empty.
    errors=json_response.get('errors', [])
    if len(errors):
        raise Exception(f'Request {payload} to {LEANIX_GRAPHQL_URL} was not successful: {errors}')
    return json_response

```

```

def create_it_component(it_component: str):
    """Create a LeanIX IT Component.

    Args:
    ----
        it_component (str): An IT component name.

    Returns:
    ----
        str: The UUID of the generated IT Component Fact Sheet.

    """
    # Create the Fact Sheet and return the `id` in order to assign it to
    # the application.
    # Mind the extra indendation as we will nest the creation of Fact Sheets
    # under one mutation.
    mutation_variables = {
        'input': {
            'name': it_component,
            'type': 'ITComponent'
        }
    }
    logging.info(f'Creating ITComponent {it_component}')
    response = make_request({'query': IT_COMPONENT_MUTATION, 'variables': mutation_variables})
    # Response contains the IDs of the ITComponents
    it_component_id = response.get('data', {}).get('createFactSheet', {}).get('factSheet', {}).get('id')
    logging.info(f'Succesfully created IT Component: {it_component} with id: {it_component_id}')
    return it_component_id


def create_applications(applications: dict):
    """Create LeanIX Application FactSheets from the provided set of data.

    Args:
    ----
        applications (dict): A dictionary of Applications with the relevant IT Components

    Returns:
    ----
        dict: A dictionary with the Application details including the Application
        Fact Sheet UUID.

    """
    for application in applications.values():
        mutation_variables = {
            'input': {
                'name': application.get('name'),
                'type': 'Application'
            },
            'patches': [
                {
                    'op': 'add',
                    'path': '/relApplicationToITComponent/new_1',
                    'value': '{\"factSheetId\": \"%s\", \"costTotalAnnual\": %s}'%(application.get('name'), application.get('costTotalAnnual'))
                }
            ]
        }
        logging.info(f'Creating Application {application}')
        response = make_request({'query': APPLICATION_MUTATION, 'variables': mutation_variables})
        application['id'] = response.get('data', {}).get('createFactSheet', {}).get('factSheet', {}).get('id')
        logging.info(f'Succesfully created application: {application.get('name')} with id: {application.get('id')}')

    return applications


def main(csv_file_path: str):
    """Read a CSV file containing LeanIX Applications and IT components
    and generate the relevant Application FactSheets and IT Components.

    Args:
    ----
        csv_file_path (str): The path to the CSV file containing the Application information

```

```

"""
with open(csv_file_path, newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    applications = dict()
    it_components = dict()
    for row in reader:
        application_name = row.get('application_name')
        it_component = row.get('it_component')
        # Validate required fields
        if not application_name or not application_name.strip():
            raise ValueError('Application name is missing or empty.')
        if not it_component or not it_component.strip():
            raise ValueError('IT component name is missing or empty.')
        # Create the IT components in order to fetch the relevant ID's
        if it_component not in it_components:
            it_components[it_component] = create_it_component(it_component)
        # Create the relevant Application pairs based on the IT Component IDs
        # Create the ID field with a null value as a placeholder for the response
        if application_name not in applications:
            applications[application_name] = {
                'name': application_name,
                'total_annual_cost': row.get('total_annual_cost', 0),
                'it_component': it_components.get(it_component),
                'id': None
            }
        # Create the application if everything was successful
    if applications:
        applications = create_applications(applications)
        # Create a JSON export of the data with the Fact Sheet information
        with open('export.json', 'w') as f:
            # Use the json.dump method to write the data to the file
            json.dump(applications, f, indent=4) # indent parameter for readability

if __name__ == '__main__':
    main(CSV_FILE)

```

Updating Total Annual Costs

To update total annual costs, export the current values using GraphQL and then import new values using a Python script.

Step 1: Retrieve Cost Values

To retrieve total annual costs, use the following GraphQL query. You can run the query in the [GraphiQL Tool](#) in your workspace.

Example query:

```

{
  allFactSheets(factSheetType: Application) {
    edges {
      node {
        ... on Application {
          name
          id
          relApplicationToITComponent {
            edges {
              node {
                factSheet {
                  name
                  id
                }
                costTotalAnnual
              }
            }
          }
        }
      }
    }
  }
}

```

```

    }
  }

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "edges": [
        {
          "node": {
            "name": "HR Management App",
            "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
            "relApplicationToITComponent": {
              "edges": [
                {
                  "node": {
                    "factSheet": {
                      "name": "Storage System",
                      "id": "a8fe4825-42b8-431b-8124-ca12c579c78b"
                    },
                    "costTotalAnnual": 125
                  }
                },
                {
                  "node": {
                    "factSheet": {
                      "name": "Web Server",
                      "id": "ed46809c-998a-4fd6-9185-4b25e4e77d9b"
                    },
                    "costTotalAnnual": 250
                  }
                }
              ]
            }
          }
        },
        {
          "node": {
            "name": "Finance Management App",
            "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
            "relApplicationToITComponent": {
              "edges": [
                {
                  "node": {
                    "factSheet": {
                      "name": "Storage System",
                      "id": "a8fe4825-42b8-431b-8124-ca12c579c78b"
                    },
                    "costTotalAnnual": 500
                  }
                },
                {
                  "node": {

```

```
        "factSheet": {
            "name": "Web Server",
            "id": "ed46809c-998a-4fd6-9185-4b25e4e77d9b"
        },
        "costTotalAnnual": 1000
    }
}
]
```

Convert the output in JSON format into a CSV file using your preferred conversion tool. The following table shows the data that we retrieved.

Application Name	Application ID	IT Component Name	IT Component ID	Total Annual Costs
HR Management App	4d121f64-116b-4ccc-a292-eb4e4f8d1b24	Storage System	a8fe4825-42b8-431b-8124-ca12c579c78b	125
HR Management App	4d121f64-116b-4ccc-a292-eb4e4f8d1b24	Web Server	ed46809c-998a-4fd6-9185-4b25e4e77d9b	250
Finance Management App	28fe4aa2-6e46-41a1-a131-72afb3acf256	Storage System	a8fe4825-42b8-431b-8124-ca12c579c78b	500
Finance Management App	28fe4aa2-6e46-41a1-a131-72afb3acf256	Web Server	ed46809c-998a-4fd6-9185-4b25e4e77d9b	1000

Example CSV file:

```
application_id,it_component_id,total_annual_cost
4d121f64-116b-4ccc-a292-eb4e4f8d1b24,a8fe4825-42b8-431b-8124-ca12c579c78b,125
4d121f64-116b-4ccc-a292-eb4e4f8d1b24,ed46809c-998a-4fd6-9185-4b25e4e77d9b,250
28fe4aa2-6e46-41a1-a131-72afb3acf256,a8fe4825-42b8-431b-8124-ca12c579c78b,500
28fe4aa2-6e46-41a1-a131-72afb3acf256,ed46809c-998a-4fd6-9185-4b25e4e77d9b,1000
```

Step 2: Run a Script to Update Cost Values

Before proceeding, prepare an input file in CSV format with updated cost values. For details, see the previous step in this tutorial.

Once you have your input data ready, run the following Python script to update cost values. The script completes the following tasks:

- Performs authentication to SAP LeanIX services
- Retrieves relations between application and IT component fact sheets
- Updates the `costTotalAnnual` attribute with values from the input file

Example script:

```

import csv
import logging
import os

import requests

logging.basicConfig(level=logging.INFO)

CSV_FILE = os.getenv('CSV_FILE')

LEANIX_API_TOKEN = os.getenv('LEANIX_API_TOKEN')
LEANIX_SUBDOMAIN = os.getenv('LEANIX_SUBDOMAIN')
LEANIX_GRAPHQL_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/pathfinder/v1/graphql'
LEANIX_OAUTH2_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token'
TIMEOUT = 20

QUERY = """
query retrieveAllFactSheets {
  allFactSheets(factSheetType: Application) {
    edges {
      node {
        id
        ... on Application {
          relApplicationToITComponent {
            edges {
              node {
                id
                factSheet {
                  id
                }
              }
            }
          }
        }
      }
    }
  }
}
"""

APPLICATION_MUTATION = """
mutation UpdateApplication($id: ID!, $patches: [Patch]!) {
  updateFactSheet(id: $id, patches: $patches) {
    factSheet {
      id
      name
      description
      type
    }
  }
}
"""

def _obtain_access_token():
    """Obtain a LeanIX Access token using the Technical User generated
    API secret.

    Returns
    -----
    Optional(str): The LeanIX OAuth2 Access Token

    """
    if not LEANIX_API_TOKEN:
        raise Exception('A valid token is required')
    response = requests.post(
        LEANIX_OAUTH2_URL,
        auth=('apitoken', LEANIX_API_TOKEN),
        data={'grant_type': 'client_credentials'},
        timeout=TIMEOUT
    )
    response.raise_for_status()
    return response.json().get('access_token')

def make_request(payload: dict):

```

"""Perform a GraphQL request to the LeanIX GraphQL API endpoint.

Args:

payload (dict): The query or the mutation to perform against the API endpoint.

Returns:

Optional(dict): The GraphQL response.

"""

Fetch the access token and set the Authorization Header

access_token = _obtain_access_token()

auth_header = f'Bearer {access_token}'

Provide the headers

```
headers = {
    'Authorization': auth_header,
}
```

response = requests.post(

LEANIX_GRAPHQL_URL,

json=payload,

headers=headers,

timeout=TIMEOUT

)

response.raise_for_status()

json_response = response.json()

GraphQL always returns a 200 response even if errors are included

as such we check if `errors` is not empty.

errors=json_response.get('errors', [])

if len(errors):

raise Exception(f'Request {payload} to {LEANIX_GRAPHQL_URL} was not successful: {errors}')

return json_response

def _parse_application_fact_sheets(query_response):

"""Loop through the query results generating a dictionary containing the information required to update the relevant application costs.

Args:

query_response (dict): The query response data from the GraphQL query

"""

applications = dict()

for edge in query_response.get('data', {}).get('allFactSheets', {}).get('edges', []):

node = edge.get('node', {})

application_id = node.get('id')

for relation_edge in node.get('relApplicationToITComponent', {}).get('edges', []):

Set an entry only if there is a relation available

applications[application_id] = dict()

relation_node = relation_edge.get('node', {})

relation_id = relation_node.get('id')

it_component_id = relation_node.get('factSheet', {}).get('id')

applications[application_id][it_component_id] = relation_id

return applications

def retrieve_application_fact_sheets():

"""Retrieve the LeanIX Application Fact Sheets, including their relational values.

"""

response = make_request({'query': QUERY})

return _parse_application_fact_sheets(response)

def update_costs(application_id: str, it_component_id: str, relation_id: str, total_annual_costs: :

"""Update LeanIX Application FactSheets from the provided set of data.

Args:

application_id (str): The UUID of the Application Fact Sheet.

it_component_id (str): The UUID of the ITComponent Fact Sheet.

relation_id (str): The UUID of the relation with the ITComponent Fact Sheet.

total_annual_costs (int): The total annual cost.

```

"""
mutation_variables = {
    'id': application_id,
    'patches': [
        {
            'op': 'replace',
            'path': f'/relApplicationToITComponent/{relation_id}',
            'value': '{"factSheetId": \"%s\"',
        }
    ]
}
logging.info(f'Updating costs for Application Fact Sheet: {application_id}')
response = make_request({'query': APPLICATION_MUTATION, 'variables': mutation_variables})
logging.info(f'Successfully updated costs: {response}')

def main(csv_file_path: str):
    """Read a CSV file containing LeanIX Applications and IT components
    and generate the relevant Application FactSheets and IT Components.

    Args:
    ----
        csv_file_path (str): The path to the CSV file containing the Application information

    """
    logging.info('Fetching Application Fact Sheets with relations')
    applications = retrieve_application_fact_sheets()
    with open(csv_file_path, newline='') as csvfile:
        reader = csv.DictReader(csvfile)
        for row in reader:
            application_id = row.get('application_id')
            it_component_id = row.get('it_component_id')
            relation_id = applications.get(application_id, {}).get(it_component_id, {})
            total_annual_costs = row.get('total_annual_costs')
            # Validate required fields
            if not application_id or not application_id.strip():
                raise ValueError('Application id is missing or empty.')
            if not it_component_id or not it_component_id.strip():
                raise ValueError('IT component id is missing or empty.')
            if not relation_id or not relation_id.strip():
                raise ValueError('Relation id is missing or empty.')
            update_costs(application_id, it_component_id, relation_id, total_annual_costs)

if __name__ == '__main__':
    main(CSV_FILE)

```

Related Information

- [Cost Management](#)
- [Using GraphQL with Python](#)

Exporting Workspace Snapshots

Export snapshots of your workspace data through the Pathfinder REST API.

Overview

While we recommend working with your inventory data and collaborating inside your workspace as much as possible, there are circumstances in which you may need to regularly download snapshots of your workspace data, for example, when it's required by your organization's policies. As an administrator, you can export your workspace data to an XLSX file in the [Export](#) section of the administration area. To learn more, see [Export](#).

This document outlines how to programmatically export workspace snapshots.

Prerequisites

- Get an API token by creating a technical user with admin permissions. For instructions, see [Technical Users](#).
- Learn how to authenticate to SAP LeanIX services. For details, see [Authentication to SAP LeanIX Services](#).
- Get the ID of your workspace. To do that, in the administration area, navigate to the **API Tokens** section, then copy the WorkspaceId value.

Step 1: Initiate an Export Job

To get started, initiate an export job by making a POST request to the following endpoint on the [Pathfinder REST API](#)  :

`https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/exports/fullExport`

Use the `exportType` query parameter to specify the export type:

- **SNAPSHOT**: Full snapshot of the workspace data that includes information from all active fact sheets, as well as information about related objects such as relations, tag groups, tags, resources (also referred to as documents), and comments. The snapshot doesn't include archived fact sheets. In the application UI, this export type appears as **Full Snapshot**.
- **AUDIT_LOG**: Data-related changes made to the workspace within a specified time frame. The default time frame is 30 days. In the application UI, this export type appears as **Changelog**.
- **ARCHIVE**: Full snapshot of the archived workspace data that includes information from all archived fact sheets, as well as information about related objects such as relations, tag groups, tags, resources (also referred to as documents), and comments. You can use this export option to view archived fact sheets before they are deleted at the end of the retention period. For more information, see [Archiving, Deleting and Recovering Fact Sheets](#).

i Note

The `/exports/fullExport` endpoint supports additional export types. However, we recommend using only the export types listed in this document.

Example request:

```
curl -X 'POST' \
  'https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/exports/fullExport?exportType=SNAPSHOT' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer {ACCESS_TOKEN}' \
  -d ''
```

Example response:

```
{
  "status": "OK",
  "type": "JobResponseData",
  "data": {
    "jobId": "job:374234:cmbf4fdiozpz8q7m4u0dc1jss",
    "message": "A new job is queued.",
    "synclogId": null
  }
}
```

The OK status in the response indicates that an export job is initiated.

Step 2: Retrieve a One-Time Download Key

Before you download a file with workspace data, retrieve a one-time key that serves as the download identifier. To do that, make a GET request to the following endpoint:

`https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/exports`

i Note

Because the export process can be time-consuming, especially for workspaces containing large volumes of data, you might need to repeatedly call the `/exports` endpoint to monitor the export status. Once the `status` updates to `COMPLETED`, a one-time `downloadKey` is generated, which can be used to download the file as outlined in the next step.

You can optionally use query parameters to filter the results returned in the response.

Example request:

```
curl -X 'GET' \
  'https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/exports?exportType=SNAPSHOT&pageSize=40&sc
  -H 'accept: application/json' \
  -H 'Authorization: Bearer {ACCESS_TOKEN}'
```

Example response:

```
{
  "status": "OK",
  "type": "Export",
  "data": [
    {
      "id": "3159d0bc-f0c1-4d6a-a541-cc534ebf7102",
      "user": {
        "id": "25d63c1a-4cc9-4c46-bb4c-a82a78cb8686",
        "firstName": null,
        "lastName": "Technical User",
        "displayName": "Technical User",
        "userName": "Technical User",
        "email": null,
        "technicalUser": true,
        "role": "ACCOUNTUSER",
        "status": "ACTIVE"
      },
      "type": "SNAPSHOT",
      "status": "COMPLETED",
      "createdAt": "2024-03-26T15:04:25.378194Z",
      "deleteAt": "2024-03-26T15:14:29.515400Z",
      "dryRun": false,
      "downloadKey": "ba25c032-f399-462e-803a-016bdf13335d",
      "factSheetTypes": null,
      "factSheetCount": null,
      "columnCount": null,
    }
  ]
}
```

```

    "bookmark": null
  },
  ...
],
"total": 9,
"endCursor": "YXJyYXk60A=="
}

```

Once the export status updates to **COMPLETED**, which indicates a successful operation, a one-time download key is generated in `downloadKey`. Save this key from the response.

Step 3: Retrieve a File with Workspace Data

With the `downloadKey` of the export, you can retrieve an XLSX file with workspace data. The `downloadKey` is a single-use key, valid for one download only. `workspaceId` is a required parameter for this request.

To retrieve a file with workspace data, make a GET request to the following endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/exports/downloads/{workspaceId}
```

Example request:

```

curl -X 'GET' \
  'https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/exports/downloads/b7d533f1-4aa0-4132-92c6-'
-H 'accept: application/octet-stream' \
-H 'Authorization: Bearer {ACCESS_TOKEN}'

```

An XLSX file is returned in the response.

Next Steps

You can automate recurring tasks, such as exporting workspace snapshots, by using cron jobs. Cron jobs are time-based job schedulers in Unix-like operating systems, which can execute commands or scripts at predefined times or intervals. For more complex scheduling or tasks, consider using task scheduling software or services that offer more advanced features and error handling.

If you want to process data from the downloaded Excel file, as a best practice, always refer to specific columns in your code using field keys (first rows in the file) instead of field descriptions (second rows in the file) or column order. Field keys are unique identifiers that remain consistent, regardless of any possible changes that can be applied to file data.

Bookmarks

Manage saved searches, reports, diagrams, and dashboards through the Pathfinder REST API.

In the context of the [Pathfinder REST API](#) , a bookmark is a data object that represents one of the following entities depending on the bookmark type: saved search, report, diagram, or dashboard.

To manage bookmarks programmatically, use the `/bookmarks` endpoints within the Pathfinder REST API.

Bookmark Types

The following table lists the types of bookmarks in the Pathfinder REST API and the corresponding objects in the application user interface (UI).

Bookmark Type in the API	Object in the Application UI
INVENTORY	Saved search within the Inventory
REPORTING	Report
VISUALIZER	Diagram
DASHBOARD	Dashboard

When you retrieve bookmarks, the `state` attribute in the response contains the bookmark data. The content of the `state` attribute varies depending on the type of bookmark.

i Note

Sharing data through the Open Data Protocol (OData) is only supported for bookmarks of the `INVENTORY` type. OData is not supported for other types of bookmarks.

Retrieving a Diagram by ID

This guide explains how to retrieve all or specific diagrams that you have access to through the Pathfinder REST API.

i Note

In the context of the Pathfinder REST API, a diagram is a type of bookmark. You can use the API endpoints listed in this guide to retrieve other types of bookmarks. For more information, visit [Bookmarks](#).

To learn how to authenticate to SAP LeanIX APIs, see [Authentication to SAP LeanIX Services](#).

Retrieving All Diagrams

To retrieve all diagrams that you have access to, make a GET request to the following API endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/bookmarks
```

To explore the endpoint schema, navigate to the [OpenAPI Explorer](#) .

The following table contains the input parameters:

Parameter	Parameter Type	Data Type	Required	Description
bookmarkType	Query	String	Required	The type of the bookmark to retrieve. For diagrams, pass <code>VISUALIZER</code> . For a list of possible values, see Bookmarks .
oDataOnly	Query	Boolean	Optional	Determines whether to return only bookmarks

Parameter	Parameter Type	Data Type	Required	Description
				for which the Open Data Protocol (OData) is enabled. OData is not supported for diagrams.
groupKey	Query	String	Optional	The key used to separate bookmarks within the same query type.

Example request:

```
curl -X 'GET' \
  'https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/bookmarks?bookmarkType=VISUALIZER' \
  -H 'accept: application/json' \
  -H 'authorization: Bearer {ACCESS_TOKEN}'
```

The **state** attribute in the response contains the diagram layout and filters.

Example response:

```
{
  "status": "OK",
  "type": "Bookmark",
  "data": [
    {
      "id": "2452d5e1-ee8b-4b7c-b7f6-7af8bc4b9a73",
      "userId": "00000000-1111-4000-2222-000000000001",
      "name": "Dataflow w/ Data Objects",
      "type": "VISUALIZER",
      "groupKey": "dataflow",
      "state": {
        "graphXml": "<mxGraphModel grid=\"1\" gridSize=\"10\" guides=\"1\" tooltips=\"1\" connect='",
        "viewport": {
          "scale": 1,
          "translate": {
            "x": 0,
            "y": 0
          }
        }
      },
    },
    "workingCopy": null,
    "description": "",
    "createdAt": "2019-03-20T15:59:54.714Z",
    "updatedAt": "2023-09-27T18:10:51.185149Z",
    "i18nKey": null,
    "predefined": true,
    "readonly": false,
    "defaultSharingPriority": 20,
    "user": {...},
    "permittedReadUserIds": [],
    "permittedWriteUserIds": [],
    "referencedFactSheetIds": [
```

```

    "a0197ca7-e614-4e93-d89f-82dd28ef819a",
    "26be83c4-2b7f-479e-e5f3-ef0006db54ba",
    ...
  ],
  "views": 11,
  "replaySequence": 6,
  "temporary": false,
  "oDataEnabled": false
},
{...}
],
"total": 4
}

```

Retrieving a Diagram by ID

To retrieve a specific diagram that you have access to by its ID, make a GET request to the following API endpoint:

`https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/bookmarks/{id}`

To explore the endpoint schema, navigate to the [OpenAPI Explorer](#) .

The following table contains the input parameters:

Parameter	Parameter Type	Data Type	Required	Description
id	Path	String	Required	The ID of the diagram to retrieve. To get the diagram ID, retrieve all diagrams and save the <code>id</code> of the desired diagram from the response. In the application user interface, you can navigate to the diagram page and copy the ID from the URL.
markAsViewed	Query	Boolean	Optional	Determines whether to mark the diagram as viewed upon executing the request. If <code>true</code> , the value of the <code>views</code> parameter is increased by one. Defaults to <code>true</code> if not specified.

Example request:

```

curl -X 'GET' \
  'https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/bookmarks/2452d5e1-ee8b-4b7c-b7f6-7af8bc4t' \
  -H 'accept: application/json' \
  -H 'authorization: Bearer {ACCESS_TOKEN}'

```

The state attribute in the response contains the diagram layout and filters.

Example response:

```
{
  "data": {
    "id": "2452d5e1-ee8b-4b7c-b7f6-7af8bc4b9a73",
    "userId": "00000000-1111-4000-2222-000000000001",
    "name": "Dataflow w/ Data Objects",
    "type": "VISUALIZER",
    "groupKey": "dataflow",
    "state": {
      "graphXml": "<mxGraphModel grid=\"1\" gridSize=\"10\" guides=\"1\" tooltips=\"1\" connect=\"1\">
      <mxgraph>
        <mxrect x1=0 y1=0 x2=1000 y2=1000 fill=#fff stroke=#333 stroke-width=1px/>
        <mxtext x=500 y=500 text=Dataflow w/ Data Objects/>
      </mxgraph>
      </mxGraphModel>"
      "viewport": {
        "scale": 1,
        "translate": {
          "x": 0,
          "y": 0
        }
      }
    },
    "workingCopy": null,
    "description": "",
    "createdAt": "2019-03-20T15:59:54.714Z",
    "updatedAt": "2024-01-05T11:51:50.876878Z",
    "il8nKey": null,
    "predefined": true,
    "readonly": false,
    "defaultSharingPriority": 20,
    "user": {...},
    "permittedReadUserIds": [],
    "permittedWriteUserIds": [],
    "referencedFactSheetIds": [
      "a0197ca7-e614-4e93-d89f-82dd28ef819a",
      "26be83c4-2b7f-479e-e5f3-ef0006db54ba",
      ...
    ],
    "views": 12,
    "replaySequence": 6,
    "temporary": false,
    "oDataEnabled": false
  },
  "status": "OK",
  "type": "Bookmark"
}
```

Updating User Permissions

Update user permissions through the MTM REST API.

Overview

This guide provides instructions on how to update user permissions through the [Multi-Tenancy Management \(MTM\) REST API](#) .

Permissions define the user status, role, and other attributes related to authorization for each workspace. Each user can only have a single permission for each workspace. Each attribute on the permission can only be set once. Some attributes, such as `role`, only allow a single value. Other attributes, such as `customerRoles`, allow multiple values. To learn more about managing users, see [Managing Users](#).

Prerequisites

- Get an API token by creating a technical user with admin permissions. For instructions, see [Technical Users](#).
- Learn how to authenticate to SAP LeanIX services. For details, see [Authentication to SAP LeanIX Services](#).
- Get the ID of your workspace. To do that, in the administration area, navigate to the **API Tokens** section, then copy the `WorkspaceId` value.

Step 1: Retrieve User Permissions

To update a permission, first retrieve a list of user permissions for a workspace. To do that, make a **GET** request to the following endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/mtm/v1/workspaces/{id}/permissions
```

The following table lists the input parameters.

Parameter	Parameter Type	Data Type	Required	Description
<code>id</code>	Path	String	Required	The ID of the workspace to retrieve user permissions for.
<code>q</code>	Query	String	Optional	Query string to filter data for a specific user. You can specify the user's first name, last name, role, or other details.
<code>email</code>	Query	String	Optional	The email address of the user to search for.
<code>status</code>	Query	String	Optional	The status of the user to search for, such as ACTIVE , INVITED , or ARCHIVED .
<code>includeTechnicalUsers</code>	Query	Boolean	Optional	Determines whether to include technical users in the response.
<code>page</code>	Query	Integer	Optional	The page number to return in the response. The default value is 1.
<code>size</code>	Query	Integer	Optional	The page size to return in the response. The

Parameter	Parameter Type	Data Type	Required	Description
				default value is 30. The maximum value is 100.
sort	Query	String	Optional	A list of sorting parameters separated by commas.

Example request:

```
curl --request GET \
  --url https://{SUBDOMAIN}.leanix.net/services/mtm/v1/workspaces/bdba4b5d-2d63-49ef-f8g4-045f02029\
  --header 'content-type: application/json' \
  --header 'authorization: Bearer {ACCESS_TOKEN}' \
```

Example response:

```
{
  "status": "OK",
  "type": "Permission",
  "errors": [],
  "total": 1,
  "data": [
    {
      "links": [...],
      "id": "04259076-8cbc-418e-5d8h-20144d5d8bda",
      "user": {
        "id": "f767b7a2-5dd8-423e-d7f4-f9ea34d36ce8",
        "account": {...},
        "userName": "john.doe@organization.com",
        "email": "john.doe@organization.com",
        "role": "ACCOUNTUSER",
        "status": "ACTIVE",
        ...
      },
      "workspace": {...},
      "workspaceId": "bdba4b5d-2d63-49ef-f8g4-045f020294bb",
      "role": "MEMBER",
      "status": "INVITED",
      "lastLogin": null,
      "invitedByUser": {...},
      "reviewedByUser": null,
      "customerRoles": null,
      "accessControlEntities": null,
      "ignoreBlacklist": false,
      "count": 0,
      "replayed": false,
      "active": false
    }
  ]
}
```

Step 2: Update User Permissions

To update user permissions, make a **POST** request to the following endpoint:

`https://{SUBDOMAIN}.leanix.net/services/mtm/v1/permissions`

There are two main use cases for managing user permissions:

- **Updating the user role:** Set `role` to one of the following values: **VIEWER**, **MEMBER**, or **ADMIN**. To learn more about these roles, see [Standard User Roles](#).
- **Updating the user status:** Set `status` to one of the following values:
 - **ARCHIVED:** Archive a user. You can't remove users from a workspace. You can only archive their permissions.
 - **ACTIVE:** Reactivate an archived user account. You can't change the status from **INVITED** to **ACTIVE**.

From the response body that you received in the previous step, copy the JSON object within the `data` array and modify relevant attributes.

The following table lists required parameters to be passed in the request body. Other parameters are optional.

Parameter	Parameter Type	Data Type	Required	Description
<code>user.id</code>	Body	String	Required	The ID of the user.
<code>workspace.id</code>	Body	String	Required	The ID of the workspace the user belongs to.
<code>role</code>	Body	String	Required	The role to be assigned to the user.
<code>status</code>	Body	String	Required	The status to be assigned to the user.

In the example request, we change the user role to **ADMIN**. The request body contains only required parameters.

Example request:

```
curl --request POST \
  --url https://{SUBDOMAIN}.leanix.net/services/mtm/v1/permissions \
  --header 'content-type: application/json' \
  --header 'authorization: Bearer {ACCESS_TOKEN}' \
  --data '{
    "user": {
      "id": "f767b7a2-5dd8-423e-d7f4-f9ea34d36ce8"
    },
    "workspace": {
      "id": "bdba4b5d-2d63-49ef-f8g4-045f020294bb"
    },
    "role": "ADMIN",
    "status": "INVITED"
  }'
```

Example request body:

```
{
  "user": {
    "id": "f767b7a2-5dd8-423e-d7f4-f9ea34d36ce8"
  },
  "workspace": {
    "id": "bdba4b5d-2d63-49ef-f8g4-045f020294bb"
  },
  "role": "ADMIN",
  "status": "INVITED"
}
```

Example response:

```
{
  "status": "OK",
  "type": "Permission",
  "errors": [],
  "total": 0,
  "data": {
    "links": [...],
    "id": "04259076-8cbc-418e-5d8h-20144d5d8bda",
    "user": {
      "id": "f767b7a2-5dd8-423e-d7f4-f9ea34d36ce8",
      "account": {...},
      "userName": "john.doe@organization.com",
      "email": "john.doe@organization.com",
      "role": "ACCOUNTUSER",
      "status": "ACTIVE",
      "technicalUser": false,
      "scimManaged": false,
      "transientUser": false,
      "links": [...],
      ...
    },
    "workspace": {...},
    "workspaceId": "bdba4b5d-2d63-49ef-f8g4-045f020294bb",
    "role": "ADMIN",
    "status": "INVITED",
    ...
  }
}
```

Retrieving Survey Results

Retrieve survey results using the Survey REST API.

Overview

Caution

The Poll API v2 is deprecated. Transition to the new Survey API v1. See [Poll API Deprecation: Transition to the New Survey API \(March 2027\)](#).

Surveys allow you to collect data about your organization's enterprise architecture landscape from various stakeholders. You can manage surveys using the Survey REST API. See [Surveys](#).

The Survey API provides access to the following resources:

- **Surveys:** A survey is a form template that survey creators send to recipients to collect specific information about fact sheets. Each survey has a unique ID.
- **Survey runs:** A survey run is an operation of sending a specific version of a survey to recipients. Each survey run has a unique ID.
- **Survey results:** A survey result stores the answers of survey recipients for a specific fact sheet. Each fact sheet included in a survey run generates a survey result with a unique ID.
- **Survey templates:** Survey forms with predefined questions. Admin users can download templates to a workspace from the SAP LeanIX extension hub.

In this guide, we retrieve surveys, survey runs, and survey results using the Survey API.

Prerequisites

- Get an API token by creating a technical user. See [Technical Users](#).
- Learn how to authenticate to SAP LeanIX APIs in [Authentication to SAP LeanIX Services](#).
- Get your SAP LeanIX subdomain `https://{SUBDOMAIN}.leanix.net`. You can get your subdomain value from the workspace URL. For details, see [Base URL](#).

Step 1: Get the Survey ID

First, get the ID of the survey you want to retrieve results from. To retrieve all surveys, make a GET request to the following endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/survey/v1/polls
```

In the example request, we retrieve a survey by its title using the `q` query parameter.

Example request:

```
curl -X GET \
--header 'Accept: application/json' \
--header 'Authorization: Bearer {ACCESS_TOKEN}' \
'https://{SUBDOMAIN}.leanix.net/services/survey/v1/polls?page=1&size=30&sort=-createdAt&q=Location'
```

Example response:

```
{
  "status": "OK",
  "type": "Poll",
  "total": 1,
```

```

"data": [
  {
    "id": "0b62781b-4266-4020-g87h-4cacb7947ba8",
    "title": "Location",
    "language": "en",
    "questionnaire": {
      "questions": [
        {
          "id": "q1-uuid",
          "text": "Where are your users located?",
          "type": "TEXT"
        }
      ]
    },
    "createdAt": "2024-01-15T10:00:00Z",
    "createdBy": {
      "id": "user-uuid",
      "firstName": "John",
      "lastName": "Doe"
    }
  }
]
}

```

From the response, save the `id` of the desired survey.

Step 2: Get the Survey Run ID

Now that you have the `id` of the survey, retrieve all runs for this survey. To do that, make a GET request to the following endpoint:

`https://{SUBDOMAIN}.leanix.net/services/survey/v1/polls/{id}/pollRuns`

Example request:

```

curl -X GET \
--header 'Accept: application/json' \
--header 'Authorization: Bearer {ACCESS_TOKEN}' \
'https://{SUBDOMAIN}.leanix.net/services/survey/v1/polls/0b62781b-4266-4020-g87h-4cacb7947ba8/pollRuns'

```

Example response:

```

{
  "status": "OK",
  "type": "PollRun",
  "total": 2,
  "data": [
    {
      "id": "ce6a4af8-7060-4adc-5ac8-2daa85efe616",
      "poll": {
        "id": "0b62781b-4266-4020-g87h-4cacb7947ba8",
        "title": "Location"
      }
    },
  ],
}

```

```

    "status": "STARTED",
    "startTime": "2024-08-24T10:55:54.693671Z",
    "factsheetsTotal": 5,
    "factsheetsCompleted": 3,
    "factsheetsSelected": 5,
    "creator": {
      "id": "user-uuid-1",
      "firstName": "Jane",
      "lastName": "Smith",
      "email": "jane.smith@example.com"
    }
  },
  {
    "id": "c0735b3b-b032-4b0d-{...}-77f5e68b5210",
    "poll": {
      "id": "0b62781b-4266-4020-g87h-4cacb7947ba8",
      "title": "Location"
    },
    "status": "STARTED",
    "startTime": "2024-08-22T06:40:08.927306Z",
    "factsheetsTotal": 5,
    "factsheetsCompleted": 5,
    "factsheetsSelected": 5,
    "creator": {
      "id": "user-uuid-2",
      "firstName": "Bob",
      "lastName": "Johnson",
      "email": "bob.johnson@example.com"
    }
  }
]
}

```

From the response, save the `id` of the desired survey run.

Step 3: Get Survey Results

You can get survey results for:

- All fact sheets included in a survey run
- A specific fact sheet included in a survey run

Get Results for All Fact Sheets in a Survey Run

With the survey run ID, you can retrieve results for all fact sheets included in the survey run. To do that, make a GET request to the following endpoint:

`https://{SUBDOMAIN}.leanix.net/services/survey/v1/pollRuns/{pollRunID}/pollResults`

Example request:

```
curl -X GET \
--header 'Accept: application/json' \
--header 'Authorization: Bearer {ACCESS_TOKEN}' \
'https://{SUBDOMAIN}.leanix.net/services/survey/v1/pollRuns/ce6a4af8-7060-4adc-5ac8-2daa85efe616/po'
```

The response contains survey results for all fact sheets included in the survey run. Each result is associated with a specific fact sheet and has a unique ID.

Example response:

```
{
  "status": "OK",
  "type": "PollResult",
  "total": 2,
  "data": [
    {
      "id": "10fd7951-1e44-4643-e6g4-ed5db8c44e29",
      "pollRunId": "ce6a4af8-7060-4adc-5ac8-2daa85efe616",
      "status": "DONE",
      "answers": [
        {
          "questionId": "cc2a3670-9d9d-5e41-6g4h-c579cdd89b1c",
          "value": "Users are based in Australia.",
          "type": "TEXT"
        }
      ],
      "users": [
        {
          "id": "user-uuid-3",
          "firstName": "Alice",
          "lastName": "Brown",
          "email": "alice.brown@example.com"
        }
      ],
      "sender": {
        "id": "user-uuid-1",
        "firstName": "Jane",
        "lastName": "Smith",
        "email": "jane.smith@example.com"
      },
      "factSheet": {
        "id": "fs-uuid-1",
        "name": "System A",
        "displayName": "Australian Office Systems",
        "type": "Application"
      }
    },
    {
      "id": "2dff1a8b-33b6-494c-b855-c8604045e333",
      "pollRunId": "ce6a4af8-7060-4adc-5ac8-2daa85efe616",
      "status": "DONE",
      "answers": [
        {

```

```

        "questionId": "cc2a3670-9d9d-5e41-6g4h-c579cdd89b1c",
        "value": "Users are based in Brazil.",
        "type": "TEXT"
    }
],
"users": [
    {
        "id": "user-uuid-4",
        "firstName": "Carlos",
        "lastName": "Silva",
        "email": "carlos.silva@example.com"
    }
],
"sender": {
    "id": "user-uuid-1",
    "firstName": "Jane",
    "lastName": "Smith",
    "email": "jane.smith@example.com"
},
"factSheet": {
    "id": "fs-uuid-2",
    "name": "System B",
    "displayName": "Brazilian Office Systems",
    "type": "Application"
}
}
]
}

```

Get Results for a Specific Fact Sheet in a Survey Run

You can retrieve a result for a specific fact sheet included in a survey run by the result ID. First, retrieve all results and save the id of the desired result from the response.

To retrieve a survey result by ID, make a GET request to the following endpoint:

`https://{SUBDOMAIN}.leanix.net/services/survey/v1/pollResults/{pollResultID}`

Example request:

```

curl -X GET \
--header 'Accept: application/json' \
--header 'Authorization: Bearer {ACCESS_TOKEN}' \
'https://{SUBDOMAIN}.leanix.net/services/survey/v1/pollResults/10fd7951-1e44-4643-e6g4-ed5db8c44e2f'

```

Example response:

```

{
  "status": "OK",
  "type": "PollResult",
  "total": 1,
  "data": {

```



```

    "id": "10fd7951-1e44-4643-e6g4-ed5db8c44e29",
    "pollRunId": "ce6a4af8-7060-4adc-5ac8-2daa85efe616",
    "status": "DONE",
    "answers": [
      {
        "questionId": "cc2a3670-9d9d-5e41-6g4h-c579cdd89b1c",
        "value": "Users are based in Australia.",
        "type": "TEXT"
      }
    ],
    "users": [
      {
        "id": "user-uuid-3",
        "firstName": "Alice",
        "lastName": "Brown",
        "email": "alice.brown@example.com"
      }
    ],
    "sender": {
      "id": "user-uuid-1",
      "firstName": "Jane",
      "lastName": "Smith",
      "email": "jane.smith@example.com"
    },
    "factSheet": {
      "id": "fs-uuid-1",
      "name": "System A",
      "displayName": "Australian Office Systems",
      "type": "Application"
    },
    "requestingUser": {
      "id": "user-uuid-5",
      "firstName": "Admin",
      "lastName": "User",
      "email": "admin@example.com"
    }
  }
}

```

Sending Alerts to Slack and Microsoft Teams

Set up a webhook to receive notifications about failed events in your integration runs.

To monitor the status of your integrations, you can set up a webhook to receive real-time notifications about events that fail during integration runs. This allows you to promptly address issues, minimizing downtime and ensuring the smooth operation of your integrations. To learn more about webhooks, see [Webhooks](#).

You can access synchronization logs for integrations in the [Synchronization Logging](#) section of the administration area. For more information, see [Synchronization Logging](#).

i Note

You can modify the webhook callback to receive notifications for specific event types, not only for failed ones.

Sending Alerts to Slack

To send alerts for failed integration events to Slack, create a webhook in SAP LeanIX. For instructions, see [Creating a Webhook](#).

Specify the following details for the webhook:

- **Type:** PUSH
- **Target URL:** You can get a target URL from the [incoming webhook](#) that you've set up in Slack.
- **Triggering Events:** Select INTEGRATION_RUN_FINISHED and INTEGRATION_RUN_ABORTED.
- **Callback:** Enter the following code to filter failed events. Replace `workspace_url` with your workspace URL that includes the workspace name, for example, `https://your-company.leanix.net/WorkspaceName`.

```
var payload = delivery.payload;
delivery.active = false;
var base_url = 'workspace_url';
if (payload.errorCount > 0) {
    delivery.active = true;
    var text = payload.type + ' with Scope: ' + payload.scope + ' has status: ' + payload.pro
}
delivery.payload = { text: text }
```

Sending Alerts to Microsoft Teams

To send alerts for failed integration events to Microsoft Teams, create a webhook in SAP LeanIX. For instructions, see [Creating a Webhook](#).

Specify the following details for the webhook:

- **Type:** PUSH
- **Target URL:** To get a target URL for event notifications, create a workflow in Microsoft Teams. Workflows enable you to automate processes that connect one or more apps to Microsoft Teams. To learn how to create workflows, refer to the [Microsoft documentation](#). If you're using Office 365 connectors, migrate your existing connectors to workflows. To learn more, see the blog entry [Retirement of Office 365 connectors within Microsoft Teams](#) on the Microsoft 365 Developer Blog.
- **Triggering Events:** Select INTEGRATION_RUN_FINISHED and INTEGRATION_RUN_ABORTED.
- **Callback:** Enter the following code to filter failed events. Replace `workspace_url` with your workspace URL that includes the workspace name, for example, `https://your-company.leanix.net/WorkspaceName`.

```
var payload = delivery.payload;
delivery.active = false;
var base_url = 'workspace_url';
if (payload.errorCount > 0) {
    delivery.active = true;
    var uri = base_url + '/admin/synclog/' + payload.synchronizationId
    var text = payload.type + ' with Scope: ' + payload.scope + ' has status: ' + payload.pro
    text += "Click **Sync Log** to check details!";
}
delivery.payload = {
    "type": "message",
    "attachments": [
        {
            "contentType": "application/vnd.microsoft.card.adaptive",
            "contentUrl": null,
            "content": {
                "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
                "type": "AdaptiveCard",

```

Explore SAP LeanIX APIs and learn when to use each API.

We provide a suite of powerful APIs that enable you to interact with and manipulate data within SAP LeanIX. These APIs offer flexible and efficient ways to integrate with other systems, manage your workspace data, and create custom reports.

The following table provides an overview of available APIs.

API	Description
REST APIs	REST APIs offer access to a wide range of resources, including the meta model, workspace settings, users, metrics, surveys, and more. These APIs are ideal for integration with systems that support RESTful interactions. You can explore and experiment with REST APIs using the OpenAPI explorer, accessible from your workspace.
GraphQL API	The GraphQL API enables you to retrieve and update fact sheets and related data using GraphQL queries and mutations. You can use the built-in GraphiQL tool in the administration area of your workspace to interactively develop and test your queries and mutations.
Integration API	The Integration API supports integrations with external platforms. It uses our custom LeanIX Data Interchange Format (LDIF) to process data, simplifying the integration process and reducing complexity. This API is ideal for building integrations between SAP LeanIX and other vendor systems.

REST APIs

SAP LeanIX REST APIs offer a versatile interface for interacting with a wide range of resources, including the meta model, workspace settings, users, metrics, surveys, and more. These APIs are designed for seamless integration with systems that support RESTful interactions.

To enhance your experience with SAP LeanIX REST APIs, we provide the [OpenAPI explorer](#) . This tool enables you to explore APIs, send requests, and view responses directly in your browser. To navigate to the OpenAPI explorer from your workspace, on the help menu, choose [Developer Tools](#) > [OpenAPI Documentation](#).

For additional information, see [REST APIs](#).

GraphQL API

SAP LeanIX chose GraphQL for managing fact sheets and related data due to its adaptability and efficiency in handling intricate data structures and relationships. Fact sheets in SAP LeanIX often encompass complex models and relationships, which can be difficult to handle with traditional REST APIs. GraphQL presents a solution to this challenge.

Rather than dealing with fixed data structures, GraphQL enables clients to specify the exact data they need, significantly reducing the volume of data transferred over the network and enhancing performance. This is particularly beneficial when dealing with complex fact sheets, as it allows for more precise and efficient data retrieval.

Moreover, GraphQL's capability to fetch related data in a single request aids in managing the relational complexity of fact sheets. With traditional REST APIs, retrieving related data often necessitates multiple requests to the server. However, with GraphQL, you can retrieve all related data in a single query, making it a powerful tool for working with the interconnected data in fact sheets.

GraphQL enables you to query data on your fact sheets as well as mutate (create, update, or delete) data. In addition, GraphQL is used to create custom reports, specifically to pull data from the workspace and present it using charts and other graphics.

We provide GraphiQL, an interactive tool that streamlines interaction with the GraphQL API, providing an intuitive interface for workspace administrators to explore the GraphQL schema, experiment with queries and mutations, and validate them in real time. To navigate to the GraphiQL tool, on the help menu, choose [Developer Tools](#) > [GraphQL Editor](#).

For additional information, see [GraphQL API](#).

Integration API

The Integration API is a powerful and flexible tool designed to streamline the integration process between SAP LeanIX and other vendor systems. One of the key features of the API is that it uses the generic LeanIX Data Interchange Format (LDIF), which minimizes complexity and eliminates the need to map logic within the code. This enables developers to effectively use the API without needing an in-depth understanding of the SAP LeanIX meta model. For more information, see [LeanIX Data Interchange Format \(LDIF\)](#).

The Integration API manages CRUD operations for any inbound data from other systems within SAP LeanIX, significantly reducing complexity. It also allows for the definition of variables and exceptions and the use of [JUEL expressions](#)  to extract or import only specific values.

Additional benefits of the Integration API include the encapsulation of SAP LeanIX configuration in every connector, the capability to integrate even when the external system doesn't allow direct access, and flexible error handling. The API mitigates failure due to single or multiple data issues by disregarding data that doesn't meet the requirements.

The Integration API also supports the import and export of data from your workspace. This feature is particularly beneficial when the built-in functionality for importing or exporting doesn't align with your needs, such as when managing attributes on relations. The API also excels in handling more complex scenarios, for example, when the data from the source system is not in a clean state.

For additional information, see [Integration API](#).

REST APIs

Get started with SAP LeanIX REST APIs.

Overview

REST stands for Representational State Transfer. REST APIs are web service APIs that adhere to the constraints of REST architecture. Learn more about [REST](#) ➡ .

SAP LeanIX offers a wide range of REST APIs. These APIs provide access to various resources, such as the meta model, users, metrics, surveys, and more. Use REST APIs to integrate with systems that support RESTful interactions.

i Note

The endpoint base URL varies for different customers. It depends on the subdomain of your workspace. In our documentation, the base URL appears as follows: `https://{SUBDOMAIN}.leanix.net`. For more information, visit [Base URL](#).

To work with your fact sheets and related data, use the [GraphQL API](#).

Available APIs

To view available APIs and reference documentation, visit the [OpenAPI explorer](#) ➡ .

Authentication

To learn how to authenticate to REST APIs, see [Authentication to SAP LeanIX APIs](#).

OpenAPI Explorer

In the [OpenAPI explorer](#) ➡ , you can view available REST APIs and try out requests right in your browser. To navigate to the OpenAPI explorer from your workspace, on the help menu (question mark icon), choose **Developer Tools** > **OpenAPI Documentation**.

i Note

Because there are multiple instances of the SAP LeanIX app, the link to the OpenAPI explorer varies for different customers. Navigate to the OpenAPI explorer from your workspace.

Use Cases

SAP LeanIX REST APIs provide access to various resources. You can use them to import and export data, set up integrations, and automate workflows for your organization.

Here are some example use cases:

- [Update user permissions through the MTM API](#)
- [Get survey results using the Poll API](#)

REST APIs Overview

Get an overview of available REST APIs and learn when to use each API.

i Note

To view available APIs and reference documentation, visit the [OpenAPI Explorer](#) .

To navigate to the OpenAPI explorer from your workspace, on the help menu (question mark icon), choose **Developer Tools > OpenAPI Documentation**.

REST API	Usage	Related Information
Apptio Connector API	Configure and run your Apptio integration.	Apptio Integration
Automations API	Manage your automation templates and retrieve information about instances and quota.	Automations
Discovery SaaS API	Manage SaaS discovery integrations and access discovered data.	SaaS Discovery
Import Export API	Interact with Inventory data through Excel files and the standardized OData capability.	• Importing Fact Sheet Data Through Excel File • Exporting Fact Sheet Data as Excel File • OData Integration
Integration API	Configure and run your custom integrations.	Integration REST API
Integration Collibra API	Configure and run your Collibra integration.	Collibra Integration
Integration ServiceNow API	Configure and run your ServiceNow integration.	ServiceNow Integration
Integration Signavio API	Configure and run your SAP Signavio integration.	SAP Signavio Integration
MTM API	Automate user management tasks for your workspaces.	• Authentication to SAP LeanIX Services • Managing Users
Metrics API	Track your own metrics and set up KPIs based on inventory filters.	Metrics
Navigation API	Manage collections of reports and diagrams.	Collections
Notifications API	Manage your notification settings.	Notifications Center
Pathfinder API	Search for and manage the lifecycle of inventory fact sheets, such as create or modify your fact sheets. We recommend that you use our GraphQL API to work with	• GraphQL API • Bookmarks

REST API	Usage	Related Information
	fact sheets. Manage bookmarks programmatically.	
Recon API	Get recommendations on potentially related fact sheets for a specific fact sheet and its description.	
Reference Data API	Maintain links between your fact sheets and our reference catalog.	Reference Catalog
Self-Built Software Discovery API	Manage microservices and their associated Software Bill of Materials (SBOMs).	Self-Built Software Discovery
Storage API	Manage and retrieve file resources for your workspace.	
Survey API	Manage your surveys, survey runs, and survey submissions.  Caution The Poll API v2 is deprecated. Transition to the new Survey API v1. For details, see Poll API Deprecation: Transition to the New Survey API (March 2027) .	Surveys
Synclog API	Get and update synchronization logs of your out-of-the-box or custom integrations.	Synchronization Logging
To-do API	Manage your to-dos and to-do assignees.	To-Dos (Administration)
Transformations API	Manage transformations and impacts.	• Transformations • Impacts
Webhooks API	Subscribe to domain events within SAP LeanIX.	Webhooks

GraphQL API

Get started with the SAP LeanIX GraphQL API.

Overview

GraphQL is a query language for APIs that allows clients to request precisely the data they need. It was developed by Facebook to provide a more efficient and flexible alternative to traditional REST APIs. To learn more about GraphQL, visit the [GraphQL website](#) .

The SAP LeanIX GraphQL API allows you to work with your fact sheets and related data. You can retrieve and manipulate fact sheet data using GraphQL [queries and mutations](#) . To explore more examples, see [Use Cases](#).

The SAP LeanIX GraphQL API adheres to the [October 2021 specification](#)  for GraphQL.

Benefits

GraphQL has several benefits, such as:

- You can summarize required related data in a single request. This allows you to focus on the business logic instead of combining call results and handling different transactions.
- For large volumes of data, such as in reports or analytics, GraphQL significantly increases performance. You can transfer exactly the attributes that you need through the network, without extra latency.
- The GraphQL schema is strongly typed. It's an advantage because strong typing implies that the API is largely self-documented. You can see what the immediate response to the query will be. The schema provides descriptions for all API components, making it easier for the user to navigate the reference documentation.
- Because everything in GraphQL is schema-driven, it's easy to extend an API. You can add new fields or types without affecting existing ones.
- In GraphQL, the `@deprecated` annotation is used to mark a field or type as deprecated. This eliminates the need for versioning an API.

Use Cases

With our GraphQL API, you can complete various tasks, such as:

- Retrieve a single fact sheet or a list of fact sheets
- Retrieve Applications with specific attributes
- Manage relations between fact sheets
- Update specific attributes on fact sheets
- Fetch data from your workspaces to build custom reports

To explore more examples, see [Sample Queries](#).

GraphQL Basics

To learn the basics of GraphQL, see the following:

- [GraphQL Basics](#): Understand how queries and mutations work.
- [Using GraphQL with Python](#): Learn how to run GraphQL queries and mutations using Python.
- [Sample Queries](#): View examples and try them out in your workspace.

For comprehensive information about GraphQL, refer to the [GraphQL documentation](#) 📖.

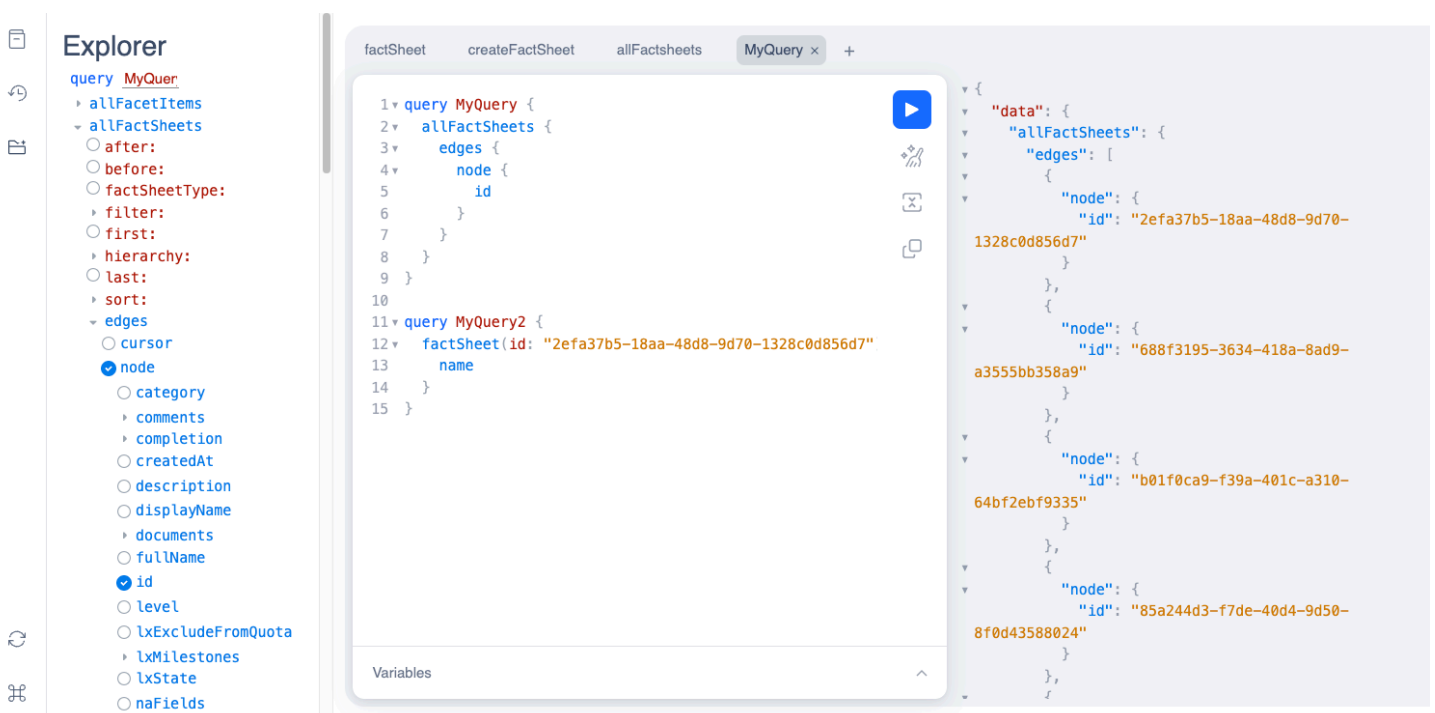
GraphiQL Tool

In the GraphiQL tool in your workspace, you can explore the GraphQL schema and interactively develop and test queries. To open the GraphiQL tool, on the help menu, choose **Developer Tools** > **GraphQL Editor**.

By default, only admins have access to the GraphiQL tool. As an admin, you can grant permission to access the GraphiQL tool to non-admin roles in the **User Roles and Permissions** section of the administration area. For more details, see [Role-Based Permissions](#).

In the GraphQL tool, you can:

- Create and execute queries in the interactive [Query Editor](#). To define variables for a query, enter them in the [Variables](#) section in JSON format.
- View the GraphQL schema. To open the schema, in the sidebar on the left, click [Show Documentation Explorer](#).
- View your request history and open your previous requests. To expand the [History](#) section, in the sidebar, click [Show History](#). You can delete specific or all entries, add requests to favorites, and edit the request labels.
- Interactively create queries and mutations in the [Explorer](#) section. To expand the section, in the sidebar, click [Show GraphQL Explorer](#). Here, you can:
 - Select queries and mutations that you want to add to the [Query Editor](#)
 - View and select available attributes for each query and mutation, without having to navigate to the GraphQL schema
 - Duplicate, rename, and delete queries and mutations



Creating Queries Using the Explorer in the GraphQL Tool

GraphQL Schema

A GraphQL schema defines the structure and capabilities of a GraphQL API.

You can explore the GraphQL schema in the [Docs](#) section in the GraphQL tool. To expand the section, in the sidebar on the left, click [Show Documentation Explorer](#).



Docs

Q ⌘ K



A GraphQL schema provides a root type for each kind of operation.



Root Types

query: **Queries**

mutation: **Mutations**

All Schema Types

ID

Boolean

SimpleBaseFactSheetConnection

Documentation Explorer in the GraphQL Tool

To reload the GraphQL schema, in the sidebar, click [Re-fetch GraphQL schema](#).

In the **Docs** section, you can:

- View a list of available queries and mutations
- Search for specific queries, mutations, fields, arguments, and other API elements

In the specification, required attributes are marked with an exclamation mark.

Retrieving the GraphQL Schema Metadata

To retrieve the GraphQL schema metadata, use the following query.

```

query {
  __schema {
    types {
      name
      kind
      interfaces {
        kind
        name
      }
      fields {
        name
      }
      inputFields {
        name
        defaultValue
      }
    }
    queryType {
      name
    }
  }
}

```

Authentication

To authenticate your requests to the GraphQL API, you need an access token. For more information, see [Authentication to SAP LeanIX Services](#).

When using the GraphQL tool in your workspace, you don't need to authenticate.

GraphQL API Endpoints

Endpoint for standard GraphQL requests:

```
https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/graphql
```

Endpoint for GraphQL file uploads:

```
https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/graphql/upload
```

Exporting Queries from the Inventory

You can export GraphQL queries from the inventory based on the current inventory data, including any applied filters and sorting options. You can copy a JSON query for external use or export it to the interactive GraphiQL tool in your workspace. By using these export options, you can create queries more quickly and save time on configuring complex filtering options.

To access query export options from the inventory, select the arrow icon on the filter bar, then choose the option you need:

- **Copy as JSON Query:** Copy a JSON query based on the current inventory data for external use. The query includes any filters and sorting options you've applied.
- **Open in GraphiQL:** Generate a GraphQL query based on the current inventory data and open it in the GraphiQL tool. The query includes any filters and sorting options you've applied.

By default, only admins can access these export options in the inventory. As an admin, you can grant permissions for one or both export options to non-admin roles in the [User Roles and Permissions](#) section of the administration area. For more details, see [Role-Based Permissions](#).

The screenshot shows the SAP LeanIX web interface. The top navigation bar includes 'SAP LeanIX', 'Dashboards', 'Inventory' (selected), 'Reports', 'Diagrams', and 'More'. A search bar is present with the text 'Search for fact sheet'. On the right, there are icons for 'Full View', a printer, a speech bubble, and a user profile. The main content area is titled 'Inventory' and shows a list of results. A filter bar at the top of the results list includes 'Add Filter with AI', 'Type: Application', and a plus sign. Below the filter bar, it says '40 of 144 results'. The first result is 'AC Management' with a description 'Application for AC management' and a progress bar showing '17% complete'. The second result is 'AC Management Cloud' with a description 'Set Your Role' and a progress bar showing '1% complete'. A context menu is open over the 'AC Management' result, showing two options: 'Copy as JSON Query' and 'Open in GraphiQL'. The menu is highlighted with a yellow box.

Query Export Options in the Inventory

Best Practices

Here are some best practices for using GraphQL:

- Focus on queries and mutations that are one-off in nature or for which you don't need to regularly (such as monthly or weekly) get up-to-date information.

- Use queries to pull relatively simple information. While you can run complex queries, there are other alternative tools that you can leverage. For example, the [Integration API](#) may be an easier and more flexible choice for some use cases. Due to a complexity limit in GraphQL, long queries are denied to prevent possible Denial of Service (DOS) attacks.
- When you query a large number of fact sheets, use pagination. To learn more, see [Pagination in GraphQL](#).
- Use GraphQL when building custom reports. GraphQL allows you to pull information presented in charts and other visual formats from your workspace. To view examples of using GraphQL with custom reports, see [Using GraphQL Queries with Custom Reports](#).

Scripts

You can find scripts that use GraphQL in our public SAP LeanIX GitHub repository. Here are some examples:

- [Update Costs](#) ➦
- [Import Metrics](#) ➦
- [Create Resources](#) ➦

GraphQL Basics

Learn the GraphQL basics. Create and execute your first queries and mutations.

This topic provides an overview of basic GraphQL concepts to help you get started. To learn about GraphQL and how it works, refer to the [GraphQL documentation](#) ➦.

To create and test your first queries and mutations, you can use the [GraphiQL Tool](#) in your workspace.

To explore the GraphQL schema, navigate to the [Documentation Explorer](#) section in the GraphiQL tool. In the schema, an exclamation mark next to an attribute indicates that the attribute is required (cannot be null).

→ Tip

To explore example queries and mutations for the SAP LeanIX GraphQL API, see [Sample Queries](#).

Queries

GraphQL queries allow you to retrieve data, similar to GET methods in REST APIs. To learn more about queries, see [Queries and Mutations](#) ➦ in the GraphQL documentation.

To explore queries available within the SAP LeanIX GraphQL API, navigate to the [Documentation Explorer](#) section in the [GraphiQL tool](#).

As an example, let's retrieve a Fact Sheet by its ID using the `factSheet` query. `id` is a required argument for this query. To get the ID of a Fact Sheet, navigate to the Fact Sheet page in the application user interface and copy the ID from the URL.

In the example, we request the following Fact Sheet attributes to be returned in the response: `id`, `name`, and `type`. You can modify the query to retrieve the data you need.

Example query:

```
{
  factSheet(id: "28fe4aa2-6e46-41a1-a131-72afb3acf256") {
```

```

      id
      name
      type
    }
  }
}

```

Example response:

```

{
  "data": {
    "factSheet": {
      "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
      "name": "AC Management",
      "type": "Application"
    }
  }
}

```

Mutations

GraphQL mutations allow you to modify data, similar to POST, PUT, PATCH, and DELETE methods in REST APIs. To learn more about mutations, see [Mutations](#) in the GraphQL documentation.

To explore mutations available within the SAP LeanIX GraphQL API, navigate to the [Documentation Explorer](#) section in the [GraphiQL tool](#).

As an example, let's create an Application Fact Sheet using the `createFactSheet` mutation. The fields `name` and `type` of the `BaseFactSheetInput` input argument are required for this mutation. We pass these fields using the `$input` variable.

To retrieve specific Fact Sheet attributes in the response, use the `factSheet` query. In the example, we retrieve the following attributes: `id`, `name`, and `type`.

Example mutation with a variable:

```

mutation ($input: BaseFactSheetInput!) {
  createFactSheet(input: $input) {
    factSheet {
      id
      name
      type
    }
  }
}

```

The mutation requires an `$input` variable. Define the variable in JSON format.

We recommend using variables in mutations with a large number of input parameters.

Variables:

```

{
  "input":{"name":"New Fact Sheet", "type": "Application"}
}

```

Alternatively, you can create a mutation without a variable.

Example mutation without a variable:

```
mutation {
  createFactSheet(input: {name: "New Fact Sheet", type: Application}) {
    factSheet {
      id
      name
      type
    }
  }
}
```

Once you have executed the request, you get a response that contains the details of the new Fact Sheet. The `id` is generated automatically.

Example response:

```
{
  "data": {
    "createFactSheet": {
      "factSheet": {
        "id": "16fdc1a9-ae59-4d0a-b6ce-bb717baa74f3",
        "name": "New Fact Sheet",
        "type": "Application"
      }
    }
  }
}
```

Including Multiple Mutations in a Single Request Using Aliases

To make your program run faster and deal with delays caused by network connection, you can include multiple mutations in one request by using mutation aliases.

When using aliases in mutations, follow these guidelines:

- Choose a unique alias for each mutation.
- Split your mutations into manageable chunks to prevent execution disruption caused by HTTP timeouts. Depending on the complexity of specific mutations, 50 is a reasonable chunk size to start with.

In the example mutation, we create two Application Fact Sheets using mutation aliases `fs1` and `fs2`.

Example mutation:

```
mutation {
  fs1: createFactSheet(input: {name: "fs1", type: Application}) {
    factSheet { id }
  },
  fs2: createFactSheet(input: {name: "fs2", type: Application}) {
    factSheet { id }
  }
}
```

Example response:

```

{
  "data": {
    "fs1": {
      "factSheet": {
        "id": "8acea607-94c6-4818-9c64-2af5ecd0be98"
      }
    },
    "fs2": {
      "factSheet": {
        "id": "6fe9bd88-dd6b-461d-ab35-db139961b48f"
      }
    }
  }
}

```

Retrieving a List of Filtering Facet Keys

The following example shows how to retrieve a list of all facet keys within a workspace.

Example query:

```

{
  allFactSheets{
    filterOptions {
      facets {
        facetKey
        results {
          name
          key
        }
      }
    }
  }
}

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "filterOptions": {
        "facets": [
          {
            "facetKey": "FactSheetTypes",
            "results": [
              {
                "name": "BusinessCapability",
                "key": "BusinessCapability"
              },
              {
                "name": "Process",
                "key": "Process"
              },
              {
                "name": "UserGroup",
                "key": "UserGroup"
              }
            ]
          }
        ]
      }
    }
  }
}

```

```

    },
    {
      "name": "Project",
      "key": "Project"
    },
    {
      "name": "Application",
      "key": "Application"
    },
    {
      "name": "Interface",
      "key": "Interface"
    },
    {
      "name": "DataObject",
      "key": "DataObject"
    },
    {
      "name": "ITComponent",
      "key": "ITComponent"
    },
    {
      "name": "Provider",
      "key": "Provider"
    },
    {
      "name": "TechnicalStack",
      "key": "TechnicalStack"
    }
  ]
},
{
  "facetKey": "hierarchyLevel",
  "results": [
    {
      "name": "1",
      "key": "1"
    },
    {
      "name": "2",
      "key": "2"
    },
    {
      "name": "3",
      "key": "3"
    }
  ]
},
{
  "facetKey": "DataQuality",
  "results": [
    {
      "name": "noResponsible",
      "key": "_noResponsible_"
    }
  ]
}

```



```

    },
    {
      "name": "qualitySealBroken",
      "key": "_qualitySealBroken_"
    },
    {
      "name": "noDescription",
      "key": "_noDescription_"
    },
    {
      "name": "noLifecycle",
      "key": "_noLifecycle_"
    }
  ]
},
{
  "facetKey": "_TAGS_",
  "results": [
    {
      "name": "Headquarter",
      "key": "aa51ccc9-db61-48df-89d1-f217f37757ea"
    },
    {
      "name": "Tag2",
      "key": "819d3d4f-f044-470a-8f5d-c11bc15eff67"
    }
  ]
}
]
}
}
},
"errors": null
}

```

Pagination in GraphQL

The SAP LeanIX GraphQL API employs the concept of [relay cursor connections](#)  for pagination, allowing for efficient and flexible data retrieval. In this pagination model, instead of using offsets, the API uses cursors to navigate through the data set. For more details, see [Pagination in GraphQL](#).

Filtering in GraphQL

Learn how filtering in GraphQL works and how to apply filters to your queries.

Data filtering is crucial for extracting relevant insights from your organization's data. GraphQL allows you to filter data accurately based on specific criteria, enhancing data retrieval and management efficiency.

This document offers an overview of data filtering in GraphQL, with a focus on its implementation in the SAP LeanIX GraphQL API. You'll learn how to fetch available filters and use them in your GraphQL queries.

→ Tip

You can export GraphQL queries from the inventory based on your inventory data, including any applied filters and sorting options. To learn more, see [Exporting Queries from the Inventory](#).

i Note

Understanding which GraphQL operations support filtering is key to using filters effectively. You can explore available filters by navigating through the GraphiQL interface in your SAP LeanIX workspace. This interface provides a comprehensive and interactive overview of the GraphQL schema. To learn more about GraphiQL and how to navigate to it, see [GraphiQL Tool](#).

Filters

The SAP LeanIX GraphQL API offers robust filtering capabilities to enhance your data retrieval operations. These filters are particularly useful when managing large data volumes, as they boost the accuracy and efficiency of your data management tasks.

The filtering capabilities are supported in specific operations through the `filter` argument associated with the corresponding API operation. This argument lets you define the criteria that data must meet to be included in the operation's results.

For example, to retrieve fact sheets with a specific technical fit, use the `filter` argument in your API operation. The system returns only the fact sheets that meet this criterion.

Logical Operators

Logical operators are essential tools for filtering complex data sets in GraphQL. They let you combine or modify filter conditions, enabling you to create more complex and precise data queries.

The table below provides an overview of the logical operators you can use in your GraphQL queries:

Logical Operator	Description
AND	Returns true if all specified conditions are met. It allows you to narrow down results by requiring multiple criteria to be satisfied.
OR	Returns true if at least one specified condition is met. It allows you to broaden results by including data that satisfies any of the given criteria.
NOR	Returns true if none of the specified conditions are met. It allows you to exclude data that satisfies any of the given criteria.

Facet Keys

i Note

This document doesn't provide a comprehensive list of all available filter options. You can view a full list of filters and facet keys in the GraphQL schema, accessible through the GraphiQL tool in your SAP LeanIX workspace. Explore filter options for each query in the [Explorer](#) section of GraphiQL and apply them interactively to your operations.

Facets are a powerful feature that enhances the precision of data filtering operations. They are the keys used to filter results in queries. These keys can be based on various attributes, such as the fact sheet type, the relation between fact sheets, and more.

Given the diverse nature of fact sheets in SAP LeanIX, facets fall into two categories:

- [Global facet keys](#)

- [Facet keys specific to fact sheet types](#)

This categorization ensures you can apply the most appropriate and relevant filters to your data.

Global Facet Keys

Global facet keys apply to all fact sheet types. These facets are based on attributes or relations common to all fact sheets, regardless of type. This consistency allows you to apply uniform filters across different fact sheet types, ensuring uniformity in your data retrieval operations. For example, `lxState` is a global facet that lets you filter fact sheets by a specific quality seal state.

The table below lists the available global facet filters. Each entry includes the facet key used in GraphQL queries and the corresponding filter name as it appears on the left-side filter pane in the inventory.

Facet Key	Filter Name in the Inventory
FactSheetTypes	Type
hierarchyLevel	Hierarchy Level
Subscriptions	Subscriptions
DataQuality	Data Quality
lxState	Quality Seal
TrashBin	Trash Bin
TAGS	Other Tags

The following image shows filters on the left-side pane of the inventory. To learn more about filtering data in the inventory, see [Searching and Filtering in the Inventory](#).

SAP LeanIX

DashboardsInventoryReportsDiagramsCollaboration

Search for Fact Sheet

SearchesFilter

Filters

Q Search for filter options

TYPE

Objective4

Platform6

Initiative23

Business Capability65

Business Context86

Organization31

Application143

Interface72

Data Object16

IT Component108

Provider46

Tech Category180

HIERARCHY LEVEL

SUBSCRIPTIONS

DATA QUALITY

QUALITY SEAL

TRASH BIN

OTHER TAGS

Inventory

+ Add Filter

40 of 780 results

Sort by: Alphabetically

AC Management

Application for AC management

Accountable · 16% complete · 2024-12-03 15:52 · Phase in

On PremiseGlobalProduction

AC Management / AC Management Cloud

12345-12345-12345

Responsible · 40% complete · 2024-11-26 18:12 · 1

GlobalProduction

AC Management to HR Admin

Set your role · 31% complete · 2024-05-27 16:28

Demo

accenture

Set your role · 21% complete · 2024-11-26 15:56 · 1

Demo

Account

Set your role · 22% complete · 2023-11-14 17:01 · 1

Demo

acctur

Set your role · 21% complete · 2023-08-16 18:48

Demo

Acoce

Acoce is a provider for graphical design tools

Set your role · 12% complete · 2024-11-26 15:58 · 1

Demo

ACTIONS

+ New Fact Sheet

Export

Import

ADDITIONAL RESOURCES

SaaS Discovery Inbox

SAP Landscape Discovery Inbox

REFERENCE CATALOG

Business Capability

IT Component

Application

RECENTLY VIEWED

AC Management

Adobe Adobe Creative Cloud ...

AC Management / AC Manage...

Filters in the Inventory

Facet Keys Specific to Fact Sheet Types

Facet keys specific to certain types of fact sheets rely on the specific attributes or relations of a particular fact sheet type. For instance, the `dataFlowDirection` facet, which indicates the data flow direction of an interface, is available only for fact sheets of the `Interface` type. This specificity helps narrow down results to a particular fact sheet type.

In the GraphQL Explorer, you can switch between different fact sheet types. Each type has its own unique set of facet filters, offering more granularity in data filtering. As you switch between types, the available facet filters on the sidebar change, allowing you to access and apply unique filters to your data.

Understanding the types of fact sheets you're working with and the filters they support is key to effective data filtering. By leveraging this knowledge, you can apply the most relevant and precise filters, ensuring your data management and analysis tasks are efficient and accurate.

Retrieving Facet Filters

You can retrieve the available filter options for specific fact sheet types. This feature helps you understand the available facets, their keys, the possible operators you can use with them, and the options for applying a specific filter.

The following GraphQL query retrieves the available facet filters for the `Application` fact sheet type:

```
query availableFaceFilters {
  allFactSheets(factSheetType: Application) {
    filterOptions {
      facets {
        facetKey
        possibleOperators
        operator
        results {
          name
          key
        }
        dateFilter {
          from
          to
          type
          minDate
          maxDate
        }
        relationFieldsFilterOperator
        relationFieldsFilter {
          fieldName
          values {
            key
            selected
          }
        }
      }
    }
  }
}
```

The query returns a list of facets for the `Application` fact sheet type. Each facet includes the following attributes:

- `facetKey`: The facet's key.
- `possibleOperators`: A list of logical operators you can use with the facet.
- `operator`: The logical operator used with the facet.
- `results`: A list of available options for using the facet. Each option has a `name` and a `key`.

- **dateFilter**: If the filter supports date ranges, this attribute provides details and options, including the range (from and to), type, and the minimum and maximum dates (minDate and maxDate).
- **relationFieldsFilterOperator**: The operator used with the relational fields filter.
- **relationFieldsFilter**: Details of the relational fields filter, including the field name (fieldName) and a list of values. Each value has a key and a selected status.

The following example response is shortened and doesn't include all available filter options.

```
{
  "data": {
    "allFactSheets": {
      "filterOptions": {
        "facets": [
          {
            "facetKey": "FactSheetTypes",
            "possibleOperators": [],
            "operator": "OR",
            "results": [
              {
                "name": "Objective",
                "key": "Objective"
              },
              {
                "name": "Platform",
                "key": "Platform"
              },
              {
                "name": "Initiative",
                "key": "Initiative"
              },
              {
                "name": "BusinessCapability",
                "key": "BusinessCapability"
              },
              {
                "name": "BusinessContext",
                "key": "BusinessContext"
              },
              {
                "name": "Organization",
                "key": "Organization"
              },
              {
                "name": "Application",
                "key": "Application"
              },
              {
                "name": "Interface",
                "key": "Interface"
              },
              {
                "name": "DataObject",
```

```

      "key": "DataObject"
    },
    {
      "name": "ITComponent",
      "key": "ITComponent"
    },
    {
      "name": "Provider",
      "key": "Provider"
    },
    {
      "name": "TechnicalStack",
      "key": "TechnicalStack"
    }
  ],
  "dateFilter": null,
  "relationFieldsFilterOperator": null,
  "relationFieldsFilter": null
},
{
  "facetKey": "DataQuality",
  "possibleOperators": [
    "OR",
    "AND",
    "NOR"
  ],
  "operator": "OR",
  "results": [
    {
      "name": "noResponsible",
      "key": "_noResponsible_"
    },
    {
      "name": "qualitySealBroken",
      "key": "_qualitySealBroken_"
    },
    {
      "name": "noDescription",
      "key": "_noDescription_"
    },
    {
      "name": "noLifecycle",
      "key": "_noLifecycle_"
    }
  ],
  "dateFilter": null,
  "relationFieldsFilterOperator": null,
  "relationFieldsFilter": null
},
{
  "facetKey": "lxState",
  "possibleOperators": [
    "OR",
    "NOR"

```

```
    ],
    "operator": "OR",
    "results": [
      {
        "name": "BROKEN_QUALITY_SEAL",
        "key": "BROKEN_QUALITY_SEAL"
      },
      {
        "name": "DRAFT",
        "key": "DRAFT"
      },
      {
        "name": "REJECTED",
        "key": "REJECTED"
      },
      {
        "name": "APPROVED",
        "key": "APPROVED"
      }
    ],
    "dateFilter": null,
    "relationFieldsFilterOperator": null,
    "relationFieldsFilter": null
  },
  ...
]
}
}
}
```

Examples

To view example queries with filters, see [Filtering Fact Sheets](#). You can adapt these examples to suit your needs based on the data you want to fetch.

In this section, we provide an example query with multiple facet filters and explain how each filter works. We aim to filter fact sheets by these parameters:

- **Fact sheet type:**Application
- **Technical fit:** unreasonable
- **Functional fit:** Any value except perfect or appropriate
- **Lifecycle phase:** phaseIn, with the start date between 2023 and 2029

The table below lists the facet filters to use in the query and specifies the operations they perform.

Facet Key	Facet Filter Object	Operation
FactSheetTypes	{ "facetKey": "FactSheetTypes", "operator": "OR",	Filters fact sheets of the Application type.

Facet Key	Facet Filter Object	Operation
	<pre> "keys": ["Application"] } </pre>	
technicalSuitability	<pre> { "facetKey": "technicalSuitability", "operator": "OR", "keys": ["unreasonable"] } </pre>	Filters fact sheets with the technical fit set to unreasonable.
functionalSuitability	<pre> { "facetKey": "functionalSuitability", "operator": "NOR", "keys": ["perfect", "appropriate"] } </pre>	Excludes fact sheets with the functional fit set to perfect or appropriate.
lifecycle	<pre> { "facetKey": "lifecycle", "operator": "OR", "keys": ["phaseIn"], "dateFilter": { "type": "RANGE", "from": "2023-01-01", "to": "2029-12-31" } } </pre>	Filters fact sheets in the phaseIn lifecycle phase with the start date between 2023 and 2029.

Additionally, we sort items in the response in descending order by the `updatedAt` field using the `sortings` variable.

Example query:

```

query filterFactSheets($filter: FilterInput!, $sortings: [Sorting]) {
  allFactSheets(filter: $filter, sort: $sortings) {
    totalCount
    edges {
      node {
        id
        displayName
        type
        ... on Application {
          technicalSuitability
          functionalSuitability
          lifecycle {
            phases {
              phase
              startDate
            }
          }
        }
        updatedAt
      }
    }
  }
}

```



```

    }
  }
}

```

Variables:

```

{
  "filter": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "Application"
        ]
      },
      {
        "facetKey": "technicalSuitability",
        "operator": "OR",
        "keys": [
          "unreasonable"
        ]
      },
      {
        "facetKey": "functionalSuitability",
        "operator": "NOR",
        "keys": [
          "perfect",
          "appropriate"
        ]
      },
      {
        "facetKey": "lifecycle",
        "operator": "OR",
        "keys": [
          "phaseIn"
        ],
        "dateFilter": {
          "type": "RANGE",
          "from": "2023-01-01",
          "to": "2029-12-31"
        }
      }
    ]
  },
  "sortings": [
    {
      "key": "updatedAt",
      "order": "desc"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "totalCount": 2,
      "edges": [
        {
          "node": {
            "id": "01740698-1ffa-4729-94fa-da6194ebd7cd",
            "displayName": "AC Management",
            "type": "Application",
            "technicalSuitability": "unreasonable",
            "functionalSuitability": "unreasonable",
            "lifecycle": {
              "phases": [
                {
                  "phase": "phaseIn",
                  "startDate": "2024-02-01"
                }
              ]
            },
            "updatedAt": "2024-12-12T15:07:59.227127386Z"
          }
        },
        {
          "node": {
            "id": "2e50479c-fd64-4800-b398-3f34c653994b",
            "displayName": "Adobe Creative Cloud",
            "type": "Application",
            "technicalSuitability": "unreasonable",
            "functionalSuitability": "insufficient",
            "lifecycle": {
              "phases": [
                {
                  "phase": "phaseIn",
                  "startDate": "2024-02-05"
                }
              ]
            },
            "updatedAt": "2024-12-03T15:45:04.375449793Z"
          }
        }
      ]
    }
  }
}

```

Pagination in GraphQL

Learn how pagination in GraphQL works and how to paginate results in your queries.

Overview

The SAP LeanIX GraphQL API employs the concept of [relay cursor connections](#) ➦ for pagination, allowing for efficient and flexible data retrieval. In this pagination model, instead of using offsets, the API uses cursors to navigate through the dataset. Cursor-based pagination allows for immediate access to specific items within large datasets, unlike offset-based pagination, which can be inefficient for high offset values.

This pagination method provides real-time data, reflecting the current state of the dataset. Paginated requests don't create static snapshots of data. Instead, data is delivered dynamically. Due to the real-time nature of the data, idempotence is not guaranteed in paginated requests. This means that the same request with the same query parameters may return different results if the underlying data has changed between requests, ensuring you always receive the most up-to-date information.

For additional information on pagination in GraphQL, refer to the [GraphQL documentation](#) ➦ .

Key Concepts

- **Cursor:** A unique identifier that points to a specific item in a dataset.
- **Connection:** A structure that provides metadata about the paginated data, including edges, nodes, page info, and more.

How Pagination Works

i Note

The SAP LeanIX GraphQL API supports forward pagination but does not support backward pagination. Therefore, the following arguments for backward pagination are not supported: `last`, `before`, `hasPreviousPage`, and `startCursor`.

The following arguments for pagination are supported:

- `first`: Specifies the number of items to be returned in the query.
- `after`: A cursor that specifies the starting point for the next set of results. To retrieve the initial set of results, you can omit this argument.

When querying a paginated field, the API returns a connection object that includes:

- `pageInfo`: An object that provides metadata about the current page, including the `endCursor` and the `hasNextPage` boolean.
- `edges`: An array of objects that contain the node (the actual data item).

Retrieving the Initial Set of Results

The following query returns the first two fact sheets from the inventory list. The `pageInfo` object contains information about the paging state:

- `hasNextPage`: Defines whether more pages are available.
- `endCursor`: Shows the position of the current paging cursor.

Example query:

```
{
  allFactSheets(first: 2) {
    pageInfo {
```

```

    hasNextPage
    endCursor
  }
  edges {
    node {
      name
    }
  }
}
}

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "pageInfo": {
        "hasNextPage": true,
        "endCursor": "bWl40jEjRFhGMVpYSjVRVzVrUm1WMFkyZ0JBQUFBQUFISUJSb1dPRmhwT0dSWFLYVlNNRU55TlZG9"
      },
      "edges": [
        {
          "node": {
            "name": "AC Management"
          }
        },
        {
          "node": {
            "name": "AC Management Cloud"
          }
        }
      ]
    }
  }
}

```

The response contains the following:

- **pageInfo:** Provides metadata about the current page.
 - **hasNextPage:** Indicates whether there are more items available.
 - **endCursor:** The cursor of the last item in the current set. This cursor is used to fetch the next set of results.
- **edges:** An array of objects containing data nodes.

Retrieving the Next Set of Results

To fetch the next set of results, use the `endCursor` from the `pageInfo` as the `after` argument in your next query.

Example query:

```

{
  allFactSheets(first: 2, after: "bWl40jEjRFhGMVpYSjVRVzVrUm1WMFkyZ0JBQUFBQUFISUJSb1dPRmhwT0dSWFLYVlNNRU55TlZG9"
    pageInfo {
      hasNextPage
      endCursor
    }
  }
}

```

```

    edges {
      node {
        name
      }
    }
  }
}

```

The response provides the next set of fact sheets and the new `endCursor` for subsequent queries.

Example response:

```

{
  "data": {
    "allFactSheets": {
      "pageInfo": {
        "hasNextPage": true,
        "endCursor": "bWl40jMjRFhGMVpYSjVRVzVrUm1WMFkyZ0JBQUFBQUFISUJoc1dPRmhWT0dSWFlYVlNNRU55TlZG9"
      },
      "edges": [
        {
          "node": {
            "name": "AC Management to HR Admin"
          }
        },
        {
          "node": {
            "name": "Accenture"
          }
        }
      ]
    }
  }
}

```

Error Handling in GraphQL

Learn how error handling in GraphQL works and how to manage GraphQL errors.

Overview

GraphQL is a powerful query language for APIs that provides a flexible and efficient approach to handling data. However, error handling in GraphQL differs from error handling in traditional REST APIs, particularly regarding the use of HTTP status codes.

Unlike REST APIs, which use HTTP status codes to indicate the success or failure of a request, GraphQL always responds with a `200 OK` HTTP status code, even when an error occurs. This happens because GraphQL requests can result in partial success and partial failure at the same time. For example, when a GraphQL query is used to request multiple fields, some fields may return data successfully, while other fields may return an error.

In GraphQL, operational errors are treated as data, not as network errors. Therefore, the HTTP status code does not reflect the success or failure of the operation. Instead, the success or failure is communicated in the response body.

Error Handling in SAP LeanIX GraphQL

When an error occurs in SAP LeanIX GraphQL, the response includes an `errors` array in the JSON payload. Each error object in the array contains a `message` field that describes the error. It may also contain a `locations` field that provides additional information.

Example GraphQL error response:

```
{
  "data": null,
  "errors": [
    {
      "message": "Validation error of type FieldUndefined: Field nonExistentField is undefined",
      "locations": [
        {
          "line": 6,
          "column": 9
        }
      ]
    }
  ]
}
```

Error Handling for Batch Mutations

When multiple mutations are included in a request and at least one mutation is invalid, none of the mutations are executed. The `data` and `errors` attributes provide details about the failed mutation.

In the example request below, the mutation `m2` for updating a fact sheet includes an invalid field: `descriptio`.

Example request:

```
mutation {
  m1: updateFactSheet(
    id: "d9506c79-5d4e-4661-9297-b3b84bd17b79"
    patches: [{ op: replace, path: "/description", value: "first description" }]
  ) {
    factSheet {
      id
      description
    }
  }
  m2: updateFactSheet(
    id: "3b2f5b03-8cff-4aa1-a619-38aa7e90d456"
    patches: [
      { op: replace, path: "/descriptio", value: "invalid field description" }
    ]
  ) {
    factSheet {
      id
      description
    }
  }
  m3: updateFactSheet(
    id: "351ee5ff-c851-48c5-92e6-3d373c6483ce"
    patches: [
      { op: replace, path: "/description", value: "third description" }
    ]
  ) {
    factSheet {
      id
      description
    }
  }
}
```

```
}
}
```

In the response, the `data` attribute returns `null`. The `errors` attribute provides details about the failed mutation. An error message indicates that the entire request hasn't been executed and none of the mutations have been applied.

Example response:

```
{
  "data" : null,
  "errors" : [ {
    "message" : "The path '/descriptio' is invalid in FactSheet schema Application.",
    "path" : [ "m2" ],
    "extensions" : {
      "errorType" : "BUSINESS_LOGIC"
    }
  }, {
    "message" : "Error in Request. Transaction is rolled back!",
    "path" : [ ],
    "extensions" : {
      "errorType" : "BUSINESS_LOGIC"
    }
  } ]
}
```

Handling Errors in Your Application

When working with the SAP LeanIX GraphQL API, ensure to design your application to handle errors in the response body, not based on the HTTP status code. Here are some steps you can follow:

1. **Check for the `errors` field in the response:** After making a GraphQL request, always check if the `errors` field is present in the response. If it's present, it means one or more errors occurred.
2. **Parse the error message:** Each error object within the `errors` array includes a `message` field. This field usually contains a description that can help you understand the nature of the error that occurred. Please note that the clarity and comprehensibility of these messages can vary, and they may not always provide a complete picture of the issue. We recommend using error messages as a starting point for troubleshooting, keeping in mind that they don't provide a definitive diagnosis of the problem.
3. **View the error location:** The `locations` field in the GraphQL error response is designed to provide specific details about where in the query or mutation the error occurred. It contains an array of objects, each with the `line` and `column` properties. The `line` property indicates the line number in the query or mutation where the error was encountered. Similarly, the `column` property specifies the character position within that line where the error was detected. In the provided error response, the `locations` field indicates that the error was found on line 6 in column 9 of the query or mutation.

Remember that the absence of the `errors` field in the response indicates that the operation was successful.

Using GraphQL with Python

Learn how to run GraphQL queries and mutations using Python.

Overview

Using Python with [GraphQL](#) allows you to efficiently retrieve and manipulate data. You can develop scripts that take full advantage of GraphQL's ability to fetch complex, interrelated data with a single request.

In this tutorial, you will learn how to create and run GraphQL queries and mutations using Python scripts. To provide examples, we will create the following scripts:

- A script with a GraphQL query that retrieves Application Fact Sheets
- A script with a GraphQL mutation that adds a comment to a specific Fact Sheet

Prerequisites

Before you start, do the following:

- Install the latest LTS version of [Python](#) ➦ .
- Install [Poetry](#) ➦ , a tool for dependency management and packaging in Python.

i Note

You can create Python virtual environments using various methods. We recommend using [Poetry](#) ➦ . It simplifies dependency management, provides a consistent environment for development, and enhances the overall project organization.

- Install the necessary dependencies. In this tutorial, we use the [requests](#) ➦ Python library. You can install it by executing the following command:

```
poetry add requests
```

- Learn how to authenticate to SAP LeanIX APIs. For more information, see [Authentication to SAP LeanIX Services](#).

This tutorial assumes you have basic knowledge of:

- Python and Python virtual environments
- GraphQL

Step 1: Create a Python Project

To get started, create a new Python project.

```
poetry new leanix-graphql-tutorial
```

Poetry generates a basic Python project within the `leanix-graphql-tutorial` directory.

Change your working directory.

```
cd leanix-graphql-tutorial
```

Step 2: Create and Run a Python GraphQL Query Script

GraphQL queries allow you to retrieve data, similar to GET methods in REST APIs.

Create and run a Python script to retrieve all Application Fact Sheets for a Workspace using a GraphQL query. In the example, we only retrieve the `id` and `name` of Fact Sheets.

Follow these steps:

1. In your integrated development environment (IDE), create a Python file named `query.py` and paste the following code into it:

query.py

```
import os
import requests
import json

LEANIX_API_TOKEN = os.getenv('LEANIX_API_TOKEN')
LEANIX_SUBDOMAIN = os.getenv('LEANIX_SUBDOMAIN')
LEANIX_GRAPHQL_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/pathfinder/v1/graphql'
LEANIX_OAUTH2_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token'

def _obtain_access_token() -> str:
    """Obtains a LeanIX Access token using the Technical User generated
    API secret.

    Returns:
        str: The LeanIX Access Token
    """
    if not LEANIX_API_TOKEN:
        raise Exception('A valid token is required')
    response = requests.post(
        LEANIX_OAUTH2_URL,
        auth=("apitoken", LEANIX_API_TOKEN),
        data={"grant_type": "client_credentials"},
    )
    response.raise_for_status()
    return response.json().get('access_token')

def main():
    """Executes a query against the LeanIX GraphQL API and prints
    the output.
    """
    access_token = _obtain_access_token()
    graphql_query = """{
        allFactSheets(factSheetType: Application) {
            totalCount
            edges {
                node {
                    id
                    name
                }
            }
        }
    }"""

    data = {'query': graphql_query}
    auth_header = f'Bearer {access_token}'

    response = requests.post(
        url=LEANIX_GRAPHQL_URL,
        headers={'Authorization': auth_header},
        data=json.dumps(data)
    )
    response.raise_for_status()
    print(response.json())

if __name__ == '__main__':
    main()
```

i Note

To minimize the possibility of unintentionally exposing sensitive data, we use environment variables for the `LEANIX_API_TOKEN` and `LEANIX_SUBDOMAIN` parameters instead of embedding them directly within the script. This approach allows you to isolate sensitive information and safeguard it from unauthorized access.

2. Launch your terminal application and navigate to the project directory. Once there, execute the following command:

```
LEANIX_SUBDOMAIN="{{YOUR_LEANIX_SUBDOMAIN}}" LEANIX_API_TOKEN="{{YOUR_LEANIX_API_TOKEN}}" poe
```

You get a response in your console that contains a list of Application Fact Sheets for your Workspace.

Example response:

```
{
  "data": {
    "allFactSheets": {
      "totalCount": 1,
      "edges": [
        {
          "node": {
            "id": "13f60713-4b8e-40b3-b680-64262d0e0e64",
            "name": "AC Management"
          }
        }
      ]
    }
  }
}
```

Step 3: Create and Run a Python GraphQL Mutation Script

GraphQL mutations allow you to update data, similar to POST, PUT, PATCH, and DELETE methods in REST APIs.

Create and run a Python script to add a comment to a Fact Sheet using a GraphQL mutation. In the example, we add a comment to the Fact Sheet that we retrieved in the previous step.

Follow these steps:

1. Within the directory that contains the `query.py` file, create a Python file named `mutate.py` and paste the following code into it:

mutate.py

```
import os
import requests
import json

LEANIX_API_TOKEN = os.getenv('LEANIX_API_TOKEN')
LEANIX_SUBDOMAIN = os.getenv('LEANIX_SUBDOMAIN')
LEANIX_GRAPHQL_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/pathfinder/v1/graphql'
LEANIX_OAUTH2_URL = f'https://{LEANIX_SUBDOMAIN}.leanix.net/services/mtm/v1/oauth2/token'

def _obtain_access_token() -> str:
    """Obtains a LeanIX Access token using the Technical User generated
    API secret.

    Returns:
        str: The LeanIX Access Token
    """
    if not LEANIX_API_TOKEN:
        raise Exception('A valid token is required')
    response = requests.post(
        LEANIX_OAUTH2_URL,
```

```

        auth=("apitoken", LEANIX_API_TOKEN),
        data={"grant_type": "client_credentials"},
    )
    response.raise_for_status()
    return response.json().get('access_token')

def main():
    """Executes a mutation against the LeanIX GraphQL API and prints
    the output.
    """
    access_token = _obtain_access_token()
    factsheet_id = '13f60713-4b8e-40b3-b680-64262d0e0e64'
    factsheet_comment = 'What successor is planned for this?'
    graphql_mutation = """
        mutation {
            createComment(factSheetId: "%s", message: "%s", status: ACTIVE) {
                id
                message
            }
        }
    """ % (factsheet_id, factsheet_comment)

    data = {'query': graphql_mutation}
    auth_header = f'Bearer {access_token}'

    response = requests.post(
        url=LEANIX_GRAPHQL_URL,
        headers={'Authorization': auth_header},
        data=json.dumps(data)
    )
    response.raise_for_status()
    print(response.json())

if __name__ == '__main__':
    main()

```

2. Launch your terminal application and navigate to the project directory. Once there, execute the following command:

```
LEANIX_SUBDOMAIN="{{YOUR_LEANIX_SUBDOMAIN}}" LEANIX_API_TOKEN="{{YOUR_LEANIX_API_TOKEN}}" poe
```

You get a response in your console that contains the comment id and message.

Example response:

```

{
  "data": {
    "createComment": {
      "id": "4f45247c-fdc5-4650-a4c7-6af39e3d7d64",
      "message": "What successor is planned for this?"
    }
  }
}

```

Step 4: Validate the Mutation Script

To validate the mutation script, you can retrieve the comment that you added using the factSheet GraphQL query. To do this, we'll modify our existing query.py script.

Follow these steps:

1. In the query.py file, replace the main function with the following code:

query.py

```

def main():
    """Executes a query against the LeanIX GraphQL API and prints
    the output.
    """
    factsheet_id = '13f60713-4b8e-40b3-b680-64262d0e0e64'
    access_token = _obtain_access_token()
    graphql_query = """
    {
      factSheet(id: "%s") {
        comments {
          edges {
            node {
              id
              message
            }
          }
        }
      }
    }
    """ % (factsheet_id)

    data = {'query': graphql_query}
    auth_header = f'Bearer {access_token}'

    response = requests.post(
        url=LEANIX_GRAPHQL_URL,
        headers={'Authorization': auth_header},
        data=json.dumps(data)
    )
    response.raise_for_status()
    print(response.json())

```

2. Run the query.py script.

```
LEANIX_SUBDOMAIN="{{YOUR_LEANIX_SUBDOMAIN}}" LEANIX_API_TOKEN="{{YOUR_LEANIX_API_TOKEN}}" poe
```

You get a response in your console that contains the comment id and message, which confirms that the mutation script works as expected.

Example response:

```

{
  "data": {
    "factSheet": {
      "comments": {
        "edges": [
          {
            "node": {
              "id": "4f45247c-fdc5-4650-a4c7-6af39e3d7d64",
              "message": "What successor is planned for this?"
            }
          }
        ]
      }
    }
  }
}

```

To view comments in the application user interface, navigate to the **Comments** tab on a Fact Sheet.

Summary

In this tutorial, you learned how to create and run GraphQL queries and mutations using Python scripts.

Migrating from the Fact Sheets REST API to the GraphQL API

Mappings of REST API endpoints to GraphQL queries and mutations.

Overview

In this migration guide, you can find examples showing how to transition from the [Fact Sheets REST API](#)  to the GraphQL API. For each operation that you can perform on fact sheets through the API, we provide an example REST call and its equivalent GraphQL query or mutation.

For the REST implementation, the result of an operation may vary depending on the provided parameters. We illustrate requests for each specific parameter with examples, providing a clear path for a seamless migration.

In the following sections are examples for each documented endpoint within the [OpenAPI Explorer](#) .

Before You Start

- In this guide, we provide examples for the fundamental combinations of existing parameters.
- Example queries are basic. They do not contain separate GraphQL variables or pagination parameters. To learn how to work with GraphQL queries and mutations, see [GraphQL Basics](#).
- All examples are based on the fact sheet type `Application`. If you want to use a different fact sheet type, change the corresponding field.
- All queries include placeholders for variables, such as `{id}` or `{toId}`. Before you make API requests, replace them with actual values.
- You can use the [GraphiQL tool](#) in your workspace to try out the provided examples. To open the tool, on the help menu, select **Developer Tools > GraphQL Editor**.
- The examples usually contain simple queries that do not include all possible parameters. If needed, you can include additional parameters in the query.
- Following GraphQL best practices, we recommend that you query minimal data to reduce the payload size and response times. For other recommendations, see [Best Practices](#).

Get All Fact Sheets

Method	Path	Reference
GET	/factSheets	Schema 

In this section, we provide an example of retrieving all fact sheets of the `Application` type and its relations to fact sheets of the `ITComponent` type.

REST call:

`https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets?relationTypes=relApplicationToITC`

GraphQL query:

```

query {
  allFactSheets(filter: {responseOptions: {maxFacetDepth: 5}, facetFilters: [
    {facetKey: "FactSheetTypes",
      keys:["Application"],
      operator: OR
    }
  ]}, sort: [{key: "displayName", order: asc}]) {
    totalCount
    edges {
      node {
        id
        displayName
        description
        rev
        type
        permissions {
          create
          read
          update
          delete
          self
        }
        qualitySeal
        lxState
        updatedAt
        completion {
          percentage
        }
        tags {
          id
          name
          description
          color
          tagGroup {
            id
            name
            shortName
            mode
          }
        }
      }
    }
    subscriptions(filter: {userId: "{userId}"}) {
      edges {
        node {
          id
          user {
            id
            firstName
            lastName
            email
          }
          type
          roles {
            id
            comment
          }
        }
      }
    }
    status
    level
    category
    ... on Application {
      relApplicationToITComponent {
        edges {
          node {
            factSheet {
              id
              name
            }
          }
        }
      }
    }
  }
}

```

```
}
  }
}
}
```

In the example request, we retrieve all fact sheets of the `Application` type, including their default fields, `completion`, `tags`, `subscriptions`, and more. For related fact sheets of the `ITComponent` type, we only fetch their `id` and `name`.

Get a Fact Sheet Hierarchy by Root ID

Method	Path	Reference
GET	/factSheets/hierarchy/{rootId}	Schema ➡

You can retrieve all fact sheets equal to or below a specific root fact sheet.

REST call:

```
https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets/hierarchy/{rootId}
```

GraphQL query:

```
query {
  allFactSheets(hierarchy: {mode: FULL_TREE} filter: {ids: ["{rootId}"]}) {
    edges {
      node {
        id
        displayName
      }
    }
  }
}
```

Get a Fact Sheet by ID

Method	Path	Reference
GET	/factSheets/{id}	Schema ➡

REST call:

```
https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets/{id}
```

GraphQL query:

```
query {
  factSheet(id: "{id}") {
    id
    displayName
    description
    rev
    type
    permissions {
      create
      read
      update
    }
  }
}
```

```

    delete
    self
  }
  qualitySeal
  lxState
  updatedAt
  completion {
    percentage
  }
  tags {
    id
    name
    description
    color
    tagGroup {
      id
      name
      shortName
      mode
    }
  }
}
comments(filter: {status: ACTIVE}) {
  totalCount
  edges {
    node {
      message
    }
  }
}
subscriptions(filter: {userId: "{userId}"}) {
  edges {
    node {
      id
      user {
        id
        firstName
        lastName
        email
      }
      type
      roles {
        id
        comment
      }
    }
  }
}
}
status
level
category
}
}

```

To limit the results to only the `id` and `name` fields of a fact sheet, use the following query:

```

query {
  factSheet(id: "{id}") {
    id
    name
  }
}

```

Get All Relations of a Fact Sheet

Method	Path	Reference
GET	/factSheets/{id}/relations	Schema ➡

In this section, we retrieve all relations of a fact sheet with the specified {id}.

REST call:

`https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets/{id}/relations`

GraphQL query:

```
query {
  factSheet(id: "{id}") {
    ... on Application {
      relToChild {
        edges {
          node {
            factSheet {
              id
              name
            }
          }
        }
      }
      relApplicationToITComponent {
        edges {
          node {
            factSheet {
              id
              name
            }
          }
        }
      }
      relToSuccessor {
        edges {
          node {
            factSheet {
              id
              name
            }
          }
        }
      }
    }
  }
}
```

In the example request, we retrieve the following relations of a fact sheet: `relToChild`, `relApplicationToITComponent`, and `relToSuccessor`. For the target fact sheet, we only fetch the `id` and `name` fields.

While the REST endpoint provides all possible fields and data in a single call, GraphQL requires you to add all the required relations and fields of the target fact sheet to your query.

Create a Fact Sheet

Method	Path	Reference
POST	/factSheets	Schema ➡

REST call:

`https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets`

Example request body:

```
{
  "name": "TestFactSheet",
  "description": "This is a demo fact sheet.",
  "type": "Application"
}
```

GraphQL mutation:

```
mutation {
  createFactSheet(input: {
    name: "TestFactSheet"
    type: Application
  }) {
    patches: [
      {
        op: add
        path: "/description"
        value: "This is a demo fact sheet."
      }
    ]
  }
}
```

To update the `description` field, use a Patch operation.

Update a Fact Sheet

Method	Path	Reference
PUT	/factSheets/{id}	Schema ➡

In this section, we provide an example of updating the description of a fact sheet.

REST call:

```
https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets/{id}
```

Example request body:

```
{
  "name": "TestFactSheet",
  "description": "This is the updated demo fact sheet description.",
  "displayName": "TestFactSheet",
  "fullName": "TestFactSheet"
}
```

GraphQL mutation:

```
mutation {
  updateFactSheet(id: "{id}"
  patches: [
```

```
{
  op: replace
  path: "/description"
  value: "This is the updated description of a demo fact sheet."
}
})) {
  factSheet {
    id
    name
    type
    description
  }
}
}
```

Unlike the REST endpoint, the GraphQL mutation allows you to update data in a more granular way. You don't need to pass fields that you don't want to modify. In this example, we only update the `description`.

Archive a Fact Sheet

Method	Path	Reference
DELETE	/factSheets/{id}	Schema ➡

REST call:

https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets/{id}

Example request body:

```
{
  "comment": "This is a test archive."
}
```

GraphQL mutation:

```
mutation {
  updateFactSheet(id: "{id}",
    comment: "This is a test archive.",
    patches: [{op: add, path: "/status", value: "ARCHIVED"}]) {
    factSheet {
      id
      name
      type
      description
      status
    }
  }
}
```

Create a Fact Sheet Relation

Method	Path	Reference
POST	/factSheets/{id}/relations	Schema ➡

REST call:

https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets/{id}/relations

Example request body:

```
{
  "displayNameToFS": "TestRelation",
  "typeFromFS": "relToPredecessor",
  "typeToFS": "relToSuccessor",
  "activeFrom": "2023-11-29",
  "activeUntil": "2024-11-29",
  "constrainingRelations": [],
  "idsOfConstrainingRelations": [],
  "status": "ACTIVE",
  "fromId": "{id}",
  "toId": "{targetId}"
}
```

GraphQL mutation:

```
mutation {
  updateFactSheet(id: "{id}", patches: [
    {
      op: add,
      path: "/relToSuccessor/new_{id}",
      value: "{\"factSheetId\":\"{targetId}\",\"activeFrom\":\"2023-11-29\",\"activeUntil\":\"2024-11-29\"}"
    }
  ]) {
    factSheet {
      id
      name
      type
    }
  }
}
```

Update a Fact Sheet Relation

Method	Path	Reference
PUT	/factSheets/{id}/relations/{relationId}	Schema ➦

REST call:

https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets/{id}/relations/{relationId}

Example request body:

```
{
  "id": "{relationId}",
  "displayNameToFS": "TestApp",
  "typeFromFS": "Application",
  "typeToFS": "Application",
  "permittedReadACL": [],
  "activeFrom": "2023-11-29",
  "activeUntil": "2025-11-29",
}
```

```
"constrainingRelations": [],
"fields": [],
"status": "ACTIVE",
"fromId": "{id}",
"toId": "{targetFactSheetId}",
"naFields": [],
"type": "relToSuccessor"
}
```

GraphQL mutation:

```
mutation updateRelation {
  updateFactSheet(id: "{id}", patches: [
    {
      op: replace,
      path: "/relToSuccessor/{relationId}",
      value: "{\"description\":\"This is a test description.\",\"factSheetId\":\"{targetFactSheetId}\"}
    }
  ]) {
    factSheet {
      id
      name
      type
      ... on Application {
        relToSuccessor {
          edges {
            node {
              id
              activeFrom
              activeUntil
              description
              factSheet {
                fullName
              }
            }
          }
        }
      }
    }
  }
}
```

In the example request, we update a relation for a fact sheet of the Application type.

Delete a Fact Sheet Relation

Method	Path	Reference
DELETE	/factSheets/{id}/relations/{relationId}	Schema ➦

REST call:

```
https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/factSheets/{id}/relations/{relationId}
```

GraphQL mutation:

```
mutation {
  updateFactSheet(id: "{id}", patches: [
    {
      op: remove,
      path: "/relToSuccessor/{relationId}",
    }
  ])
}
```

```

    value: ""
  }
  ] ) {
    factSheet {
      id
      name
      type
      ... on Application {
        relToSuccessor {
          edges {
            node {
              id
              factSheet {
                fullName
              }
            }
          }
        }
      }
    }
  }
}

```

In the example request, we delete a relation for a fact sheet of the `Application` type.

Sample GraphQL Queries

Get started with the GraphQL API using example queries and mutations.

In this section, you can find sample queries and mutations for the SAP LeanIX GraphQL API. You can try them out in the [GraphiQL Tool](#) in your workspace.

To learn more about GraphQL queries and mutations, see [GraphQL Basics](#).

Retrieving Fact Sheets (GraphQL)

Example queries for retrieving fact sheets through the GraphQL API.

In the example queries, we only retrieve basic fact sheet attributes, such as `id`, `name`, or `type`. Modify the queries to retrieve the data you need.

i Note

Within the GraphiQL interface in your workspace, there is a response limit of 18,006 nodes. For workspaces with large data volumes, this node limit may result in not all fact sheets being included in the response. We recommend implementing pagination and limiting your retrieval to no more than 1,000 fact sheets per page. To learn more about pagination, see [Pagination in GraphQL](#).

Retrieving a Fact Sheet by ID

To retrieve a fact sheet by ID, use the `factSheet` query.

The `id` argument is required for this query. To find the ID of a fact sheet, navigate to the fact sheet page and copy the ID from the URL.

Example query:

```
{
  factSheet(id: "28fe4aa2-6e46-41a1-a131-72afb3acf256") {
    id
    name
    type
  }
}
```

Example response:

```
{
  "data": {
    "factSheet": {
      "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
      "name": "AC Management",
      "type": "Application"
    }
  }
}
```

Retrieving All Fact Sheets

To retrieve all fact sheets available in the inventory, use the `allFactSheets` query.

→ Tip

You can apply filters to your queries to narrow down results. For example queries with filters, see [Filtering Fact Sheets](#).

Example query:

```
{
  allFactSheets {
    totalCount
    edges {
      node {
        id
        name
        type
      }
    }
  }
}
```

Example response:

```
{
  "data": {
    "allFactSheets": {
      "totalCount": 132,
      "edges": [
        {
          "node": {
            "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
            "name": "AC Management",
            "type": "Application"
          }
        }
      ]
    }
  }
}
```

```

    },
    {
      "node": {
        "id": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
        "name": "AC Management Cloud",
        "type": "Application"
      }
    },
    {
      "node": {
        "id": "688f3195-3634-418a-8ad9-a3555bb358a9",
        "name": "AC Management to HR Admin",
        "type": "Interface"
      }
    },
    ...
  ]
}
}
}

```

Retrieving Custom Fact Sheet Attributes

You can retrieve data specific to a certain fact sheet type using inline fragments. To understand how inline fragments work in GraphQL, refer to [Inline Fragments](#) in the GraphQL documentation.

Fact sheets come with standard attributes defined in the meta model configuration. You can also add custom attributes to a fact sheet to tailor the configuration to your needs. For more details, see [Adding Custom Attributes](#).

The following example query returns all fact sheets with these attributes:

- Standard fields: `id`, `name`, and `type`
- Custom fields specific to IT component fact sheets: `capEx` and `opEx`

We use an inline fragment `... on ITComponent` to request custom fields for IT component fact sheets. These fields aren't returned for other types of fact sheets.

Example query:

```

{
  allFactSheets {
    totalCount
    edges {
      node {
        id
        name
        type
        ... on ITComponent {
          capEx
          opEx
        }
      }
    }
  }
}

```

Example response:


```

{
  "data": {
    "allFactSheets": {
      "totalCount": 120,
      "edges": [
        {
          "node": {
            "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
            "name": "AC Management",
            "type": "Application"
          }
        },
        {
          "node": {
            "id": "9deb9733-5701-42f1-8c52-c165acaa6487",
            "name": "App Maintenance & Support Service",
            "type": "ITComponent",
            "capEx": 2000,
            "opEx": 1500
          }
        },
        ...
      ]
    }
  }
}

```

Retrieving Related Fact Sheets

You can retrieve related fact sheets for a specific fact sheet using an inline fragment.

In the example, we retrieve the following data for a specific fact sheet of the IT component type:

- All related fact sheets of the tech category type, referred to as `TechnicalStack` in the GraphQL API
- All child fact sheets and their related fact sheets of the tech category type

Example query:

```

{
  factSheet(id: "afd31c81-3fff-4367-9d90-458eecb3efa7") {
    name
    type
    ... on ITComponent {
      relITComponentToTechnologyStack {
        edges {
          node {
            id
            factSheet {
              name
            }
          }
        }
      }
    }
    relToChild {
      edges {
        node {
          id
          factSheet {

```


Example queries for filtering fact sheets through the GraphQL API.

Filtering Fact Sheets by Type

Example query:

```
query retrieveFactSheetsByType($filter: FilterInput!) {
  allFactSheets(filter: $filter) {
    totalCount
    edges {
      node {
        id
        displayName
        type
      }
    }
  }
}
```

```
{
  "filter": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "Application",
          "ITComponent",
          "BusinessCapability"
        ]
      }
    ]
  }
}
```

```

    ]
  }
}

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "totalCount": 291,
      "edges": [
        {
          "node": {
            "id": "3c86fe91-df46-4fde-8191-6c67253ca91a",
            "displayName": "Corporate Services",
            "type": "BusinessCapability"
          }
        },
        {
          "node": {
            "id": "b5e0fc18-bef0-414d-a38a-3409a2f8c01f",
            "displayName": "Corporate Services / Fleet Management",
            "type": "BusinessCapability"
          }
        },
        ...
        {
          "node": {
            "id": "58f554bc-b8f2-487e-b598-df27b4a8e870",
            "displayName": "AC Management",
            "type": "Application"
          }
        },
        {
          "node": {
            "id": "91a18208-aade-4a64-ae58-11a6a307a26e",
            "displayName": "AC Management Cloud",
            "type": "Application"
          }
        },
        ...
        {
          "node": {
            "id": "645a6128-6f47-45da-bd88-51ed548c6c44",
            "displayName": "Microsoft .NET Framework 1.0 Service Pack 3",
            "type": "ITComponent"
          }
        },
        {
          "node": {
            "id": "5ca2c6e0-7414-4887-9dee-5d0f388cf89f",
            "displayName": "Microsoft .NET Framework 1.1",
            "type": "ITComponent"
          }
        }
      ]
    }
  }
}

```

```

    },
    ...
  ]
}
}
}

```

For a single fact sheet type, you can pass the `factSheetType` argument in the `allFactSheets` query.

Example query:

```

{
  allFactSheets(factSheetType: Application) {
    totalCount
    edges {
      node {
        id
        name
        type
      }
    }
  }
}

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "totalCount": 50,
      "edges": [
        {
          "node": {
            "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
            "name": "AC Management",
            "type": "Application"
          }
        },
        {
          "node": {
            "id": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
            "name": "AC Management Cloud",
            "type": "Application"
          }
        },
        ...
      ]
    }
  }
}

```

Filtering Fact Sheets by Type and Technical Fit

To filter fact sheets by type and technical fit, use the following facet keys:

- FactSheetTypes: Filter for fact sheet types.
- technicalSuitability: Filter for the technical fit attribute.

In the example, we retrieve Application fact sheets with the unreasonable technical fit.

Example query:

```
query TechnicalFit($filter: FilterInput!) {
  allFactSheets(filter: $filter) {
    edges {
      node {
        ... on Application {
          id
          displayName
          technicalSuitability
        }
      }
    }
  }
}
```

Variables:

```
{
  "filter": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "Application"
        ]
      },
      {
        "facetKey": "technicalSuitability",
        "operator": "OR",
        "keys": [
          "unreasonable"
        ]
      }
    ]
  }
}
```

Example response:

```
{
  "data": {
    "allFactSheets": {
      "edges": [
        {
          "node": {
            "id": "01740698-1ffa-4729-94fa-da6194ebd7cd",
            "displayName": "AC Management",
            "technicalSuitability": "unreasonable"
          }
        }
      ]
    }
  }
}
```

```

    }
  },
  {
    "node": {
      "id": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
      "displayName": "AC Management Cloud",
      "technicalSuitability": "unreasonable"
    }
  }
]
}
}
}
}

```

Filtering Fact Sheets by Type and Lifecycle Phase

To filter fact sheets by type and lifecycle phase, use the following facet keys:

- **FactSheetTypes**: Filter for fact sheet types.
- **lifecycle**: Filter for the lifecycle phase attribute. Adding **dateFilter** is required for this facet.

In the example, we retrieve **Application** fact sheets in the **phaseIn** phase with start dates within the specified time frame (between 2023 and 2029).

Example query:

```

query filterLifeCycle($filter: FilterInput!) {
  allFactSheets(filter: $filter) {
    edges {
      node {
        ... on Application {
          id
          name
          ApplicationLifecycle: lifecycle {
            asString
            phases {
              startDate
            }
          }
        }
      }
    }
  }
}

```

Variables:

```

{
  "filter": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "Application"
        ]
      }
    ],
  },
}

```

```

{
  "facetKey": "lifecycle",
  "operator": "OR",
  "keys": [
    "phaseIn"
  ],
  "dateFilter": {
    "type": "RANGE",
    "from": "2023-01-01",
    "to": "2029-12-31"
  }
}
]
}
}

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "edges": [
        {
          "node": {
            "id": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
            "name": "AC Management Cloud",
            "ApplicationLifecycle": {
              "asString": "phaseIn",
              "phases": [
                {
                  "startDate": "2023-11-01"
                }
              ]
            }
          }
        }
      ]
    }
  }
}

```

Filtering Fact Sheets by Type and Tags

To filter fact sheets by type and assigned tags, use the following facet keys:

- **FactSheetTypes:** Filter for fact sheet types.
- **_TAGS_:** Filter for tags assigned to fact sheets. You can use this facet key to filter tags from any tag group, including those that don't belong to any group. In the keys array, pass the tag IDs, not names. To retrieve tag IDs, use the `allTags` query.

To apply a filter for tags from a specific tag group, pass the tag group ID in the facet key instead of `_TAGS_`. To retrieve tag group IDs, use the `allTagGroups` query.

In the example, we filter Platform fact sheets with both Production and Beta tags assigned.

To sort the results by the most recently updated fact sheets, use the `sortings` argument with the `updatedAt` key and `desc` value.

Example query:

```
query platformTag($filter: FilterInput!, $sortings: [Sorting]) {
  allFactSheets(filter: $filter, sort: $sortings) {
    totalCount
    edges {
      node {
        ... on Platform {
          id
          displayName
          type
          updatedAt
          tags {
            id
            name
          }
        }
      }
    }
  }
}
```

Variables:

```
{
  "filter": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "Platform"
        ]
      },
      {
        "facetKey": "_TAGS_",
        "operator": "AND",
        "keys": [
          "6c8d1073-0724-4582-97bd-c972e85be0cb",
          "1c9c71b0-db60-453e-b607-05471c4f839a"
        ]
      }
    ]
  },
  "sortings": [
    {
      "key": "updatedAt",
      "order": "desc"
    }
  ]
}
```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "totalCount": 2,
      "edges": [
        {
          "node": {
            "id": "dc78b30e-8b16-4a60-a1b0-837a1663d6d8",
            "displayName": "Design Platform",
            "type": "Platform",
            "updatedAt": "2024-03-25T14:42:47.803291477Z",
            "tags": [
              {
                "id": "6c8d1073-0724-4582-97bd-c972e85be0cb",
                "name": "Beta"
              },
              {
                "id": "1c9c71b0-db60-453e-b607-05471c4f839a",
                "name": "Production"
              }
            ]
          }
        },
        {
          "node": {
            "id": "07fac0d6-76bc-4e46-88c0-f4a1596172cc",
            "displayName": "BI Platform",
            "type": "Platform",
            "updatedAt": "2024-03-25T14:40:32.422964977Z",
            "tags": [
              {
                "id": "1c9c71b0-db60-453e-b607-05471c4f839a",
                "name": "Production"
              },
              {
                "id": "6c8d1073-0724-4582-97bd-c972e85be0cb",
                "name": "Beta"
              }
            ]
          }
        }
      ]
    }
  }
}

```

Filtering Fact Sheets by External ID

You can filter fact sheets with a specific external ID. In the example, we filter fact sheets where the `externalId` is set to 1234.

Example query:

```

{
  allFactSheets(filter: {externalIds: ["externalId/1234"]}) {
    totalCount
    edges {
      node {
        id
        name
        ... on Application {
          externalId {
            externalId
          }
        }
      }
    }
  }
}

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "totalCount": 1,
      "edges": [
        {
          "node": {
            "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
            "name": "AC Management",
            "externalId": {
              "externalId": "1234"
            }
          }
        }
      ]
    }
  }
}

```

Filtering Archived Fact Sheets

To filter archived fact sheets, use the `TrashBin` facet key.

Example query:

```

query allFactSheetsQuery($filter: FilterInput!) {
  allFactSheets(filter: $filter) {
    edges {
      node {
        id
        name
        type
        status
      }
    }
  }
}

```

Variables:

```
{
  "filter": {
    "responseOptions": {
      "maxFacetDepth": 5
    },
    "facetFilters": [
      {
        "facetKey": "TrashBin",
        "operator": "OR",
        "keys": [
          "archived"
        ]
      }
    ]
  }
}
```

Example response:

```
{
  "data": {
    "allFactSheets": {
      "edges": [
        {
          "node": {
            "id": "65e5f779-b7f0-4a1f-9c88-9f0e3ae9bed9",
            "name": "Application Hosting",
            "type": "ITComponent",
            "status": "ARCHIVED"
          }
        }
      ]
    }
  }
}
```

Filtering Fact Sheets by Fields on Relations

In the following example, we filter application fact sheets that support specific business capabilities. The support type is `leading`.

Use the `relApplicationToBusinessCapability` facet key and specify the IDs of business capability fact sheets in the `keys` array. To return only the relations that match the applied relation field filters, set the `relationFieldsFilterOperator` parameter to `INCLUSIVE`.

Example query:

```
query filterFactSheets($filter: FilterInput!) {
  allFactSheets(filter: $filter) {
    totalCount
    edges {
      node {
        id
        displayName
        type
      }
    }
  }
}
```

```

... on Application {
  relApplicationToBusinessCapability {
    edges {
      node {
        type
        supportType
        factSheet {
          id
          name
        }
      }
    }
  }
}

```

Variables:

```

{
  "filter": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "Application"
        ]
      },
      {
        "facetKey": "relApplicationToBusinessCapability",
        "operator": "OR",
        "keys": [
          "5634544d-4cd3-4533-b190-3f947a99e752",
          "0c53c875-feab-4a87-9006-c0b3191aa65f"
        ]
      },
      {
        "relationFieldsFilterOperator": "INCLUSIVE",
        "relationFieldsFilter": [
          {
            "fieldName": "supportType",
            "values": [
              "leading"
            ]
          }
        ]
      }
    ]
  }
}

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "totalCount": 18,

```

```

    "edges": [
      {
        "node": {
          "id": "01740698-1ffa-4729-94fa-da6194ebd7cd",
          "displayName": "AC Management",
          "type": "Application",
          "relApplicationToBusinessCapability": {
            "edges": [
              {
                "node": {
                  "type": "RelApplicationToBusinessCapability",
                  "supportType": "leading",
                  "factSheet": {
                    "id": "5634544d-4cd3-4533-b190-3f947a99e752",
                    "name": "Customer Relationship"
                  }
                }
              }
            ]
          }
        }
      }
    ]
  }
}

```

Filtering Fact Sheets by Subscriptions

To filter fact sheets by subscriptions, use the `Subscriptions` facet key with the `allFactSheets` query. To learn more about subscriptions, refer to:

- [Fact Sheet Subscription](#)
- [Subscription Roles](#)

In the following example, we filter fact sheets by user ID, subscription type, and subscription role. We aim to retrieve fact sheets where a specific user is subscribed as `ACCOUNTABLE` (subscription type) and `Solution Architect` (custom subscription role).

Subscription types are predefined, with possible values of `ACCOUNTABLE`, `RESPONSIBLE`, or `OBSERVER`. Workspace admins define subscription roles, which are associated with unique IDs. To get the IDs of subscription roles, use the `allSubscriptionRoles` query.

Example query:

```

query getSubscriptionRoles {
  allSubscriptionRoles {
    edges {
      node {
        id
        name
      }
    }
  }
}

```

```
}
}
```

Example response:

```
{
  "data": {
    "allSubscriptionRoles": {
      "edges": [
        {
          "node": {
            "id": "8d1aab90-4d86-43e5-9a75-bafccee81732",
            "name": "Data Architect"
          }
        },
        {
          "node": {
            "id": "a523b819-208c-49ef-a0c6-8d8b84464f6e",
            "name": "Solution Architect"
          }
        }
      ]
    }
  }
}
```

To filter fact sheets that a specific user is subscribed to, pass the user `id` in the `keys` array. You can provide multiple user IDs.

- To get user IDs, retrieve all workspace users by making a GET request to the following endpoint on the MTM REST API. For the endpoint schema, visit the [API documentation](#) ➡.

`https://{SUBDOMAIN}.leanix.net/services/mtm/v1/workspaces/{id}/users`

- To find your user ID, go to the [API Tokens](#) section in the admin area and look for the `UserId` value.

In the example, we use the `FactSheetTypes` facet key to return fact sheets of the application type.

Example query:

```
query retrieveFactSheetsBySubscriber($filter: FilterInput!) {
  allFactSheets(filter: $filter) {
    totalCount
    edges {
      node {
        id
        displayName
        type
      }
    }
  }
}
```

Variables:

```
{
  "filter": {
```

```

"facetFilters": [
  {
    "facetKey": "FactSheetTypes",
    "operator": "OR",
    "keys": [
      "Application"
    ]
  },
  {
    "facetKey": "Subscriptions",
    "operator": "OR",
    "keys": [
      "87678c21-98a2-9567-aaaa-fc66ff2b9d56"
    ],
    "subscriptionFilter": {
      "type": "ACCOUNTABLE",
      "roleId": "a523b819-208c-49ef-a0c6-8d8b84464f6e"
    }
  }
]
}
}

```

Example response:

```

{
  "data": {
    "allFactSheets": {
      "totalCount": 2,
      "edges": [
        {
          "node": {
            "id": "01740698-1ffa-4729-94fa-da6194ebd7cd",
            "displayName": "AC Management",
            "type": "Application"
          }
        },
        {
          "node": {
            "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
            "displayName": "AC Management Cloud",
            "type": "Application"
          }
        }
      ]
    }
  }
}

```

Creating a Fact Sheet (GraphQL)

Create a fact sheet through the GraphQL API.

To create a fact sheet, use the `createFactSheet` mutation. The fields `name` and `type` of the `BaseFactSheetInput` input argument are required for this mutation.

In the example, we create a fact sheet of the `Application` type and add the following attributes to it through the `add patch` operation:

- `description`: The description of a fact sheet.
- `externalId`: The ID of a fact sheet in an external system.
- `alias`: The alternative name for a fact sheet that is also used in full-text search.

Example mutation:

```
mutation ($input: BaseFactSheetInput!, $patches: [Patch]!) {
  createFactSheet(input: $input, patches: $patches) {
    factSheet {
      id
      name
      description
      type
      ... on Application {
        externalId {
          externalId
        }
        alias
      }
    }
  }
}
```

Variables:

```
{
  "input": {
    "name": "AC Management",
    "type": "Application"
  },
  "patches": [
    {
      "op": "add",
      "path": "/description",
      "value": "Application for AC management"
    },
    {
      "op": "add",
      "path": "/externalId",
      "value": "{\"type\":\"ExternalId\",\"externalId\":\"1234567890\"}"
    },
    {
      "op": "add",
      "path": "/alias",
      "value": "AC App"
    }
  ]
}
```

Example response:

```
{
  "data": {
    "createFactSheet": {
      "factSheet": {
        "id": "1c680bb5-2323-4aca-afce-52dab5d7de57",
        "name": "AC Management",
        "description": "Application for AC management",
        "type": "Application",
        "externalId": {
          "externalId": "1234567890"
        },
        "alias": "AC App"
      }
    }
  }
}
```

i Note

You can create multiple fact sheets in batch. For more information, see [Including Multiple Mutations in a Single Request Using Aliases](#).

Updating a Fact Sheet (GraphQL)

Update a fact sheet through the GraphQL API.

To add, change, or delete fact sheet attributes, use the `updateFactSheet` mutation. Provide the attribute details using patch operations.

Some patch operations are not supported for specific attributes. For example, you can only use the `replace` operation for the `name` attribute.

`id` is a required argument for the `updateFactSheet` mutation. To learn how to get the ID of a fact sheet, see [Retrieving Fact Sheets](#).

Updating the Name and Description on a Fact Sheet

In the following example, we change the fact sheet name and description using the `replace` patch operation.

Example mutation:

```
mutation ($patches: [Patch]!) {
  updateFactSheet(id: "28fe4aa2-6e46-41a1-a131-72afb3acf256", patches: $patches) {
    factSheet {
      id
      name
      description
    }
  }
}
```

Variables:

```
{
  "patches": [
    {
      "op": "replace",
      "path": "/name",
      "value": "AC Management Cloud"
    },
    {
      "op": "replace",
      "path": "/description",
      "value": "Application for AC management"
    }
  ]
}
```

Example response:

```
{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
        "name": "AC Management Cloud",
        "description": "Application for AC management"
      }
    }
  }
}
```

Updating the Quality Seal on a Fact Sheet

In the following example, we set the quality seal on a fact sheet to APPROVED using the replace patch operation. For more information, see [Quality Seal](#).

Example mutation:

```
mutation ($patches: [Patch]!) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", patches: $patches) {
    factSheet {
      id
      name
      lxState
    }
  }
}
```

Variables:

```
{
  "patches": [
    {
      "op": "replace",
      "path": "/lxState",
      "value": "APPROVED"
    }
  ]
}
```

```

    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
        "name": "AC Management Cloud",
        "lxState": "APPROVED"
      }
    }
  }
}

```

Updating the External ID on a Fact Sheet

In the following example, we update the external ID on a fact sheet using the `replace` patch operation.

Example mutation:

```

mutation ($patches: [Patch]!) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", patches: $patches) {
    factSheet {
      id
      name
      ... on Application {
        externalId {
          externalId
        }
      }
    }
  }
}

```

Variables:

```

{
  "patches": [
    {
      "op": "replace",
      "path": "/externalId",
      "value": "{\"type\":\"ExternalId\",\"externalId\":\"123456789\"}"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {

```

```

    "factSheet": {
      "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
      "name": "AC Management Cloud",
      "externalId": {
        "externalId": "123456789"
      }
    }
  }
}
}
}

```

Deleting the External ID from a Fact Sheet

In the following example, we delete the external ID from a fact sheet using the `remove` patch operation.

Example mutation:

```

mutation ($patches: [Patch]!) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", patches: $patches) {
    factSheet {
      id
      name
      ... on Application {
        externalId {
          externalId
        }
      }
    }
  }
}

```

Variables:

```

{
  "patches": [
    {
      "op": "remove",
      "path": "/externalId",
      "value": ""
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
        "name": "AC Management Cloud",
        "externalId": null
      }
    }
  }
}

```

```

    }
  }
}

```

Updating the Lifecycle on a Fact Sheet

In the following example, we update the lifecycle on a fact sheet by changing the start dates of specific phases through the `replace` patch operation.

Ensure to include all phases and their corresponding values in the input, even if you only want to update a single phase. Any phases that are not included in the input are deleted. For example, if you only include the `plan` phase in the input, the mutation will update this phase and delete all others.

The `remove` patch operation deletes all phases, regardless of the specific phases included in the input.

Example mutation:

```

mutation ($patches: [Patch]!) {
  updateFactSheet(id: "85be63bf-6347-4f20-a306-2d06a10dc6f3", patches: $patches) {
    factSheet {
      ... on Initiative {
        lifecycle {
          phases {
            phase
            startDate
          }
        }
      }
    }
  }
}

```

Variables:

```

{
  "patches": [
    {
      "op": "replace",
      "path": "/lifecycle",
      "value": "{\"phases\": [{\"phase\": \"plan\", \"startDate\": \"2019-01-12\"}, {\"phase\": \"phase\"}]}"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "lifecycle": {
          "phases": [
            {
              "phase": "plan",
              "startDate": "2019-01-12",
              "milestoneId": null
            },

```

```

    {
      "phase": "phaseIn",
      "startDate": "2020-03-12",
      "milestoneId": null
    },
    {
      "phase": "active",
      "startDate": "2021-05-10",
      "milestoneId": null
    },
    {
      "phase": "phaseOut",
      "startDate": "2025-03-30",
      "milestoneId": null
    },
    {
      "phase": "endOfLife",
      "startDate": "2027-01-01",
      "milestoneId": null
    }
  ]
}
}
}
}
}

```

Archiving a Fact Sheet

You may need to archive fact sheets to remove irrelevant or duplicate data from the inventory. Archiving does not immediately delete a fact sheet but removes it from the default Inventory list. You can recover an archived fact sheet within the retention period, after which it is permanently deleted. For more information, visit [Archiving, Deleting and Recovering Fact Sheets](#).

You can archive or recover a fact sheet by updating the `status` attribute.

To archive a fact sheet, set the `status` attribute to `ARCHIVED` and specify a reason for archiving in `comment`.

Example mutation:

```

mutation ($patches: [Patch]!) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", comment: "Irrelevant application", patches: $patches) {
    factSheet {
      id
      status
    }
  }
}

```

Variables:

```

{
  "patches": [
    {
      "op": "add",
      "path": "/status",

```

```

      "value": "ARCHIVED"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
        "status": "ARCHIVED"
      }
    }
  }
}

```

Recovering an Archived Fact Sheet

To recover an archived fact sheet, set the `status` attribute to `ACTIVE` and specify a reason for restoring in `comment`. You can recover an archived fact sheet within the retention period, after which it is permanently deleted.

Example mutation:

```

mutation ($patches: [Patch]!) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", comment: "Recover the application from archive") {
    factSheet {
      id
      status
    }
  }
}

```

Variables:

```

{
  "patches": [
    {
      "op": "add",
      "path": "/status",
      "value": "ACTIVE"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",

```



```

    "status": "ACTIVE"
  }
}
}
}

```

Updating a Custom Attribute on a Fact Sheet

To update a custom attribute on a fact sheet, specify the attribute ID in `path`. You can view the attribute ID (key) in the fact sheet configuration.

In the following example, we update the custom attribute `serviceNowId` on an application fact sheet using the `replace` patch operation.

Example mutation:

```

mutation ($patches: [Patch!]) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", patches: $patches) {
    factSheet {
      id
      displayName
      ... on Application {
        serviceNowId
      }
    }
  }
}

```

Variables:

```

{
  "patches": [
    {
      "op": "replace",
      "path": "/serviceNowId",
      "value": "SN-123456"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
        "displayName": "AC Management Cloud",
        "serviceNowId": "SN-123456"
      }
    }
  }
}

```

Updating an Attribute of the Multiple Select Type on a Fact Sheet

To update an attribute of the **Multiple Select** type on a fact sheet, provide the values in an array.

In the following example, we update the custom attribute supportedPlatforms on an application fact sheet with two values: macOS and windows.

Example mutation:

```
mutation ($patches: [Patch]!) {
  updateFactSheet(id: "2efa37b5-18aa-48d8-9d70-1328c0d856d7", patches: $patches) {
    factSheet {
      ... on Application {
        supportedPlatforms
      }
    }
  }
}
```

Variables:

```
{
  "patches": [
    {
      "op": "replace",
      "path": "/supportedPlatforms",
      "value": "[\"macOS\", \"windows\"]"
    }
  ]
}
```

Example response:

```
{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "supportedPlatforms": [
          "macOS",
          "windows"
        ]
      }
    }
  }
}
```

Managing Fact Sheet Relations (GraphQL)

Manage relations between fact sheets and filter fact sheets for relation validity using the GraphQL API.

Creating Fact Sheet Relations

To create relations between Fact Sheets, use the `updateFactSheet` mutation. Apply the `add` patch operation with a path that identifies the relation attribute. You can create multiple relations with one request.

To get individual error messages for each relation in case of possible errors, add a suffix in the following format `/new_1` to the path values. Use a unique suffix for each relation.

Example mutation:

```
mutation ($patches: [Patch]!) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", patches: $patches) {
    factSheet {
      id
      name
      ... on Application {
        relApplicationToITComponent {
          edges {
            node {
              id
            }
          }
        }
      }
    }
  }
}
```

Variables:

```
{
  "patches": [
    {
      "op": "add",
      "path": "/relApplicationToITComponent/new_1",
      "value": "{\"factSheetId\":\"9deb9733-5701-42f1-8c52-c165acaa6487\"}"
    },
    {
      "op": "add",
      "path": "/relApplicationToITComponent/new_2",
      "value": "{\"factSheetId\":\"a8fe4825-42b8-431b-8124-ca12c579c78b\"}"
    }
  ]
}
```

Example response:

```
{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
        "name": "AC Management Cloud",
        "relApplicationToITComponent": {
          "edges": [
            {
              "node": {
                "id": "41ff8420-05d2-46e8-9912-975a9263f27e"
              }
            }
          ]
        }
      }
    }
  }
}
```

Example error response:

```
{
  "data": {
    "updateFactSheet": null
  },
  "errors": [
    {
      "message": "[FS_VALIDATION_RELATION_NOT_UNIQUE_OUTGOING] Outgoing relation occurred twice! re
      "path": [
        "result"
      ],
      "extensions": {
        "entityType": "FACT_SHEET",
        "objectId": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
        "objectPath": "/relApplicationToITComponent/fbec0812-ec6b-4c2c-8df2-525d5d3f3121",
        "errorType": "MODEL_COMPLIANT"
      }
    }
  ]
}
```

In the example, we update an existing relation by unlinking a Fact Sheet and linking a new one.

```
mutation ($patches: [Patch]!) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", patches: $patches) {
    factSheet {
      id
      name
      ... on Application {
        relApplicationToITComponent {
          edges {
```

```

    node {
      id
    }
  }
}
}
}
}
}
}
}

```

Variables:

```

{
  "patches": [
    {
      "op": "replace",
      "path": "/relApplicationToITComponent/41ff8420-05d2-46e8-9912-975a9263f27e",
      "value": "{$\"factSheetId\": \"ed46809c-998a-4fd6-9185-4b25e4e77d9b\", \"description\": \"Update\"}"}
    ]
  }
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
        "name": "AC Management Cloud",
        "relApplicationToITComponent": {
          "edges": [
            {
              "node": {
                "id": "8a039155-774b-4c09-94b1-e221eb0fee1c"
              }
            },
            {
              "node": {
                "id": "41ff8420-05d2-46e8-9912-975a9263f27e"
              }
            }
          ]
        }
      }
    }
  }
}

```

Deleting Fact Sheet Relations

Before you delete relations between Fact Sheets, get the `id` of relations. To do that, use the `factSheet` query and copy the `id` of the desired relations from the response.

Example query:

```
{
  factSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24") {
    name
    type
    ... on Application {
      relApplicationToITComponent {
        edges {
          node {
            id
          }
        }
      }
    }
  }
}
```

Example response:

```
{
  "data": {
    "factSheet": {
      "name": "AC Management Cloud",
      "type": "Application",
      "relApplicationToITComponent": {
        "edges": [
          {
            "node": {
              "id": "8a039155-774b-4c09-94b1-e221eb0fee1c"
            }
          },
          {
            "node": {
              "id": "41ff8420-05d2-46e8-9912-975a9263f27e"
            }
          }
        ]
      }
    }
  }
}
```

Once you get the `id` of relations that you want to delete, use the `updateFactSheet` mutation. Provide the relation `id` in the `remove` patch operation. In the example, we delete two relations that we created.

Example mutation:

```
mutation ($patches: [Patch]!) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", patches: $patches) {
    factSheet {
      id
      name
      ... on Application {
        relApplicationToITComponent {
          edges {
            node {
              id
            }
          }
        }
      }
    }
  }
}
```

```

    }
  }
}

```

Variables:

```

{
  "patches": [
    {
      "op": "remove",
      "path": "/relApplicationToITComponent/8a039155-774b-4c09-94b1-e221eb0fee1c"
    },
    {
      "op": "remove",
      "path": "/relApplicationToITComponent/41ff8420-05d2-46e8-9912-975a9263f27e"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
        "name": "AC Management Cloud",
        "relApplicationToITComponent": {
          "edges": []
        }
      }
    }
  }
}

```

Setting an Empty Relation on a Fact Sheet

You can intentionally set a relation on a Fact Sheet as empty. This implies that the Fact Sheet is not related to or dependent on other Fact Sheets and this data is not missing. Setting an empty relation affects the completion score of a Fact Sheet because the relation field is treated as filled.

To set a relation as empty, use the `updateFactSheet` mutation and apply a patch operation. The `naFields` attribute indicates fields and relations that are intentionally left blank. In the example, we set the `relToParent` relation as empty, which means that the Fact Sheet is not linked to any parent Fact Sheets.

Example mutation:

```

mutation ($patches: [Patch]!) {
  updateFactSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24", patches: $patches) {
    factSheet {
      id
      name
      naFields
    }
  }
}

```

```

    }
  }
}

```

Variables:

```

{
  "patches": [
    {
      "op": "add",
      "path": "/naFields",
      "value": "[\"relToParent\"]"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
        "name": "AC Management",
        "naFields": [
          "relToParent"
        ]
      }
    }
  }
}

```

Filtering Fact Sheets for Relation Validity

Relation validity ("activeFrom" and "activeUntil") is the time interval during which a relation between fact sheets is deemed to exist. The two values are dates in ISO format like 2018-07-01(no time).

Any of them may be unspecified, which means the Relation validity is not bounded in this direction of time. If both of these values of the relation are not set, then the Relation is considered to always exist.

For this purpose, the `DateFilterInput` argument type contains the fields `from` and `to` and the `DateFilterType` in field `type`.

These are the possible values of the `type` field:

- Of main importance is only the date filter type `RANGE`.
- `RANGE_START` and `RANGE_ENDS` are not implemented for relation validity filtering and behave like `RANGE` for now.
- `POINT` behaves like `RANGE`, but the given value for `to` is ignored and always considered to be equal to `from`.
- `TODAY`, `END_OF_MONTH`, and `END_OF_YEAR` are special cases of `POINT` where the given values of `from` and `to` are ignored and considered to be equal to the day corresponding to the type.

The values of the fields `from` and `to` are dates in ISO format (no time) and define the filter interval. Any of the two values may be null, meaning the interval is unbounded in that direction.

The relation validity filtering semantics is the following: a relation is matched by the filter if and only if the relation validity interval has a non-empty intersection with the filter interval. In other words, if the time span defined by the relation's `activeFrom` and `activeUntil` fields and the time span defined by the filter's `from` and `to` fields overlap.

In the GraphQL API, you can filter Fact Sheets for relation validity in the:

- Relation facets
- Relation validity filter

Filtering Relations in the Relation Facet

You can filter relations in the relation facets (`filter -> facetFilter -> dateFilter` argument of the `allFactSheets` top level query field).

For now, instead of the `dateFilter` on the individual relation facets, the `dateFilter` on the lifecycle facet is used. This will change in a future version. The key field of this lifecycle filter facet is ignored for the purpose of relation validity filtering.

Example query:

```
{
  allFactSheets(filter:
    {facetFilters: [
      {facetKey:"FactSheetTypes" keys:["UserGroup"]}
      {facetKey: "relUserGroupToApplication" keys: ["e3a08f5f-5c3e-4dc1-8444-ed87bdd48634"]}
      {facetKey:"lifecycle" keys:"__any__" dateFilter:{type:RANGE from:"2018-07-01" to:"2018-07-01"}}
    ])
  {
    edges {
      node {
        type
        displayName
      }
    }
  }
}
```

Filtering Relations in the Relation Validity Filter

This is a filter that restricts the relations that are returned in the relation fields of a concrete Fact Sheet type, for example, field `relApplicationToUserGroup` in type `Application`. Those fields accept an argument `validityFilter` containing the fields `activeFrom` and `activeUntil`, which behave like `from` and `to` in `DateFilterInput` (see above).

Example query:

```
{
  factSheet(id: "e3a08f5f-5c3e-4dc1-8444-ed87bdd48634") {
    displayName
    ... on Application {
      relApplicationToUserGroup(validityFilter: {activeFrom: "2018-07-01", activeUntil: "2018-07-01"}) {
        edges {
          node {
            activeFrom
            factSheet {
              displayName
            }
          }
        }
      }
    }
  }
}
```

```

    }
  }
}

```

Managing Fact Sheet Tags (GraphQL)

Manage tags on a fact sheet using the GraphQL API.

Tags allow you to categorize fact sheets by attributes and relations that are not present in the standard configuration. For more information, see [Tags](#).

Adding Tags to a Fact Sheet

To add tags to a fact sheet, use the `updateFactSheet` mutation. Provide the tag details through the `add patch` operation.

`id` is a required argument for this mutation. To learn how to get the ID of a Fact Sheet, see [Retrieving Fact Sheets](#).

Adding Existing Tags to a Fact Sheet

Before you add existing tags to a fact sheet, get the `id` of tags. To do that, retrieve all tags using the `allTags` query and copy the `id` of the desired tags from the response.

Example query:

```

{
  allTags {
    edges {
      node {
        id
        name
      }
    }
  }
}

```

Example response:

```

{
  "data": {
    "allTags": {
      "edges": [
        {
          "node": {
            "id": "c26a8509-330e-4d8f-ba48-2defcdc910f7",
            "name": "Europe"
          }
        },
        {
          "node": {
            "id": "7b27fc19-585e-4d95-acae-65891575a42f",
            "name": "Cloud"
          }
        }
      ]
    }
  }
}

```

```

    }
  }
}

```

Once you get the `id` of tags that you want to add to a Fact Sheet, use the `updateFactSheet` mutation.

Example mutation:

```

mutation ($patches: [Patch]!) {
  updateFactSheet(id: "2efa37b5-18aa-48d8-9d70-1328c0d856d7", patches: $patches) {
    factSheet {
      id
      name
      tags {
        id
        name
      }
    }
  }
}

```

Variables:

```

{
  "patches": [
    {
      "op": "add",
      "path": "/tags",
      "value": "[{"tagId\": \"7b27fc19-585e-4d95-acae-65891575a42f\"}, {"tagId\": \"c26a8509-330e-4d8f-ba48-2defcdc910f7\"}]"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
        "name": "AC Management Cloud",
        "tags": [
          {
            "id": "c26a8509-330e-4d8f-ba48-2defcdc910f7",
            "name": "Europe"
          },
          {
            "id": "7b27fc19-585e-4d95-acae-65891575a42f",
            "name": "Cloud"
          }
        ]
      }
    }
  }
}

```

Adding New Tags to a Fact Sheet

In the following example, we create two new tags that don't belong to any tag group and add them to a Fact Sheet.

Example mutation:

```
mutation ($patches: [Patch]!) {
  updateFactSheet(id: "2efa37b5-18aa-48d8-9d70-1328c0d856d7", patches: $patches) {
    factSheet {
      id
      name
      tags {
        id
        name
        tagGroup {
          name
        }
      }
    }
  }
}
```

Variables:

```
{
  "patches": [
    {
      "op": "add",
      "path": "/tags",
      "value": "[{\"tagName\":\"Development\"}, {\"tagName\":\"Demo\"}]"
    }
  ]
}
```

Example response:

```
{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
        "name": "AC Management Cloud",
        "tags": [
          {
            "id": "2ddc9122-4513-40e2-afd8-4b28f41474a4",
            "name": "Demo",
            "tagGroup": null
          },
          {
            "id": "0eb40008-28ac-467c-adc5-5b995ca47769",
            "name": "Development",
            "tagGroup": null
          }
        ]
      }
    }
  }
}
```

```
}
}
```

Removing Tags from a Fact Sheet

Before you remove tags from a Fact Sheet, get the `id` of tags. To do that, retrieve all tags using the `allTags` query and copy the `id` of the desired tags from the response.

i Note

Removing a tag from a Fact Sheet does not delete the tag from your workspace.

To remove tags from a Fact Sheet, use the `updateFactSheet` mutation and apply the `remove` patch operation.

Example mutation:

```
mutation ($patches: [Patch]!) {
  updateFactSheet(id: "2efa37b5-18aa-48d8-9d70-1328c0d856d7", patches: $patches) {
    factSheet {
      id
      name
      tags {
        id
        name
        tagGroup {
          name
        }
      }
    }
  }
}
```

Variables:

```
{
  "patches": [
    {
      "op": "remove",
      "path": "/tags",
      "value": "[{\\"tagId\\":\\"7b27fc19-585e-4d95-acae-65891575a42f\\"}, {\\"tagId\\":\\"c26a8509-330e-4"
    }
  ]
}
```

Example response:

```
{
  "data": {
    "updateFactSheet": {
      "factSheet": {
        "id": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
        "name": "AC Management Cloud",
        "tags": []
      }
    }
  }
}
```

```

    }
  }
}

```

Managing Fact Sheet Resources (GraphQL)

Manage fact sheet resources using the GraphQL API.

To store additional information on a fact sheet, you can add the following resources to it: links, files, diagrams, and logos. To learn how to work with resources in the application UI, see [Storing Resources on Fact Sheets](#).

i Note

In the SAP LeanIX GraphQL API, fact sheet resources are referred to as documents.

Creating a Resource on a Fact Sheet

To create a resource on a fact sheet, use the `createDocument` mutation. `factSheetId` and `name` are required arguments for this mutation. To learn how to get the ID of a fact sheet, see [Retrieving Fact Sheets](#).

In the following example, we add a link to a fact sheet as a resource.

Example mutation for adding a link:

```

mutation createDocument {
  createDocument(
    factSheetId: "01740698-1ffa-4729-94fa-da6194ebd7cd"
    name: "Website link"
    url: "https://www.leanix.net/"
    description: "Link to the SAP LeanIX website"
  ) {
    id
    name
    url
    description
    factSheetId
  }
}

```

Example response:

```

{
  "data": {
    "createDocument": {
      "id": "fc87eb51-0f2e-4b77-9df3-f6325b6836fd",
      "name": "Website link",
      "url": "https://www.leanix.net/",
      "description": "Link to the SAP LeanIX website",
      "factSheetId": "01740698-1ffa-4729-94fa-da6194ebd7cd"
    }
  }
}

```

To be able to upload files and logos as resources to a fact sheet, activate the [Uploading Files on Fact Sheets](#) feature in [Optional Features](#) in the administration area. If this feature is deactivated, uploading files and logos through the GraphQL API is not possible.

To create resources of the **File** and **Logo** types using the `createDocument` mutation, provide the GraphQL request and file stream as form data. To upload a file to a fact sheet as a resource, use the GraphQL endpoint for file uploads:

`https://{SUBDOMAIN}.leanix.net/services/pathfinder/v1/graphql/upload`

i Note

For a **Logo** resource, the `documentType` argument is required and should be set to `logo`. You can only upload one logo to a fact sheet.

For **File** resources, the `documentType` argument is optional. You can upload multiple files to a fact sheet.

Example mutation for uploading a logo:

```
mutation createDocument {
  createDocument(
    factSheetId: "01740698-1ffa-4729-94fa-da6194ebd7cd"
    name: "leanix.png"
    documentType: "logo"
  ) {
    id
    name
    url
    factSheetId
  }
}
```

Example response:

```
{
  "data": {
    "createDocument": {
      "id": "ec87eb51-0f2e-4b77-9df3-f6325b6836fd",
      "name": "leanix.png",
      "url": "<<storage location="" of="" the="" logo="">>",
      "factSheetId": "01740698-1ffa-4729-94fa-da6194ebd7cd"
    }
  }
}
```

Retrieving Fact Sheet Resources

To retrieve resources stored on a specific fact sheet, use the `factSheet` query. The fact sheet `id` is a required argument for this query.

Example query:

```
{
  factSheet(id: "4d121f64-116b-4ccc-a292-eb4e4f8d1b24") {
    id
    name
    documents {
      edges {
        node {
          origin
          url
          name
          id
        }
      }
    }
  }
}
```

```

    }
  }
}

```

Example response:

```

{
  "data": {
    "factSheet": {
      "id": "4d121f64-116b-4ccc-a292-eb4e4f8d1b24",
      "name": "AC Management Cloud",
      "documents": {
        "edges": [
          {
            "node": {
              "origin": "CUSTOM_LINK",
              "url": "https://www.leanix.net/",
              "name": "Website link",
              "id": "22b9ed8f-5dea-48e8-b9c4-d7db97a811e4"
            }
          }
        ]
      }
    }
  }
}

```

Updating a Resource on a Fact Sheet

To update a resource on a fact sheet, use the `updateDocument` mutation. `id` is a required argument for this mutation. To get the resource ID, retrieve all fact sheet resources and copy the `id` from the response. To learn more, see [Retrieving Fact Sheet Resources](#).

To update specific fields on a resource, apply patch operations. In the example, we change the link URL using the `replace` patch operation.

Example mutation:

```

mutation updateDocument($patches: [Patch]!) {
  updateDocument(
    id: "22b9ed8f-5dea-48e8-b9c4-d7db97a811e4"
    patches: $patches
  ) {
    id
    name
    url
  }
}

```

Variables:

```

{
  "patches": [
    {

```



```

      "op": "replace",
      "path": "/url",
      "value": "https://www.leanix.net/de/"
    }
  ]
}

```

Example response:

```

{
  "data": {
    "updateDocument": {
      "id": "22b9ed8f-5dea-48e8-b9c4-d7db97a811e4",
      "name": "Website link",
      "url": "https://www.leanix.net/de/"
    }
  }
}

```

Deleting a Resource from a Fact Sheet

To delete a resource from a fact sheet, use the `deleteDocument` mutation. `id` is a required argument for this mutation. To get the resource ID, retrieve all fact sheet resources and copy the `id` from the response. To learn more, see [Retrieving Fact Sheet Resources](#).

Example mutation:

```

mutation {
  deleteDocument(id: "a1904f80-f9b8-4fd6-848c-bad1f55192d2") {
    name
  }
}

```

Example response:

```

{
  "data": {
    "deleteDocument": null
  }
}

```

Managing Fact Sheet Subscriptions (GraphQL)

Manage fact sheet subscriptions using the GraphQL API.

Subscriptions let you configure a set of custom permissions and responsibilities in relation to fact sheets for various user groups within your organization. To learn more about subscriptions, see [Fact Sheet Subscription](#).

Get Subscription Roles

Before you start, get subscription roles configured in your workspace using the `allSubscriptionRoles` query.

Example query:

```
{
  allSubscriptionRoles {
    edges {
      node {
        id
        name
      }
    }
  }
}
```

Example response:

```
{
  "data": {
    "allSubscriptionRoles": {
      "edges": [
        {
          "node": {
            "id": "1d7f3a77-9a33-4029-9077-d21ae4562575",
            "name": "Solution Architect"
          }
        },
        {
          "node": {
            "id": "b4ccabdc-f0c4-4386-8ff0-b54e0882605f",
            "name": "Application Manager"
          }
        }
      ]
    }
  }
}
```

Create a Fact Sheet Subscription

To create a subscription to a fact sheet for a user, use the `createSubscription` mutation. The following arguments are required for this mutation:

- **factSheetId**: The ID of the fact sheet to subscribe the user to. To learn how to get the ID of a fact sheet, see [Retrieving Fact Sheets](#).
- **user**: The user to be subscribed to a fact sheet. Pass one of the following attributes in the object: `id` or `email`.
- **type**: The subscription type. Possible values:
 - **ACCOUNTABLE**: User who carries overall accountability for a fact sheet.
 - **RESPONSIBLE**: User responsible for keeping the fact sheet data accurate and up to date.
 - **OBSERVER**: User who follows fact sheet updates.

To specify subscription roles, use the `roleIds` argument. To retrieve the IDs of subscription roles, use the `allSubscriptionRoles` query. Once you get all the required parameters, create a subscription using the `createSubscription` mutation. In the example, we subscribe a user as **RESPONSIBLE** with the **Application Manager** subscription role.

Example mutation:

```

mutation {
  createSubscription(factSheetId: "01740698-1ffa-4729-94fa-da6194ebd7cd",
    user: {id: "e45f0d10-c59e-4c80-bd56-56b9e7325gf6"},
    type: RESPONSIBLE,
    roleIds: ["b4ccabdc-f0c4-4386-8ff0-b54e0882605f"]) {
    id
    user {
      id
      email
    }
    type
    roles {
      id
      name
    }
  }
}

```

Example response:

```

{
  "data": {
    "createSubscription": {
      "id": "abd9ca16-b4a7-4199-a0ad-36e5a6de117b",
      "user": {
        "id": "e45f0d10-c59e-4c80-bd56-56b9e7325gf6",
        "email": "john.doeC@organization.com"
      },
      "type": "RESPONSIBLE",
      "roles": [
        {
          "id": "b4ccabdc-f0c4-4386-8ff0-b54e0882605f",
          "name": "Application Manager"
        }
      ]
    }
  }
}

```

Get Fact Sheet Subscriptions

To retrieve all subscriptions for a specific fact sheet, use the `factSheet` query. `id` is a required argument for this query. To learn how to get the ID of a fact sheet, see [Retrieving Fact Sheets](#).

Example query:

```

query {
  factSheet(id: "01740698-1ffa-4729-94fa-da6194ebd7cd") {
    id
    subscriptions {
      edges {
        node {
          id
          user {
            id
            email
          }
          type
          roles {
            id
            name
          }
        }
      }
    }
  }
}

```

```

    comment
  }
}
}
}
}
}
}

```

Example response:

```

{
  "data": {
    "factSheet": {
      "id": "01740698-1ffa-4729-94fa-da6194ebd7cd",
      "subscriptions": {
        "edges": [
          {
            "node": {
              "id": "abd9ca16-b4a7-4199-a0ad-36e5a6de117b",
              "user": {
                "id": "e45f0d10-c59e-4c80-bd56-56b9e7325gf6",
                "email": "john.doeC@organization.com"
              },
              "type": "RESPONSIBLE",
              "roles": [
                {
                  "id": "b4ccabdc-f0c4-4386-8ff0-b54e0882605f",
                  "name": "Application Manager",
                  "comment": null
                }
              ]
            }
          ]
        ]
      }
    }
  }
}

```

Update a Fact Sheet Subscription

Before you update a fact sheet subscription, get the subscription id. To learn how to retrieve all fact sheet subscriptions, see [Retrieving Fact Sheet Subscriptions](#).

To update a subscription, use the `updateSubscription` mutation. The following arguments are required for this mutation:

- **id**: The ID of the subscription.
- **user**: The user subscribed to a fact sheet. Pass one of the following attributes in the object: **id** or **email**.
- **type**: The subscription type. Possible values:
 - **ACCOUNTABLE**: User who carries overall accountability for a fact sheet.
 - **RESPONSIBLE**: User responsible for keeping the fact sheet data accurate and up to date.

- OBSERVER: User who follows fact sheet updates.

In the example, we change the subscription role of the user from **Application Manager** to **Solution Architect** using the `roleIds` argument. You can optionally add a comment.

Example mutation:

```
mutation {
  updateSubscription(
    id: "abd9ca16-b4a7-4199-a0ad-36e5a6de117b"
    user: {id: "e45f0d10-c59e-4c80-bd56-56b9e7325gf6"}
    type: RESPONSIBLE
    roles: [{id: "1d7f3a77-9a33-4029-9077-d21ae4562575", comment: "Changing the subscription role"}]
  ) {
    id
    user {
      id
      email
    }
    type
    roles {
      id
      name
      comment
    }
  }
}
```

Example response:

```
{
  "data": {
    "updateSubscription": {
      "id": "abd9ca16-b4a7-4199-a0ad-36e5a6de117b",
      "user": {
        "id": "e45f0d10-c59e-4c80-bd56-56b9e7325gf6",
        "email": "john.doeC@organization.com"
      },
      "type": "RESPONSIBLE",
      "roles": [
        {
          "id": "1d7f3a77-9a33-4029-9077-d21ae4562575",
          "name": "Solution Architect",
          "comment": "Changing the subscription role"
        }
      ]
    }
  }
}
```

Delete a Fact Sheet Subscription

Before you delete a fact sheet subscription, get the subscription `id`. To learn how to retrieve all fact sheet subscriptions, see [Retrieving Fact Sheet Subscriptions](#).

To delete a subscription, use the `deleteSubscription` mutation.

Example mutation:

```
mutation {
  deleteSubscription(id: "abd9ca16-b4a7-4199-a0ad-36e5a6de117b") {
    id
    name
  }
}
```

Example response:

```
{
  "data": {
    "deleteSubscription": {
      "id": "01740698-1ffa-4729-94fa-da6194ebd7cd",
      "name": "AC Management"
    }
  }
}
```

Retrieving Event Logs for a Fact Sheet (GraphQL)

Retrieve event logs for a fact sheet using the GraphQL API.

i Note

You can retrieve event logs for fact sheets only through the GraphQL API. Our REST APIs don't offer a dedicated endpoint for this operation.

All updates to a fact sheet are recorded in the event logs. You can use event logs to create audit trails and get insights into the dynamics of your workspace. In the application user interface, event logs appear on the **Last Update** tab of the fact sheet page.

To retrieve event logs through the GraphQL API, use the `allLogEvents` query. `factSheetId` is a required argument for this query. To learn how to get the ID of a fact sheet, see [Retrieving Fact Sheets](#).

To filter logs by a specific event type, add the `eventTypes` argument to the `allLogEvents` query. To view the full list of event types, navigate to the GraphQL tool in your workspace and search for the `EventTypes` schema type. To include multiple event types in the query, use an array of values, as shown in the following example: `eventTypes: [FACT_SHEET_TAG_ADDED, FACT_SHEET_TAG_REMOVED]`.

→ Tip

You can automate the routine retrieval of event logs by setting up cron jobs or using automation tools. To fetch event logs for all fact sheets or for specific types of fact sheets, create a script with a loop that iterates over the fact sheet IDs. To obtain all fact sheet IDs, use the `allFactSheets` query. These IDs can then be used as input for the `allLogEvents` query. To learn how to fetch fact sheets, see [Retrieving Fact Sheets](#).

To receive real-time updates for specific events automatically without pulling them, consider using webhooks. To learn more, see [Webhooks](#).

In the following example query, we retrieve events of the `QUALITY_SEAL_APPROVED` type, which indicates the approval of the quality seal on a fact sheet.

The `createdAt` attribute in the response denotes the time stamp of the event. By using this value, you can identify when the quality seal is set to expire. For example, if the expiration time of the quality seal is set to three months, it means that it will expire

three months after the `createdAt` date — unless a breaking event occurs earlier. You can use this information to get a snapshot of fact sheets whose quality seals will expire soon.

As an administrator, you can set the expiration time of the quality seal in the fact sheet configuration. To learn more, see [Quality Seal](#).

Example query:

```
{
  allLogEvents(
    factSheetId: "2efa37b5-18aa-48d8-9d70-1328c0d856d7"
    eventTypes: QUALITY_SEAL_APPROVED
  ) {
    edges {
      node {
        id
        eventType
        path
        oldValue
        newValue
        message
        secondsPast
        createdAt
        user {
          id
          firstName
          lastName
          displayName
          email
          technicalUser
        }
      }
    }
  }
}
```

Example response:

```
{
  "data": {
    "allLogEvents": {
      "edges": [
        {
          "node": {
            "id": "42574",
            "eventType": "QUALITY_SEAL_APPROVED",
            "path": "/qualitySeal",
            "oldValue": null,
            "newValue": "APPROVED",
            "message": "The quality seal was approved by user 'John Doe'",
            "secondsPast": 4928370,
            "createdAt": "2024-02-13T14:44:06.299466554Z",
            "user": {
              "id": "5ee78c21-98a2-4377-acaa-fc66ff2b78fg",
              "firstName": "John",
              "lastName": "Doe",
              "displayName": "John Doe",
              "email": "john.doe@example.com",
              "technicalUser": false
            }
          }
        }
      ]
    }
  }
}
```

```

    }
  ]
}
}
}

```

Integration API

Overview of the Integration API.

Overview

The Integration API provides the ability to import and export data using a generic LeanIX Data Interchange Format (LDIF). LDIF is a JSON format with a very simple structure described in the following sections. All mapping and processing of the incoming and outgoing data is done using "Data Processors" that are configured behind the API. Configuration of the processors can be done using the UI, The configurations can be managed using the Integration API as well.

To learn how to configure processors in the application UI, see [Getting Started with the Integration API](#).

Advantages

- Complexity is minimized as developers no longer have to understand the SAP LeanIX data model
- Configuration for SAP LeanIX is contained in every connector
- No longer having to map logic inside of code
- Integration now possible even if the external system does not allow direct access
- No need for a direct REST connection
- Flexible error handling, no failure due to a single or even multiple data issues. Data that does not meet the requirements, is simply ignored
- Cross-cutting concerns are part of the API and in each connector

Inbound and Outbound Data Processors

Definitions

Inbound Data Processors read the incoming LDIF format, process and convert it into a set of commands the Integration API can understand. Then it executes it against the SAP LeanIX backend database (Pathfinder) or in the case of inboundMetrics the Metrics API.

Outbound Data Processors read data from the workspace and **export** it to an LDIF. The content of the produced LDIF will depend on the configured data processors.

General Attributes

There are attributes that can be set within the Processors, whether they be outbound or inbound.


```
63     ],
64     "enabled": true,
65     "variables": {},
66     "logLevel": "debug"
67 },
```

The flags are set at the end of the data processors

Attribute	Allowed Values	Details
logLevel	- off (default) - warning - debug	When writing Data Processors, the log level can be set to show detailed output for all data objects processed by a data processor. In order to do this, a key "logLevel" needs to be set to "debug" for the processor. debugInfo in addition to the actual error information also contains the name of the processor and the index. The "warning" logLevel also provides the processor name as well as the index. Default setting is "warning". In this setting detailed information will be provided in case an issue happened processing the data. There may be cases (e.g. incoming data is not consistent/of poor quality) where issues are expected and must not flood the result report. In such cases, the "logLevel" can be set to "off". No errors will be reported for the data processor.
enabled	- true - false	Turns the whole processor on and off. This can be helpful to analyze the impact of certain processors on the result without having to completely remove and add back in them.

The following example shows an output with warnings that contain the processor name and index. These attributes are also included when the logLevel is set to debug. In this case, the debug information is included in the debugInfo section of the output log.

OUTPUT LOG

```

1 {
2   "results": null,
3   "warnings": [
4     {
5       "processor": {
6         "name": "ITComponent Rel to Prov",
7         "index": 0
8       },
9       "content": {
10        "message": "Pathfinder GraphQL response contains errors",
11        "detail": {
12          "message": "An unexpected error occurred. The following error code helps the LeanIX support
13        },
14        "category": "mutationRequest"
15      },
16    },
17    {
18      "processor": {
19        "name": "ITComponent Rel to Prov",
20        "index": 0
21      },
22      "content": {
23        "message": "Pathfinder GraphQL response contains errors",

```

Support

Example Output of a Processor Run

LeanIX Data Interchange Format (LDIF)

All data [sent](#) (inbound Data Processor) to the SAP LeanIX Integration API needs to be in a standard format, called LDIF. For synchronization of data from SAP LeanIX to other systems (outbound Data Processor), the Integration API will provide all data in the same format as well.

The LDIF contains the following information:

- Data sent from the external system to SAP LeanIX. Or in the case of outbound, data extracted from SAP LeanIX.
- Metadata information to identify the connector instance that wrote the LDIF. The metadata is used to define ownership of entities in SAP LeanIX if we need to ensure name spacing/deletion
- Identification of the target workspace and the target API version
- Allow customers adding some arbitrary description for any kind of grouping, notification or any unstructured notes for display purposes that is not processed by the API

Mandatory Attributes

The following table lists mandatory attributes for data input in LDIF format.

Attribute	Details
connectorType	Contains a string identifying a connector type in the workspace. A connector type identifies code that can be deployed to multiple locations and is able to communicate with a specific external program. In the above example, "IxKubernetes" identifies a connector that was written to read data from a Kubernetes installation and can be deployed into various Kubernetes clusters. In conjunction with "connectorID", the Integration API will match configurations the administrators created and stored in SAP LeanIX and use it when processing incoming data.

Attribute	Details
	Incoming data cannot be processed if the corresponding connectorType has not been configured for the SAP LeanIX workspace.
connectorVersion (connector Version is not mandatory for configurations where the version numbers follow the pattern number . number . number)	In case the version number follows the pattern "number.number.number", Integration API provides automatic grouping and handling of different versions of a configuration. The Admin UI will automatically group all configurations by their version numbers and display the highest number on the overview page. In the details users can select the specific version in a drop down. Configurations that follow the pattern above can be executed without sending a specific version number in the LDIF. The Integration API will always process with the highest found version number if no version number was given in the input LDIF. In order to use a specific version for processing, the exact version can be provided in the LDIF. "connectorVersion" may contain any other string content. In such cases, automatic detection of a version does not work and the exact string must be provided in the LDIF
connectorId	Contains a string to identify a specific deployment of a connector type. As an example: A Kubernetes connector might be deployed multiple times and collect different data from the different Kubernetes clusters. In conjunction with "connectorID", the Integration API will match configurations administrators created and stored in SAP LeanIX and use it when processing incoming data. Administrators in SAP LeanIX can manage each instance of a connector by creating and editing processing configuration, monitor ongoing runs (progress, status) and interact (pause, resume, cancel...). One data transfer for each instance can run at any point of time. Incoming data cannot be processed if the corresponding connectorId has not been configured for the SAP LeanIX workspace.
lxVersion	Defines the version of the Integration API the connector expects to send data to and will be used to ensure that a component grabbing LDIF files from a cloud storage will send it to the right integration API version (in case no direct communication is available).
content	The content section contains a list of Data Objects. This section contains the data collected from the external system or the data to be sent to the external system. Each Data Object is defined by an "id", a type to group the source information and a map of data elements (flat or hierarchical maps). Values of map entries may be single string values or lists of string values.

i Note

Incoming data cannot be processed if the corresponding connectorType has not been configured for the SAP LeanIX workspace.

Content Array

The content section contains a list of Data Objects. **This section contains the data collected from the external system or the data to be sent to the external system.** Each Data Object is defined by an "id" (typically a UUID in the source system), a type to group the source information and a map of data elements. The keys "id", "type" and "data" are mandatory. The data map may be flat or hierarchical maps. Values of map entries may be single string values, maps or lists.

Mandatory Attributes

The following table lists mandatory attributes for the content array in the input data.

Attribute	Description
id	Contains a string typically representing a unique identifier that allows to track back the item in the source system and to ensure updates in SAP LeanIX always go to the same Fact Sheet. SAP LeanIX Data processors will provide an efficient matching option to allow configuration of specific mapping rules based on IDs or groups of IDs that can be identified by patterns.
type	A string representing any required high level structuring of the source data. Example content in case of Kubernetes are e.g. "Cluster" (containing data that identifies the whole Kubernetes instance) or "Deployment" (which can represent a type of application in Kubernetes) Will typically be used to create different types or subtypes of Fact Sheets or relations from. SAP LeanIX Data processors will provide an efficient matching option to allow configuration of specific mapping rules based on Type or groups of Type strings that can be identified by patterns.
data	The data extracted from the source system. The format is simple: All data has to be in a Map. Each map can contain a single string as a value for a key, a list of strings as a value or contain a map. The map again has to follow the rules just described.

Optional Attributes

The following table lists optional attributes for the content array in the input data.

Attribute	Description
processingMode	May contain "PARTIAL" (default if not existing) or "FULL". Full mode allows to automatically remove all Fact Sheets that match a configured query and are not touched by the integration.
chunkInformation	If existing, it contains "firstDataObject", "lastDataObject" and "maxDataObject". Each value is a number defining what is in this potentially chunked LDIF.
description	The description can contain any string that may help to identify source or type of data. Sometimes it is helpful to add some information to analysis purposes or when setting up configuration on SAP LeanIX side
customFields	This optional section may contain a map of fields and values defined by the producer of the LDIF. All data can be referenced in

Attribute	Description
	any data processor. It will be used for globally available custom meta data.
lxWorkspace	Defines the SAP LeanIX workspace the data is supposed to be sent to or received from. The content will be used for additional validation by the integration API to check if data will be sent to the right workspace. The content has to match the string visible in the URL when accessing the workspace in a browser. Users need to enter the UUID of the workspace in order to make use of this additional security mechanism. The UUID can e.g. be read from the administration page where API Tokens are created. Example "lxWorkspace": "19fcfab-cb8a-4b7c-97fe-7c779345e20e"

i Note

- Additional fields in the LDIF that do not match the requirements of defined here will be silently ignored.
- Each of the values listed above, except the values in the "content" section must not be longer than 500 characters.

Example input data for importing fact sheets:

```
{
  "connectorType": "clouddockit",
  "connectorId": "CDK",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "lxWorkspace": "workspace-id",
  "description": "Imports Clouddockit data into LeanIX",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "content": [
    {
      "type": "ITComponent",
      "id": "b6992b1d-4e4d",
      "data": {
        "name": "Gatsby.js",
        "description": "Gatsby is a free and open source framework based on React that helps develop",
        "category": "sample_software",
        "provider": "gcp",
        "applicationId": "28db27b1-fc55-4e44"
      }
    },
    {
      "type": "ITComponent",
      "id": "cd4fab6c-4336",
      "data": {
        "name": "Contentful",
        "description": "Beyond headless CMS, Contentful is an API-first content management infrastru",
        "category": "cloud_service",
        "provider": "gcp",
        "applicationId": "28db27b1-fc55-4e44"
      }
    }
  ]
}
```

```

    }
  },
  {
    "type": "ITComponent",
    "id": "3eaaa629-5338-41f4",
    "data": {
      "name": "GitHub",
      "description": "GitHub is a tool that provides hosting for software development version control systems.",
      "category": "cloud_service",
      "provider": "gcp",
      "applicationId": "28db27b1-fc55-4e44"
    }
  },
  {
    "type": "Application",
    "id": "28db27b1-fc55-4e44",
    "data": {
      "name": "Book a Room Internal",
      "description": "Web application that's used internal to book rooms for a meeting."
    }
  }
]
}

```

Example input data for importing lifecycle information:

```

{
  "connectorType": "clouddockit",
  "connectorId": "CDK",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "lxWorkspace": "workspace-id",
  "description": "Imports Clouddockit data into LeanIX",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "content": [
    {
      "type": "Application",
      "id": "company_app_1",
      "data": {
        "name": "TurboTax",
        "plan": null,
        "phaseIn": "2016-12-29",
        "active": "2019-12-29",
        "phaseOut": "2020-06-29",
        "endOfLife": "2020-12-29"
      }
    },
    {
      "type": "Application",
      "id": "company_app_2",
      "data": {
        "name": "QuickBooks",
        "plan": null,

```

```

    "phaseIn": "2016-11-29",
    "active": "2019-11-29",
    "phaseOut": "2020-05-29",
    "endOfLife": "2020-11-29"
  }
}
]
}

```

Integration REST API

You can interact with the Integration API through REST API endpoints. For more information, see [Integration REST API](#).

Grouped Execution of Multiple Configurations

A new key "executionGroups" added to the main section now allows Integration API to execute multiple Integration API configurations as if they were one. This is advanced functionality, not available in the UI but only when using the REST API. For details, navigate to the OpenAPI Explorer.

Example processor with grouped execution:

```

{
  "processors": [
    {
      "processorType": "inboundTag",
      "processorName": "Set TIME tags",
      "processorDescription": "Sets the TIME tags based on existing Fact Sheet values",
      "filter": {
        "advanced": "${integration.contentIndex==0}"
      },
      "identifier": {
        "search": {
          "scope": {
            "ids": [],
            "facetFilters": [
              {
                "keys": [
                  "Application"
                ],
                "facetKey": "FactSheetTypes",
                "operator": "OR"
              }
            ]
          }
        }
      },
      "run": 0,
      "updates": [
        {
          "key": {
            "expr": "name"
          },
          "values": [

```

```

    {
      "expr": "${((lx.factsheet.functionalSuitability=='unreasonable' || lx.factsheet.funci
    },
    {
      "expr": "${((lx.factsheet.functionalSuitability=='appropriate' || lx.factsheet.funci:
    },
    {
      "expr": "${((lx.factsheet.functionalSuitability=='unreasonable' || lx.factsheet.funci
    },
    {
      "expr": "${((lx.factsheet.functionalSuitability=='appropriate' || lx.factsheet.funci:
    },
    {
      "expr": "No Data"
    }
  ]
},
{
  "key": {
    "expr": "group.name"
  },
  "values": [
    {
      "expr": "Time Model"
    }
  ]
}
],
"logLevel": "warning",
"read": {
  "fields": [
    "technicalSuitability",
    "name",
    "functionalSuitability"
  ]
}
}
],
"executionGroups": [
  "customGroupA",
  "anotherGroup",
  "myGroup"
]
}

```

Calling /synchronizationRuns/createSynchronizationRunWithExecutionGroup with parameter customGroupA will run all configurations that contain customGroupA in the list of executionGroups defined.

Caution

Advanced functionality: use with careful management of configurations.

The system will merge the configurations and use all processors from all configurations. There will be no ordering applied. Processors from different configurations with same run levels will run in parallel.

Conflicting settings like mixing inbound and outbound configurations will result in an error. Other special logic like settings for e.g. a data consumer will be taken from any configuration. If multiple ones are found, the run will be executed with the last found configuration.

In order to detect the "last found configuration", Integration API configurations with a matching tag will be sorted alphabetically and merged one by one. Sorting happens by type, then id, then version.

Comments in Processors

Integration API allows to solve simple mapping tasks as well solving highly complex processing and aggregation use cases.

Documentation of more complex processing is key for maintenance work in case of changes. Integration API allows to add "description" keys in the locations listed in the below examples. The free text will be stored with the configuration but not be used for any processing. It is just used to document more complex expressions used.

Example inbound processor with descriptions:

```
{
  "identifier": {
    "description": "We iterate over all Fact Sheets of type 'Application'"
  },
  "variables": [
    {
      "description": "Write a variable for each found child to copy description to that later",
      "key": "collectedChild_${integration.variables.valueOfForEach.target.id}",
      "value": "${lx.factsheet.description}",
      "forEach": {
        "elementOf": "${lx.relations}"
      }
    },
    {
      "description": "Write all children to one variable to iterate over in later processor",
      "key": "collectedChildren",
      "value": "${integration.variables.valueOfForEach.target.id}",
      "forEach": {
        "elementOf": "${lx.relations}"
      }
    }
  ],
  "updates": [
    {
      "description": "Values to writing the field Business Fit are taken from a variable",
      "values": [
        {
          "description": "The source variable is names Businessfit_[currentFactSheetId]"
        }
      ]
    }
  ]
}
```

Example outbound processor with descriptions:

```
{
  "output": [
    {
      "description": "Example description",
      "key": {
        "expr": "content.id"
      },
      "values": [
        {
          "description": "Example description"
        }
      ]
    }
  ]
}
```

Getting Started with the Integration API

Overview

To get started with the Integration API, use the built-in user interface in the [Integration API](#) section of the administration area. Here, you can:

- View, modify, and run preconfigured processors.
- Create and manage processors.
- Complete test runs without the risk of overwriting or deleting workspace data.
- Run processors to import or export data to and from your workspace.
- Get the data model of your workspace in JSON format, which includes the keys of meta model attributes.

Search by item name

To-Dos

Transformations

DISCOVERY AND INTEGRATIONS

API Tokens

Developer Tools

Integrations

Integration API

Metrics

Reference Catalog

Sync Logging

Technical Users

Webhooks

Integration API

New Processor Configuration

Search inside all processor configurations

Connector Type	Connector Id	Connector Version	Processing Direction	Processing Mode	Integrated Tool ⓘ	Last Time Used
Import Applications Example	--	--	--	--	--	--
Import Hierarchy Example	--	--	--	--	--	--
CMDB Connector Example	--	--	--	--	--	--
Import Links to Applications	leanix-pandasCSV	1.0.1	inbound	partial		2025-02-17 16:05:02
inboundDocument	inbound-document-connector	1.0.0	inbound	partial		2024-12-14 22:07:52

Integration API Page in the Administration Area

Key Concepts

- **Processor**: A processor defines an action to be executed on each content object of the incoming LDIF.

- **Connector:** A connector is not an entity within the Integration API but a middleware that creates an LDIF. It defines a set of meta properties the Integration API uses to bind incoming LDIF to a processor configuration. These properties are: `connectorType`, `connectorId`, and `connectorVersion`.
- **Processing direction:** The direction of data processing:
 - **Inbound:** Data is imported from an external system to SAP LeanIX in LDIF format. For more information, see [Inbound Processors](#).
 - **Outbound:** Data is exported from SAP LeanIX to an external system in LDIF format. For more information, see [Outbound Processors](#).
- **Processing Modes:** The mode of data processing:
 - **Full:** The connector expects the complete set of data. You can apply `deletionScope`, and any data not sent as part of the run can be archived from the workspace.
 - **Partial:** The connector expects a subset of data.
- **Integrated Tool:** The third-party system from which the data is imported or to which it's exported.

i Note

Even though you can use outbound processors, it's recommended to use the [writeToLdif](#) inbound processor to export data in LDIF format instead.

Preconfigured Processors

To get started, you can review the configuration of preconfigured processors marked with the lock icon on the [Integration API](#) page. These processors cover typical scenarios for importing data into your workspace:

- **Import Applications Example:** This processor imports application fact sheets with basic attributes, tags, subscriptions, and relations.
- **Import Hierarchy Example:** This processor imports business capability fact sheets with basic attributes and relation hierarchy, including parent and child fact sheets.
- **CMDB Connector Example:** This processor imports data from a CMDB to your workspace, including:
 - IT component, provider, and application fact sheets
 - Relations from IT components to applications and providers
 - Links added as resources (documents) on fact sheets

You can perform test runs for these processors and preview the results in the [Output Log](#) section. Use these examples as a basis to create your own processors by copying the configuration and adjusting the scope to your needs.

Search by item name

To-Dos

Transformations

DISCOVERY AND INTEGRATIONS

API Tokens

Developer Tools

Integrations

Integration API

Metrics

Reference Catalog

Sync Logging

Technical Users

Webhooks

Integration API

New Processor Configuration

Search inside all processor configurations

Connector Type	Connector Id	Connector Version	Processing Direction	Processing Mode	Integrated Tool ⓘ	Last Time Used
🔒 Import Applications Example	--	--	--	--	--	--
🔒 Import Hierarchy Example	--	--	--	--	--	--
🔒 CMDB Connector Example	--	--	--	--	--	--
Import Links to Applications	leanix-pandasCSV	1.0.1	inbound	partial		2025-02-17 16:05:02
inboundDocument	inbound-document-connector	1.0.0	inbound	partial		2024-12-14 22:07:52

Preconfigured Processors

Configuring a Processor

Follow these steps:

1. In the administration area, go to the **Integration API** section.
2. Click **New Processor Configuration**.
3. Specify the processor details, then click **Save**. Once you save the processor, you can add new versions and update the integrated tool. You can't change any other details.

← Create new Processor Configuration

Connector Type *

Import applications

Connector Id *

import-apps

Connector Version *

1.0.0

Processing Direction *

inbound

Processing Mode *

partial

Integrated Tool ⓘ *

None

Cancel

Create

- Creating a Processor
4. On the left side of the configuration page, define the processor configuration. You can add multiple processors.
 5. In the **Input** section, enter the following depending on the processor direction:
 - **Inbound**: Enter the data to be imported in LDIF format. For more details, see [LeanIX Data Interchange Format \(LDIF\)](#).
 - **Outbound**: To define the scope of data to export in LDIF format, in the upper-right corner, click the three-dot icon > **Define Scope**, then select fact sheets using inventory filters. The **scope** in the processor configuration on the left side is populated accordingly.
 6. Click **Save**.

After saving a processor, you can test and run it, as described in the following sections.

Import applications



1.0.0

Input

...

```
1 {
2   "processors": [
3     {
4       "processorType": "inboundFactSheet",
5       "processorName": "Create applications",
6       "processorDescription": "Creates application fact sheets",
7       "type": "Application",
8       "filter": {
9         "type": "Application"
10      },
11      "identifier": {
12        "external": {
13          "id": {
14            "expr": "${content.id}"
15          },
16          "type": {
17            "expr": "externalId"
18          }
19        }
20      },
21      "run": 0,
22      "updates": [
23        {
24          "key": {
25            "expr": "name"
26          },
27          "values": [
28            {
```

```
1 {
2   "connectorType": "Basic-ImportApplications",
3   "connectorId": "ImportApplications",
4   "connectorVersion": "1.0.0",
5   "lxVersion": "1.0.0",
6   "description": "",
7   "processingDirection": "inbound",
8   "processingMode": "full",
9   "customFields": {},
10  "content": [
11    {
12      "type": "Application",
13      "id": "APP-0001"
```

Output Log

```
1 {}
```

Save

Stop

Documentation

Preview

Test run

Run

Close

Processor Configuration Page

i Note

Processor configuration and data are strictly separated in the Integration API to ensure synchronization runs are reliable, repeatable, and auditable. Whenever you save the configuration of processors, only the processors are stored. The system does not store any LDIF data.

Testing a Processor

To verify the processor configuration, you can complete a test run. This doesn't apply any changes to your workspace.

To complete a test run, click **Test Run** at the bottom of the processor page. The results of the test run appear in the **Output Log** section. Review the output for possible errors and update the processor configuration or input as needed.

1.0.1

Input

```

1 {
2   "processors": [
3     {
4       "processorType": "inboundFactSheet",
5       "processorName": "Create applications",
6       "processorDescription": "Creates application fact sheets",
7       "type": "Application",
8       "filter": {
9         "type": "Application"
10      },
11      "identifier": {
12        "external": {
13          "id": {
14            "expr": "${content.id}"
15          },
16          "type": {
17            "expr": "externalId"
18          }
19        },
20      },
21      "run": 0,
22      "updates": [
23        {
24          "key": {
25            "expr": "name"
26          },
27          "values": [
28            {
29              "expr": "${data.name}"
30            }
31          ]
32        }
33      ]
34    }
35  ]
36 }

```

Save Stop Documentation

Preview **Test Run** Run

```

1 {
2   "connectorType": "Basic-ImportApplications",
3   "connectorId": "ImportApplications",
4   "connectorVersion": "1.0.0",
5   "lxVersion": "1.0.0",
6   "description": "",
7   "processingDirection": "inbound",
8   "processingMode": "full",
9   "customFields": {},
10  "content": [
11    {
12      "type": "Application",
13      "id": "APP-0001",
14      "data": {
15        "name": "AC Management",
16        "description": "AC Management description"
17      }
18    }
19  ]
20 }

```

Output Log

```

1 {
2   "results": null,
3   "warnings": [],
4   "debugInfo": [],
5   "debugVariables": [],
6   "statistics": {
7     "statusChanges": [
8       {
9         "status": "CREATED",
10        "timestamp": "2025-05-05T15:39:30.010764076Z"
11      },
12      {
13        "status": "PENDING",
14        "timestamp": "2025-05-05T15:39:30.112649592Z"
15      }
16    ]
17   }
18 }

```

Close

Testing a Processor and Previewing Results in the Output Log

Running a Processor

After successfully testing a processor, you can run it. This applies changes to your workspace, so always ensure to test your processors first.

To run a processor, click **Run** at the bottom of the processor page. The results appear in the **Output Log** section.

Saving New Processor Versions

You can update the processor configuration and save it as a new version without deleting the previous version. This allows you to maintain a history of changes.

Always version processors using the `number.number.number` format, such as `1.0.0`. This convention helps maintain the processor's version history and supports versioning during configuration updates.

To save a configuration to a new version, in the upper-right corner of the processor page, click the three-dot icon > **Save New Version**, then enter the version details and click **OK**.

Input

...

Save new version

Get Data Model

Delete configuration

```
1 {
2   "connectorType": "Basic-ImportApplications",
3   "connectorId": "ImportApplications",
4   "connectorVersion": "1.0.0",
5   "lxVersion": "1.0.0",
6   "description": "",
7   "processingDirection": "inbound",
8   "processingMode": "full",
9   "customFields": {},
10  "content": [
11    {
12      "type": "Application",
13      "id": "APP-0001"
```

Saving a New Processor Version

You can switch between the processor versions by using the drop-down list at the top of the processor page.

Import applications

1.0.1

1.0.1

1.0.0

Input

```
1 {
2   "processors": [
3     {
4       "processorType": "inboundFactSheet",
5       "processorName": "Create applications",
6       "processorDescription": "Creates application fact sheets",

```

Processor Versions

Getting the Data Model of Your Workspace

The data model of your workspace provides the meta model attributes in JSON format. Each fact sheet attribute is uniquely identified by an attribute key, also known as the technical name. To learn more about attribute keys in the meta model configuration, see [Attribute Key](#).

To get the data model of your workspace, in the upper-right corner of the processor page, click the three-dot icon > **Get Data Model**.

The data model in JSON format is returned in the **Output Log** section. You can copy attribute keys from here and use them in your configurations.

1.0.1

Input

...

```

1 {
2   "processors": [
3     {
4       "processorType": "inboundFactSheet",
5       "processorName": "Create applications",
6       "processorDescription": "Creates application fact sheets",
7       "type": "Application",
8       "filter": {
9         "type": "Application"
10      },
11      "identifier": {
12        "external": {
13          "id": {
14            "expr": "${content.id}"
15          },
16          "type": {
17            "expr": "externalId"
18          }
19        },
20      },
21      "run": 0,
22      "updates": [
23        {
24          "key": {
25            "expr": "name"
26          },
27          "values": [
28            {

```

```

1 {
2   "connectorType": "Basic-ImportApplications",
3   "connectorId": "ImportApplications",
4   "connectorVersion": "1.0.0",
5   "lxVersion": "1.0.0",
6   "description": "",
7   "processingDirection": "inbound",
8   "processingMode": "full",
9   "customFields": {},
10  "content": [
11    {
12      "type": "Application",
13      "id": "APP-0001"

```

Output Log

Close

```

1 {
2   "factSheets": {
3     "objective": {
4       "projectStatus": {
5         "values": [
6           "red",
7           "yellow",
8           "green"
9         ],
10        "type": "PROJECT_STATUS"
11      },
12      "objectiveLifecycle": {
13        "values": [
14          "planned",
15          "active",

```

Save

Stop

Documentation

Preview

Test run

Run

Save new version

Get Data Model

Delete configuration

Getting the Data Model of a Workspace

Managing Processors

- **Updating the configuration:** To update the processor configuration, make the necessary changes and then click **Save**. Note that the system doesn't save input data.
- **Updating the integrated tool:** You can update the integrated tool associated with the processor from the **Integration API** page.
- **Deleting a processor:** Be cautious with this action, as it cannot be undone. To delete a processor, click the three-dot icon in the upper-right corner of the processor page, then select **Delete Configuration**.

Inbound Processors

Overview

The integration API reads provided [LDIF](#) and for each data object, it walks over all the configured Data Processors and checks if the filter configured for each Data Processor matches. In case of a match, the Processor executes the transformation of data and writes to an SAP LeanIX entity.

For each data object in the content section of the LDIF, inbound Data Processors, depending on the type of Data Processor, create Fact Sheets, Relations, Subscriptions, Metrics and Links (Resources tab of Fact Sheets formerly known as Documents). This includes setting and updating values in fields from certain data keys and values in the data object.

Extended to fully support various sources and keep the connector code simple. SAP LeanIX provides a powerful mapping component that allows to (partly) map, add and combine information from multiple metadata elements and/or type or id. Creation of a relation that depends on certain key value pairs or keys (in case of simple tags), as well as combining input from different tags is supported. The configuration even allows to set fix values in certain fields to cover cases where not all data points in the source system have values.

Each Data Object will potentially be processed by multiple Data Processors (in case filters of multiple Data Processors match). To prevent data inconsistency in the processing run like creating a relation to a not yet existing object it is possible to order the execution by assigning each Data Processor a numeric run key, e.g. run : 0 when creating a Fact Sheet, run : 1 when creating a relation to the new Fact Sheet.

i Note

The processor name should not include any reserved characters which are as operators e.g. +,*,>,<= ...

Types of Inbound Data Processors

Inbound Data Processor Type	General	Details
inboundFactSheet	Is used to manage fact sheets (create, update, delete). Example configuration can be found in the Admin UI.	The configuration contains an additional "type" key to define the target Fact Sheet Type to create/update. The configuration needs to provide the name of the Fact Sheet to create and the external ID in case of updating a Fact Sheet. The following field types in Fact Sheet fields can be updated (using the update section): STRING, SINGLE_SELECT, MULTIPLE_SELECT, DOUBLE, INTEGER, LOCATION, LIFECYCLE, EXTERNALID, PROJECT, MILESTONE, FACTSHEETSTATE Changing the Data Processor mode to "delete" will mark the Fact Sheet "archived", so behave the same way as if users selected "Delete" from the UI.
inboundRelation	Is used to manage relations between Fact Sheets (create, update, delete). Example configuration can be found in the Admin UI.	The configuration needs to provide an external or internal ID of the two Fact Sheets that need to be connected. The type of the relation needs to be provided. Fields configured for the relation can be updated. Supported types are the same as for the inboundFactSheet data processor. ActiveFrom and activeUntil can as well be updated. The expected date format is following the ISO 8601 format (e.g. "2019-08-02T09:03:49+00:00").
inboundSubscription	Is used to create, update, or delete subscriptions on fact sheets.	The processor adds a new subscription to a fact sheet. This doesn't apply to subscriptions of the Accountable type, for which the processor doesn't replace the current subscription. To add a new subscription of the Accountable type, delete the existing subscription first. You can delete subscriptions in one of the following ways:

Inbound Data Processor Type	General	Details
		- Using the Integration API: See Deletion of Subscriptions. - Using the GraphQL API: See Delete a Fact Sheet Subscription. - In the application UI: Navigate to the Subscriptions tab on a fact sheet and delete the subscriptions that you need. Variables to be set in the output section: "user" (mandatory), "subscriptionType" (mandatory), "subscriptionRoles" (optional, depending on workspace configuration), "comment" (optional, will be ignored if no role given).
inboundDocument	Is used to create, update or delete documents linked to Fact Sheets (create, update, delete). The structure is the same as for the inboundFactSheet data processor. Same matching logic for Fact Sheets applies. The found Fact Sheet will not be modified but a linked document changed according to mode (default is "createOrUpdate").	The updates section must contain a key "name" or the processor will fail. Other potential keys to be set are "description", "url", "origin" to complete the information for a document that is linked to a Fact Sheet.
inboundTag	Is used to manage Tag Groups (create, update), Tags (create, update, delete) and assignments from Tags to Fact Sheets (create). Example configuration can be found in the Admin UI.	Tags can only be removed completely currently. There is no way to remove tags for given Fact Sheets only. If removal of flags is wanted, a Fact Sheet processor in run 0 needs to be created to delete the tags. In run 1 a processor can then add them back in. If no Tag Group is provided, the processor assumes the default Tag Group ("Other tags"). The processor automatically removes all Fast Sheet assignments if a Tag is deleted.
inboundMetrics	Is used to write a single new point to a configured metrics endpoint.	Metrics can store time series data in SAP LeanIX and become interactive charts to display data. In addition the data can be linked to Fact Sheets, presented to the Dashboard or be displayed within the "Reports" area.
inboundImpact	Is used to write BPM impacts using integration API.	All standard integration API functionality is available. A processor always writes a single impact. In case a list of input data is given where "forEach" can iterate over, multiple impacts can be written by a single processor. Different types of impacts should be split into different processors in order to keep a configuration readable as different impacts have different parameters.

Inbound Data Processor Type	General	Details
inboundToDo	Is used to write To-do using integration API.	

Execution of the Started iAPI Runs

When triggering the start of an iAPI run via API call, the Integration API scheduled the run to be executed next when an available processing slot becomes available. While the Integration API service is able to scape on demand, there may be heavy load where the underlying system needs to be protected to be still available for standard user operations.

Integration API tries to add as much robustness to the process as possible. Most temporary outages of underlying infrastructure is covered and in case of issues, iAPI continues at the last "save point". Save points are generated behind the scenes and ensure, not all work has to be done again in case of a temporary failure.

When the customer calls a specific iAPI configuration multiple times, the order of execution is not guaranteed. The second call might be processed first and in case of a failure, a newer execution might be picked up first before the retry of the older one starts.

Configuring the Order of Runs

For cases where the order of execution is important, administrators can add a key/value pair to the root path of the iAPI configuration to ensure that there is only one run for the same configuration executed at a time, and in case of failures, the first triggered run is always picked up first.

While this might be mandatory if an external system sends updates to same information shortly after each other, it may lead to small updates having to wait for big ones.

In case you want to ensure sequential processing in order of starting time of a run, please specify the below key value pair in your configuration

Example of using the `sequentialExecution` attribute for a sequential execution of runs:

```
{
  "processors": [...],
  "sequentialExecution": true
}
```

General Structure

Filters

Filter section is where you define if the Data Processor should work on the found data object.

i Note

Data Processors provide filter capabilities to configure on which Data Object the data processor will work on (match the filter) and what Data Objects to skip (not match the filter)

Types of filters that can be configured:

Filter Type	Details
exactType (type)	The exactType and type filters work differently when comparing strings with the type field of a data object.

Filter Type	Details
	The <code>exactType</code> filter requires an exact match between the string and the <code>type</code> field. This means the entire string must match the <code>type</code> field for the filter to be applied. We recommend using this filter type to ensure accurate matching. The <code>type</code> filter performs a comparison based on the first characters of the string, following the logic of a regular expression. This means the filter is applied if the initial characters of the string match the <code>type</code> field of the data object, without considering the remaining characters.
<code>id</code>	If configured, the string is interpreted as a regular expression and matched against the "id" field of the Data Object
<code>advanced</code>	If configured, the field contains a JUEL expression that may evaluate to "true" for a match or "false". This filter allows to filter even for combinations of certain key and values in the Data Object
<code>onRead</code>	Behaves like the advanced filter but uses results of eventually configured read sections to filter based on existence of a Fact Sheet or based on specific values on an existing Fact Sheet. You can find an example of how to use this filter in Advanced Use Cases for the Integration API .
<code>writeToLdif</code>	Using this processor, Administrators can configure an inbound Integration API run to write a new LDIF file. The resulting LDIF will be available using the <code>/results</code> and the <code>/resultsUrl</code> endpoints same as with outbound Integration API runs
<code>updatedInDuration</code>	Used to filter on items that have changed recently. Integration API can target Fact Sheets that have been changed recently. Only Fact Sheets that pass this criteria will be processed. The feature is most helpful to generate proper output with the "writeToLdif" processor that only contains e.g. Fact Sheets changed since last export. It is used the following way: "updatedInDuration": "P3D". You can find an example in Outbound Processors .

i Note

All configured filters need to match in order to start the Data Processor on the Data Object (AND logic).

Example of combining multiple filters:

```
{
  "filters": {
    "exactType": "ITComponent",
    "updatedInDuration": "P3D"
  }
}
```

Identifier Section

Identifier section defines the pathfinder entity in scope of the processor. Depending on the processor it can be called "identifier" (all processors with one Fact Sheet in scope) or "from" and "to" for the inboundRelation processor.

Identification of the target Fact Sheet happens by defining the internal ID, the external ID or a "search scope".

Only one field must be filled as a value of key "identifier":

- `internalId`: JUEL expression, replace RegEx

Example of identification by an internal ID:

```
{
  "identifier": {
    "internal": "${content.id}"
  }
}
```

- `externalId`: JUEL expression, replace RegEx (id/name of Fact Sheet or other entity)

Example of Identification by an external ID:

```
{
  "identifier": {
    "external": {
      "id": {
        "expr": "${content.id}"
      },
      "type": {
        "expr": "externalId"
      }
    }
  }
}
```

Using the external key, it is possible to create an object in case it is not found. This happens transparently without any need to distinguish between create or update when configuring the processor.

When using the "search" based identification of the Fact Sheet that are supposed to be updated by the incoming data object, then the section may contain a section to limit the scope of searched Fact Sheets and an expression filtering the Fact Sheets that should be updated. Details can be found on the "Advanced" page of this documentation.

The below processor will update all descriptions of Application Fact Sheets that have a tag "AsiaPacific" in the tag group "Region". The full example can be found on the "Advanced" page.

Example processor for identifying Fact Sheets using search and updating them with incoming data:

```
{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Update all Cloud Apps",
      "processorDescription": "Updates all Apps with tag 'Cloud'",
      "type": "Application",
      "filter": {
        "exactType": "AppUpdate"
      }
    },
  ],
}
```

```

"identifier": {
  "search": {
    "scope": {
      "facetFilters": [
        {
          "facetKey": "FactSheetTypes",
          "operator": "OR",
          "keys": [
            "Application"
          ]
        },
        {
          "facetKey": "${integration.tags.getTagGroupId('Region')}",
          "operator": "OR",
          "keys": [
            "${integration.tags.getTagId('Region','AsiaPacific')}"
          ]
        }
      ],
      "ids": []
    },
    "filter": "${true}",
    "multipleMatchesAllowed": true
  }
},
"logLevel": "debug",
"updates": [
  {
    "key": {
      "expr": "description"
    },
    "values": [
      {
        "expr": "External sync executed ${data.dateTime}"
      }
    ]
  }
]
}

```

Update Section

The Update Section provides the ability to write to fields or further metadata to the targeted entity (depending on the processor).

Multiple values can be written. Each value consists of a JUEL and a RegEx for building the name of the target key to be written and a list of potential values to be written.

Some keys might be mandatory depending on the processor (see the processor description for details).

The following field types in Fact Sheet fields can be updated (using the update section):

Attribute	Description
STRING	Is a basic text field with no functionality. This field has no configurable formatting like displaying clickable links or bold formatting
SINGLE_SELECT	Allows for the selection of one value from a dropdown list. This list of values can be changed at any point in time without data loss. This attribute can be filtered in the inventory and used as a view in the reports
MULTIPLE_SELECT	Allows for the selection of multiple values from a predefined list. Once defined this list cannot be changed again without incurring data loss
DOUBLE	There is no explicit currency field in SAP LeanIX, but this type can display a currency icon
INTEGER	Represents a numeric value without decimal places
LOCATION	Will be sent to the location service to resolve a valid location from the given input string. Setting the location will fail if the given data is not specific and results in multiple possible locations. In case a "#" is found as the first one, the API will pick the first result returned by the location service and use this. This is helpful if comma separated coordinates are being provided
LIFECYCLE	Content needs to follow the date format specifications "yyyy-mm-dd". Each field in the life cycle can be addressed with "."-syntax like e.g. lifecycle.active
EXTERNALID	External Ids can only be written if they are not marked readonly in the data model. The following fields can be written using "."-syntax: externalId.externalId, externalId.externalUrl, externalId.comment, externalId.status, externalId.externalVersion. The "externalId" left of the "." may be changed with the name of the external id field.
PROJECT	Project status values will always written as a full set that replaces the currently set project status values. In order to add to existing values, you need to add the field to the read section. This returns an object you would then iterate over using inner forEach (see advanced section for usage). While iterating a filter could be applied to not write back all found status values but selected only. In addition by defining more values new values can be added in the same step. The structure of the required map can be copied from a read result (e.g. output to a description field for testing): <pre> { "updates": [{ "key": { "expr": "projectStatus" }, "values": [{ "map": [{ "key": "id", "value": "myId" }, { "key": "date", "value": "2020-07-25" }, { "key": "status", "value": "green" }] }] }] } </pre>

Attribute	Description
	<pre> }, { "key": "progress", "value": "20" }] }] }] }] } </pre>
FACTSHEETSTATE	Integration API reads and writes the content of a FactSheetState field as a String, which makes it easy to handle. Please be aware to only send allowed values into a state field.
MILESTONE	Milestones can be written following the same logic as "project" data is written by the Integration API. All content will just replace the current content by default. iAPI takes care to ensure the final state in Pathfinder reflects what was sent in the processor. There is no need to decide if items need to be deleted or modified or added new. The data sent by iAPI will always reflect the full state after writing. This allows to process without knowledge of the current state and does not interfere with potential other operations happening in parallel. <pre> { "key": { "expr": "lxMilestones" }, "values": [{ "forEach": { "elementOf": "\${data.milestones}", "filter": "\${true}" }, "map": [{ "key": "name", "value": "\${integration.output.valueOfForEach.n}" }, { "key": "date", "value": "\${integration.output.valueOfForEach.d}" }, { "key": "description", "value": "\${integration.output.valueOfForEach.type}" }] }] } </pre>

Example updates section for an inbound data processor:


```

{
  "updates": [
    {
      "key": {
        "expr": "name",
        "regexReplace": {
          "match": "",
          "replace": ""
        }
      },
      "values": [
        {
          "expr": "${data.app}"
        }
      ]
    },
    {
      "key": {
        "expr": "description"
      },
      "values": [
        {
          "expr": "${header.processingMode}",
          "regexMatch": "abc"
        },
        {
          "expr": "${header.processingMode}_2"
        }
      ],
      "optional": true
    }
  ]
}

```

Modes

Changing the Data Processor mode to "delete" will mark the Fact Sheet "archived", so behave the same way as if users selected "Delete" from the UI.

Delete mode:

```

{
  "mode": "delete"
}

```

Example connector and input data using the delete mode:

```

{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Delete Data sent in the Input",
      "processorDescription": "",

```

```

    "type": "Application",
    "filter": {
      "exactType": "Deployment"
    },
    "identifier": {
      "external": {
        "id": {
          "expr": "${content.id}"
        },
        "type": {
          "expr": "externalId"
        }
      }
    },
    "updates": [],
    "mode": "delete"
  }
],
"variables": {}
}

```

Example input:

```

{
  "connectorType": "example",
  "connectorId": "deleteMode",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "description": "Delete Data using mode",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "Deployment",
      "id": "634c16bf-198c-1129-9d08-92630b573fbf",
      "data": {
        "app": "HR Service",
        "version": "1.8.4",
        "maturity": "3",
        "clusterName": "westeurope",
        "tags": []
      }
    }
  ]
}

```

In case a Fact Sheet is updated with a standard mode that has been set to "archived", there are two potential behaviors:

- In case the Fact Sheet was matched using the external ID, then a new Fact Sheet will be created
- In case the reference was done by using the internal ID, then the old Fact Sheet will be used and set back to "active"

Inbound Fact Sheet

Example inboundFactSheet processor:

```
{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Create IT Components",
      "processorDescription": "One Processor for IT Components",
      "enabled": true,
      "type": "ITComponent",
      "identifier": {
        "external": {
          "id": {
            "expr": "${content.id.replaceAll('/', '_')}"
          },
          "type": {
            "expr": "externalId"
          }
        }
      },
    },
    "filter": {
      "exactType": "ITComponent"
    },
    "updates": [
      {
        "key": {
          "expr": "name"
        },
        "values": [
          {
            "expr": "${data.name}"
          }
        ]
      },
      {
        "key": {
          "expr": "cloudProvider"
        },
        "values": [
          {
            "expr": "${data.provider}"
          }
        ]
      },
      {
        "key": {
          "expr": "category"
        },
        "values": [
          {
            "expr": "${data.category}",
            "regexMatch": "(cloud_service)",

```

```

        "regexReplace": {
            "match": "^.*$",
            "replace": "cloudService"
        }
    },
    {
        "expr": "${data.category}",
        "regexMatch": "(sample_software)",
        "regexReplace": {
            "match": "^.*$",
            "replace": "software"
        }
    }
]
}
],
"vars": []
}
]
}

```

Lifecycle Management

Writing to fields of type lifecycle needs to be split into different write operations (lines in the data processor. The value of the "key" field has to use the "." syntax. E.g. "lifecycle.plan", "lifecycle.phaseIn". Other default values are "phaseOut" and "endOfLife".

Example inboundFactSheet processor with life cycle data:

```

{
    "processorType": "inboundFactSheet",
    "processorName": "Lifecycle Example",
    "processorDescription": "Creates an Application with lifecycle information",
    "type": "Application",
    "filter": {
        "exactType": "Application"
    },
    "identifier": {
        "external": {
            "id": {
                "expr": "${content.id}"
            },
            "type": {
                "expr": "externalId"
            }
        }
    },
    "run": 0,
    "updates": [
        {
            "key": {
                "expr": "name"
            },
            "values": [

```

```

    {
      "expr": "${data.name}"
    }
  ],
  {
    "key": {
      "expr": "description"
    },
    "values": [
      {
        "expr": "${data.name} is an application that carries lifecycle information"
      }
    ]
  },
  {
    "key": {
      "expr": "lifecycle.plan"
    },
    "values": [
      {
        "expr": "${data.plan == null ? '2014-01-01' : data.plan}"
      }
    ]
  },
  {
    "key": {
      "expr": "lifecycle.phaseIn"
    },
    "values": [
      {
        "expr": "${data.phaseIn}"
      }
    ]
  },
  {
    "key": {
      "expr": "lifecycle.active"
    },
    "values": [
      {
        "expr": "${data.active}"
      }
    ]
  },
  {
    "key": {
      "expr": "lifecycle.phaseOut"
    },
    "values": [
      {
        "expr": "${data.phaseOut}"
      }
    ]
  }
]

```

```

    },
    {
      "key": {
        "expr": "lifecycle.endOfLife"
      },
      "values": [
        {
          "expr": "${data.endOfLife}"
        }
      ]
    }
  ]
}

```

Location Management

Writing to fields of type "location" will require a single string as input. The string will be sent to the location service (open street map). In case a single result was returned, the location will be written to the field with all meta data returned by "open street map". Providing latitudes and longitude works by simply passing the coordinates in that order, separated by comma: "50.11, 8.682".

In case of no or multiple locations returned, the field will not be populated and an error shown in the log for this field. Other updates by the data processor may still be valid and pass.

i Note

When writing locations, the used open street map service may return multiple results. Default behaviour is to not set any location. In case the value provided to the Location starts with a # character, the first result from open street map will be used (same logic as we see when providing coordinates)

Inbound Subscription

Variables to be set in the output section:

Variable	Required	Notes
user	Required	Users' email (either user or newUser" needs to be present.
newUser	Required	Works like "user" but creates a new user if not existing
subscriptionType	Required	
subscriptionRoles	Optional	It may or may not be required because it is based on the specific configuration set in each workspace
addSubscriptionRoles	Optional	Same as "subscriptionRoles" but adds to existing roles instead of completely replacing all existing roles
optional	Optional	Boolean(Disables warnings related to new user creation in case inboundSubscription only works for existing users)

Variable	Required	Notes
comment	Optional	

Example inboundSubscription processor:

```
{
  "processorType": "inboundSubscription",
  "processorName": "Subscription creation",
  "processorDescription": "Creates subscriptions",
  "filter": {
    "exactType": "ITComponent"
  },
  "identifier": {
    "external": {
      "id": {
        "expr": "${content.id}"
      },
      "type": {
        "expr": "externalId"
      }
    }
  },
  "updates": [
    {
      "key": {
        "expr": "user"
      },
      "values": [
        {
          "expr": "jane.doe@leanix.net"
        }
      ]
    },
    {
      "key": {
        "expr": "subscriptionType"
      },
      "values": [
        {
          "expr": "RESPONSIBLE"
        }
      ]
    },
    {
      "key": {
        "expr": "subscriptionRoles"
      },
      "values": [
        {
          "map": [
            {
              "key": "roleName",
```

```

        "value": "Business Owner"
      },
      {
        "key": "comment",
        "value": "This person is the business owner"
      }
    ]
  }
]
},
{
  "key": {
    "expr": "newUser.userName"
  },
  "values": [
    {
      "expr": "jane.doe@leanix.net"
    }
  ]
},
{
  "key": {
    "expr": "newUser.email"
  },
  "values": [
    {
      "expr": "jane.doe@leanix.net"
    }
  ]
},
{
  "key": {
    "expr": "newUser.firstName"
  },
  "values": [
    {
      "expr": "Jane"
    }
  ]
},
{
  "key": {
    "expr": "newUser.lastName"
  },
  "values": [
    {
      "expr": "Doe"
    }
  ]
}
]
}

```


Inbound Relation

The "inboundRelation" processor requires identification of two Fact Sheets. In this processor the "identifier" is replaced by two fields named "from" and "to". The potential values of the "from" and the "to" fields are identical with the "identifier" values and can handle internal and external ids as well.

Allowed values: JUEL expression plus optional replace RegEx map (available for each expression, for internal external and from and to in case of the inboundRelation processor).

Example RegEx in ID mapping:

```
{
  "identifier": {
    "external": {
      "id": {
        "expr": "${content.id}",
        "regexReplace": {
          "match": "",
          "replace": ""
        }
      },
    },
    "type": {
      "expr": "externalId"
    }
  }
}
```

Please replace the type "relApplicationToITComponent" with the name of the relation that needs to be created or updated (e.g. "relToParent").

Example inboundRelation processor:

```
{
  "processors": [
    {
      "processorType": "inboundRelation",
      "processorName": "Rel from Apps to ITComponent",
      "processorDescription": "Creates LeanIX Relations between the created or updated Application:",
      "type": "relApplicationToITComponent",
      "filter": {
        "exactType": "Deployment"
      },
      "from": {
        "external": {
          "id": {
            "expr": "${content.id}"
          },
        },
        "type": {
          "expr": "externalId"
        }
      }
    },
  ],
}
```

```

    "to": {
      "external": {
        "id": {
          "expr": "${data.clusterName}"
        },
        "type": {
          "expr": "externalId"
        }
      }
    },
    "run": 1,
    "updates": [
      {
        "key": {
          "expr": "description"
        },
        "values": [
          {
            "expr": "Relationship Description"
          }
        ]
      },
      {
        "key": {
          "expr": "activeFrom"
        },
        "values": [
          {
            "expr": "2019-08-02T09:03:49+00:00"
          }
        ]
      },
      {
        "key": {
          "expr": "activeUntil"
        },
        "values": [
          {
            "expr": "2020-08-02T09:03:49+00:00"
          }
        ]
      }
    ],
    "logLevel": "debug"
  }
]
}

```

i Note

Referencing "from" and "to" fact sheets for relations by internal IDs.

The inboundRelation processor as well supports referencing source and target Fact Sheets by their internal id as well. The syntax is the same we see for the identifier for inboundFactsheet processor: "internal": "\${content.id}"

i Note

To create a Fact Sheet using the inboundFactsheet processor, providing an externalId is mandatory.

Inbound Relations Constraints

The relation processor allows to set constraining relations as well. In order to do so, a target key "constrainingRelations" needs to be defined (in the output section similar to the example target key "description in the example above"). All values of the resulting values list will be written as constraints. Existing ones will be removed. Alternatively the key "addConstrainingRelations" may be used to add constraints to existing ones.

Example configuration to read information of constraining relations from an LDIF input:

```
{
  "key": {
    "expr": "constrainingRelations"
  },
  "values": [
    {
      "forEach": {
        "elementOf": "${integration.valueOfForEach.rels.constrainingRelations.relations}"
      },
      "map": [
        {
          "key": "type",
          "value": "${integration.output.valueOfForEach.type}"
        },
        {
          "key": "targetExternalIdType",
          "value": "externalId"
        },
        {
          "key": "targetExternalIdValue",
          "value": "${integration.output.valueOfForEach.target.externalId}"
        }
      ]
    }
  ]
}
```

Example configuration to generate an LDIF output with information about constraining relations:

```
{
  "scope": {
    "ids": [
      "7750c7ba-5d24-4849-a1b4-564bc6c874a0"
    ],
    "facetFilters": [
      {
        "keys": [
          "Application"
        ],
        "facetKey": "FactSheetTypes",

```

```

      "operator": "OR"
    }
  ]
},
"processors": [
  {
    "processorType": "outboundFactSheet",
    "processorName": "Export to LDIF",
    "processorDescription": "This is an example how to use the outboundFactSheet processor",
    "enabled": true,
    "fields": [
      "lifecycle",
      "name",
      "location",
      "createdAt",
      "technicalSuitabilityDescription",
      "description"
    ],
    "relations": {
      "filter": [
        "relApplicationToProcess"
      ],
      "fields": [
        "description"
      ],
      "targetFields": [
        "displayName",
        "externalId"
      ],
      "constrainingRelations": true
    },
    "output": [
      {
        "key": {
          "expr": "content.id"
        },
        "mode": "selectFirst",
        "values": [
          {
            "expr": "${lx.factsheet.id}"
          }
        ]
      },
      {
        "key": {
          "expr": "content.type"
        },
        "mode": "selectFirst",
        "values": [
          {
            "expr": "${lx.factsheet.type}"
          }
        ]
      }
    ]
  },

```

```
{
  "key": {
    "expr": "Name"
  },
  "values": [
    {
      "expr": "${lx.factsheet.name}"
    }
  ],
  "optional": true
},
{
  "key": {
    "expr": "relations"
  },
  "mode": "list",
  "values": [
    {
      "forEach": {
        "elementOf": "${lx.relations}",
        "filter": "${true}"
      },
      "map": [
        {
          "key": "relationName",
          "value": "${integration.output.valueOfForEach.type}"
        },
        {
          "key": "object",
          "value": "${integration.output.valueOfForEach}"
        }
      ]
    }
  ]
}
]
```

Inbound Metrics

The `inboundMetrics` processor is used to write a single new point to a configured metrics endpoint.

The `update` section of the processor must contain the following keys with values:

Required Attribute	Description
measurement	Name of the configured metrics measurement a points needs to be added to
time	The date and time of the point in ISO formatting (e.g. 2019-09-09T08:00:00.000000Z)

Required Attribute	Description
fieldKey	Name of the field to store the point value in
fieldValueNumber	The value you want to store for that field and point of time
tagKey	Name of the tag
tagValue	Value of the tag. You may want to write the internal ID of a specific Fact Sheet here to allow assignment of the data to a specific Fact Sheet as a rule in the created chart for the measurement (go to admin/metrics to configure)

The output section of the inboundMetrics data processor should be configured the same as other inbound processors. The keys will be written to the corresponding variables.

Example inboundMetrics processor:

```
{
  "processorType": "inboundMetrics",
  "processorName": "Metrics data for measurement",
  "processorDescription": "Metrics processor configuration",
  "filter": {
    "exactType": "Metrics",
    "advanced": "${data.measurement.equals('measurement')}}"
  },
  "run": 1,
  "updates": [
    {
      "key": {
        "expr": "measurement"
      },
      "values": [
        {
          "expr": "${data.measurement}"
        }
      ]
    },
    {
      "key": {
        "expr": "time"
      },
      "values": [
        {
          "expr": "${data.time}"
        }
      ]
    },
    {
      "key": {
        "expr": "fieldKey"
      },
      "values": [
        {
          "expr": "${data.fieldKey}"
        }
      ]
    }
  ]
}
```

```

    }
  ]
},
{
  "key": {
    "expr": "fieldValueNumber"
  },
  "values": [
    {
      "expr": "${data.fieldValueNumber}"
    }
  ]
},
{
  "key": {
    "expr": "tagKey"
  },
  "values": [
    {
      "expr": "${data.tagKey}"
    }
  ]
},
{
  "key": {
    "expr": "tagValue"
  },
  "values": [
    {
      "expr": "${data.tagValue}"
    }
  ]
},
{
  "key": {
    "expr": "tags"
  },
  "values": [
    {
      "map": [
        {
          "key": "key",
          "value": "${data.tagKey}_1"
        },
        {
          "key": "value",
          "value": "${data.tagValue}_1"
        }
      ]
    }
  ],
  {
    "map": [
      {
        "key": "key",

```

```
        "value": "${data.tagKey}_2"
      },
      {
        "key": "value",
        "value": "${data.tagValue}_2"
      }
    ]
  }
]
}
],
"logLevel": "debug"
}
```

Inbound Document

The `inboundDocument` processor is used to create, update, or delete documents linked to Fact Sheets.

The structure is the same as for the `inboundFactSheet` data processor. Same matching logic for Fact Sheets applies. The found Fact Sheet will not be modified but a linked document changed according to mode (default is "createOrUpdate")

Keys Specific to the <code>inboundDocumentProcessor</code>	Details
<code>description</code>	Description of the document
<code>origin</code>	From what department or person does this originate from
<code>url</code>	Link to the document
<code>documentType</code>	A string containing information how to display the link on the Resource tab. Values are dynamic. It is suggested to first read the links for an item, then copy the values for writing similar links. Some examples of the value but this can change based on your specific configuration: <code>policy</code> , <code>decision</code> , <code>jira</code> , <code>documentation</code> , <code>website</code> , <code>support_ticket</code> , <code>faq</code> , <code>additional_help</code> , <code>task</code> , <code>roadmap</code>
<code>metadata</code>	A string containing information how to display the link on the Resource tab. Values are dynamic. It is suggested to first read the links for an item, then copy the values for writing similar links

 Caution

The `updates` section must contain the `key name`, otherwise the run will fail.

Example `inboundDocument` processor:

```
{
  "processorType": "inboundDocument",
  "processorName": "My link to Integration API docs",
  "processorDescription": "Contains the link that will point to the documentation for the Integrat:
  "identifier": {
    "external": {
      "id": {
```



```

      "expr": "${content.id}"
    },
    "type": {
      "expr": "externalId"
    }
  },
  "filter": {
    "exactType": "ITComponent"
  },
  "run": 1,
  "updates": [
    {
      "key": {
        "expr": "name"
      },
      "values": [
        {
          "expr": "Integration API Document"
        }
      ]
    },
    {
      "key": {
        "expr": "documentType"
      },
      "values": [
        {
          "expr": "website"
        }
      ]
    },
    {
      "key": {
        "expr": "origin"
      },
      "values": [
        {
          "expr": "CUSTOM_LINK"
        }
      ]
    },
    {
      "key": {
        "expr": "url"
      },
      "values": [
        {
          "expr": "https://docs-eam.leanix.net/reference/integration-api"
        }
      ]
    }
  ]
}

```

Inbound Tag

Tag Sent as an Array

In the below example if you just specify the name of the tag without other attributes the Tag by the name specified will be created under "Other Tags" and attached to the Fact Sheet.

Example inboundTag processor:

```
{
  "processorType": "inboundTag",
  "processorName": "Tag creation",
  "processorDescription": "Creates tags and tag groups",
  "factSheets": {
    "external": {
      "ids": "${content.id}",
      "type": {
        "expr": "externalId"
      }
    }
  },
  "run": 1,
  "updates": [
    {
      "key": {
        "expr": "name"
      },
      "values": [
        {
          "expr": "${integration.valueOfForEach}"
        }
      ]
    },
    {
      "key": {
        "expr": "description"
      },
      "values": [
        {
          "expr": "${integration.valueOfForEach}"
        }
      ]
    },
    {
      "key": {
        "expr": "color"
      },
      "values": [
        {
          "expr": "#123456"
        }
      ]
    }
  ],
  {
```

```

    "key": {
      "expr": "group.name"
    },
    "values": [
      {
        "expr": "Kubernetes Tags"
      }
    ]
  },
  {
    "key": {
      "expr": "group.shortName"
    },
    "values": [
      {
        "expr": "k8s"
      }
    ]
  },
  {
    "key": {
      "expr": "group.description"
    },
    "values": [
      {
        "expr": "Tags relevant for Kubernetes"
      }
    ]
  },
  {
    "key": {
      "expr": "group.mode"
    },
    "values": [
      {
        "expr": "MULTIPLE"
      }
    ]
  },
  {
    "key": {
      "expr": "group.restrictToFactSheetTypes"
    },
    "values": [
      {
        "expr": "Application"
      },
      {
        "expr": "ITComponent"
      }
    ]
  }
],
"forEach": "${data.tags}",

```

```
"logLevel": "debug"
}
```

Example input in LDIF format for importing tags:

```
{
  "connectorType": "ee",
  "connectorId": "Kub Dev-001",
  "connectorVersion": "1.2.0",
  "lxVersion": "1.0.0",
  "description": "Imports kubernetes data into LeanIX",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "Deployment",
      "id": "784616bf-198c-11f9-9da8-9263b0573fbe",
      "data": {
        "app": "Finance Service",
        "version": "10.5",
        "maturity": "5",
        "clusterName": "westeurope",
        "tags": [
          "Important"
        ]
      }
    }
  ]
}
```

Example input in LDIF format for importing tag groups and tags:

```
{
  "connectorType": "Report Technology Radar",
  "connectorId": "Technology Radar Tags",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "Deployment",
      "id": "1",
      "data": {
        "taggroups": [
          {
            "name": "Technology radar - Quadrant",
            "shortname": "TRQ",
            "description": "Beschreibung Quadrant",
            "mode": "SINGLE",
            "factsheettype": "ITComponent"
          }
        ]
      }
    }
  ]
}
```

```

    },
    {
      "name": "Technology radar - Ring",
      "shortname": "TRR",
      "description": "Beschreibung Ring",
      "mode": "SINGLE",
      "factsheettype": "ITComponent"
    }
  ],
  "tags": [
    {
      "groupname": "Technology radar - Quadrant",
      "name": "Architecture Concepts",
      "description": "Beschreibung Architecture Concepts",
      "color": "#ff0000"
    },
    {
      "groupname": "Technology radar - Quadrant",
      "name": "Platforms",
      "description": "Beschreibung Platforms",
      "color": "#00ff00"
    },
    {
      "groupname": "Technology radar - Quadrant",
      "name": "Techniques",
      "description": "Beschreibung Techniques",
      "color": "#0000ff"
    },
    {
      "groupname": "Technology radar - Quadrant",
      "name": "Tools & Infrastructure",
      "description": "Beschreibung Tools & Infrastructure",
      "color": "#000000"
    },
    {
      "groupname": "Technology radar - Ring",
      "name": "Hold",
      "description": "Beschreibung Hold",
      "color": "#ff0000"
    },
    {
      "groupname": "Technology radar - Ring",
      "name": "Incubating",
      "description": "Beschreibung Incubating",
      "color": "#00ff00"
    },
    {
      "groupname": "Technology radar - Ring",
      "name": "Emerging",
      "description": "Beschreibung Emerging",
      "color": "#0000ff"
    },
    {
      "groupname": "Technology radar - Ring",

```



```

    },
    {
      "key": {
        "expr": "group.mode"
      },
      "values": [
        {
          "expr": "${integration.valueOfForEach.mode}"
        }
      ]
    },
    {
      "key": {
        "expr": "group.restrictToFactSheetTypes"
      },
      "values": [
        {
          "expr": "${integration.valueOfForEach.factsheettype}"
        }
      ]
    }
  ],
  "logLevel": "warning",
  "enabled": true
},
{
  "processorType": "inboundTag",
  "processorName": "Tag creation",
  "processorDescription": "Creates tags",
  "run": 1,
  "forEach": "${data.tags}",
  "updates": [
    {
      "key": {
        "expr": "group.name"
      },
      "values": [
        {
          "expr": "${integration.valueOfForEach.groupname}"
        }
      ]
    },
    {
      "key": {
        "expr": "name"
      },
      "values": [
        {
          "expr": "${integration.valueOfForEach.name}"
        }
      ]
    }
  ],
  {
    "key": {

```

```

        "expr": "description"
      },
      "values": [
        {
          "expr": "${integration.valueOfForEach.description}"
        }
      ]
    },
    {
      "key": {
        "expr": "color"
      },
      "values": [
        {
          "expr": "${integration.valueOfForEach.color}"
        }
      ]
    }
  ],
  "logLevel": "warning",
  "enabled": true
}
]
}

```

Example input in LDIF format for tag groups and tags:

```

{
  "connectorType": "Report Technology Radar",
  "connectorId": "Technology Radar Tags",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "Deployment",
      "id": "1",
      "data": {
        "taggroups": [
          {
            "name": "Technology radar - Quadrant",
            "shortname": "TRQ",
            "description": "Beschreibung Quadrant",
            "mode": "SINGLE",
            "factsheettype": "ITComponent"
          },
          {
            "name": "Technology radar - Ring",
            "shortname": "TRR",
            "description": "Beschreibung Ring",
            "mode": "SINGLE",
            "factsheettype": "ITComponent"
          }
        ]
      }
    }
  ]
}

```



```

    }
  ],
  "tags": [
    {
      "groupname": "Technology radar - Quadrant",
      "name": "Architecture Concepts",
      "description": "Beschreibung Architecture Concepts",
      "color": "#ff0000"
    },
    {
      "groupname": "Technology radar - Quadrant",
      "name": "Platforms",
      "description": "Beschreibung Platforms",
      "color": "#00ff00"
    },
    {
      "groupname": "Technology radar - Quadrant",
      "name": "Techniques",
      "description": "Beschreibung Techniques",
      "color": "#0000ff"
    },
    {
      "groupname": "Technology radar - Quadrant",
      "name": "Tools & Infrastructure",
      "description": "Beschreibung Tools & Infrastructure",
      "color": "#000000"
    },
    {
      "groupname": "Technology radar - Ring",
      "name": "Hold",
      "description": "Beschreibung Hold",
      "color": "#ff0000"
    },
    {
      "groupname": "Technology radar - Ring",
      "name": "Incubating",
      "description": "Beschreibung Incubating",
      "color": "#00ff00"
    },
    {
      "groupname": "Technology radar - Ring",
      "name": "Emerging",
      "description": "Beschreibung Emerging",
      "color": "#0000ff"
    },
    {
      "groupname": "Technology radar - Ring",
      "name": "Mature",
      "description": "Beschreibung Mature",
      "color": "#000000"
    }
  ]
}

```

```

    ]
  }
}

```

Tag Sent as a Comma-Separated List

Data in the tags looks like "Important,Mature". Helper function(toList) below will convert the comma-separated string to an Array and the Output of the below processor will be "Other Tags" : Important and Mature attached to Deployment "Finance Service".

Example inboundTag processor:

```

{
  "processorType": "inboundTag",
  "processorName": "Tag creation",
  "processorDescription": "Creates tags and tag groups",
  "factSheets": {
    "external": {
      "ids": "${content.id}",
      "type": {
        "expr": "externalId"
      }
    }
  },
  "run": 1,
  "updates": [
    {
      "key": {
        "expr": "name"
      },
      "values": [
        {
          "expr": "${integration.valueOfForEach.trim()}"
        }
      ]
    }
  ],
  "forEach": "${helper:toList(data.tags.split(','))}",
  "logLevel": "debug"
}

```

Example LDIF input for tags:

```

{
  "connectorType": "ee",
  "connectorId": "Kub Dev-001",
  "connectorVersion": "1.2.0",
  "lxVersion": "1.0.0",
  "description": "Imports kubernetes data into LeanIX",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "Deployment",

```

```

    "id": "784616bf-198c-11f9-9da8-9263b0573fbe",
    "data": {
      "app": "Finance Service",
      "version": "10.5",
      "maturity": "5",
      "clusterName": "westeurope",
      "tags": "Important,Mature"
    }
  }
]
}

```

The inboundTag processor does automatically creates tags that do not exist for a tag group (the tag processor does not create new tag groups).

The inbound tagProcessor can be configured to not create any tags nor change metadata of existing tags but only to assign Fact Sheets to existing tags. To use this functionality, an additional key "tagsReadOnly" needs to be configured in the updates section as shown in the example:

Example of enabling the "read only" mode for the inboundTag processor:

```

{
  "updates": [
    {
      "key": {
        "expr": "tagsReadOnly"
      },
      "values": [
        {
          "expr": "${true}"
        }
      ]
    },
    {
      "key": {
        "expr": "name"
      },
      "values": [
        {
          "expr": "${data.myTagName}"
        }
      ]
    }
  ]
}

```

i Note

The "optional" flag avoids warning messages if the input data may not contain values for all fields and this is expected.

i Note

Is used to only write values to internal variables. This will be used for aggregation use cases where the LDIF content needs to be used to only collect values without directly writing anything to SAP LeanIX.

Write to LDIF

The processor allows Administrators to configure an inbound Integration API run to write a new LDIF file. The resulting LDIF will be available using the /results and the /resultsUrl endpoints same as with outbound Integration API runs.

With this functionality, inbound runs can be used in all combinations to read, process, update Pathfinder entities and even write a new LDIF in one step. Integrations that write to SAP LeanIX and read data from SAP LeanIX can be written and managed in one Integration API configuration executed with a single call.

The new processor can even be used to only export data or just transform an LDIF into another LDIF.

In a configuration, all defined processors will write to a single, globally defined LDIF. This allows to collect all kinds of data objects to the target LDIF from multiple processors including content from aggregations and other processing (e.g. variables).

The LDIF header definition needs to be set as a global key in the Integration API configuration. All fields can be freely configured and will be evaluated to a String using JUEL. Exception is the "customFields" key. If defined, the value will be interpreted as an object and passed to the target LDIF. Please ensure the expression always results in a map object to not break the LDIF format.

Example writeToLdif processor:

```
{
  "processors": [
    {
      "processorType": "writeToLdif",
      "updates": [
        {
          "key": {
            "expr": "content.id"
          },
          "values": [
            {
              "expr": "${content.id}"
            }
          ]
        },
        {
          "key": {
            "expr": "content.type"
          },
          "values": [
            {
              "expr": "${content.type}"
            }
          ]
        },
        {
          "key": {
            "expr": "description"
          },
          "values": [
```

```

        {
            "expr": "Just a test. Could be any read content or JUEL calculation"
        }
    ]
}
],
"targetLdif": {
    "dataConsumer": {
        "type": "leanixStorage"
    },
    "ldifKeys": [
        {
            "key": "connectorType",
            "value": "myNewLdif"
        },
        {
            "key": "connectorId",
            "value": "mycreatedId"
        },
        {
            "key": "customFields",
            "value": "${integration.toObject('{\"anyKey\": \"anyValue\"'})}"
        }
    ]
}
}
}

```

A more advanced example shows how to read content from Pathfinder and write it to an LDIF using the processor. It requires an input LDIF with at least one data object but content is not relevant. The data object is only used to trigger.

`writeToLdif` can write variables as well (run levels are supported and variables are available one run after creation). You may even define multiple `writeToLdif` processors. All content will be collected and written to one resulting LDIF.

Please ensure to adjust the search scope to your workspace and change the id to an existing one.

Advanced example of the `writeToLdif` processor:

```

{
    "processors": [
        {
            "processorType": "writeToLdif",
            "filter": {
                "advanced": "${integration.contentIndex==0}",
                "onRead": "${lx.factsheet.description!=''}"
            },
            "identifier": {
                "search": {
                    "scope": {
                        "ids": [
                            "8de51ff7-6f13-47df-8af8-9132ada2e74d"
                        ],
                        "facetFilters": []
                    }
                }
            }
        }
    ]
}

```

```

    },
    "filter": "${true}",
    "multipleMatchesAllowed": true
  }
},
"run": 0,
"updates": [
  {
    "key": {
      "expr": "content.id"
    },
    "values": [
      {
        "expr": "${lx.factsheet.id}"
      }
    ]
  },
  {
    "key": {
      "expr": "content.type"
    },
    "values": [
      {
        "expr": "${lx.factsheet.name}"
      }
    ]
  },
  {
    "key": {
      "expr": "description"
    },
    "values": [
      {
        "expr": "${lx.factsheet.description}"
      }
    ]
  }
],
"logLevel": "warning",
"read": {
  "fields": [
    "description"
  ]
}
},
"targetLdif": {
  "dataConsumer": {
    "type": "leanixStorage"
  },
  "ldifKeys": [
    {
      "key": "connectorType",
      "value": "${header.connectorType}_export"
    }
  ]
}

```

```

    },
    {
      "key": "connectorId",
      "value": "${header.connectorId}_export"
    },
    {
      "key": "description",
      "value": "Enriched imporeted LDIF for Applications"
    }
  ]
}
}

```

Example input in LDIF format:

```

{
  "connectorType": "example",
  "connectorId": "example",
  "lxVersion": "1.0.0",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "content": [
    {
      "type": "",
      "id": "",
      "data": {}
    }
  ]
}

```

Inbound Impact

The processor is used to write impacts to the SAP LeanIX backend.

A processor always writes a single impact. In case a list of input data is given where "forEach" can iterate over, multiple impacts can be written by a single processor. Different types of impacts should be split into different processors in order to keep a configuration readable as different impacts have different parameters.

The example below shows how to define Impacts. Please be aware that each type of impacts may require a different set of keys to be configured.

→ Tip

To find out about the different keys required for each Impact type, create the needed types of impacts in the UI, then export using an outbound processor or by using the "read" section in an inbound processor and either export to LDIF or write to a field like "description"

Example inboundImpact processor:

```

{
  "processors": [
    {

```

```

"processorType": "inboundImpact",
"updates": [
  {
    "key": {
      "expr": "groupName"
    },
    "values": [
      {
        "expr": "G1"
      }
    ]
  },
  {
    "key": {
      "expr": "description"
    },
    "values": [
      {
        "expr": "Group Description 2"
      }
    ]
  },
  {
    "key": {
      "expr": "impacts"
    },
    "values": [
      {
        "map": [
          {
            "key": "type",
            "value": "FACTSHEET_SET"
          },
          {
            "key": "factSheetId",
            "value": "28fe4aa2-6e46-41a1-a131-72afb3acf256"
          },
          {
            "key": "fieldName",
            "value": "functionalSuitabilityDescription"
          },
          {
            "key": "fieldValue",
            "value": "${data.value}"
          }
        ]
      }
    ]
  }
],
"identifier": {
  "internal": "6d8acf0c-fa4e-40ed-9986-97da860f3414"
},
"logLevel": "warning"

```



```
    }
  ]
}
```

Inbound To-dos

This processor can be used import to-dos into the workspace.

Keys specific to the <code>inboundToDo</code> Processor	Details
<code>title</code>	Title of the To-do
<code>category</code>	Category of the To-do (Sample values: ANSWER,ACTION_ITEM)
<code>description</code>	Description of the To-do
<code>status</code>	Status of the To-do (OPEN, CLOSED)
<code>resolution</code>	When closing you also have an option to provide the resolution(ACCEPTED,REJECTED,REVERTED)

Below example will need to be adapted as per specific type of To-dos.

Example `inboundToDo` processor:

```
{
  "processors": [
    {
      "processorType": "inboundToDo",
      "processorName": "Create toDos",
      "processorDescription": "Creates toDos from incoming data",
      "filter": {
        "exactType": "ActionItem"
      },
      "identifier": {
        "external": "${content.id}"
      },
      "run": 1,
      "updates": [
        {
          "key": {
            "expr": "factSheetId"
          },
          "values": [
            {
              "expr": "${data.factSheetId}"
            }
          ]
        },
        {
          "key": {
            "expr": "title"
          },
          "values": [
```

```

    {
      "expr": "${data.title}"
    }
  ],
  {
    "key": {
      "expr": "description"
    },
    "values": [
      {
        "expr": "${data.description}"
      }
    ]
  },
  {
    "key": {
      "expr": "category"
    },
    "values": [
      {
        "expr": "${data.category}"
      }
    ]
  },
  {
    "key": {
      "expr": "dueDate"
    },
    "values": [
      {
        "expr": "${data.dueDate}"
      }
    ]
  }
],
"variables": [
  {
    "key": "category",
    "value": "${lx.todo.id}_${lx.todo.category}"
  },
  {
    "key": "state",
    "value": "${lx.todo.state}"
  }
],
"logLevel": "debug"
}
],
"variables": {}
}

```

Example input in LDIF format:

```

{
  "connectorType": "todoReadType",
  "connectorId": "todoReadId",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "description": "",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "ActionItem",
      "id": "E-100",
      "data": {
        "title": "test abc",
        "description": "Updated by iAPI",
        "todoId": "ec25a364-66df-4313-9127-44e429df81ad",
        "dueDate": "2021-08-19",
        "category": "ACTION_ITEM",
        "varValue": "value1",
        "creatorId": "275617d6-2538-466c-b210-961ef2cb554a",
        "factSheetId": "28fe4aa2-6e46-41a1-a131-72afb3acf256"
      }
    }
  ]
}

```

Outbound Processors

Caution

We recommend to use the [Write to LDIF](#) inbound processor to export data as LDIF.

Overview

Outbound Processors reads data from SAP LeanIX and exports it to an LDIF. Please see the [Integration API](#) main page for more information on LDIF's including a detailed breakdown of the different key-value pairs.

When creating an outbound configuration, the user can retrieve an LDIF after executing a synchronization run. The content of the produced LDIF will depend on the configured data processors. Only outbound processors are available in outbound configurations.

Each Outbound Data Processors will always have the same output fields (targeting the LDIF standard) but depending on type we will see other filter elements and have different data available in scope of the JUEL parsers

Outbound processors will always create valid LDIF files. This includes that mandatory LDIF fields in the header and for each data object (type, id) are always present. This information is automatically taken from the outbound configuration.

i Note

The processor name should not include any reserved characters which are as operators e.g. +, *, >, = ...

The processor configurations (outbound Data Processors) need to contain the names of the Fact Sheet fields that will be written (for performance reasons, only selected fields will be read) and names of the relations including the requested fields of the relations. For Tags users can filter by tag group and retrieve all tags in this group. For Subscriptions, the user will filter by the type of a subscription.

Setting the Scope

The outbound configuration needs to contain a scope. **The scope defines the set of Fact Sheets that will be looked at when iterating over the configured outbound processors and creating content in the resulting LDIF.** However, while the scope section needs to exist, it can be left completely empty as the example above shows, this means that it will look at all Fact Sheets.

The scope defined in the below example will iterate over all Fact Sheets. **The UI provides a visual interface (accessible via the Set Scope button) which allows for the selection of the Fact Sheet data required and sets this data as in "scope" into the configuration.** The default scope used for all processors will be defined in the global section. Each processor may contain a scope section to overwrite the global scope settings.

Scope configuration:

```
{
  "scope": {},
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Export to LDIF",
      "processorDescription": "This is an example how to use the processor",
      "scope": {
        "ids": [],
        "facetFilters": [
          {
            "keys": [
              "Application"
            ],
            "facetKey": "FactSheetTypes",
            "operator": "OR"
          }
        ]
      }
    }
  ]
}
```

Setting the Fields

The processor needs the "fields" section to define the names of the fields that will be read from each Fact Sheet and can then be used in the output section using "lx.factsheet.fieldName".

The "description" field was marked "optional". This means in case a Fact Sheet does not have any description, there will be no warning generated but the Description key silently omitted as it was configured to be expected that description may be missing.

The above configuration leads to a resulting LDIF similar to this.

Outbound processor:

```

{
  "scope": {
    "facetFilters": [],
    "ids": []
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Export to LDIF",
      "processorDescription": "This is an example how to use the processor",
      "enabled": true,
      "fields": [
        "createdAt",
        "description"
      ],
      "output": [
        {
          "key": {
            "expr": "content.id"
          },
          "values": [
            {
              "expr": "${lx.factsheet.id}"
            }
          ]
        },
        {
          "key": {
            "expr": "content.type"
          },
          "values": [
            {
              "expr": "${lx.factsheet.type}"
            }
          ]
        },
        {
          "key": {
            "expr": "Description"
          },
          "values": [
            {
              "expr": "${lx.factsheet.description}"
            }
          ],
          "optional": true
        },
        {
          "key": {
            "expr": "creationDateTime"
          },
          "mode": "selectFirst",
          "values": [
            {

```

```

    "expr": "${lx.factsheet.createdAt}"
  }
]
}
]
}
]
}

```

LDIF output:

```

{
  "results": {
    "connectorType": "Test connector",
    "connectorId": "--",
    "connectorVersion": "1.0.0",
    "processingDirection": "INBOUND",
    "processingMode": "PARTIAL",
    "lxWorkspace": null,
    "lxVersion": "1.0.0",
    "description": "The resulting LDIF of the outbound run c86faf41-cac5-4309-a98a-bb0:",
    "content": [
      {
        "type": "Application",
        "id": "fc2b0641-6172-40cf-9dbb-10e514c7e341",
        "data": {
          "Description": "",
          "creationDateTime": "2018-04-17T21:44:27.214Z"
        }
      },
      {
        "type": "Application",
        "id": "716b9d27-3df7-42ca-ae6e-e2ef788064dd",
        "data": {
          "Description": "SAP Supply Chain Management handles everyth",
          "creationDateTime": "2018-04-17T21:44:27.214Z"
        }
      },
      {
        "type": "Application",
        "id": "1a610104-6088-498a-856f-bb6a8298bcd4",
        "data": {
          "Description": "Mailsnake is supporting newsletter mailing",
          "creationDateTime": "2018-04-17T21:44:27.212Z"
        }
      },
      {
        "type": "BusinessCapability",
        "id": "58aaff75-1649-447c-bdec-01860db7026f",
        "data": {
          "Description": "",
          "creationDateTime": "2018-04-17T21:44:28.663Z"
        }
      }
    ]
  }
}

```

```

    ],
    "chunkInformation": {
        "firstDataObject": 0,
        "lastDataObject": 394,
        "maxDataObject": 394
    },
    },
    "warnings": [],
    "debugInfo": [],
    "debugVariables": [],
    "statistics": {
        "statusChanges": [
            {
                "status": "CREATED",
                "timestamp": "2022-03-10T07:24:01.44829307Z"
            },
            {
                "status": "PENDING",
                "timestamp": "2022-03-10T07:24:01.768868487Z"
            },
            {
                "status": "IN_PROGRESS",
                "timestamp": "2022-03-10T07:24:03.194204924Z"
            },
            {
                "status": "FINISHED",
                "timestamp": "2022-03-10T07:24:04.614733108Z"
            }
        ],
        "processorStatistics": [
            {
                "processorIndex": 0,
                "processorName": "Export to LDIF",
                "networkDuration": "PT0.955467958S",
                "processingDuration": "PT0.085116706S",
                "itemsInScopeCount": 395,
                "processedContentCount": 395,
                "errorCount": 0
            }
        ],
        "totalBlockingTime": "PT0S"
    },
    "resultsUrl": {}
}

```

Setting Filters

Filter section is where you define if the Data Processor should work on the found data object. It can be set to null to indicate no filter is being provided.

Example outbound processor with filters:

```
{
  "scope": {},
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "User Groups that Begin with Br",
      "processorDescription": "",
      "filter": {
        "advanced": "${lx.factsheet.name.startsWith('Br')}}"
      }
    }
  ]
}
```

Filter Type	Details
type	If configured, the string is interpreted as a regular expression and matched against the "type" of the Fact Sheet
id	If configured, the string is interpreted as a regular expression and matched against the "id" d of the Fact Sheet
advanced	If configured, the field contains a JUEL expression that may evaluate to "true" for a match or "false". This filter allows to filter even for combinations of certain key and values in the Fact Sheet

i Note

Data Processors provide filter capabilities to configure on which Data Object the data processor will work on (match the filter) and what Data Objects to skip (not match the filter)

Below example also includes the option to modify the choices as per external System.

Example single outbound processor:

```
{
  "processorType": "outboundFactSheet",
  "processorName": "<Unnamed processor>",
  "processorDescription": "",
  "filter": null,
  "enabled": true,
  "fields": [
    "nonExistingField",
    "lifecycle",
    "location",
    "createdAt",
    "businessCriticality",
    "technicalSuitabilityDescription"
  ],
  "output": [
    {
      "key": {
        "expr": "content.id"
      },
    },
  ],
}
```



```

"mode": "selectFirst",
"values": [
  {
    "expr": "${lx.factsheet.id}"
  }
],
},
{
  "key": {
    "expr": "content.type"
  },
  "mode": "selectFirst",
  "values": [
    {
      "expr": "${lx.factsheet.type}"
    }
  ]
},
{
  "key": {
    "expr": "lifecycle.times"
  },
  "mode": "list",
  "values": [
    {
      "expr": "${lx.factsheet.lifecycle.endOfLife}"
    },
    {
      "expr": "${lx.factsheet.lifecycle.active}"
    },
    {
      "expr": "${lx.factsheet.lifecycle.phaseOut}"
    },
    {
      "expr": "${lx.factsheet.lifecycle.phaseIn}"
    }
  ]
},
{
  "key": {
    "expr": "location"
  },
  "mode": "selectFirst",
  "values": [
    {
      "expr": "${lx.factsheet.location.rawAddress}, with place id: ${lx.factsheet.location.placeId}"
    }
  ]
},
{
  "key": {
    "expr": "businessCriticality"
  },
  "values": [

```

```

{
  "expr": "${lx.factsheet.businessCriticality}",
  "regexMatch": "administrativeService",
  "regexReplace": {
    "match": "^.*$",
    "replace": "Administrative Service"
  }
},
{
  "expr": "${lx.factsheet.businessCriticality}",
  "regexMatch": "businessOperational",
  "regexReplace": {
    "match": "^.*$",
    "replace": "Business Operational"
  }
}
],
{
  "key": {
    "expr": "creationTime"
  },
  "mode": "selectFirst",
  "values": [
    {
      "expr": "${lx.factsheet.createdAt}"
    }
  ]
},
{
  "key": {
    "expr": "technicalSuitabilityDescription"
  },
  "mode": "selectFirst",
  "values": [
    {
      "expr": "${lx.factsheet.technicalSuitabilityDescription}"
    }
  ]
}
]
}

```

Outbound Fact Sheet

This processor is capable writing Fact Sheet, relation, tag, document, subscription and metrics information from SAP LeanIX to LDIF.

In order to write LDIF, the scope of total Fact Sheets to read data from needs to be defined by a search. The Integration API UI allows to set the scope and add this scope to the configuration of an outbound configuration. Users then only have to regularly request the LDIF and will receive one LDIF with most current information matching the defined query. The exported data is configurable using same mechanisms as we do the inbound way.

Scopes can be defined globally for all processors or inside each processor. In case both was defined, the scope in the processor takes priority over the global used as a fallback only if the processor has not defined any.

Basic structure of the outboundFactSheet processor.

Example outboundFactSheet processor:

```
{
  "scope": {
    "facetFilters": [],
    "ids": []
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Export to LDIF",
      "processorDescription": "This is an example how to use the processor",
      "enabled": true,
      "fields": [
        "lifecycle",
        "location",
        "createdAt",
        "description",
        "technicalSuitabilityDescription"
      ],
      "relations": {
        "filter": [
          "relToParent",
          "relApplicationToITComponent"
        ],
        "fields": [
          "description"
        ],
        "targetFields": [
          "displayName",
          "externalId"
        ],
        "constrainingRelations": false
      },
      "tags": {
        "groups": [
          "Other tags",
          "Cloud Transformation"
        ]
      },
      "subscriptions": {
        "types": [
          "RESPONSIBLE"
        ]
      },
      "documents": {
        "filter": ".*"
      },
      "output": [
```

```

{
  "key": {
    "expr": "content.id"
  },
  "values": [
    {
      "expr": "${lx.factsheet.id}"
    }
  ]
},
{
  "key": {
    "expr": "content.type"
  },
  "values": [
    {
      "expr": "${lx.factsheet.type}"
    }
  ]
},
{
  "key": {
    "expr": "Description"
  },
  "values": [
    {
      "expr": "${lx.factsheet.description}"
    }
  ],
  "optional": true
},
{
  "key": {
    "expr": "creationDateTime"
  },
  "mode": "selectFirst",
  "values": [
    {
      "expr": "${lx.factsheet.createdAt}"
    }
  ]
}
]
}
]
}

```

Exporting Relations

In case **relations** need to be available to write information about them to the LDIF, a section "relations" needs to exist. The section contains three parts: "filter", where users will configure the names of the relations they are interested in to export, the fields and the targetFields key to define what fields on the relation and what fields from the target Fact Sheet will be available in the output

section. All fields on relations can be configured in the "targetFields" key which is a value within the "relations" key. A processor iterating over the target Fact Sheet types needs to be configured to read all fields.

Example configuration of relations:

```
{
  "relations": {
    "filter": [
      "relToParent",
      "relApplicationToITComponent"
    ],
    "fields": [
      "description"
    ],
    "targetFields": [
      "displayName",
      "externalId"
    ],
    "constrainingRelations": false
  }
}
```

In case relations are found, they are all available as an array in "lx.relations". An example on how to access this information fill follow below. The example above only exports the creation date of all Fact Sheets in scope.

Making information about [tags and subscriptions](#) available to be used in the output section works following the same pattern.

Exporting All Relations and Fields within Relations

In the example below all of the relations that exist in a workspace have been associated to the processor. Along with all of the attributes available within the relations, such as the "Active From" and "Active Until" fields and "Total Annual Cost". Any custom relations and or attributes can be added to the appropriate object within the LDIF.

Example processor configuration with all relations and fields within relations:

```
{
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Export to LDIF",
      "processorDescription": "This is an example how to use the outboundFactSheet processor",
      "enabled": true,
      "fields": [
        "displayName",
        "externalId"
      ],
      "relations": {
        "filter": [
          "relToParent",
          "relToRequires",
          "relToSuccessor",
          "relApplicationToUserGroup",
          "relApplicationToDataObject",

```

```

    "relApplicationToITComponent",
    "relApplicationToProject",
    "relProviderApplicationToInterface",
    "relConsumerApplicationToInterface",
    "relApplicationToProcess",
    "relApplicationToBusinessCapability",
    "relITComponentToTechnologyStack",
    "relITComponentToUserGroup",
    "relITComponentToProvider",
    "relInterfaceToDataObject",
    "relInterfaceToITComponent",
    "relProcessToBusinessCapability",
    "relProjectToBusinessCapability",
    "relProjectToITComponent",
    "relProjectToProcess",
    "relProjectToUserGroup",
    "relProjectToProvider"
  ],
  "fields": [
    "activeFrom",
    "activeUntil",
    "description",
    "functionalSuitability",
    "numberOfUsers",
    "usageType",
    "usage",
    "costTotalAnnual",
    "technicalSuitability",
    "serviceLevel",
    "supportType",
    "resourceClassification",
    "orderNo",
    "orderedCapex",
    "orderedOpex"
  ],
  "targetFields": [
    "displayName",
    "externalId",
    "category"
  ]
},
"output": [
  {
    "key": {
      "expr": "content.id"
    },
    "mode": "selectFirst",
    "values": [
      {
        "expr": "${lx.factsheet.id}"
      }
    ]
  }
],
{

```

```

    "key": {
      "expr": "content.type"
    },
    "mode": "selectFirst",
    "values": [
      {
        "expr": "${lx.factsheet.type}"
      }
    ]
  },
  {
    "key": {
      "expr": "displayName"
    },
    "mode": "selectFirst",
    "values": [
      {
        "expr": "${lx.factsheet.displayName}"
      }
    ]
  },
  {
    "key": {
      "expr": "externalId"
    },
    "mode": "selectFirst",
    "values": [
      {
        "expr": "${lx.factsheet.externalId}"
      }
    ]
  },
  {
    "key": {
      "expr": "relations"
    },
    "mode": "list",
    "values": [
      {
        "forEach": {
          "elementOf": "${lx.relations}",
          "filter": "${true}"
        },
        "map": [
          {
            "key": "relationName",
            "value": "${integration.output.valueOfForEach.type}"
          },
          {
            "key": "object",
            "value": "${integration.output.valueOfForEach}"
          }
        ]
      }
    ]
  }
}

```

```

    ]
  }
]
},
"scope": {
  "facetFilters": [],
  "ids": []
}
}

```

Exporting Constraining Relations

All information about constraining relations may be made available as content in "lx.relations" to be ready for export on demand by setting the key "constrainingRelations" to true. If the key is set to false or is not defined, no information about constraining relations will be present in the variable "lx.relations"

When exporting, the following field structure may be used when iterating over relations e.g. using `forEach`:

"integration.valueOfForEach.constrainingRelations" and write all content to a map target or filter in a `forEach` section applied in the "values" segment.

Example configuration of constraining relations:

```

{
  "constrainingRelations": {
    "relations": [
      {
        "id": "60f57ffd-6dc8-42a5-9099-283256e627ad",
        "target": {
          "id": "1c3ef333-5cc1-414d-ba3e-108af7f0ffc4",
          "type": "UserGroup",
          "leanixV3IdUserGroup": "120000002"
        }
      },
      {
        "id": "a4661164-a948-4112-82de-270c31da4297",
        "target": {
          "id": "d973d8a8-6435-4952-988d-2e29feeafd57",
          "type": "UserGroup",
          "externalId": "Europe",
          "leanixV3IdUserGroup": "120000052"
        }
      }
    ],
    "totalCounts": [
      {
        "type": "relApplicationToUserGroup",
        "totalCount": 2
      }
    ]
  }
}

```


Exporting Tags and Subscriptions

For Tags, users can define a list of tag groups they are interested in using for export purposes. All tags in the listed tag groups will be part of the output scope and can be used in JUEL expressions "lx.tags" and "lx.subscriptions". As for relations, the tags and subscriptions are collected in a list. This list can be processed to output LDIF in multiple ways.

Iterating with an outer "forEach" allows to create separate data objects for every tag/subscription. Using the inner "forEach" in the output section allows to add the information to the data object written for the Fact Sheet.

Below is an example that shows multiple ways to convert the information to an output LDIF. It is recommended to potentially first try an output with the full tag/subscription object. This will allow you to see and review all of the contained fields. After which point, you could update the processor by limiting the output to the very specific content that needs to be part of the result LDIF.

Example outbound processor that exports relations, tags, and subscriptions into LDIF format:

```
{
  "scope": {
    "facetFilters": [
      {
        "keys": [
          "Application"
        ],
        "facetKey": "FactSheetTypes",
        "operator": "OR"
      },
      {
        "keys": [
          "__any__"
        ],
        "facetKey": "lifecycle",
        "operator": "OR",
        "dateFilter": {
          "to": "2024-12-31",
          "from": "2018-01-01",
          "type": "RANGE",
          "maxDate": "2021-08-06",
          "minDate": "2010-11-07"
        }
      },
      {
        "keys": [],
        "facetKey": "relApplicationToITComponent",
        "operator": "OR"
      }
    ],
    "ids": []
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Export to LDIF",
      "processorDescription": "This is an example how to use the outboundFactSheet processor",
      "enabled": true,
      "fields": [
```

```

    "lifecycle",
    "location",
    "createdAt",
    "technicalSuitabilityDescription",
    "description",
    "name"
  ],
  "relations": {
    "filter": [
      "relToParent",
      "relApplicationToITComponent"
    ],
    "fields": [
      "description"
    ],
    "targetFields": [
      "displayName",
      "externalId"
    ]
  },
  "tags": {
    "groups": [
      "Other tags",
      "Cloud Transformation"
    ]
  },
  "subscriptions": {
    "types": [
      "RESPONSIBLE"
    ]
  },
  "output": [
    {
      "key": {
        "expr": "content.id"
      },
      "values": [
        {
          "expr": "${lx.factsheet.id}"
        }
      ]
    },
    {
      "key": {
        "expr": "content.type"
      },
      "values": [
        {
          "expr": "${lx.factsheet.type}"
        }
      ]
    }
  ],
  {
    "key": {

```

```

    "expr": "Description"
  },
  "values": [
    {
      "expr": "${lx.factsheet.description}"
    }
  ],
  "optional": true
},
{
  "key": {
    "expr": "lifecycleTimes"
  },
  "mode": "list",
  "values": [
    {
      "expr": "endOfLife:${lx.factsheet.lifecycle.endOfLife}",
      "regexMatch": ".*:.*$"
    },
    {
      "expr": "active:${lx.factsheet.lifecycle.active}",
      "regexMatch": ".*:.*$"
    },
    {
      "expr": "phaseOut:${lx.factsheet.lifecycle.phaseOut}",
      "regexMatch": ".*:.*$"
    },
    {
      "expr": "phaseIn:${lx.factsheet.lifecycle.phaseIn}",
      "regexMatch": ".*:.*$"
    }
  ]
},
{
  "key": {
    "expr": "creationDateTime"
  },
  "mode": "selectFirst",
  "values": [
    {
      "expr": "${lx.factsheet.createdAt}"
    }
  ]
},
{
  "key": {
    "expr": "technicalSuitabilityDescription"
  },
  "values": [
    {
      "expr": "${lx.factsheet.technicalSuitabilityDescription}"
    }
  ]
},

```

```

{
  "key": {
    "expr": "relations"
  },
  "mode": "list",
  "values": [
    {
      "forEach": {
        "elementOf": "${lx.relations}",
        "filter": "${true}"
      },
      "map": [
        {
          "key": "relationName",
          "value": "${integration.output.valueOfForEach.type}"
        },
        {
          "key": "objectForFindingRelevantFields",
          "value": "${integration.output.valueOfForEach}"
        }
      ]
    }
  ],
},
{
  "key": {
    "expr": "tags"
  },
  "mode": "list",
  "values": [
    {
      "forEach": {
        "elementOf": "${lx.tags}",
        "filter": "${true}"
      },
      "map": [
        {
          "key": "tagGroup",
          "value": "${integration.output.valueOfForEach.tagGroup.name}"
        },
        {
          "key": "tagName",
          "value": "${integration.output.valueOfForEach.name}"
        }
      ]
    }
  ],
},
{
  "key": {
    "expr": "subscriptions"
  },
  "mode": "list",
  "values": [

```

```

{
  "forEach": {
    "elementOf": "${lx.subscriptions}",
    "filter": "${true}"
  },
  "map": [
    {
      "key": "subscriptionObjectForTestOnly",
      "value": "${integration.output.valueOfForEach}"
    },
    {
      "key": "subscriptionType",
      "value": "${integration.output.valueOfForEach.type}"
    },
    {
      "key": "subscriptionRoles",
      "value": "${integration.output.valueOfForEach.roles}"
    }
  ]
}
]
}
]
}
]
}
]
}

```

Below is a sample of what the `lx.subscriptions` object looks like, which can be accessed in the list as defined above.

Example `lx.subscriptions` object:

```

"lx.subscriptions": [
  {
    "id": "76cf02fa-c358-4653-af09-e1d9729960fa",
    "type": "RESPONSIBLE",
    "user": {
      "id": "2d9b1cbb-158f-4679-b9b8-60136a52de3e",
      "email": "22345@leanix.net",
      "lastName": "User",
      "userName": "anonymized.user@leanix.net",
      "firstName": "Anonymized",
      "displayName": "Anonymized User"
    },
    "roles": [
      {
        "id": "1b3e71fe-781a-4fd9-988e-56ba42b33a51",
        "name": "Business Owner"
      }
    ],
    "createdAt": "2022-12-12T09:04:40.214367Z"
  },
  {
    "id": "94fbbaf1-8c75-4cf2-a6ac-98d92e277877",

```

```
"type": "RESPONSIBLE",
"user": {
  "id": "8b189683-7852-4ff7-9845-f1725e324670",
  "email": "abel.tuter@example.com",
  "lastName": "tuter",
  "userName": "Abel.tuter@example.com",
  "firstName": "abel",
  "displayName": "abel tuter"
},
"roles": [
  {
    "id": "60d53ae5-34f0-4f8c-8ca3-9aa7f674f41f",
    "name": "Application Owner Deputy",
    "comment": "Deputy Comment"
  }
],
"createdAt": "2023-04-10T11:34:40.145262Z"
}
```

i Note

Outbound Data Processors need to have mandatory fields configured.

An outbound Fact Sheet processor will fail if the mandatory output fields "content.id" and "content.type" are not defined as they are mandatory part of any LDIF.

i Note

The created file may contain more optional fields. The above displays a short version for understanding the mandatory structure. See detailed description above

Write LDIF using Modes, Map and ForEach

Produced LDIF may not only contain simple key-value pairs in the data section but lists and maps as well. The below illustrated how to configure a data processor to write these data types.

Mode	Details
list	Collects all entries and writes the result as a list.
selectFirst	Is the default setting, it selecting the first value that is not empty and uses it to write to the target key.

LDIF from Processor Set to Mode: List

In this case, the mode was switched to "list". The default ("selectFirst") would pick the first value that is not empty and use it to write to the target key. Mode list collects all entries and writes the result as a list.

outboundFactSheet processor configuration writing values to list:

```
{
  "scope": {
```

```

"facetFilters": [],
"ids": []
},
"processors": [
{
  "processorType": "outboundFactSheet",
  "processorName": "Export to LDIF",
  "fields": [
    "lifecycle"
  ],
  "output": [
    {
      "key": {
        "expr": "content.id"
      },
      "values": [
        {
          "expr": "${lx.factsheet.id}"
        }
      ]
    },
    {
      "key": {
        "expr": "content.type"
      },
      "values": [
        {
          "expr": "${lx.factsheet.type}"
        }
      ]
    },
    {
      "key": {
        "expr": "lifecycleTimes"
      },
      "mode": "list",
      "values": [
        {
          "expr": "endOfLife:${lx.factsheet.lifecycle.endOfLife}",
          "regexMatch": ".*:.*$"
        },
        {
          "expr": "active:${lx.factsheet.lifecycle.active}",
          "regexMatch": ".*:.*$"
        },
        {
          "expr": "phaseOut:${lx.factsheet.lifecycle.phaseOut}",
          "regexMatch": ".*:.*$"
        },
        {
          "expr": "phaseIn:${lx.factsheet.lifecycle.phaseIn}",
          "regexMatch": ".*:.*$"
        }
      ]
    }
  ]
}
]

```

```

    }
  ]
}
]
}

```

LDIF result with list values:

```

{
  "connectorType": "lxExport",
  "connectorId": "007",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "content": [
    {
      "type": "Application",
      "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
      "data": {
        "lifecycleTimes": [
          "endOfLife:2021-08-06",
          "active:2018-08-06",
          "phaseOut:2021-05-06",
          "phaseIn:2018-05-06"
        ]
      }
    },
    {
      "type": "Application",
      "id": "001d3e51-a3be-4d5b-90f7-5b7a9e0b9458",
      "data": {
        "lifecycleTimes": [
          "active:2018-08-06",
          "phaseIn:2018-05-06"
        ]
      }
    }
  ]
}

```

Caution

Empty values are not written to the LDIF.

Please note that the list in the result LDIF does not always contain all elements but only the ones that are populated. This is done by additional configuration of a "regexMatch" key eliminating all values with no content. By removing this, the second data object would contain all four elements including the empty ones with values like "phaseOut:"

LDIF from Processor Writing Relations: Map

The desired LDIF output and sample outbound processor are:

Sample outbound processor configuration to write relations:


```

{
  "scope": {
    "facetFilters": [],
    "ids": []
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Export to LDIF",
      "processorDescription": "This is an example how to use the outboundFactSheet processor",
      "enabled": true,
      "fields": [],
      "relations": {
        "filter": [
          "relApplicationToITComponent"
        ],
        "fields": [
          "description"
        ],
        "targetFields": [
          "displayName",
          "externalId"
        ]
      },
      "output": [
        {
          "key": {
            "expr": "content.id"
          },
          "mode": "selectFirst",
          "values": [
            {
              "expr": "${lx.factsheet.id}"
            }
          ]
        },
        {
          "key": {
            "expr": "content.type"
          },
          "mode": "selectFirst",
          "values": [
            {
              "expr": "${lx.factsheet.type}"
            }
          ]
        },
        {
          "key": {
            "expr": "relations"
          },
          "mode": "list",
          "values": [
            {

```

```

    "forEach": {
      "elementOf": "${lx.relations}",
      "filter": "${true}"
    },
    "map": [
      {
        "key": "relationName",
        "value": "${integration.output.valueOfForEach.type}"
      },
      {
        "key": "object",
        "value": "${integration.output.valueOfForEach}"
      }
    ]
  }
]
}
]
}
]
}
}
}

```

LDIF result containing relations:

```

{
  "connectorType": "lxExport",
  "connectorId": "007",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "content": [
    {
      "type": "Application",
      "id": "28fe4aa2-6e46-41a1-a131-72afb3acf256",
      "data": {
        "relations": [
          {
            "object": {
              "id": "5da3529f-2d40-4f00-9a22-ab8ddb4fc216",
              "type": "relApplicationToITComponent",
              "target": {
                "id": "20be4187-c384-4557-a6eb-c7a3d1e5eb40",
                "type": "ITComponent",
                "displayName": "Visual Studio 2011"
              },
              "description": ""
            },
            "relationName": "relApplicationToITComponent"
          },
          {
            "object": {
              "id": "51663856-ca7e-4a53-8412-14e6811df0ab",
              "type": "relApplicationToITComponent",
              "target": {
                "id": "5719d96b-4c45-497a-b006-c77e0d8e767b",

```

```

    "type": "ITComponent",
    "displayName": "meshlab IT Application Hosting"
  },
  "description": ""
},
"relationName": "relApplicationToITComponent"
}
]
}
},
{
  "type": "Application",
  "id": "001d3e51-a3be-4d5b-90f7-5b7a9e0b9458",
  "data": {
    "relations": [
      {
        "object": {
          "id": "509511e4-b5ff-43b1-b523-1ae5eb0522e0",
          "type": "relApplicationToITComponent",
          "target": {
            "id": "1d05f6ca-2d81-4998-8e6a-be70a0c33378",
            "type": "ITComponent",
            "displayName": "Test Cisco Router 3925"
          },
        },
        "description": ""
      },
      "relationName": "relApplicationToITComponent"
    ]
  }
}
]
}
}
}
}
}
}

```

In this case, we again use the output mode "list" to generate a list and not only select the first valid result (We expect potentially multiple relations). Output of maps will work with mode "selectFirst" as well and export a single map as a value.

In the above example we use two new concepts:

Definitions: map & forEach

Data Type	Details
map	Here we define the set of key names and value names we want to see in the output. Both are evaluated dynamically and can contain JUEL expressions referencing available data like Fact Sheet data.
forEach	*Following the same pattern we see for the forEach in inbound processors, we iterate over a given list variable. In the example the list of found relations. For each relation entry we expect to see one map with the configured keys and values.

i Note

Expression-based filtering for Outbound Data Processors.

The `forEach` as well provides an expression based filtering to e.g. only output specific relations

(`${integration.output.valueOfForEach.type=='relApplicationToITComponent'}`) in case multiple different relations have been requested in the relations section. More useful filtering may be done on target id or specific fields of a Fact Sheet or relation.

To learn more about `forEach` and other advanced functionality please see [Advanced Use Cases for the Integration API](#).

Use Relations to Create Separate Data Object

Last example shows how to use the relation (alternatively tag or subscription) information to create separate data object from it.

The example as well shows, how to filter specific relations from being written to the LDIF. In the example all relations with description "skipme" will not be exported. Please see the [Setting Filters](#) section for more information on filters.

Example outbound processor that uses relations to create a separate data object:

```
{
  "scope": {
    "facetFilters": [
      {
        "keys": [
          "Application"
        ],
        "facetKey": "FactSheetTypes",
        "operator": "OR"
      }
    ],
    "ids": []
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Relations relFromApp",
      "processorDescription": "Export all relations for Apps",
      "filter": {
        "advanced": "${integration.valueOfForEach.description!='skipme'}"
      },
      "enabled": true,
      "fields": [
        "description",
        "name"
      ],
      "relations": {
        "filter": [
          "relApplicationToBusinessCapability"
        ],
        "fields": [
          "description",
          "activeFrom",
          "activeUntil",
          "functionalSuitability"
        ],
        "targetFields": []
      }
    }
  ]
}
```

```

"forEach": "${lx.relations}",
"output": [
  {
    "key": {
      "expr": "content.id"
    },
    "values": [
      {
        "expr": "0"
      }
    ]
  },
  {
    "key": {
      "expr": "content.type"
    },
    "values": [
      {
        "expr": "lxRelApplicationToBusinessCapability"
      }
    ]
  },
  {
    "key": {
      "expr": "lxId"
    },
    "values": [
      {
        "expr": "${lx.factsheet.id}"
      }
    ]
  },
  {
    "key": {
      "expr": "lxTargetId"
    },
    "values": [
      {
        "expr": "${integration.valueOfForEach.target.id}"
      }
    ]
  },
  {
    "key": {
      "expr": "lxFsName"
    },
    "values": [
      {
        "expr": "${lx.factsheet.name}"
      }
    ]
  }
]
}

```

```

    ]
  }
}

```

Export Metrics

Integration API allows to export metrics information visible in a chart on a Fact Sheet to an LDIF file. This is advanced configuration and requires deeper understanding of Metrics data and configuration in SAP LeanIX. The Fact Sheet processor will contain one or more definitions to point to series of data points stored in the Metrics and displayed as a diagram attached to the Fact Sheet.

The configuration allows flexibility to customize the selection and aggregation of data points even different from the current display on the Fact Sheet as Integration API accesses the raw data not the processed data visible in the UI. Multiple data series as e.g. displayed on a stacked bar chart need to be configured as separate configurations in the processor and will be accessed separately when writing to the LDIF.

This example shows how to export Metrics data and needs to be adjusted to the defined metrics series in the workspace.

Multiple different configurations all need to define different values for the "name" key. This value is then used to access the collected data. Please use only characters that are supported by JUEL, avoid any special characters. In the example below "variableName" is used in the definition and then in the output section to access the data and write it to the LDIF.

- "comparator" currently only supports "=" and intended for future extension.
- "aggregationFunction" does support all standard influx aggregations supporting a column as parameter.

Example outboundFactsheet processor that exports metrics:

```

{
  "scope": {
    "ids": [
      "28fe4aa2-6e46-41a1-a131-72afb3acf256"
    ],
    "facetFilters": []
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Export to LDIF",
      "processorDescription": "This is an example how to use the processor",
      "enabled": true,
      "fields": [
        "lifecycle",
        "location",
        "createdAt",
        "technicalSuitabilityDescription",
        "description"
      ],
      "relations": {
        "filter": [
          "relToParent",
          "relApplicationToITComponent"
        ],
        "fields": [
          "description"
        ]
      }
    }
  ]
}

```

```

    "targetFields": [
      "displayName",
      "externalId"
    ],
    "constrainingRelations": false
  },
  "tags": {
    "groups": [
      "Other tags",
      "Cloud Transformation"
    ]
  },
  "subscriptions": {
    "types": [
      "RESPONSIBLE"
    ]
  },
  "documents": {
    "filter": ".*"
  },
  "metrics": [
    {
      "name": "variableName",
      "measurement": "money",
      "fieldName": "dollars_per_day",
      "aggregationFunction": "MEAN",
      "groupBy": "1h",
      "start": "2020-01-20T00:00:00Z",
      "duration": "P0DT24H30M",
      "rules": {
        "key": "factSheetId",
        "comparator": "=",
        "compareWith": "${lx.factsheet.id}"
      }
    }
  ],
  "output": [
    {
      "key": {
        "expr": "content.id"
      },
      "values": [
        {
          "expr": "${lx.factsheet.id}"
        }
      ]
    },
    {
      "key": {
        "expr": "content.type"
      },
      "values": [
        {
          "expr": "${lx.factsheet.type}"
        }
      ]
    }
  ]
}

```

```

    }
  ]
},
{
  "key": {
    "expr": "values"
  },
  "values": [
    {
      "expr": "${integration.toJson(lx.metrics.variableName.values)}"
    }
  ],
  "optional": true
},
{
  "key": {
    "expr": "fields"
  },
  "values": [
    {
      "expr": "${integration.toJson(lx.metrics.variableName.fields)}"
    }
  ],
  "optional": true
},
{
  "key": {
    "expr": "series"
  },
  "values": [
    {
      "expr": "${integration.toJson(lx.metrics.variableName.series)}"
    }
  ],
  "optional": true
},
{
  "key": {
    "expr": "metrics"
  },
  "mode": "list",
  "values": [
    {
      "forEach": {
        "elementOf": "${lx.metrics.variableName.series}",
        "filter": "${true}"
      },
      "map": [
        {
          "key": "time",
          "value": "${integration.output.valueOfForEach.time}"
        },
        {
          "key": "value",

```



```

    "value": "${integration.output.valueOfForEach.values}"
  }
]
}
]
}
]
}
]
}
]
}

```

Filtering on items that have changed recently

Integration API can produce 'delta' extracts, reflecting only recently updated Fact Sheets. This functionality is scoped to updates for entries in the audit history for specific Fact Sheets; It does not work for changes on documents or relations.

To filter, just add an additional key "updatedInDuration" to the filter section as shown below in the example:

Example processor that filters items by the change date:

```

{
  "scope": {
    "ids": [
      "28fe4aa2-6e46-41a1-a131-72afb3acf256"
    ],
    "facetFilters": []
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Export to LDIF",
      "processorDescription": "This is an example how to use the processor",
      "filter": {
        "updatedInDuration": "${header.customFields.age}"
      },
      "enabled": true,
      "fields": [
        "lifecycle",
        "location",
        "createdAt",
        "technicalSuitabilityDescription",
        "description"
      ],
      "relations": {
        "filter": [
          "relToParent",
          "relApplicationToITComponent"
        ],
        "fields": [
          "description"
        ],
        "targetFields": [
          "displayName",

```

```

    "externalId"
  ],
  "constrainingRelations": false
},
"tags": {
  "groups": [
    "Other tags",
    "Cloud Transformation"
  ]
},
"subscriptions": {
  "types": [
    "RESPONSIBLE"
  ]
},
"documents": {
  "filter": ".*"
},
"output": [
  {
    "key": {
      "expr": "content.id"
    },
    "values": [
      {
        "expr": "${lx.factsheet.id}"
      }
    ]
  },
  {
    "key": {
      "expr": "content.type"
    },
    "values": [
      {
        "expr": "${lx.factsheet.type}"
      }
    ]
  },
  {
    "key": {
      "expr": "updatedAt"
    },
    "mode": "selectFirst",
    "values": [
      {
        "expr": "${lx.factsheet.updatedAt}"
      }
    ]
  }
]
}
]
}

```

To pass the value for `updatedAtDuration`, just use the below LDIF when calling the outbound run. The example exports all Fact Sheets that changed in the last 3 days. Of course it is possible to add the P3D directly into the configuration if this does not need to be a parameter.

i Note

Duration is encoded in ISO 8601 duration format.

Example LDIF that contains the duration for updates to be exported:

```
{
  "connectorType": "test",
  "connectorId": "test",
  "connectorVersion": "1.0.0",
  "processingDirection": "outbound",
  "processingMode": "partial",
  "customFields": {
    "age": "P3D"
  }
}
```

Translate list entries

A first use case covers the need to translate all entries of a list of values read from a Fact Sheet. Combining the option to iterate over a list of values in the output section and being able to configure this multiple times to collect results to output and on top of this filter by specific entries of the list.

We iterate over the list of entries multiple times. One time for each known option we want to translate. And we skip all other entries of the list. As a result, we collected a list of translated values.

The solution can be used for inbound and outbound processors both.

Example of translating all entries of a list to be exported:

```
{
  "key": {
    "expr": "myGermanValues"
  },
  "mode": "list",
  "values": [
    {
      "forEach": {
        "elementOf": "${myList}",
        "filter": "${integration.output.valueOfForEach=='english value 1'}"
      },
      "expr": "Deutscher Wert 1"
    },
    {
      "forEach": {
        "elementOf": "${myList}",
        "filter": "${integration.output.valueOfForEach=='english value 2'}"
      },
      "expr": "Deutscher Wert 2"
    }
  ]
}
```

```

    }
  ]
}

```

How to export Fact Sheets that are archived

To include archived Fact Sheets when using outbound processors, a flag "omitArchivedFactSheets" needs to be set to false as in the sample below. Default for Integration API is to ignore all archived Fact Sheets when exporting.

Example scope definition to export Fact Sheets including archived ones:

```

{
  "scope": {
    "omitArchivedFactSheets": false,
    "ids": [],
    "facetFilters": [
      {
        "keys": [
          "BusinessCapability"
        ],
        "facetKey": "FactSheetTypes",
        "operator": "OR"
      },
      {
        "keys": [
          "archived"
        ],
        "facetKey": "TrashBin",
        "operator": "OR"
      }
    ]
  },
  "processors": [...]
}

```

How to export large amounts of data

To export large amounts of data, in case this is not possible to read data from the `/synchronizationRuns/{id}/results` and the response is too big the option which you have is to fetch data from blob storage.

```

{
  "scope": {},
  "processors": [],
  "dataConsumer": {
    "type": "leanixStorage"
  }
}

```

Fetch the `resultsUrl` by using the below and then use this to stream results from the Azure Blob storage.

- **Endpoint**: `https://{customerdomain}.leanix.net/services/integration-api/v1/synchronizationRuns/{id}/resultsUrl`
- **Method**: GET

The response can look like this below:

```
{
  "url": "https://leanixsomething.blob.core.windows.net/xxxx"
}
```

Advanced Use Cases for the Integration API

Explore advanced scenarios and examples for the Integration API.

Field and Value Processing

Data Processors allow to map the incoming data found in the LDIF to fields of the specific entity in SAP LeanIX (e.g. a Fact Sheet) or the other way round in case of "Outbound Data Processors". The configuration allows to map incoming values to different types of SAP LeanIX fields (single value, float, multi value, life cycle...).

Applying Field and Value mappings may result in errors in case of source fields are not existing. Reason may be low input data quality, optional data in the source system. This types of "error" is expected and would only be noted in "test mode". Processing for next configuration lines of the data processor and next data processor always continues.

Type conversions to proper output type (variable for SAP LeanIX, String for LDIF) happen transparently. JUEL provides implicit type conversion and allows even calculations on Strings that contain numbers.

JUEL and RegEx

To provide high flexibility, predictability and at the same time guarantee to easily understand the configuration, all relevant configuration options for the Data Processors always support a combination of JUEL and regEx configuration executed after each other. For additional information, visit the [JUEL site](#) ➡ .

JUEL allows to access and combine all input fields and values of the incoming data and data of the target entity (e.g. Fact Sheet).

RegEx allows final string mapping on the JUEL result.

All conversion of data types happens transparently.

While a most simple JUEL is always required to define the value to be used as an output, the RegEx replace may be empty if no value conversion is supposed to happen. Both methods provide in parts overlapping functionality. This is wanted and allows the user to focus on a potential solution based on technical knowledge.

Fields and value mapping in the Data Processors are configured as a list of single field configurations. Each configuration allows a JUEL/RegEx for the Key and a list of JUEL/RegEx Match/RegEx Replace for the value(s). This allows multi value field support.

Each JUEL Expression returns data.

This logic allows for configurations that fit many types of scenarios.

Processing of each field configuration works following this specification:

Value Type	Details
In case of a "List" (multi select field in pathfinder)	Each item in the List is tested against the regEx Match. If it matches, the regEx replace is executed and the result added to the list of target values for the configured field
In case of a single value,	It is tested against the RegEx Match. If it matches, the regEx replace is executed and the result added to the list of target values for the configured field All Strings in the list of target values are written to the configured target field
In case of a Multi value target field	All non-empty Strings will be written to Pathfinder
In case of a Single Value Field	the first non-empty String will be written to Pathfinder
In case no regEx match is configured	The match is considered to be true
In case no regEx replace is configured	The original String will be part of the output list The Logic allows to configure all kinds of scenarios.

RegEx and JUEL

All RegEx filters allow negation and case insensitivity. The Java RegEx syntax can be applied: To match all but "notMe", `"^((?!notMe).)*$"` would be used. To ensure matching in a case insensitive manner, you'd add `"(?i)"` to the beginning of the regular expression.

Each inbound Data Processor JUEL expression contain the following references to the data object the is in scope for processing:

Reference	Example	Details
* header	"header.customFields.myGlobaldata1"	the value of myGlobaldata1" would be useable in any expression, given such a global value is provided in the JUEL. If not present (no customFields section or no defined key), this will always evaluate to an empty string.
* content	"\${content.id}"	"688c16bf-198c-11e9-9d08-926310573fbf"
* data	"\${data.chart}"	ill result in a string "chartmuseum-1.8.4" (given the first data object in the above LDIF is being processed)
	"\${header.connectorId}"	would result in an evaluated string "Kub Dev-001".
	"\${content.id}"	will result in a string "688c16bf-198c-11e9-9d08-926310573fbf"
	"\${data.chart}"	"chartmuseum-1.8.4" (given the first data object in the above LDIF is being processed)

Each of them allow to access all data elements in the same or in subsections. It allows to e.g. access the id of the connector creating the LDIF. `"${header.connectorId}"` would result in an evaluated string "Kub Dev-001".

Using the "header" section, there is as well access to the global custom data section. Using `"header.customFields.myGlobaldata1"` the value of "myGlobaldata1" would be useable in any expression, given such a global value is provided in the JUEL. If not present

(no customFields section or no defined key), this will always evaluate to an empty string.

Users can use any type of operation that can be executed on String objects in Java. Documentation of all the Java String methods is not in scope of this documentation. For more information on methods for Java 8, refer to the [Java documentation](#) .

Advanced JUEL

JUEL Advanced functions	Details
Working with keys that contain spaces. Sometimes the keys in LDIF may contain spaces. That means that "." syntax "data.key with space" does not work.	Instead the syntax "data['key with space']" can be used.
Capitalize an incoming value	<code>\${data.name.toUpperCase().charAt(0)}\${data.name.substring(1)}</code>
How to use different data based on a condition to map into a field	<code>\${data.name1.length()>3 ? data.name1 : data.name2}</code>
Display all list values of a key in LDIF as comma separated string (e.g. input in LDIF: "architecture": ["amd64","Intel"])	<code>\${data.architecture}</code> and configure the regexReplace section like this: "regexReplace": { "match": "(\\[\\])", "replace": "" } (the regex matches all characters '[' and ']' and replaces with an empty string. Result will be "amd64, Intel"
Add a Hash value to make something unique	<code>\${data.name} (\${data.app.hashCode()}>0 ? data.app.hashCode() : data.app.hashCode()*-1)}</code>
Combine two fields into one** (here the second is in brackets)	<code>\${data.name} (\${data.app})</code>
Replace some characters with something else	<code>\${data.name.replace('chart','xx')}</code>
Remove characters	<code>\${data.name.replace('chart','')}</code>
Use one entry of a string containing values separated by a certain value (in this example a comma)	<code>\${data.clusterName.split(',')[1].trim()}</code> (given clusterName has a value of "abc, def, ghi", the resulting string will be "def")
Map a comma separated String found in LDIF to a multi value field in SAP LeanIX	<code>\${data.clusterName.split(',')}</code> (given clusterName has a value of "abc,def,ghi", the multivalue field in SAP LeanIX will be filled with these values. An additional regEx replace may be used to remove unwanted space characters if existing in each field
Fill defined values based on some prefix of incoming data	<code>\${data.clusterName.toLowerCase().startsWith('lean') ? 'High' : 'Low'}</code>
Accessing hierarchical data in LDIF data section. Given a data section like this: "data": {"level0": {"level1a": "abc", "level1b": "def"}}	<code>\${data.level0.level1a}</code> will result in a string "abc"
How to efficiently check if a source value is not null and not an empty string.	This could be done by <code>"\${data.myKey != null && data.myKey != ""}</code> . But it can be combined into a short expression: <code>\${not empty data.myKey}</code>
How to do a filter that finds a certain word in a multi line text field like in description	"onRead": <code>"\${!x.factsheet.alias.matches('(?!s).\\bwordToSearch\\b.')}}"</code>

JUEL Use Cases

Scenario	Input From LDIF	Configured JUEL	Regex Match	Regex Replace	Target Field	Result
Mixed input from single and	"Home Country": "D"	<code>"\${data['Home Country']}"</code>			multi value	D UK

Scenario	Input From LDIF	Configured JUEL	Regex Match	Regex Replace	Target Field	Result
multi value field written to multi value field	"Other Countries": ["UK","DK"]	"\${data.['Other countries']}"				DK
Multi value input in LDIF to multi value in SAP LeanIX with mapping of defined input values to alternative multi values in SAP LeanIX, filtering out any undefined values	"Area": [" EU ","US "," APAC "," MARS "]	"\${data.Area.trim()}" "\${data.Area.trim()}" "\${data.Area.trim()}"	^EU\$ ^US\$ ^APAC\$	EU / Europe US / United States APAC / Asia Pacific	multi value	D UK DK
Multi value input data in LDIF to multi value field in SAP LeanIX	"flag": ["Important","Urgent"]	"\${data.flag}"			multi value	Important Urgent
Multiple single value Fields in LDIF to one multi value field in SAP LeanIX	"importance": "High" "urgency": "High"	"\${data.importance}" Importance "\${data.urgency}" Urgency			multi value	High Importance High Urgency
Multi value input data into single value field in SAP LeanIX (first matching will be selected)	"importance": "High" "urgency": "High"	"\${data.importance}" Importance "\${data.urgency}" Urgency			single value	High Importance
Multi value input data into single value field in SAP LeanIX (first matching will be selected, matching on second configured input happens. Importance would only match if value started with "Top")	"importance": "High" "urgency": "High"	"\${data.importance}" Importance "\${data.urgency}" Urgency	^Top . *		single value	High Urgency
Single value input data in LDIF to single	"importance": "high"	"\${data.importance}"			single value	high

Scenario	Input From LDIF	Configured JUEL	Regex Match	Regex Replace	Target Field	Result
value field in SAP LeanIX						
Single value input data into multi value field in SAP LeanIX	"importance": "high"	"\${data.importance}"			multi value	high
Single field to single field but only write if the input data contains defined value(s)	"importance": "high"	"\${data.importance}"	^very high		multi value	nothing written

Best Name for Fact Sheet

There are situations where it is not easy to find the best potential name for a Fact Sheet based on incoming data. The best name may not be available as it is not unique. Another use case might be that the source may provide different candidates for a name where we want to select from best possible option to lower ranked options automatically based on information availability for each data object.

On the other hand we want to ensure that we do not change names of already created Fact Sheets all of a sudden just because a better name option became available during an update.

All the above use cases can be covered simply by providing a list of potential name candidates. Every candidate that results in a null (evaluated in the 'values' section) or is already taken by another Fact Sheet will be skipped. In case I want to keep a name once set and not changed after creation, admins configure to read current Fact Sheet content and use the existing name as a first option. This will automatically be skipped if the Fact Sheet is not yet existing.

Example

Please see the example of a processor and a sample LDIF. You may test play around with matching against existing Fact Sheet names and remove/rename some of the keys from the source data and do test runs:

Best fact sheet name selection:

```
{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Apps from Deployments",
      "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
      "type": "Application",
      "filter": {
        "exactType": "Deployment"
      },
      "identifier": {
        "external": {
          "id": {
            "expr": "${content.id}"
          }
        }
      }
    }
  ]
}
```

```

    "type": {
      "expr": "externalId"
    }
  },
  "run": 0,
  "updates": [
    {
      "key": {
        "expr": "name"
      },
      "values": [
        {
          "expr": "${lx.factsheet.name}"
        },
        {
          "expr": "${data.app}"
        },
        {
          "expr": "${data.app2}"
        },
        {
          "expr": "${data.app3}"
        }
      ]
    }
  ],
  "read": {
    "fields": [
      "name"
    ]
  },
  "logLevel": "debug"
}
]
}

```

Sample LDIF for best name suggestion:

```

{
  "connectorType": "ee",
  "connectorId": "Kub Dev-001",
  "connectorVersion": "1.2.0",
  "lxVersion": "1.0.0",
  "description": "Imports kubernetes data into LeanIX",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "Deployment",
      "id": "634c16bf-198c-1129-9d08-92630b573fbf",
      "data": {
        "app3": "veryLongAndUnhandyNameIDoNotWantToSeeIfPossible",

```

```
"app2": "littleBitBetterNameButStillNotGood",
"app": "Best Name"
}
}
]
}
```

Using "Integration" Object

Expression	Details	More Examples
"\${integration.now}"	Contains the information about the date and time the synchronization run started. "integration.now" contains a Java LocalDateTime object and allows to call methods with parameters of types String or long. E.g.	integration.now.plus Content like the date \${integration.now.ge The values can be us processor.
"\${integration.contentIndex}"	Contains the index number of the currently processed data object. This could be used to e.g. create a filter for a data processor to always run for the first data object of a synchronization run.	
"\${integration.maxContentIndex}"	Contains the contentIndex of the last data object in scope of the sync run. Matching this in an advanced filter for a data processor would ensure the processor only runs e.g. when processing the last data object.	
"\${integration.toJson(data.Properties)}"	Offers a helper method to convert any given section from the LDIF (data.Properties in the example) into a valid JSON string. The JSON can be used to be rendered in a Fact Sheet without any option to search but dump arbitrary data.	
"\${integration.toObject(data.Properties)}"	The opposite of "toJson". The method converts any Json String back to the corresponding object representation. This might be lists or maps e.g..	Given a String "{"key "\${integration.toObj the JUEL expression
"\${helper:toActiveLifecyclePhase(lx.factsheet.lifecycle, integration.now)}"	Offers a helper method to read the name of the lifecycle phase at a given point of time. Potential parameter for the	"\${helper:toActiveLif "\${helper:toActiveLif Note: In case custom "helper:toActiveLifec

Expression	Details	More Examples
	current date may be "integration.now"	occurr on the same c
<code>\${helper:toActiveLifecyclePhaseOrdered(lx.factsheet.lifecycle, integration.now, helper:toList("planned","phaseIn","active","phaseOut","endOfLife"))}</code>	Same as "helper:toActiveLifecyclePhase" with an additional parameter to define the order of the phases.	"\${helper:toActiveLi helper:toList("phase
<code>"\${helper:toList('default','optionHighPrio','optionMediumPrio','optionLowPrio')}"</code>	Converts a set of strings into a list to be used as parameters in a java String method	This works as well if :
<code>integration.processing.*</code>	The methods <code>sum()</code> , <code>distinct()</code> , <code>average()</code> , <code>max()</code> , <code>min()</code> , and <code>getNumbers()</code> allow to operate on every list in the JUEL scope in order to aggregate data and work with all lists in a way that is already supported for variables	Even chaining like da how many different r
<code>integration.processing.mergeList(firstList, secondList)</code>	The method merges two source lists and can be used to iterate over all values in multiple different input lists using one <code>forEach</code> loop	Even merging multip
<code>integration.tags.getTagGroupId(tagGroupName)"</code>	Allows to resolve internally used tag group ids from their external name	The conversion is us name and not their in workspace to worksp
<code>integration.tags.getTagId(tagGroupName,tagName)</code>	Allows to resolve internally used tag ids from their external name	The conversion is us name and not their in workspace to worksp
<code>integration.tags.getAllTagGroups()</code>	Allows to work with an object containing all tag groups and all tags defined in the workspace	See example configu
<code>helper:localDateTimeFromString('2016-03-04 11:30','yy-MM-dd HH:mm','yyyy-MM-dd HH:mm')</code>	Returns a <code>LocalDateTime</code> object representing the time 2016-03-04 11:30	Can be used to conver methods can then be • LocalDateTir • SimpleDateF
<code>helper:localDateTimeToString(integration.now, 'yy-MM-dd HH:mm')</code>	Returns a string "20-07-02 11:30". The helper allows to convert any <code>localDateTime</code> object back to a string in the required format	See Java documenta
<code>helper:getDuration(localDateTime,localDateTime)</code>	Returns a <code>Duration</code> object to allow flexible work with the result. See java documenta to <code>Duration</code> class methods. Parameters can be the return of the <code>localDateTimeFromString</code> or <code>integration.now</code> as well as	The helper will be us in days to a field is or returned "duration" c

Expression	Details	More Examples
	localDateTimeObjects returned by other methods called.	
math:nameOfTheMethod	calculates abs, round and other functions always on the highest available precision (double). In case a method returns a double, users can add ".intValue()" to convert to an integer for display purposes or to push to an integer field	For details on support
lx.toOrdinal('fieldName')	returns the position of the currently set value for a SINGLE_SELECT field. This allows to do calculations on SINGLE_SELECT fields given the order of the select options reflect a kind of order like "low", "medium", "high" would be returned as 0, 1, 2	See the configuration aware that the fields as shown in the example
helper:getFromMap(myMap,myKeyInTheMapAsString)	returns the object referenced by the specified key in the provided map. The helper can be used to access keys with spaces like a map containing a key names "my Important Value", which cannot be referenced by dot Syntax.	The helper allows to E.g. in cases where the value is determined dynamically.

To test the below example, please change the id in "ids" to an existing internal id of a Fact Sheet in your workspace. You may just open a Fact Sheet and copy the id from the browser URL.

In real world scenarios, you may not want to export the whole object, but iterate over tag groups or export a subset of the information.

Example for exporting all tag groups:

```
{
  "scope": {
    "ids": [
      "869ee28b-c60a-4e88-8d18-f9e4ff466456"
    ],
    "facetFilters": []
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "processorName": "Export tag groups and tags",
      "processorDescription": "Sample how to export all available tag groups and all tags as part of a",
      "fields": [
        "name"
      ],
      "output": [
```

```

{
  "key": {
    "expr": "content.id"
  },
  "values": [
    {
      "expr": "${lx.factsheet.id}"
    }
  ]
},
{
  "key": {
    "expr": "content.type"
  },
  "values": [
    {
      "expr": "${lx.factsheet.type}"
    }
  ]
},
{
  "key": {
    "expr": "name"
  },
  "values": [
    {
      "expr": "${lx.factsheet.name}"
    }
  ]
},
{
  "key": {
    "expr": "tagGroupsAndTags"
  },
  "values": [
    {
      "object": "${integration.tags.getAllTagGroups()}"
    }
  ]
}
]
}
]
}

```

A similar configuration will help gathering all tag groups where a specific Fact Sheet has tags set. The `lx.tagGroups` list will be filled with all tag groups where the Fact Sheet has at least one tag set. And inside each tag group element there will be a list of the found tags for the Fact Sheet. The below example filters and returns only a sub set (default tag group). Just adding `"${true}"` in the filter will ensure to return all tag groups and included tags the Fact Sheet has set.

Reading all tag groups for a fact sheet:

```

{
  "scope": {

```

```

"ids": [
  "bb8b0b74-f737-4f1b-a937-a06bddf3fe47"
],
"facetFilters": [
  {
    "keys": [
      "Application"
    ],
    "facetKey": "FactSheetTypes",
    "operator": "OR"
  }
],
},
"processors": [
  {
    "processorType": "outboundFactSheet",
    "processorName": "Export to LDIF",
    "processorDescription": "This is an example how to use the processor",
    "enabled": true,
    "fields": [
      "lifecycle",
      "location",
      "createdAt",
      "technicalSuitabilityDescription",
      "description"
    ],
    "relations": {
      "filter": [
        "relToParent",
        "relApplicationToITComponent"
      ],
      "fields": [
        "description"
      ],
      "targetFields": [
        "displayName",
        "externalId"
      ],
      "constrainingRelations": false
    },
    "tags": {
      "groups": [
        "SomeTagGroupName"
      ],
      "multipleGroups": "${dm.tagGroup.name == 'Other tags'}"
    },
    "subscriptions": {
      "types": [
        "RESPONSIBLE"
      ]
    },
    "documents": {
      "filter": ".*"
    },
  },

```

```

"output": [
  {
    "key": {
      "expr": "content.id"
    },
    "values": [
      {
        "expr": "${lx.factsheet.id}"
      }
    ]
  },
  {
    "key": {
      "expr": "content.type"
    },
    "values": [
      {
        "expr": "${lx.factsheet.type}"
      }
    ]
  },
  {
    "key": {
      "expr": "Description"
    },
    "values": [
      {
        "expr": "${integration.toJson(lx.tagGroups).toString()}"
      }
    ],
    "optional": true
  }
]
}

```

Calculating with single-select fields

This processor enables calculations with single-select fields

Example:

```

{
  "connectorId": "id-92476445-10b3-40f7-9386-6f13c61e4b89",
  "connectorType": "ee",
  "connectorVersion": "1.2.0",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "processors": [
    {
      "enabled": true,
      "filter": {

```



```

    "type": "DataObject"
  },
  "identifier": {
    "internal": "${content.id}"
  },
  "logLevel": "debug",
  "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
  "processorName": "Apps from Deployments",
  "processorType": "inboundFactSheet",
  "read": {
    "fields": [
      "businessValue",
      "projectRisk",
      "dataClassification"
    ],
    "noNullForOrdinal": true
  },
  "run": 0,
  "type": "DataObject",
  "updates": [
    {
      "key": {
        "expr": "description"
      },
      "values": [
        {
          "expr": "No null - ${lx.toOrdinal('dataClassification')}}"
        }
      ]
    }
  ],
  "variables": [
    {
      "key": "deploymentMaturity",
      "value": "${data.maturity}"
    }
  ]
}
]
}

```

Updating multi_select fields from an array

This processor supports updating multi_select fields in SAP LeanIX.

Example. Pre-requisite: Have the multi-select field myMultiSelect available in the workspace with options: FOO and BAR. Update myMultiSelect field with values from myField in input.

Connector:

```

{
  "processors": [
    {

```

```

"processorType": "inboundFactSheet",
"processorName": "Apps from Deployments",
"processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
"type": "Application",
"filter": {
  "exactType": "Deployment"
},
"identifier": {
  "external": {
    "id": {
      "expr": "${content.id}"
    },
    "type": {
      "expr": "externalId"
    }
  }
},
"updates": [
  {
    "key": {
      "expr": "name"
    },
    "values": [
      {
        "expr": "${data.app}"
      }
    ]
  },
  {
    "key": {
      "expr": "myMultiSelect"
    },
    "values": [
      {
        "expr": "${integration.output.valueOfForEachElementOf}"
        "forEach": {
          "elementOf": "${data['myField']}"
        }
      }
    ]
  }
]
}
]
}
}

```

LDIF input:

```

{
  "connectorType": "Kubernetes",
  "connectorId": "Kub Dev-001",
  "connectorVersion": "1.2.0",
  "lxVersion": "1.0.0",
  "description": "Imports Kubernetes data into LeanIX",

```

```

    "processingDirection": "inbound",
    "processingMode": "partial",
    "customFields": {},
    "content": [
      {
        "type": "Deployment",
        "id": "634c16bf-198c-1129-9d08-92630b573fbf",
        "data": {
          "app": "HR Service",
          "tags": [],
          "myField": [
            "F00"
          ],
          "version": "1.8.4",
          "maturity": "3",
          "clusterName": "westeurope"
        }
      },
      {
        "type": "Deployment",
        "id": "784616bf-198c-11f9-9da8-9263b0573fbe",
        "data": {
          "app": "Finance Service",
          "tags": [
            "Important"
          ],
          "myField": [
            "F00",
            "BAR"
          ],
          "version": "10.5",
          "maturity": "5",
          "clusterName": "westeurope"
        }
      }
    ]
  }
}

```

Remove content from fields

The integration API can be used to remove content. In case Values array contains "null" values after evaluating all configured elements in the Values Array, the Integration API will try to reset the configured field to an initial "not filled" state. This is specifically helpful for Single or multi select fields. String fields can simply be cleaned by passing an empty String. Numbers may rather be set to 0.

To avoid a warning, that no value could be found, ensure the "optional" field is used.

i Note

variableProcessor

Is used to only write values to internal variables. This will be used for aggregation use cases where the LDIF content needs to be used to only collect values without directly writing anything to SAP LeanIX.

Write-back to Fact Sheet

inboundFactsheet processor, the inboundRelation and the writeToLdif processor allow to read information from the Fact Sheet (currently supported: fields, relations, subscriptions, tags, documents and metrics) and use the information when writing back to the Fact Sheet. In case you need to work with read information in other processors, please write results to a variable first. The below example shows two use case examples, where a cost field is increased by the incoming value and an update of the risk section will only be done if the description is not starting with a key word "manually".

The example as well contains information how to use this feature. In case you define the read section for the inboundRelation processor, the fields will be read for the Fact Sheet defined in the "from" section. You can still read the fields from the target Fact Sheet using the "relations/targetFields" as shown below.

For the example to work, the workspace needs to contain a Project Fact Sheet with external ID "12345". Or change the LDIF data to an external ID of a Project Fact Sheet existing in the workspace.

Write to fact sheet based on current content:

```
{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Apps from Deployments",
      "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
      "type": "Project",
      "filter": {
        "exactType": "prj"
      },
      "identifier": {
        "external": {
          "id": {
            "expr": "${content.id}"
          },
          "type": {
            "expr": "externalId"
          }
        }
      },
      "run": 0,
      "updates": [
        {
          "key": {
            "expr": "budgetOpEx"
          },
          "values": [
            {
              "expr": "${lx.factsheet.budgetOpEx+data.monthlyOpEx}"
            }
          ]
        },
        {
          "key": {
            "expr": "projectRisk"
          },
          "values": [
```

```

    {
      "expr": "${(lx.tags.toString().contains('\\"name\\":\\"MANUAL_INPUT'))?null:data.risk}",
      "regexMatch": ".*"
    }
  ],
  "optional": true
},
{
  "key": {
    "expr": "projectRiskDescription"
  },
  "values": [
    {
      "expr": "${(lx.tags.toString().contains('\\"name\\":\\"MANUAL_INPUT'))?null:data.riskDescriptio",
      "regexMatch": ".*"
    }
  ],
  "optional": true
},
{
  "key": {
    "expr": "metrics"
  },
  "values": [
    {
      "expr": "${integration.toJson(lx.metrics.variableName.values)}"
    }
  ]
}
],
"logLevel": "debug",
"read": {
  "fields": [
    "budgetOpEx"
  ],
  "tags": {
    "groups": [
      "Other tags"
    ]
  },
  "relations": {
    "filter": [
      "relToParent",
      "relApplicationToITComponent"
    ],
    "fields": [
      "description"
    ],
    "targetFields": [
      "displayName",
      "externalId",
      "location"
    ]
  }
},

```

```

    "subscriptions": {
      "types": [
        "RESPONSIBLE"
      ]
    },
    "metrics": [
      {
        "name": "variableName",
        "measurement": "money",
        "fieldName": "dollars_per_day",
        "aggregationFunction": "MEAN",
        "groupBy": "1h",
        "start": "2020-01-20T00:00:00Z",
        "duration": "P0DT24H30M",
        "rules": {
          "key": "factSheetId",
          "comparator": "=",
          "compareWith": "${lx.factsheet.id}"
        }
      }
    ],
    "impacts": {
      "readAll": true
    }
  }
]
}

```

Sample LDIF:

```

{
  "connectorType": "showcaseUpdate",
  "connectorId": "showcaseUpdate",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "content": [
    {
      "type": "prj",
      "id": "12345",
      "data": {
        "monthlyOpEx": 50000,
        "risk": "lowProjectRisk",
        "riskDescription": "The risk is considered to be low."
      }
    }
  ]
}

```

Example to access fields on relations and on the target Fact sheet of a relation:

```

{
  "lx.relationsElement": {

```

```

    "id": "9316291b-361a-4050-ac79-bf9f96811fb1",
    "type": "relApplicationToITComponent",
    "target": {
      "id": "161abc0d-7bed-4440-b756-5c14a741e1ad",
      "name": "Application Development",
      "type": "ITComponent"
    },
    "activeFrom": "2021-01-18",
    "description": ""
  }
}

```

i Note

Access to fields on relations and relation target fields.

By defining the fields on the relations and on the target Fact Sheets of a relation, admins can use the values in JUEL expressions in the output section. The found relations need to be iterated using "forEach". Each element will then contain the standard information about e.g. name and type of a relation plus the requested fields. They can be accessed following the structure shown in the below example

Dynamic definition of Fact Sheet fields to read

Sometimes it is helpful to decide at run time which fields from a Fact Sheet to read and not hard code the names of the fields in the configuration.

For this purpose, Integration API allows to define a key "multipleFields" and a value that is a JUEL expression and will be resolved to boolean true and false. The Integration API will iterate over all available fields taken from the data model and allow the expression to do any filter logic required. As input value, the currently iterated field can be used with "dm.factSheetField.name". The type of the field can be identified with "dm.factSheetField.type"

In following JUEL expressions like forEach or update section, the list of read fields can be used with dm.factSheetFields, which is an object with keys: name, type and factsheetType

The below example would read all fields of type "STRING" from a Fact Sheet.

Dynamic fact sheet fields:

```

{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Apps from Deployments",
      "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
      "type": "Application",
      "filter": {
        "exactType": "Deployment"
      },
      "identifier": {
        "external": {
          "id": {
            "expr": "${content.id}"
          }
        }
      }
    },
  ],
}

```

```

    "type": {
      "expr": "externalId"
    }
  },
  "read": {
    "multipleFields": "${dm.factSheetField.type=='STRING'}"
  },
  "updates": [
    {
      "key": {
        "expr": "${integration.valueOfForEach.name}"
      },
      "values": [
        {
          "expr": "${lx[integration.valueOfForEach.name]} - added by Integration API"
        }
      ]
    }
  ],
  "forEach": "${dm.factSheetFields}",
  "logLevel": "debug"
}
]
}

```

In the sample, all String fields of the Fact Sheet get a string " - added by Integration API" appended.

multipleFields can be used for relations as well.

After evaluation of dynamic relation reading, fields dm.relationFilters[], lx.relationTargetFields[] and lx.relationFields[] are available.

Multiple fields on relations:

```

"multipleFields": "${data.fieldsToRead.contains(dm.relationField.name) && dm.relationField.type=='I'}",
"multipleTargetFields": "${data.fieldsToRead.contains(dm.factSheetField.name) && dm.relationType=='I'}",
"multipleFilters": "${data.myDynamicRelationList.contains(dm.relationType)}",

```

Same way, dynamic definition of Tags can happen: (after execution, the collected tag groups are available in a "lx.tagGroups" list.

In the filter "multipleGroups", the object of the currently iterated tag group "dm.tagGroup" can be used.

Dynamic tag group definition:

```

{
  "read": {
    "tags": {
      "multipleGroups": "${true}",
      "groups": [
        "Cloud Transformation"
      ]
    }
  }
}

```



```

    ]
  }
}
}

```

i Note

Availability of information read from the Fact Sheet.

Information read from the Fact Sheet is available in the output section. The information is not available in the outer `forEach`, in the identifier and the filter section. The reason for this is, that at the time when the content in these sections is evaluated, the target Fact Sheet is not yet identified.

Auto deletion with inboundProcessors

Integration API supports the processing mode "full" mode when creating the configuration. Only in case, the configuration is set to mode "full", a section "deletionScope" is read from the processor configuration. The following operations are supported:

- Deletion of Fact Sheets If that section does contain a key "factSheets", all Fact Sheets matching the scope query inside will be removed if they are not found in the processed LDIF
- All Fact Sheets that match the deletion scope but are not touched by an inbound Data Processor during processing, will be removed (set to "Archived")
- Relations can be automatically removed as well. The structure to define relations to be deleted is similar. See an example configuration below. The example removes all relations but by narrowing the scope to fewer Fact Sheets, only for these Fact Sheets relations will be removed
- Documents can be deleted by defining a scope of Fact Sheets and adding a regular expression pattern to match the documents by name that may be removed if they are no longer referenced by the incoming LDIF data

How does this work? Do I need to delete any data manually or first delete in a processor?

The concept of deletion is unique with the Integration API in that there is no active deletion needed by the user. All the deletion is done by the iAPI automatically through the configuration of a 'deletion scope'.

The logic of deletion works as follows:

1. In an inbound 'Run', factsheets and other data like tags or relations are created and updated. These actions of creating or updating mark artifacts as 'touched'.
2. Upon finishing a the 'Run', the API determines which artifacts within the defined deletion scope have not been touched at all in the course of the run and these untouched artifacts are deleted.
3. This prevents constant "delete-create-delete-create" cycles that would be visible in the audit log of factsheets and avoids any manual work deletion any data

i Note

Sending data for an archived Fact Sheet in the SyncRun.

In case an archived Fact Sheet exists in the Workspace with the same externalId as sent in one of the input LDIFs, Integration API will create a new FactSheet rather than recovering the deleted Fact Sheet, this is done in order to avoid unexpected data on the archived Fact Sheet reflected on the Active Fact Sheet.

This scenario can only happen when the "uniqueFactSheet" attribute for a Fact Sheet is set to "false" when set to "true" externalIds are already cleared while archiving a Fact Sheet.

i Note

Please note that you can define multiple sets of deletion scopes for every type (e.g. 2 Fact Sheet deletion scopes and 3 relation deletion scopes). Processed items during synchronization runs will be compared against each set separately. Any item in each deletion scope definition will be removed if not touched during processing. It is allowed to even define overlapping scopes. Each item will be handled once only.

Example

The example processor in this section removes the following data from a workspace:

- All project fact sheets that are no longer part of the incoming LDIF data
- All relations from applications to IT components
- All documents prefixed with "MyDocs_"

⚠ Caution

Before running this example processor, consider implications and always proceed with caution. The processor is configured to delete all project fact sheets present in the workspace. To limit the deletion scope, modify the code so that a tag such as "TEST_PRJ" is assigned to test projects. You can add this tag as a `facetFilter` to the deletion scope definition as shown below:

```
{
  "facetKey": "${integration.tags.getTagGroupId('Testing')}",
  "operator": "OR",
  "keys": [
    "${integration.tags.getTagId('Testing','TEST_PRJ')}"
  ]
}
```

Run this example processor multiple times to first create all projects, then remove one item and try again.

Sample configuration for processingMode "full" to remove projects automatically:

```
{
  "deletionScope": {
    "maximumDeletionRatio": {
      "relations": 40,
      "factSheets": 30
    },
    "factSheets": [
      {
        "scope": {
          "facetFilters": [
            {
              "keys": [
                "Project"
              ],
            },
          ],
        },
      },
    ],
  },
}
```

```

    "facetKey": "FactSheetTypes",
    "operator": "OR"
  }
],
"ids": []
}
}
],
"relations": [
{
  "relationTypes": [
    "relApplicationToITComponent"
  ],
  "scope": {
    "facetFilters": [],
    "ids": []
  }
}
],
"documents": [
{
  "documentMatches": [
    "^MyDocs_.*"
  ],
  "scope": {
    "facetFilters": [
      {
        "keys": [
          "Project"
        ],
        "facetKey": "FactSheetTypes",
        "operator": "OR"
      }
    ],
    "ids": []
  }
}
]
},
"processors": [
{
  "processorType": "inboundFactSheet",
  "processorName": "Apps from Deployments",
  "processorDescription": "Creates LeanIX Applications from Deployments",
  "type": "Project",
  "filter": {
    "exactType": "prj"
  },
  "identifier": {
    "external": {
      "id": {
        "expr": "${content.id}"
      },
      "type": {

```

```

    "expr": "externalId"
  }
},
"run": 0,
"updates": [
  {
    "key": {
      "expr": "name"
    },
    "values": [
      {
        "expr": "${data.name}"
      }
    ]
  }
],
"enabled": true,
"logLevel": "debug"
}
]
}

```

Sample LDIF:

```

{
  "connectorType": "prjFull",
  "connectorId": "prjFull",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "content": [
    {
      "type": "Project",
      "id": "prj-42",
      "data": {
        "name": "Project 42"
      }
    },
    {
      "type": "Project",
      "id": "prj-43",
      "data": {
        "name": "Project 43"
      }
    },
    {
      "type": "Project",
      "id": "prj-44",
      "data": {
        "name": "Project 44"
      }
    }
  ]
}

```

```

    ]
  }
}

```

Remove tags

Deletion of Tags works similar. One or more deletion scope sections of type "tags" needs to be configured. The scope defines the set of Fact Sheets to be looked at when removing tags and allows to configure a tag group and a tag name to be deleted. Tag names support regular expression matching to allow removal of tags based on name patterns. Please note that tags will be removed from the Fact Sheets where no longer referenced by processors adding them but tags themselves will not be deleted.

The following example processor removes tags and subscriptions from fact sheets.

Example configuration to remove tags and subscriptions from fact sheets:

```

{
  "deletionScope": {
    "maximumDeletionRatio": {
      "tags": 40
    },
    "tags": [
      {
        "tagScopes": [
          {
            "group": "myGroup",
            "tag": "Prefix_.*"
          }
        ]
      },
      {
        "scope": {
          "facetFilters": [
            {
              "keys": [
                "Project"
              ],
              "facetKey": "FactSheetTypes",
              "operator": "OR"
            }
          ],
          "ids": []
        }
      }
    ],
    "advanced": "${lx.tag.tagGroup.name==null}"
  },
  "subscriptions": [
    {
      "subscriptionScopes": [
        {
          "type": "RESPONSIBLE",
          "roles": [
            "My Role"
          ]
        }
      ]
    },
    {
      "scope": {

```

```

    "facetFilters": [
      {
        "keys": [
          "Project"
        ],
        "facetKey": "FactSheetTypes",
        "operator": "OR"
      }
    ],
    "ids": []
  }
}
],
"processors": []
}

```

Preventing removal of items

The Integration API can be configured to not delete any configured item if the ratio of to be deleted items in any deletion scope exceeds a defined ratio compared to the total items in scope for the deletion scope. This can be used to protect the existing data in case of erroneously sending incomplete data into a full scope synchronisation with deletion defined. by defining "maximumDeletionRatio" for each type of deletion scope, this mechanism can be used. A format example can be found in the examples above. A threshold of 30 means that all deletion will be stopped if 30% or more items would be deleted for a given deletion scope.

i Note

To create valid JSON content to define the scope of Fact Sheets to be deleted if they no longer exist in the incoming LDIF, admins may want to use an outbound configuration. Using this configuration, a Button "Scope" is available that opens the facet filter UI. Once confirmed, the scope is automatically pasted to the processor configuration. Admins may copy and paste it into the inbound configuration where they need to use automatic deletion.

Advanced deletion

The functionality to delete content for elements we do no longer see references in the LDIF, can be used in an even more advanced was if required. The deletion scope for Relations, Tags and Documents may contain an optional key "advanced". If configured in the specific deletion scope, the value is evaluated as a JUEL expression resulting in "true" or "false". Only such elements will be added to the scope of elements potentially to be deleted if not touched where the JUEL expression evaluates to "true".

Multiple deletion scopes of same or different types can be defined. Please note that deletions of Fact Sheets will always happen last. This allows to use the Fact Sheet meta data `lx.factsheet.*` even for Fact Sheets that will be deleted. If allows to even use the owner field of the Fact Sheet deletion and evaluate in a relation, tag, document or subscription deletion scope in the state before the owner of the current run will be removed from the field (see ownership concept of advanced Fact Sheet deletion)

Please note, that currently no content from fields of type `projectStatus` is available for advanced deletion.

The information about the current Fact Sheet (for relations always the source Fact Sheet of a relation) is available using `"lx.factsheet.*"`. All fields of the Fact Sheet can be used. In addition, all meta data fields of relations, documents and tags can be used in the JUEL for the related type of deletion scope.

Sample configuration to execute an advanced filter on document meta data:

```

{
  "deletionScope": {
    "factSheets": [
      {
        "scope": {
          {
            "facetFilters": [
              {
                "keys": [
                  "Project"
                ],
                "facetKey": "FactSheetTypes",
                "operator": "OR"
              }
            ]
          }
        }
      },
    ],
    "relations": [
      {
        "relationTypes": [
          "relProjectToITComponent"
        ],
        "scope": {
          "ids": [],
          "facetFilters": []
        },
        "advanced": "${lx.relation.description.contains('from hr service')}}"
      }
    ],
    "documents": [
      {
        "documentMatches": [
          ".*"
        ],
        "scope": {
          "facetFilters": [
            {
              "keys": [
                "Project"
              ],
              "facetKey": "FactSheetTypes",
              "operator": "OR"
            }
          ]
        },
        "advanced": "${lx.document.documentType.equals('jira') || lx.document.name.equals('someName') }"
      }
    ],
  },
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Read Projects",
    }
  ]
}

```

```

    "processorDescription": "Creates LeanIX Projects from Project Management Solution",
    "type": "Project",
    "filter": {
      "exactType": "prj"
    },
    "identifier": {
      "external": {
        "id": {
          "expr": "${content.id}"
        },
        "type": {
          "expr": "externalId"
        }
      }
    }
  }
}

```

Deletion for multiple external sources

Advanced deletion is as well available for Fact Sheets but works slightly different. Advanced Fact Sheet deletion supports multiple external sources for one SAP LeanIX Fact Sheet. Deletion would not happen unless the last referenced source of a Fact Sheet does not longer contain information about the Fact Sheet.

This functionality is helpful in cases where a Fact Sheet might be created and again removed by potentially more than one foreign system providing separate LDIF to update the SAP LeanIX side.

In cases, where the Fact Sheet was no longer referenced by one of the sources, a deletion would not be a valid solution unless all sources no longer contain the information.

For such cases, the advanced deletion allows every sources to set a unique id as a marker. If this marker, called "owner", was found in the configuration, the Integration API first checks the field with all markers and only removes (archives) the Face Sheet if the list of markers found in the field is empty. The field to store the markers needs to be created in the data model as a standard String type. Integration API will read the content ant treat as a JSON list.

The below example shows example usage including the way to add the marker for a specific owner as part of the output section.

Please note that the example uses the "alias" field to store the owner information. This is for testing and demonstration as it allows easy inspection and required no data model work. For production usage, this needs to be written to a new created field not visible in the UI.

Advanced fact sheet deletion:

```

{
  "deletionScope": {
    "factSheets": [
      {
        "scope": {
          "ids": [],
          "facetFilters": [
            {
              "keys": [

```



```

    "Process"
  ],
  "facetKey": "FactSheetTypes",
  "operator": "OR"
}
],
},
"owner": {
  "fieldName": "alias",
  "ownerId": "myOwner"
}
}
],
},
"processors": [
{
  "processorType": "inboundFactSheet",
  "processorName": "Apps from Deployments",
  "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
  "type": "Process",
  "identifier": {
    "external": {
      "id": {
        "expr": "fullSyncOwnerTest ${integration.valueOfForEach}"
      },
      "type": {
        "expr": "externalId"
      }
    }
  },
},
"run": 0,
"updates": [
{
  "key": {
    "expr": "name"
  },
  "values": [
    {
      "expr": "Full Sync Owner Test ${integration.valueOfForEach}"
    }
  ]
},
{
  "key": {
    "expr": "alias"
  },
  "values": [
    {
      "expr": "${helper:addIfNotExisting(lx.factsheet.alias, 'myOwner')}}"
    }
  ]
}
],
"enabled": false,

```

```

    "forEach": "${data.tags}",
    "logLevel": "debug",
    "read": {
      "fields": [
        "alias"
      ]
    }
  }
}
]
}

```

Delete tags of multiple groups

Use case: Remove all tags of two tag groups from Fact Sheets of a given type can be performed as easy as shown in the below example. The deletion scope marks all Fact Sheets with tags in the tag groups and the given Fact Sheet type to be deleted if not touched. Then the processor configuration stays empty. No Fact Sheet will be touched when processing. At the end of the run, all tags will be removed.

Please do not forget to increase the deletion limit set to 50% by default to 101 to allow removing even 100% of the tags.

Removing all tags from all fact sheets of a specific type:

```

{
  "deletionScope": {
    "maximumDeletionRatio": {
      "tags": 101
    },
    "tags": [
      {
        "tagScopes": [
          {
            "group": "Cloud:Region",
            "tag": ".*"
          }
        ],
        "scope": {
          "facetFilters": [
            {
              "facetKey": "FactSheetTypes",
              "operator": "OR",
              "keys": [
                "CloudComponent"
              ]
            }
          ],
          "ids": []
        }
      }
    ],
    "tagScopes": [
      {
        "group": "Cloud:Cloud Service",
        "tag": ".*"
      }
    ]
  }
}

```

```

    }
  ],
  "scope": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "CloudComponent"
        ]
      }
    ],
    "ids": []
  }
},
{
  "tagScopes": [
    {
      "group": "Cloud:Tech Category",
      "tag": ".*"
    }
  ],
  "scope": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "CloudComponent"
        ]
      }
    ],
    "ids": []
  }
}
]
},
"processors": []
}

```

Deletion of Subscriptions

Example configuration to use advanced deletion on subscriptions. Here, to remove all subscriptions on application Fact Sheets that have an anonymised user as a subscriber for a subscription of type "RESPONSIBLE".

Please note, that the below configuration does not configure any processors, thus can work with an empty LDIF as input to trigger Integration API.

Delete all RESPONSIBLE subscriptions of anonymized users for application fact sheets:

```

{
  "deletionScope": {
    "subscriptions": [

```

```

{
  "subscriptionScopes": [
    {
      "type": "RESPONSIBLE",
      "roles": []
    }
  ],
  "scope": {
    "facetFilters": [
      {
        "keys": [
          "Application"
        ],
        "facetKey": "FactSheetTypes",
        "operator": "OR"
      }
    ]
  },
  "advanced": "${lx.subscription.user.userName=='AnonymizedUser'}"
}
],
"processors": []
}

```

Empty LDIF for advanced deletion:

```

{
  "connectorId": "subscription",
  "connectorType": "subscription",
  "connectorVersion": "1.0.0",
  "content": [],
  "lxVersion": "1.0.0"
}

```

i Note

Using External IDs in pathfinder search scopes.

When working with integrations, specifically with deletion scopes it is handy to know that pathfinder is capable filtering documents by their external id. and not only by internal id which is most of the times not known to foreign systems.

To filter by external ids, just use the field "externalIds" instead of "ids" in the search scope definition.

Please note that Pathfinder required a special syntax when defining external ids using the name of the externalID field, a slash ("/") then value of the externalId.

Example for the default externalId field:

```
"externalIds": ["${'externalId/'.concat(header.customFields.myExternalId)}"]
```

Data exchange and Aggregation between Objects

In some situations it may be required to use information from multiple Data Objects and store a joint result in another entity like a Fact Sheet or Relation. Even Creating specific relations if we find certain value combinations in different Data Objects is possible.

In order to perform such operations, a "variables" section is available to write and add to, while iterating over Data Objects. Data Processors in the following Runs (!! Not in the same Run) can then read the values and perform defined operations on them.

This works in the following steps:

Working with Variables	Details	Example
Define the variable with a default value	This avoids errors if a variable was never written but later a try to access is configured (example in the admin section of the UI)	
Write additional values to the variable	This is available on all Data Processors by adding a "variables" section (same structure as in step 1) and assigning a value to the variable.	
In a subsequent "Run", processors can access the variable and perform operations on it or even use the variable in the "forEach" section (see below) to execute steps for every entry for the variable	Variables can have dynamic names based on content. In combination with the "forEach" feature, this allows powerful use cases.	As an example, the user needs to collect cost data from various data objects. The cost data needs to be grouped by the subscription they belong to. Each data object contains the cost in field "cost" and the id of the subscription in a field "subscriptionId". The user simply needs to collect all subscriptions in a variable "subscriptionList" and add each found cost to another variable named "_cost". in the next run, a data processor iterates over all unique entries in "subscriptionList" ("forEach": "\${variables.subscriptionList.distinct()}"). Then the aggregated cost variable can be accessed by using the name taken from "integration.valueOfForEach" plus "_cost". Please see the example below.

Writing Variables using Expressions

The following example shows how to write variables using expressions to define names and values.

```
{
  "variables": [
    {
      "key": "prefix_${dataMyNameFromDataObjectValue}",
      "value": "${data.myValueFromDataObject}"
    }
  ]
}
```

Below is an example of a processor with a matching LDIF that shows how variables work. In the first run which is marked by the processor with "run": 0, the variable section can be called on with the key aggregatedCosts and gathers together all the costs in the data section of each entry of an LDIF that is gathered by the filter in place. In this case, that filter is a Fact Sheet of type

ITComponent. In the next run marked with "run": 1, the processor is calling the sum function on the variable aggregatedCosts and writing the sum to the description field of the Fact Sheets that fall under the specified filter, which in this case is all the Applications in the LDIF.

The example's result is three Fact Sheets created, two IT Components, and one Application with a description of 11. Note that the costs of the IT Components were not written in the IT Component's Fact Sheets.

Using variables in processors:

```
{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Create IT Components",
      "processorDescription": "One Processor for IT Components",
      "enabled": true,
      "type": "ITComponent",
      "identifier": {
        "external": {
          "id": {
            "expr": "${content.id}"
          },
          "type": {
            "expr": "externalId"
          }
        }
      },
      "run": 0,
      "filter": {
        "exactType": "ITComponent"
      },
      "updates": [
        {
          "key": {
            "expr": "name"
          },
          "values": [
            {
              "expr": "${data.name}"
            }
          ]
        }
      ],
      "variables": [
        {
          "key": "aggregatedCosts",
          "value": "${data.cost}"
        }
      ],
      "processorType": "inboundFactSheet",
      "processorName": "Create Applications",
      "processorDescription": "Aggregated IT Costs in Application's Description",
    }
  ]
}
```

```

    "enabled": true,
    "type": "Application",
    "identifier": {
      "external": {
        "id": {
          "expr": "${content.id}"
        },
        "type": {
          "expr": "externalId"
        }
      }
    },
    "run": 1,
    "filter": {
      "exactType": "Application"
    },
    "updates": [
      {
        "key": {
          "expr": "name"
        },
        "values": [
          {
            "expr": "${data.name}"
          }
        ]
      },
      {
        "key": {
          "expr": "description"
        },
        "values": [
          {
            "expr": "${variables.aggregatedCosts.sum()}"
          }
        ]
      }
    ]
  },
  "variables": {}
}

```

Sample LDIF:

```

{
  "connectorType": "ee",
  "connectorId": "Kub Dev-001",
  "connectorVersion": "1.2.0",
  "lxVersion": "1.0.0",
  "content": [
    {
      "type": "ITComponent",
      "id": "itc1",

```

```

    "data": {
      "name": "IT1",
      "cost": 5
    }
  },
  {
    "type": "ITComponent",
    "id": "itc2",
    "data": {
      "name": "IT2",
      "cost": 6
    }
  },
  {
    "type": "Application",
    "id": "app",
    "data": {
      "name": "My App"
    }
  }
]
}

```

Dynamic Variable Handling

Example for dynamic variables:

```

${variables[integration.valueOfForEach.concat('_cost')].sum()}
(which is same as: variables['12345_cost'].sum() in case valueOfForEach is "12345")

```

Supported Dynamic Variable Operations

Supported operations are listed below. Each invalid entry will be counted as "0" when calculating.

Method	Details
myVariable.sum()	Creates a number adding all values in the variable
myVariable.get()	Reads the variable as a single value (first value)
myVariable.join(String delimiter)	Creates a String concatenating all values using the passed string. E.g. myVariable=["1","2","3"] will be converted to "1, 2, 3" by variables.myVariable.join(', ')
myVariable.distinct()	Returns the same list of values but with duplicate entries removed. The result can be used to do further calculations like e.g. variables.myVariable.distinct().join(', ') to show all unique entries
myVariable.contains(String value)	Returns a boolean that e.g. can be used in advanced filters for Data Processors to execute a Data Processor only if certain values occur in a variable
myVariable.count()	Returns a number of entries in the variable
myVariable.average()	Calculates the math average of all values. non numerical values will be ignored
myVariable.toList()	Converts to a Java-List in order to execute standard java list methods

Method	Details
myVariable.max()	Selects the highest number value in the variable and returns it
myVariable.min()	selects the lowest number value in the variable and returns it
myVariable.getNumbers()	Filters out all non-numeric values and returns a list of values other methods explained can be executed on. In the variable and allows to safely calculate average, min, max.. avoiding errors with values added that cannot be converted to a number myVar.getNumbers().average() uses the numbers only that have been added to the variable
myVariable.selectFirst()	Picks the first available String from method parameters that match any of the values of myVariable. If nothing was matched, the first parameter will be selected (default). Please note that the list of options to match needs to be provided as a list as JUEL does not allow a parameter list variable parameters. A helper function was added to allow creation of a list from any string split result (array). Example: variables.myVariable.selectFirst(helper.toList('default','optionHighPrio','optionMediumPrio','optionLowPrio'))

forEach Logic

Each data processor provides additional capabilities to handle values that are lists. Using the standard functionality, every data processor will be executed exactly one time for each data object sent to the Integration API.

Sometimes however, there is a need to update multiple Fact Sheets or multiple fields in a Fact Sheet for each value we find in a list of values found in the LDIF.

Iterate over lists in Data Processor Configuration:

```
{
  "data": {
    "attachedDocuments": [
      {
        "extension": "vsdx",
        "name": "thediagram.vsdx",
        "displayName": "Diagram",
        "url": "sotrage.azure.com/123/thediagram.vsdx",
        "content": null
      },
      {
        "extension": "docx",
        "name": "thedoc.docx",
        "displayName": "Documentation",
        "url": "sotrage.azure.com/123/thedoc.docx",
        "content": null
      },
      {
        "extension": "html",
        "name": "webpage.html",
        "displayName": "Web Page",
        "url": null,
        "content": "<body>the vm 789 ...</body>"
      }
    ],
    "version": "1.8.4",
    "myForEachField": "attachedDocuments",
    "maturity": "3",
  }
}
```

```

    "note": "I did the first comment here",
    "Home Country": "D",
    "Other Country": "UK",
    "clusterName": "leanix-westeuropa-int-aks"
  }
}

```

inboundFactSheet ForEach example:

```

{
  "processorType": "inboundFactSheet",
  "processorName": "Deployment To Application",
  "processorDescription": "The processor creates or updates an Application from every data object of type",
  "type": "Application",
  "name": "My Awesome App",
  "run": 0,
  "enabled": true,
  "identifier": {
    "external": {
      "id": {
        "expr": "${content.id}"
      },
      "type": {
        "expr": "externalId"
      }
    }
  },
  "filter": {
    "exactType": "Deployment"
  },
  "forEach": "${data.myForEachField}",
  "updates": [
    {
      "key": {
        "expr": "name"
      },
      "values": [
        {
          "expr": "${data.attachedDocuments[integration.indexOfForEach].name}", // or in short: ${integration.indexOfForEach}
          "regexReplace": {
            "match": "",
            "replace": ""
          }
        }
      ],
      "expr": "${data.value}"
    }
  ]
}

```

Using the "forEach" section in each data processor as in the example above. Will result in executing the data processor "Deployment To Application" four times for the given data object and each run will allow the user to use the index of the current iteration in all expressions (integration.indexOfForEach).

To fill some output field of the data processor with the specific url (see example above), the configuration would look like this: `${data.attachedDocuments[integration.indexOfForEach].name}`. This will generate the three different names of the attached documents in each run of the data processor. This could be used to create separate Fact Sheets and relations from the source data.

There is another way to access the value of the element referenced by the current index:

`${integration.valueOfForEach}`

Which is the same as:

`${data.attachedDocuments[integration.indexOfForEach].name}`

The index variable however can be used to reference the same index of another list element e.g. Important note: The admin can configure a "regexReplace" section in the forEach section. This will allow to manipulate the JSON representation of the value object resulting from the expression. In case such a manipulation is configured, it will have impact on the "integration.valueOfForEach" and not alter the original data one may reference using the indexOfForEach variable in the original data and reference manually.

Of course, the logic could be used to always execute a data processor n times. Just add '[1,2,3]' as configuration and the data processor will execute three times with the index variable integration.indexOfForEach set to 0-2 for reference.

In case the field 'attachedDocuments' is not available or contains an empty list, the data processor will not execute (operate on an empty list). In case the url is a single value and no list, the data processor will execute once.

The Integration API allows to iterate over list values and map values. In case of iterating over a map, indexOfForEach will always return -1 as maps are not sorted. For maps there is an additional variable "keyOfForEach" available providing access to the name of the key. The value will be accessed with "valueOfForEach"

Example: forEach

Sample LDIF with a map:

```
{
  "connectorType": "ee",
  "connectorId": "Kub Dev-001",
  "connectorVersion": "1.2.0",
  "lxVersion": "1.0.0",
  "content": [
    {
      "type": "Deployment",
      "id": "634c16bf-198c-1129-9d08-92630b573fbf",
      "data": {
        "app": "HR Service",
        "version": "1.8.4",
        "myList": [
          "lValue1",
          "lValue2"
        ],
        "myMap": {
          "key1": "value1",
```

```

    "key2": "value2"
  }
}
}
]
}

```

Iterate over maps:

```

{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Apps from Deployments",
      "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
      "type": "Application",
      "filter": {
        "exactType": "Deployment"
      },
      "identifier": {
        "external": {
          "id": {
            "expr": "${content.id}"
          },
          "type": {
            "expr": "externalId"
          }
        }
      },
      "run": 0,
      "updates": [
        {
          "key": {
            "expr": "name"
          },
          "values": [
            {
              "expr": "${data.app}"
            }
          ]
        },
        {
          "key": {
            "expr": "description"
          },
          "values": [
            {
              "expr": "${integration.keyOfForEach}: ${integration.valueOfForEach}"
            }
          ]
        }
      ],
      "forEach": "${data.myMap}",
      "logLevel": "debug"
    }
  ]
}

```

```

    }
  ]
}

```

For each logic can as well be applied inside the value section for and update key. The list or map will be iterated and the values result will contain n entires that are then mapped to the defined key. Please ensure to set the mode to "list" if not only the first value is to be used as a result (default mode for every key in the update section of a processor is "selectFirst" to only take the first non null result from what was defined in the values array. The "inner forEach" is behaving exactly as if the admin defined a fix number of elements in the "values" section.

Example: Nested forEach

Using all three options to iterate with "forEach" functionality, Integration API now allows to ingest data in LDIF where data structures are nested up to three levels.

Example configuration of "inner forEach":

```

{
  "key": {
    "expr": "targetITComponents"
  },
  "mode": "list",
  "values": [
    {
      "forEach": {
        "elementOf": "${lx.relations}",
        "filter": "${true}"
      },
      "map": [
        {
          "key": "id",
          "value": "${integration.output.valueOfForEach.target.id}"
        },
        {
          "key": "type",
          "value": "${integration.output.valueOfForEach.target.type}"
        },
        {
          "key": "name",
          "value": "${integration.output.valueOfForEach.target.displayName}"
        }
      ]
    }
  ]
}

```

Given the processor was configured to read relations (read section) and put the results into a list "lx-relations", the above example of an outboundFactsheet processor, will output all relation results into an array as value of a key named "targetITCmponents".

Please note that admins may configure the filter JUEL expression evaluating to boolean in order to not have some of the input list elements in the output. The JUEL may contain references to "integration.output.valueOfForEach" and filter on any content.

A third option to iterate using "forEach" is to add the key at the level of "updates". It allows to create a dynamic set of field updates to be pushed to e.g. a Fact Sheet. In the below example the fields to be updated will be read from the incoming LDIF. In order to execute the example, the referenced Fact Sheet needs to already exist. Most easy way is to one time execute the "starter example" configuration on the workspace.

Example: forEach to update dynamic fields

Example how to use forEach to update a dynamic set of fact sheet fields:

```
{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Apps from Deployments",
      "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
      "type": "Application",
      "filter": {
        "exactType": "Deployment"
      },
      "logLevel": "debug",
      "identifier": {
        "external": {
          "id": {
            "expr": "${content.id}"
          },
          "type": {
            "expr": "externalId"
          }
        }
      },
      "updates": [
        {
          "key": {
            "expr": "${integration.updates.keyOfForEach}"
          },
          "values": [
            {
              "expr": "${integration.updates.valueOfForEach}"
            }
          ],
          "forEach": {
            "elementOf": "${data}",
            "filter": "${integration.updates.valueOfForEach!='toBeFiltered'}"
          }
        }
      ]
    }
  ]
}
```

Sample LDIF:

```

{
  "connectorType": "Kubernetes",
  "connectorId": "Kub Dev-001",
  "connectorVersion": "1.2.0",
  "lxVersion": "1.0.0",
  "description": "Imports Kubernetes data into LeanIX",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "Deployment",
      "id": "634c16bf-198c-1129-9d08-92630b573fbf",
      "data": {
        "name": "HR Service",
        "version": "toBeFiltered",
        "description": "test description"
      }
    }
  ]
}

```

Use "object" key to output all objects

The "object" key might be used instead of "map" or "expr" to retrieve a representation of any potential input object as a defined value. This allows to easily export all information without need to know about the details inside the object.

Use Object key Value:

```

{
  "scope": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "Application"
        ]
      }
    ],
    "ids": [
      "90a8296c-92fe-4009-a4cf-21db710719ec"
    ]
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",
      "logLevel": "debug",
      "fields": [
        "lifecycle"
      ],
      "output": [
        {

```

```

    "key": {
      "expr": "content.id"
    },
    "values": [
      {
        "expr": "${lx.factsheet.id}"
      }
    ]
  },
  {
    "key": {
      "expr": "content.type"
    },
    "values": [
      {
        "expr": "${lx.factsheet.type}"
      }
    ]
  },
  {
    "key": {
      "expr": "description"
    },
    "values": [
      {
        "object": "${lx.factsheet.lifecycle}"
      }
    ]
  }
]
}
]
}

```

Filter Processor execution based on current Fact Sheet content

Using the onRead filter in an inbound processor allows to execute a processor based on currently existing Fact Sheets. The processor can be configured to only execute if a Fact sheet already exists or if the Fact sheet has defined values in some fields.

Or exactly the other way round: The Processor may only be executed if the Fact Sheet does not yet exist.

The following Example shows a Processor that will execute if a Fact Sheet exists that has a defined name and is flagged with a certain Tag. It adds " (Cloud)" to the name of a Fact Sheet if current the name is exactly as defined in the data object and a tag "Public Cloud" in a tag group "Cloud Transformation" is set on the Fact Sheet.

Processor configuration to demonstrate the onRead filter:

```

{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Apps from Deployments",
      "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",

```



```

"type": "Application",
"filter": {
  "exactType": "Deployment",
  "onRead": "${lx.factsheet.name==data.name && lx.tags.size()>0 && lx.tags[0].name=='Public Cloud'
},
"identifier": {
  "external": {
    "id": {
      "expr": "${content.id}"
    },
    "type": {
      "expr": "externalId"
    }
  }
},
"updates": [
  {
    "key": {
      "expr": "name"
    },
    "values": [
      {
        "expr": "${lx.factsheet.name} (Cloud)"
      }
    ]
  }
],
"logLevel": "debug",
"read": {
  "fields": [
    "name"
  ],
  "tags": {
    "groups": [
      "Cloud Transformation"
    ]
  }
}
}
]
}

```

Caution

The onRead filter is only available for inbound processors. The outbound processor will ignore setting this filter configuration.

Functionality is available same way outbound however as it is possible to use read content in the "advanced" filter like e.g. "filter": {"advanced": "\${lx.relations.size()>0}"}, for cases where you only want to export if any requested relation was found

i Note

Order of RegEx execution.

Using the replace regEx will allow to modify the output after applying the match regEx

Load large LDIFs

Depending on the source of the incoming data, LDIF files can be very large. The Integration API may not accept LDIF larger than 50 MB. If your file is bigger, it needs to be provided as a URL reference on an Azure Blob storage.

The configuration can be added to the Processor configuration part:

Read LDIF from Azure Blob Storage:

```
{
  "dataProvider": {
    "url": "${header.customFields.url}"
  },
  "processors": [
  ]
}
```

The value can be configured fix or as in the example passed in as part of the custom fields information in the LDIF. Please ensure to not send the "content" section in case you want to read from Azure. If content is part of the API call, this content will be used instead of the content in the Azure storage.

The URL needs to contain the path to the blob storage entry plus the Azure SAS token. For details, refer to the [Azure documentation](#) .

i Note

No support for IP whitelisting.

Please note that we process from our Azure infrastructure where IP addresses can dynamically change. Reading data from a URL only works if no IP whitelisting is set. Instead SaS tokens with limited ttl can be used.

Long running API calls

The default behaviour of Integration API is to execute all changes with the user that is provided by the API client when logging in. E.g. the history of factsheets will show this user as the one executing the changes as if the user logged into the SAP LeanIX UI and did the changes manually. This is easy to understand and communicate. In some situations however, it may not be sufficient.

Use Cases may be:

- The process takes more than 60 minutes.
- All changes the API does are caused by different login users, but should rather be shown to the user as a change the Integration API did instead of showing all the different API users.

In order to support long running inbound or outbound processes, an API Token may be provided in the configuration. This will be used instead of the access token that comes with the call from the API (as it will expire after one hour and does not contain any refresh token to grant access to the workspace data for more than 60 minutes)

Configure API Token as part of the Processor configuration:

```
{
  "credentials": {
    "apiToken": "...",
  },
}
```

```
"processors": [
]
}
```

As an alternative, the Integration API can be configured to use a "Technical User" for accessing the SAP LeanIX Pathfinder backend. This user called INTEGRATION_API will be automatically created in the workspace by the API if it is not existing.

To use the Technical Users, please add the following section.

Usage of the default Technical User by the Integration API:

```
{
  "credentials": {
    "useTechnicalUser": true
  },
  "processors": [
  ]
}
```

Executing Processor Configurations with Custom Technical Users

Besides using the default technical user that is created by the integration, it is possible to use technical users that are already set up in the workspace. To execute a processor configuration with a custom technical user, the user ID of the technical user has to be added to the credentials section of the configuration. In the following example, the CUSTOM_TECHNICAL_USER_ID placeholder would need to be replaced by the ID of the technical user that will be used when reading data from or writing data to your workspace.

Usage of a Custom Technical User by the Integration API:

```
{
  "credentials": {
    "technicalUserId": "CUSTOM_TECHNICAL_USER_ID"
  },
  "processors": [
  ]
}
```

Restricting the Execution of Processor Configurations

To restrict the execution of processor configurations to specific users, you can add the executionRestrictions object next to the processors of the configuration.

Restricting the execution of Integration API processor configurations:

```
{
  "executionRestrictions": {
    "defaultTechnicalUser": true,
    "userIds": ["USER_ID"]
  },
  "processors": [
```

```

    ]
  }

```

The `defaultTechnicalUser` parameter specifies whether the default technical user should be allowed to execute the configuration, while the `userIds` parameter specifies the user IDs of the users that should be allowed to execute the configuration. To restrict execution to multiple users, simply add their user IDs to the `userIds` array.

Both parameters are optional and when the `executionRestrictions` object is specified and kept empty no user will be able to execute the processor.

Search Based Matching of Fact Sheets

When using the "search" based identification of the Fact Sheet that are supposed to be updated by the incoming data object, then the section may contain a section to limit the scope of searched Fact Sheets and an expression filtering the Fact Sheets that should be updated.

In case Integration API iterates over a search result, two variables can be used in all JUEL expressions: `search.resultSize` (indicating the total number of items we iterate over in the processor) and `search.resultIndex` (number of current item being iterated)

When configuring an inbound Factsheet processor, a key "search" is now allowed. the value of this key is an object as defined in the example below. One or more Fact Sheets may be identified by the search and be updated based on the same data object in the LDIF.

The search works in two steps:

1. The "scope" defines a search against the pathfinder backend and limits the number of Fact Sheets to be matched. A valid scope can e.g. be created by using an outbound Integration API configuration and click on "set scope". Then the Scope can be copied from there.
2. After reducing the scope of Fact Sheets potentially in scope as good as possible with pathfinder filtering options, an additional JUEL expression is being executed to further narrow down the scope (key: "filter"). This is the far more costly process but allows much more flexibility identifying the right Fact Sheets. Admins should always try to limit the scope in phase 1 as far as possible to avoid long processing times. In the JUEL expression all fields defined in the "read" section can be used for filtering (e.g. `${lx.factsheet.description.startsWith('Autoupdate:')}`)

As the Outcome, all identified Fact Sheets will be processed as if they had been found by the processor one by one.

The "search" key can be used in conjunction with the "external" key. In this scenario, the Integration API first tries to find based on the external ID. If that fails, the search will be executed. If that fails as well, the integration API will try to create the Fact Sheet based on information in the "external" value. The last step can be avoided if creation is not allowed for the use case by adding an `onRead` filter and check for `"lx.factsheet"` not being null.

From a use case perspective, it allows to search for a Fact Sheet with a specific external id, if not existing then search for a Fact Sheet with e.g. a specific name and add the external id in case it is existing. OR create a new Fact Sheet in case a Fact Sheet with the name was not found.

The key "multipleMatchesAllowed" allows to define the API behaviour in case multiple Fact Sheets are matching the search criteria. Some use cases may only want to update if exactly one Fact Sheet was found (then the value will be set to "false"). By allowing multiple matches, bulk updates on multiple Fact Sheets are possible. Default if not existing is "true"

The following example processor will update all descriptions of Application Fact Sheets that have a tag "AsiaPacific" in the tag group "Region".

Search based on fact sheet identification:

```

{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Update all Cloud Apps",
      "processorDescription": "Updates all Apps with tag 'Cloud'",
      "type": "Application",
      "filter": {
        "exactType": "AppUpdate"
      },
      "identifier": {
        "search": {
          "scope": {
            "facetFilters": [
              {
                "facetKey": "FactSheetTypes",
                "operator": "OR",
                "keys": [
                  "Application"
                ]
              },
              {
                "facetKey": "${integration.tags.getTagGroupId('Region')}}",
                "operator": "OR",
                "keys": [
                  "${integration.tags.getTagId('Region','AsiaPacific')}}"
                ]
              }
            ],
            "ids": []
          },
          "filter": "${true}",
          "multipleMatchesAllowed": true
        }
      },
      "logLevel": "debug",
      "updates": [
        {
          "key": {
            "expr": "description"
          },
          "values": [
            {
              "expr": "External sync executed ${data.dateTime}"
            }
          ]
        }
      ]
    }
  ]
}

```

Sample LDIF:

```

{
  "connectorType": "searchBasedScope",
  "connectorId": "searchBasedScope",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "description": "Updates external sync date",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "AppUpdate",
      "id": "apps",
      "data": {
        "dateTime": "06/08/2019"
      }
    }
  ]
}

```

Extending the example is easy to e.g. only match update the Fact Sheets where the description already starts with a specific text, indicating that an automatic update was allowed:

Extended example with filter option set:

```

{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Update all Cloud Apps",
      "processorDescription": "Updates all Apps with tag 'Cloud'",
      "type": "Application",
      "filter": {
        "exactType": "AppUpdate"
      },
      "identifier": {
        "search": {
          "scope": {
            "facetFilters": [
              {
                "facetKey": "FactSheetTypes",
                "operator": "OR",
                "keys": [
                  "Application"
                ]
              }
            ],
            "facetKey": "${integration.tags.getTagGroupId('Region')}",
            "operator": "OR",
            "keys": [
              "${integration.tags.getTagId('Region','AsiaPacific')}"
            ]
          }
        }
      }
    }
  ]
}

```

```

    ],
    "ids": []
  },
  "filter": "${lx.factsheet.description.startsWith('External sync')}",
  "multipleMatchesAllowed": true
}
},
"logLevel": "debug",
"read": {
  "fields": [
    "description"
  ]
},
"updates": [
  {
    "key": {
      "expr": "description"
    },
    "values": [
      {
        "expr": "External sync executed ${data.dateTime}"
      }
    ]
  }
]
}
]
}
}

```

Include Archived Fact Sheets in scope result

It is even possible to tell iAPI to include archived Fact Sheets in the result by enabling a specific flag. Just set "omitArchivedFactSheets" to false in the scope:

Include archived fact sheets in search results:

```

{
  "scope": {
    "omitArchivedFactSheets": false,
    "ids": [],
    "facetFilters": [
      {
        "keys": [
          "BusinessCapability"
        ],
        "facetKey": "FactSheetTypes",
        "operator": "OR"
      },
      {
        "keys": [
          "archived"
        ],
        "facetKey": "TrashBin",

```

```

        "operator": "OR"
      }
    ]
  },
  "processors": [
    ....
  ]
}

```

i Note

Using ExternalIDs in search based scope (and in deletion scope).

Scope filters may not only filter for items by their internal ids (key "ids") but by external ids as well. To use this, the key "externalIds" needs to be defined and contain an array of searched external ids. Each need to be prefixed with the name of the external id field and a slash. See example below

The Below example can even be dynamic and inject content from the LDIF custom fields:

```
"externalIds": ["${'externalId/'.concat(header.customFields.myExternalId)}"]
```

Example how to use external ids in search based scope:

```

{
  "scope": {
    "facetFilters": [],
    "externalIds": [
      "externalId/Ext-ID-0m0NiY6Z"
    ]
  }
}

```

Bookmarks to define search scope

It is possible to not hardcode the search scope into the Integration API configuration but allow users with access to specific bookmarks to dynamically change the scope of Integration API runs by modifying the Bookmark in the Frontend of the application. In Integration API the ID or name of a Bookmark can be configured to be used to set the scope for search based scoping as shown in the below example:

If a bookmark is used, the whole bookmark object is available to be used in JUEL expressions in the processor. The below example shows this by accessing the bookmark name in the updates section.

Example how to use a Bookmark to define the search based scope:

```

{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Apps from Deployments",
      "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
      "type": "Application",
      "filter": {
        "exactType": "Deployment"
      }
    }
  ]
}

```



```

    },
    "identifier": {
      "search": {
        "filter": "${true}",
        "multipleMatchesAllowed": true,
        "scopeFromBookmark": "${integration.bookmarks.getBookmarkId('book-1')}}"
      }
    },
    "updates": [
      {
        "key": {
          "expr": "description"
        },
        "values": [
          {
            "expr": "bookmark name: '${bookmark.name}' AND id=${integration.bookmarks.getBookmarkId('book-1')}"
          }
        ]
      }
    ]
  }
]
}

```

i Note

Find out details about Bookmarks.

You may output the bookmark object into the description field to inspect the structure and available information in a Bookmark. Use the "test run" mode to not alter any Fact Sheets.

Read-Only FactSheet Processor

In case you want to collect information from the resulting FactSheets into variables, you can enable the "readonly" mode on the inboundFactsheet processor. In the example below, all entries of the "releases" field of each application are collected into a variable releases. Results could then be used in a next run.

Using the "filter": "\${myExpression}", the set of Fact Sheet values collected can be even more narrowed down according to the use case. This feature comes handy to save processing time as the processor does not need to prepare any write operation.

Example how to set the "readonly" mode on an inboundFactSheet processor:

```

{
  "processorType": "inboundFactSheet",
  "processorName": "Process variables with Search Scope",
  "processorDescription": "Collect deploymentMaturity ",
  "type": "Application",
  "filter": {
    "exactType": "Deployment"
  },
  "identifier": {
    "search": {
      "scope": {

```

```

    "ids": [],
    "facetFilters": [
      {
        "keys": [
          "Application"
        ],
        "facetKey": "FactSheetTypes",
        "operator": "OR"
      }
    ]
  }
},
"run": 0,
"enabled": true,
"variables": [
  {
    "key": "releases",
    "value": "${lx.release}"
  }
],
"logLevel": "debug",
"readOnly": true
}

```

Accessing the "Hierarchy Level" of a Fact Sheet

It is easily possible to access the hierarchy level of a Fact Sheet just by reading the field "level" provided by the pathfinder backend. This information can be used to filter for certain hierarchy levels or do calculations.

In the below example, it is used to filter for Level 2 Project Fact Sheets for export.

This could of-course have been done by just applying the restriction to the pathfinder scope query. The below is just to show case and allow extending for more advanced filtering.

Export Only Processes In Hierarchy Level 2:

```

{
  "scope": {
    "facetFilters": [
      {
        "facetKey": "FactSheetTypes",
        "operator": "OR",
        "keys": [
          "Project"
        ]
      }
    ],
    "ids": []
  },
  "processors": [
    {
      "processorType": "outboundFactSheet",

```

```

"processorName": "Export Projects L2",
"processorDescription": "Exports only Level 2",
"enabled": true,
"filter": {
  "advanced": "${lx.factsheet.level==2}"
},
"fields": [
  "name",
  "level"
],
"output": [
  {
    "key": {
      "expr": "content.id"
    },
    "values": [
      {
        "expr": "${lx.factsheet.id}"
      }
    ]
  },
  {
    "key": {
      "expr": "content.type"
    },
    "values": [
      {
        "expr": "project}"
      }
    ]
  },
  {
    "key": {
      "expr": "name"
    },
    "values": [
      {
        "expr": "${lx.factsheet.name}"
      }
    ]
  },
  {
    "key": {
      "expr": "level"
    },
    "values": [
      {
        "expr": "${lx.factsheet.level}"
      }
    ]
  }
]
}

```

```

]
}

```

Integration REST API

How to connect to the Integration API from an external system.

Overview

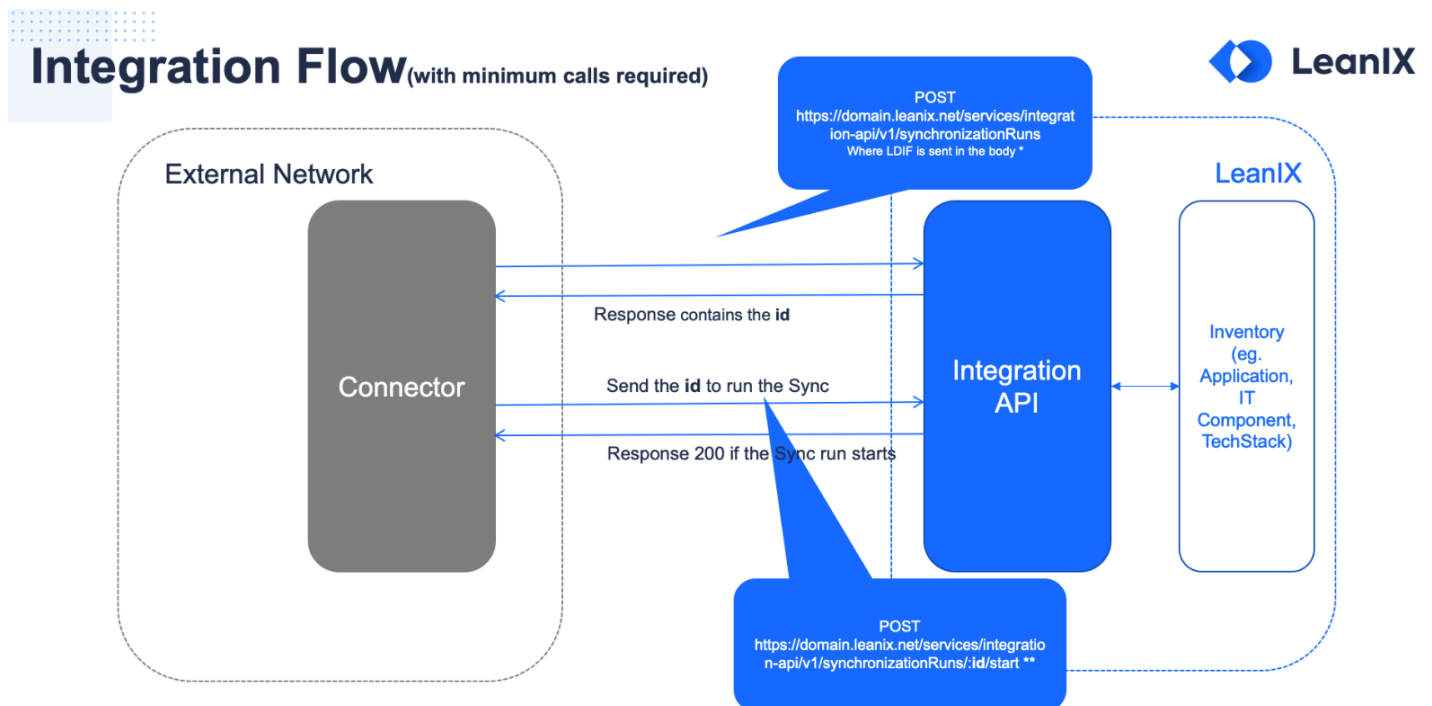
In the previous documents in this series the focus is on utilizing the Integration API via the built-in UI. This is an excellent way to quickly get started, and to test different scenarios. In addition, it can be used with real data in "one-off" situations where you quickly need to get an answer to a question using the Outbound Processor, or even doing a simple one time import via the Inbound Processors. However, in other more complex cases such as when you need to connect an "other vendor" system to SAP LeanIX, it will eventually be necessary to build a REST API.

If you haven't done so already please see [Integration API](#) in order to familiarize yourself with the basics of the Integration API.

To view reference documentation for the Integration REST API, navigate to the [OpenAPI Explorer](#) . When you make API requests, ensure that you use the right base URL. For more information, see [Base URL](#).

To learn how to authenticate to SAP LeanIX APIs, see [Authentication to SAP LeanIX Services](#).

Integration runs in two steps and a minimum two calls are required to start a Sync run. Please see below for the integration flow including calls.



Creating a Processor

Create a Processor under Integration API, with the below details:

- **Connector Type:** importProjectsExample
- **Connector Id:** importProjectsExample

- **Connector Version:** 1.0.0
- **Processing Direction:** inbound

Place the below details in the processor :

```
{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Projects from prj",
      "processorDescription": "Creates LeanIX Projects from external LDIF",
      "type": "Project",
      "filter": {
        "type": "prj|subp"
      },
      "identifier": {
        "external": {
          "id": {
            "expr": "${content.id}"
          },
          "type": {
            "expr": "externalId"
          }
        }
      },
      "run": 0,
      "updates": [
        {
          "key": {
            "expr": "name"
          },
          "values": [
            {
              "expr": "${data.displayName} (${content.id})"
            }
          ]
        },
        {
          "key": {
            "expr": "description"
          },
          "values": [
            {
              "expr": "${data.processDescription}"
            }
          ]
        },
        {
          "key": {
            "expr": "lifecycle.active"
          },
          "values": [
            {
```

```

    "expr": "${data.startActive}"
  }
]
},
"enabled": true,
"variables": [],
"logLevel": "warning"
}
],
"variables": {}
}

```

Creating a Synchronization Run

To create a synchronization run, make a **POST** request to the following endpoint and include the processor details in the request body:

<https://{SUBDOMAIN}.leanix.net/services/integration-api/v1/synchronizationRuns>

Example request body:

```

{
  "connectorType": "importProjectsExample",
  "connectorId": "importProjectsExample",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "processingDirection": "inbound",
  "description": "Imports Projects from default project connector",
  "content": [
    {
      "type": "prj",
      "id": "0123-459",
      "data": {
        "displayName": "Save the company",
        "processDescription": "Most important project of our 'Herkömmliche AG'",
        "startActive": "2019-12-12",
        "tags": [
          "Importance:High",
          "Some invalid Content to be ignored",
          "Effort:Medium"
        ]
      }
    }
  ]
}

```

In the response, the **id** of the synchronization run is returned. The **id** is required to start a synchronization run.

Example response:

```
{
  "id": "xxxx523c-2c03-4b16-a1f1-xxxxc9dc6e6df"
}
```

Starting a Synchronization Run

With the `id` of the synchronization run, you can start a synchronization by making a **POST** request to the following endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/integration-api/v1/synchronizationRuns/{id}/start
```

Canceling a Synchronization Run

To cancel a synchronization run in progress, make a **POST** request to the following endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/integration-api/v1/synchronizationRuns/runId/stop
```

Retrieving the Status of a Synchronization Run

To get the status of the synchronization run that you initiated, make a **GET** request to the following endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/integration-api/v1/synchronizationRuns/{id}/status
```

Example response:

```
{
  "id": "xxxx523c-2c03-4b16-a1f1-xxxxc9dc6e6df",
  "status": "FINISHED"
}
```

Retrieving the Details of a Synchronisation Run

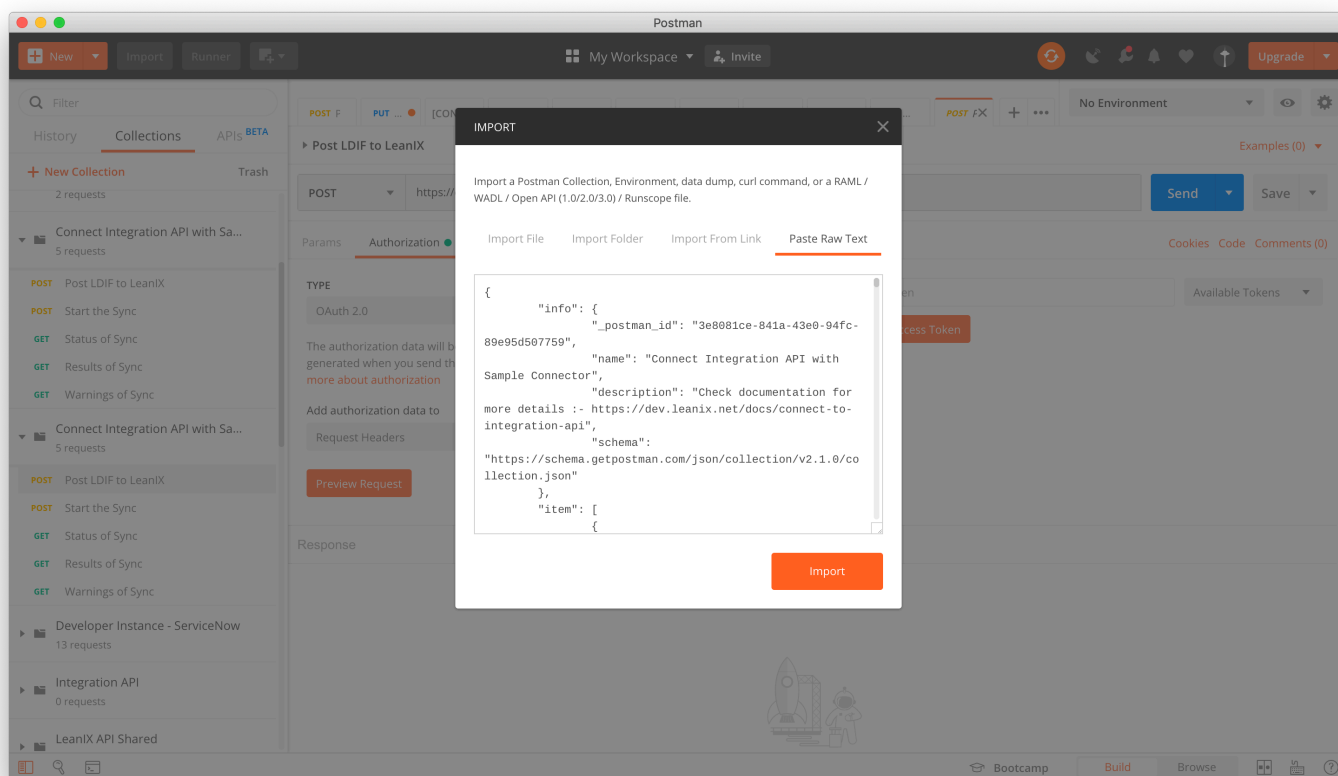
With the `id` of a synchronization run, you can get additional details about it. To do that, use the following endpoints listed in the table.

Method	Endpoint	Details
GET	<code>https://{SUBDOMAIN}.leanix.net/services/integration-api/v1/synchronizationRuns/runId/status</code>	Get the status of a synchronization run.
GET	<code>https://{SUBDOMAIN}.leanix.net/services/integration-api/v1/synchronizationRuns/runId/results</code>	Get the results of a synchronization run in LDIF format. The LDIF output includes all Fact Sheet data which is defined in the corresponding processor declared in a workspace.
GET	<code>https://{SUBDOMAIN}.leanix.net/services/integration-api/v1/synchronizationRuns/runId/warnings</code>	Get warning messages for a synchronization run. For example, if an attribute that is included in a processor does

Method	Endpoint	Details
		not exist in a workspace, this information is returned in a warning message.

Importing a Collection to Postman

To make API calls to the Integration REST API in Postman, you can import the following file into it.



```
{
  "info": {
    "_postman_id": "d5fc07f1-cef1-4b28-b965-97f2c662eb4b",
    "name": "Connect Integration API with Sample Connector",
    "description": "Check documentation for more details :- https://docs-eam.leanix.net/reference/int",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "Post LDIF to LeanIX",
      "request": {
        "method": "POST",
        "header": [
          {
            "key": "Content-Type",
            "name": "Content-Type",
            "type": "text",
            "value": "application/json"
          }
        ]
      }
    ]
  }
}
```



```

    "body": {
      "mode": "raw",
      "raw": ""
    },
    "url": {
      "raw": "https://{{base_url}}/services/integration-api/v1/synchronizationRuns",
      "protocol": "https",
      "host": [
        "{{base_url}}"
      ],
      "path": [
        "services",
        "integration-api",
        "v1",
        "synchronizationRuns"
      ]
    }
  },
  "response": []
},
{
  "name": "Start the Sync",
  "request": {
    "method": "POST",
    "header": [],
    "url": {
      "raw": "https://{{base_url}}/services/integration-api/v1/synchronizationRuns/:id/start",
      "protocol": "https",
      "host": [
        "{{base_url}}"
      ],
      "path": [
        "services",
        "integration-api",
        "v1",
        "synchronizationRuns",
        ":id",
        "start"
      ],
      "variable": [
        {
          "key": "id",
          "value": "e546e34b-cbe5-43f2-9f43-2cbcda4c5405"
        }
      ]
    }
  },
  "response": []
},
{
  "name": "Status of Sync",
  "request": {
    "auth": {
      "type": "oauth2",

```

```

"oauth2": [
  {
    "key": "accessToken",
    "value": "",
    "type": "string"
  },
  {
    "key": "tokenType",
    "value": "bearer",
    "type": "string"
  },
  {
    "key": "addTokenTo",
    "value": "header",
    "type": "string"
  }
],
"method": "GET",
"header": [],
"url": {
  "raw": "https://{{base_url}}/services/integration-api/v1/synchronizationRuns/:id/status",
  "protocol": "https",
  "host": [
    "{{base_url}}"
  ],
  "path": [
    "services",
    "integration-api",
    "v1",
    "synchronizationRuns",
    ":id",
    "status"
  ],
  "variable": [
    {
      "key": "id",
      "value": "e546e34b-cbe5-43f2-9f43-2cbcda4c5405"
    }
  ]
},
"response": []
},
{
  "name": "Results of Sync",
  "request": {
    "method": "GET",
    "header": [],
    "url": {
      "raw": "https://{{base_url}}/services/integration-api/v1/synchronizationRuns/:id/results",
      "protocol": "https",
      "host": [
        "{{base_url}}"
      ],

```

```

    ],
    "path": [
      "services",
      "integration-api",
      "v1",
      "synchronizationRuns",
      ":id",
      "results"
    ],
    "variable": [
      {
        "key": "id",
        "value": "e546e34b-cbe5-43f2-9f43-2cbcda4c5405"
      }
    ]
  },
  "response": []
},
{
  "name": "Warnings of Sync",
  "request": {
    "method": "GET",
    "header": [],
    "url": {
      "raw": "https://{{base_url}}/services/integration-api/v1/synchronizationRuns/:id/warnings",
      "protocol": "https",
      "host": [
        "{{base_url}}"
      ],
      "path": [
        "services",
        "integration-api",
        "v1",
        "synchronizationRuns",
        ":id",
        "warnings"
      ],
      "variable": [
        {
          "key": "id",
          "value": "e546e34b-cbe5-43f2-9f43-2cbcda4c5405"
        }
      ]
    }
  },
  "response": []
}
],
"auth": {
  "type": "oauth2",
  "oauth2": [
    {
      "key": "accessToken",

```

```

    "value": "",
    "type": "string"
  },
  {
    "key": "tokenType",
    "value": "bearer",
    "type": "string"
  },
  {
    "key": "addTokenTo",
    "value": "header",
    "type": "string"
  }
]
},
"event": [
  {
    "listen": "prerequest",
    "script": {
      "id": "ec73502c-9e67-4cf0-85cf-cf8170200181",
      "type": "text/javascript",
      "exec": [
        ""
      ]
    }
  },
  {
    "listen": "test",
    "script": {
      "id": "74a45df7-6d1e-4161-9f91-cf050d515c92",
      "type": "text/javascript",
      "exec": [
        ""
      ]
    }
  }
],
"variable": [
  {
    "id": "99861109-e4a2-4607-9ed3-75857900a6ab",
    "key": "base_url",
    "value": "app.leanix.net",
    "type": "string"
  }
],
"protocolProfileBehavior": {}
}

```

Follow these steps:

1. Import the collection to Postman.
2. Within the Postman UI, edit the collection at the Parent level, updating the Authorization by clicking Get New Access Token, and providing an API token from your SAP LeanIX workspace.

3. Updated the raw body of the 'Post LDIF to LeanIX 'step to reflect the identifiers of the Integration API connector you would like to test.
4. Send the Post LDIF to LeanIX request, and notice that an id is returned in the response { "id": "xx" }
5. Pass this id to Start the Sync, and each subsequent step as a Path Variable, id. Note: Start the Sync returns no Body, just a status 200 OK.

Initiating Integration API Runs with Python

Overview

The goal of this tutorial is to provide a working example of how 'runs' of the SAP LeanIX Integration API can be initiated from a 3rd party system using Python.

i Note

Command Line Interface (CLI) Version

This tutorial involves modifying/assigning variables in a Python script. For a version of this that can be executed in the CLI without directly modifying code, please visit our Public Scripts [repository](#) 🐘 .

i Note

Processor Details

Detailed documentation about configuring Integration API connectors are not covered in this tutorial. Comprehensive documentation of the Integration API can be found [here](#).

Prerequisites

- The Integration API enabled in your SAP LeanIX workspace (for this, contact your Customer Success Manager).
- An outbound and/or inbound processor configured in your workspace to test out, depending on your requirements.
- Python 3 installed on your system.
- Python Libraries installed: json, requests, time.

Initial Script Setup

The script initiateOutboundRun.py below provides a template which can be customized to initiate either inbound and outbound run to/from the Integration API.

i Note

Outbound vs Inbound

This tutorial focuses initially on initiating an outbound run. Please continue to the bottom of the page for instructions on adapting the script for inbound purposes.

- Download the script below open it in a text editor of your choosing (we use Visual Studio Code).

- Update the request_url and auth_url variables on lines 5 and 6 with your workspace domain. You can find this by looking at your SAP LeanIX workspace URL.
- Generate an API Token within your workspace (Admin > API Tokens) and assign in to the api_token variable (line 9)

initiateOutboundRun.py

```
import json
import requests
import time

auth_url = 'https://<your domain>.leanix.net/services/mtm/v1/oauth2/token'
request_url = 'https://<your domain>.leanix.net/services/integration-api/v1/'

api_token = '<your api token>'

response = requests.post(auth_url, auth=('apitoken', api_token),
                        data={'grant_type': 'client_credentials'})

response.raise_for_status()
access_token = response.json()['access_token']
auth_header = 'Bearer ' + access_token

header = {'Authorization': auth_header, 'Content-Type': 'application/json'}

def call_post(endpoint, data=False):
    response = requests.post(url=request_url + endpoint, headers=header, data=data)
    response.raise_for_status()
    return response

def call_get(endpoint):
    response = requests.get(url=request_url + endpoint, headers=header)
    response.raise_for_status()
    return response

def create_run(run_config):
    result = call_post("synchronizationRuns", json.dumps(run_config))
    return json.loads(result.text)['id']

def start_run(run_id):
    start_run_endpoint = 'synchronizationRuns/%s/start' % (run_id)
    result = call_post(start_run_endpoint)
    return result.status_code

def check_run_status(run_id, status_response=None):
    print('checking status')
    status_endpoint = 'synchronizationRuns/%s/status' % (run_id)
    status_response = call_get(status_endpoint)
    status_response = json.loads(status_response.text)['status']
    print(status_response)
    if status_response != 'FINISHED':
        time.sleep(5)
        return check_run_status(run_id, status_response)
    else:
        return True

def fetch_results(run_id):
    results_endpoint = 'synchronizationRuns/%s/results' % (run_id)
    results_response = call_get(results_endpoint)
    return json.loads(results_response.text)

def handle_run(ldif_data):
    run_id = create_run(ldif_data)
    if start_run(run_id) == 200:
        if check_run_status(run_id) == True:
            if ldif_data['processingDirection'] == "outbound":
                return fetch_results(run_id)
```

```
elif ldif_data['processingDirection'] == "inbound":
    return f"inbound run: {run_id} finished successfully"
```

```
connectorType = ""
connectorId = ""
connectorVersion = ""
lxVersion = "1.0.0"
processingDirection = ""
#inbound or outbound
processingMode = ""
#partial or full
description = ""
contents = []
### contents array will be left empty for outbound runs

run_config = {
    "connectorType": connectorType,
    "connectorId": connectorId,
    "connectorVersion": connectorVersion,
    "lxVersion": lxVersion,
    "description": description,
    "processingDirection": processingDirection,
    "processingMode": processingMode,
    "content": contents
}

run_results = handle_run(run_config)

with open('leanixOutboundData' + connectorVersion + '.json', 'w') as outfile:
    json.dump(run_results, outfile, ensure_ascii=False, indent=4)
```

Adjust Connector Variables

The variables on lines 69 - 74 are used to identify the proper Workspace and Connector to be triggered by the script.

- Variables connectorType, connectorId, connectorVersion, processingDirection, and processingMode should all be adapted to match the Connector you want to trigger and can be found within the Integration API user interface (Admin > Integration API).
- lxWorkspace should be set to your workspace ID, which can be found in Admin > API Tokens.

Integration API Custom integrations made easy

Add

Search inside all processor configurations

Connector Type	Connector Id	Connector Version	Processing Direction	Processing Mode
 Starter Example	--	--	--	--
 Advanced Example	--	--	--	--
Export Applications With BC	applications_with_BC_rel	1.0.0	outbound	partial
outboundFactsheet	qualityseal-to-metric	1.0.0	outbound	partial

i Note

For detailed information about the LDIF configuration elements, visit [Mandatory Attributes](#).

With these variables in place, you can run the script and it will call the Integration API, initiating a 'run' of the processor indicated per your configured variables. The data retrieved from the run will be saved in the same directory as your copy of

Adapt for Inbound

If you would like to submit an LDIF to an inbound connector, this is possible with the following steps:

- Adjust your configuration variables to point the script at an inbound processor
- Add the key "contents": [] to the run_config object on line 87
- Submit your LDIF [formatted data](#) within the contents array (see example below)

Inbound LDIF:

```
contents:[
  {
    "type": "ITComponent",
    "id": "b6992b1d-4e4d",
    "data": {
      "name": "Gatsby.j",
      "description": "Gatsby is a free and open source framework based on React that helps develop",
      "category": "sample_software",
      "provider": "gcp",
      "applicationId": "28db27b1-fc55-4e44"
    }
  },
  {
    "type": "ITComponent",
    "id": "cd4fab6c-4336",
    "data": {
      "name": "Contentful",
      "description": "Beyond headless CMS, Contentful is an API-first content management infrastru",
      "category": "cloud_service",
      "provider": "gcp",
      "applicationId": "28db27b1-fc55-4e44"
    }
  }
]
```

Caution

You will likely not want to 'hard code' your LDIF in the run_config in any substantial implementation, as this would be inflexible and not scalable.

If you are considering implementing an integration with SAP LeanIX Integration API, please don't hesitate to reach out to your Customer Success Manager to arrange a meeting with a member of our Engineering team for a consultation.

Configuring Automated Nightly Runs for Inbound Processors

Adjust the configuration of inbound Integration API processors to enable automated nightly runs.

Overview

This guide explains how to enable automated nightly runs for an inbound Integration API processor by adjusting the processor configuration.

Enabling nightly runs allows you to ensure that your processors work without disruptions after the deprecation of workflows, which occurred on February 15, 2024. Previously, you could configure a custom schedule for a processor using workflows.

i Note

Nightly runs only work on processors that don't require input in LDIF format and directly handle workspace data.

You can configure nightly runs only for inbound processors. For outbound processors, nightly runs are not supported.

Prerequisites

Get administrator access to your workspace.

→ Tip

For processors with custom scheduled workflows, please contact support to remove the workflows first. Then you can configure automated nightly runs for these processors as described in this guide.

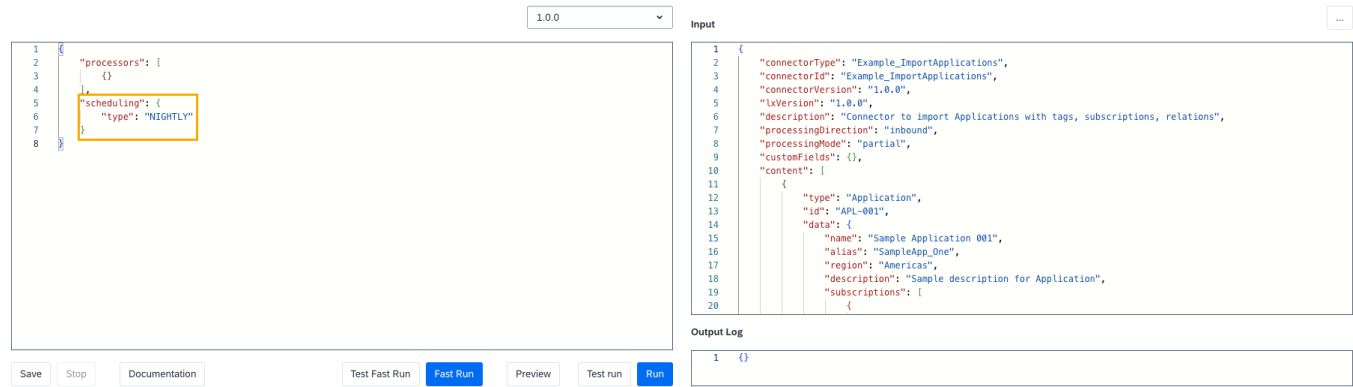
To contact support, submit a ticket from the [SAP LeanIX support portal](#) ➡. If you're an SAP customer, use the [SAP for Me](#) ➡ portal.

Procedure

1. In the administration area, navigate to the **Integration API** section.
2. On the **Integration API** page, select an inbound processor that you want to modify.
3. Within the processor configuration object, add the `scheduling` object with the `type` attribute set to `NIGHTLY`, as shown in the following code. Click **Save**.

```
{
  "processors": [
    {},
    {},
    {}
  ],
  "scheduling": {
    "type": "NIGHTLY"
  }
}
```

The following image shows an example processor configuration with the `scheduling` object.



scheduling Object in the Configuration of an Integration API Processor

Once configured, the processor will automatically run daily between 2 AM and 6 AM local time, as determined by your workspace location.

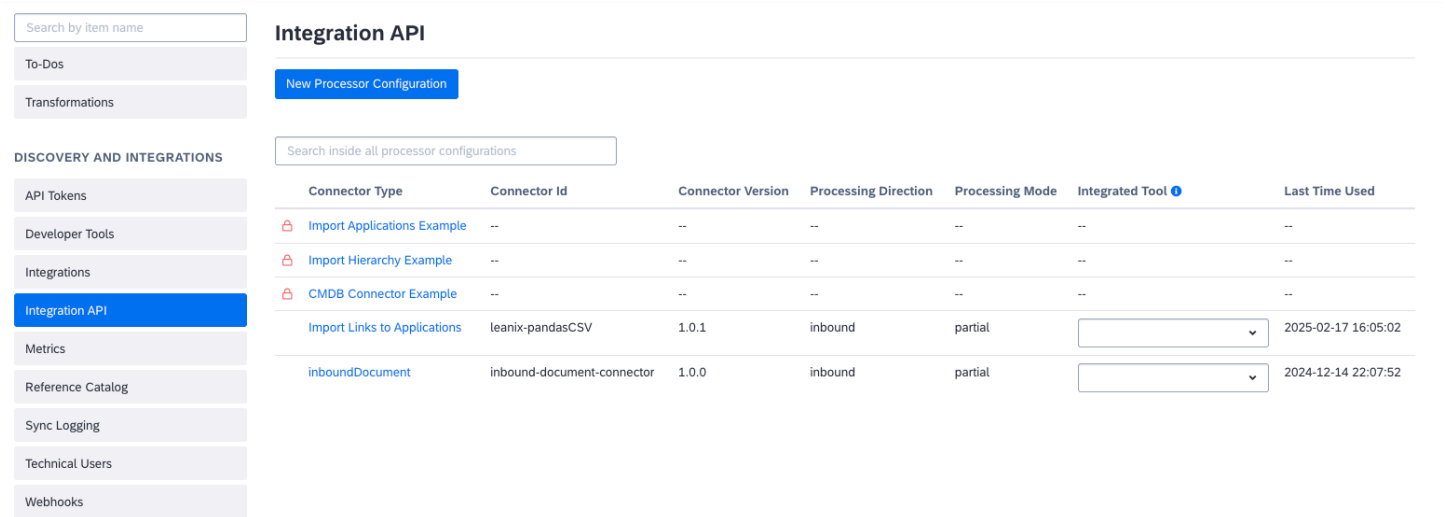
Best Practices for the Integration API

Best practices and tips to optimize your usage of the Integration API.

Overview

Integration API offers a wide range of functionality. Expanding scope of Integration possibilities with SAP LeanIX. Here is a collection of some suggested best practices and tips to optimize usage of Integration API.

To get access to the Integration API, navigate to the [Integration API](#) section in the admin area.



Integration API Section in the Admin Area

Use the UI

While we also have our [REST API](#) available, it is highly recommended to setup your processors in the UI before addressing the API.

advancedExample

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

```
"processors": [{
  {
    "processorType": "inboundFactSheet",
    "processorName": "Apps from Deployments",
    "processorDescription": "Creates LeanIX Applications from Kubernetes Deployments",
    "type": "Application",
    "filter": {
      "exactType": "Deployment"
    },
    "identifier": {
      "external": {
        "id": {
          "expr": "${content.id}"
        },
        "type": {
          "expr": "externalId"
        }
      }
    },
    "run": 0,
    "updates": [
      {
        "key": {
          "expr": "name"
        },
        "values": [
          {
            "expr": "${data.app}"
          }
        ]
      },
      {
        "key": {
          "expr": "description"
        }
      }
    ]
  }
}]
```

Stop

Preview

Test run

Run

Input

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

```
{
  "connectorType": "Kubernetes",
  "connectorId": "Kub Dev-001",
  "connectorVersion": "1.2.0",
  "lxVersion": "1.0.0",
  "description": "Imports kubernetes data into LeanIX",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "Deployment",
      "id": "634c16bf-198c-1129-9d08-92630b573fbf",
      "data": {
        "app": "HR Service",
        "version": "1.8.4",
      }
    }
  ]
}
```

Output Log

1

Close

Utilizing the UI allows for quick testing and debugging of processors.

Check Scope Before Firing Test Runs

Test runs on a large workspace can take up a lot of time and unnecessary processing. Setting a scope while doing testing ensures that the testing is quick and does not overload the system.

Integration API



1.0.0

Input

- Save new version
- Define scope
- Get Data Model
- Delete configuration

```

1 {
2   "processorType": "InboundSubscription",
3   "processorName": "Subscription creation",
4   "processorDescription": "Creates subscriptions",
5   "filter": {
6     "exactType": "ITComponent"
7   },
8   "identifier": {
9     "external": {
10      "id": {
11        "expr": "${content.id}"
12      },
13      "type": {
14        "expr": "externalId"
15      }
16    }
17  },
18  "updates": [
19    {
20      "key": {
21        "expr": "user"
22      },
23      "values": [
24        {
25          "expr": "jane.doe@leanix.net"
26        }
27      ]
28    },
29    {
30      "key": {
31        "expr": "subscriptionType"
32      },
33      "values": [
34        {
35          "expr": "RESPONSIBLE"

```

Save

Stop

Preview

Test run

Run

Output Log

```

1 {}

```

Close

Integration API



Define scope

Filters

Type: Application x +

TYPE

Application 118

BUSINESS CAPABILITIES

☐ Customer Relationship 16☐ HR 15☐ Manufacturing 12☐ Information Technology 12☐ Marketing 12☐ n/a 9[More](#)

USER GROUPS

☐ Australia 35☐ Spain 25☐ Headquarter 19☐ France 16☐ USA 13☐ n/a 68[More](#)

LIFECYCLE

☐ Any☐ Plan☐ Phase in☐ Active

118 Results

AC Management

Assessment Center Management supports all major functions for planning, organizing and managing our assessment center. This is usable for external and internal applicants.

Application

AC Management Cloud

Test application for QA

Application

Adobe Suite

Application

Adobe Suite / Adobe Premiere Pro

Application

AttorneyNet

AttorneyNet is an inventory of all lawyers around the world we are working together.

Application

audimex

To perform Audit projects audimex is the companion for managing all aspects of such projects, COBIT compliant.

Application

AutoCAD

In Development and construction we use AutoCAD as the CAD program. It is connected with everything to ensure most effective and efficient use.

Application

Bonus Card Americas

This is the special version for Americas to increase customer loyalty

Application

Bonus Card Asia/Pacific

Due to different requirements in the regions for each region a dedicated bonus program exist, this is for Asia/Pacific

Application

Bonus Card Europe

Due to different requirements in the regions for each region a dedicated bonus program exist, this is for Europe

Application

Bonus Card Use Watch

With this application the use of the bonus card per customer is tracked

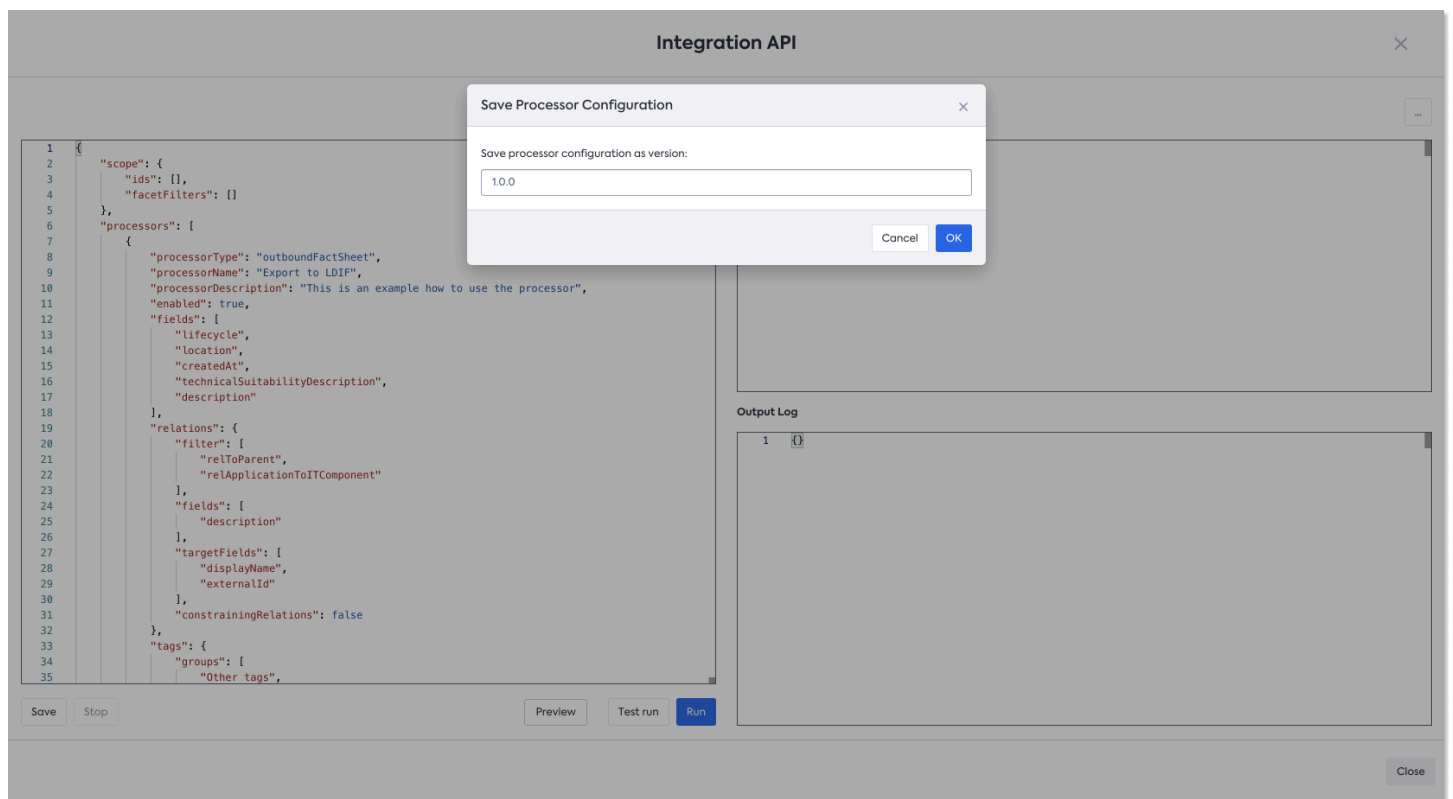
Application

[Use Fact Sheet Filter](#)

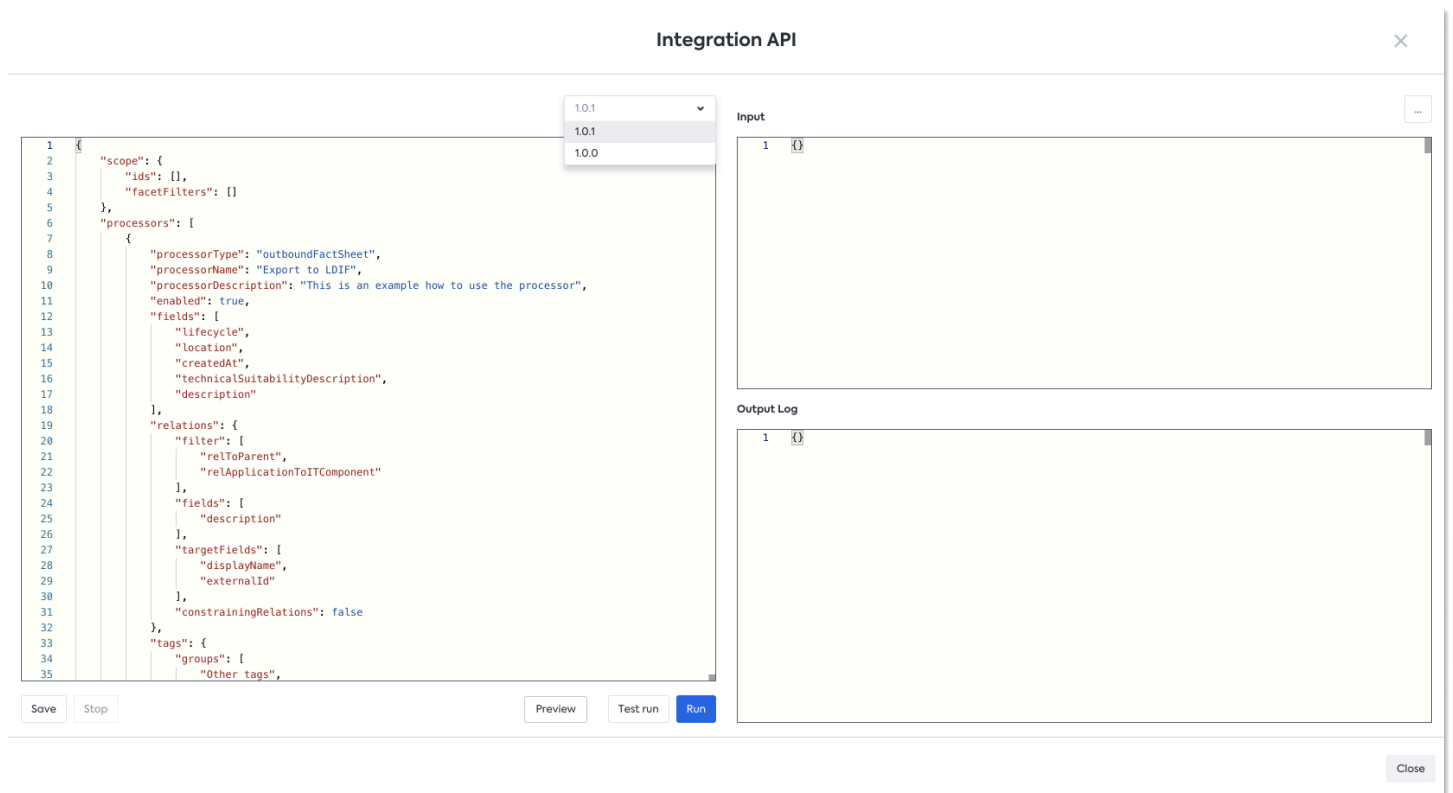
Close

Adhere to Proper Processor Versioning Convention

Integration API Processors should always be versioned using the number.number.number format. (e.g. 1.0.0). Following this convention allows for maintaining versioning support in case there is an update to the processor.



Following the convention allows for version history of the processor to be saved and be accessible from the UI itself.



Use Webhooks

Our tutorial on [Sending Alerts to Slack and Microsoft Teams](#) can be used to set up alerts for Integration API runs that occur on a recurring basis.

One way to do this could be to have a separate slack channel wherein successful or unsuccessful runs are sent with a different emoji in their payload.



incoming-webhook APP 8:07 PM

⚠ Workspace: ██████████ **Connector Id:** CDK-AWS **Scope:** FULL **Connector Direction:** INBOUND **has Status:** ABORTED and error count: 0 Synclog link : <https://ad-4b9a-8f3f-1459703b5d6e>

```
{
  "deliveryType": "PUSH",
  "id": "",
  "identifier": "Integration Statistics to Slack",
  "tagSets": [
    [
      "integrations",
      "statistics"
    ]
  ],
  "createdAt": "2020-06-10T15:17:11.064809Z",
  "workspaceId": "[Workspace ID from API Token section]",
  "userId": "[User ID from API Token Section]",
  "targetUrl": "[SLACK TARGET URL]",
  "targetMethod": "POST",
  "authorizationHeader": "",
  "callback": "var payload = delivery.payload;delivery.active = false;var base_url = 'https://eu.leanix.net/';",
  "lastDeliveryStatus": 200,
  "ignoreError": false,
  "maxBatchSize": 512,
  "workspaceConstraint": "WORKSPACE",
  "active": true,
  "errorCount": 0,
  "firstTimeDeliveryFailed": null,
  "payloadMode": "DEFAULT"
}
```



incoming-webhook APP 9:02 PM

✅ Workspace: ██████████ **Connector Id:** CDK-Azure **Scope:** FULL **Connector Direction:** INBOUND **Connector Version:** 1.0.0 **has Status:** FINISHED and error count: 0 Synclog link : <https://eu.leanix.net/>

Leverage Deletion Scope

Deletion scope can be used to set up automatic deletion when loading data using inbound Data Processors.

For more information, see [Auto deletion with inboundProcessors](#).

⚠ Caution

Examples provided can archive Fact Sheets.

Executing examples in the section above needs to be done with care. All potentially existing project Fact Sheets in the workspace will be in scope for deletion. To limit, you may want to change the sample that a tag "TEST_PRJ" or similar will be set for the test projects. This tag can be added as filter criteria to the deletion scope definition.

Cover All Edge Cases to Eliminate Warnings and Errors

Whenever value mapping is defined in the processor, it is highly suggested to add a "catch-all" option at the end of the value mapping list. By doing so, warnings that arise from the processor on not finding any matching value from other value mapping list can be removed.

```
{
  "leanixType": "tag",
  "fields": [
    {
      "leanixFieldName": "group",
      "inboundPropertyPath": "Process Frequency",
      "valueMappings": []
    },
    {
      "leanixFieldName": "name",
      "inboundPropertyPath": "${editordata.model.properties["meta-processfrequency"]}",
      "valueMappings": [
        {
          "outputExpression": "High",
          "regexMatch": "ci1560953961659160925663"
        },
        {
          "outputExpression": "Medium",
          "regexMatch": "ci1560953961659758129772"
        },
        {
          "outputExpression": "Low",
          "regexMatch": "ci1560953961659682886651"
        },
        {
          "outputExpression": "None",
          "regexMatch": "ci1560953961659123251220"
        },
        {
          "outputExpression": "Other value",
          "regexMatch": ".*"
        }
      ]
    }
  ]
}
```

In the example above, by using the last value mapping of **"Other Value"**, the Integration will not output a warning in the sync log when it does not find any of the matching values defined.

Configuring Automated Nightly Runs

You can configure automated nightly runs for inbound processors. For more information, see [Configuring Automated Nightly Runs for Inbound Processors](#).

Integration API Tutorials

Explore practical examples of using the Integration API.

- [Importing and Updating Fact Sheets](#)
- [Calculating Scores from Numeric Fields and Updating Custom Fact Sheet Fields](#)
- [Importing Resources to Fact Sheets](#)

Importing and Updating Fact Sheets

Learn how to import and update application fact sheets using the `inboundFactSheet` processor of the Integration API.

Overview

You can import fact sheets into SAP LeanIX using the `inboundFactSheet` processor of the Integration API. By the end of this tutorial, you'll be able to:

- Import application fact sheets into SAP LeanIX, including basic information like names and descriptions.
- Update imported fact sheets with business criticality values.
- Set up value mappings for fact sheet fields in the processor configuration.

Prerequisites

This tutorial assumes you have basic knowledge of:

- [LeanIX Data Interchange Format \(LDIF\)](#) of the Integration API
- [Inbound processors](#) of the Integration API, including the `inboundFactSheet` processor
- [Fact sheets](#) in SAP LeanIX

Step 1: Import Fact Sheets with Basic Information

In the first step, import application fact sheets with the following attributes:

Attribute Key	Type	Description
name	STRING	The name of the fact sheet.
description	STRING	The description of the fact sheet.

→ Tip

To get attribute keys, go to the processor configuration page, click the three-dot icon in the upper-right corner, then select **Get Data Model**. The data model of the workspace is returned in the **Output Log** section, where you can find the keys of specific attributes.

inboundFactSheet



1.0.0

Input



- Save new version
- Get Data Model
- Delete configuration

```
1 {
2   "processors": [],
3   "variables": {}
4 }
```

1 {}

To configure and run a connector, follow these steps:

1. Go to the [Integration API](#) section in the administration area.
2. Click [New Processor Configuration](#), fill in the required details, and then click [Create](#).
3. On the connector configuration page, enter the processor configuration in the left-side section.
4. In the [Input](#) section on the right, enter the input data.
5. To start a test run, click [Test run](#). If successful, proceed to the next step.
6. To start a run, click [Run](#).
7. To save the connector, click [Save](#) in the lower-left corner.

Two application fact sheets are imported into the workspace with their names and descriptions. You can check the created fact sheets in the inventory. To learn how to use the inventory, see [Inventory](#).

Processor Configuration

Example processor configuration:

```
{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "Create applications",
      "processorDescription": "Creates application fact sheets",
      "type": "Application",
      "filter": {
        "type": "Application"
      },
      "identifier": {
        "external": {
          "id": {
            "expr": "${content.id}"
          },
          "type": {
            "expr": "externalId"
          }
        }
      },
      "run": 0,
      "updates": [
        {
          "key": {
            "expr": "name"
          },
          "values": [
            {
              "expr": "${data.name}"
            }
          ]
        }
      ]
    }
  ]
}
```

```

    },
    {
      "key": {
        "expr": "description"
      },
      "values": [
        {
          "expr": "${data.description}"
        }
      ]
    }
  ],
  "enabled": true,
  "variables": [],
  "logLevel": "warning"
}
]
}

```

Input Data

i Note

The `id` passed in the `content` array represents the record ID in the external system. This ID is written to the `externalId` field (**External ID**). A unique fact sheet ID is generated automatically upon fact sheet creation.

Example input data:

```

{
  "connectorType": "Basic-ImportApplications",
  "connectorId": "ImportApplications",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "description": "",
  "processingDirection": "inbound",
  "processingMode": "full",
  "customFields": {},
  "content": [
    {
      "type": "Application",
      "id": "APP-0001",
      "data": {
        "name": "AC Management",
        "description": "AC Management description"
      }
    },
    {
      "type": "Application",
      "id": "APP-0002",
      "data": {
        "name": "AC Management Cloud",
        "description": "AC Management Cloud description"
      }
    }
  ]
}

```

```
}  
]  
}
```

Step 2: Update Fact Sheets with Business Criticality Values

In this step, update the imported fact sheets with business criticality values. The names and descriptions remain unchanged.

Attribute Key	Type	Description
businessCriticality	SINGLE_SELECT	Indicates the importance of an application to the functioning and success of a business. Possible values: <code>missionCritical</code> (Mission critical), <code>businessCritical</code> (Business critical), <code>businessOperational</code> (Business operational), or <code>administrativeService</code> (Administrative service).

Consider a scenario where business criticality values differ between the external customer's system and SAP LeanIX. To address this, create a mapping of business criticality values and provide these mappings in the processor configuration. This setup lets you import business criticality values from your external system by aligning them with SAP LeanIX values.

External Value	SAP LeanIX Value
1-Critical	<code>missionCritical</code>
2-High	<code>businessCritical</code>
3-Low	<code>businessOperational</code>
0-NonCritical	<code>administrativeService</code>

In the connector you created in the previous step, update the processor configurations and input data. Then, run the connector. This updates two application fact sheets you imported earlier. Business criticality values are mapped from the external system to SAP LeanIX.

Processor Configuration

Example processor configuration:

```
{  
  "processors": [  
    {  
      "processorType": "inboundFactSheet",  
      "processorName": "Create applications",  
      "processorDescription": "Creates application fact sheets",  
      "type": "Application",  
      "filter": {  
        "type": "Application"  
      },  
      "identifier": {  
        "external": {
```

```

    "id": {
      "expr": "${content.id}"
    },
    "type": {
      "expr": "externalId"
    }
  },
  "run": 0,
  "updates": [
    {
      "key": {
        "expr": "name"
      },
      "values": [
        {
          "expr": "${data.name}"
        }
      ]
    },
    {
      "key": {
        "expr": "description"
      },
      "values": [
        {
          "expr": "${data.description}"
        }
      ]
    },
    {
      "key": {
        "expr": "businessCriticality"
      },
      "values": [
        {
          "expr": "${data.businessCriticality}",
          "regexMatch": "1-Critical",
          "regexReplace": {
            "match": "^.*$",
            "replace": "missionCritical"
          }
        },
        {
          "expr": "${data.businessCriticality}",
          "regexMatch": "2-High",
          "regexReplace": {
            "match": "^.*$",
            "replace": "businessCritical"
          }
        },
        {
          "expr": "${data.businessCriticality}",
          "regexMatch": "3-Low",

```

```

        "regexReplace": {
            "match": "^.*$",
            "replace": "businessOperational"
        }
    },
    {
        "expr": "${data.businessCriticality}",
        "regexMatch": "0-NonCritical",
        "regexReplace": {
            "match": "^.*$",
            "replace": "administrativeService"
        }
    }
]
}
],
"enabled": true,
"variables": [],
"logLevel": "warning"
}
]
}

```

Input Data

Example input data:

```

{
  "connectorType": "Basic-ImportApplications",
  "connectorId": "ImportApplications",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "description": "",
  "processingDirection": "inbound",
  "processingMode": "full",
  "customFields": {},
  "content": [
    {
      "type": "Application",
      "id": "APP-0001",
      "data": {
        "name": "AC Management",
        "description": "AC Management description",
        "businessCriticality": "1-Critical"
      }
    },
    {
      "type": "Application",
      "id": "APP-0002",
      "data": {
        "name": "AC Management Cloud",
        "description": "AC Management Cloud description",
        "businessCriticality": "2-High"
      }
    }
  ]
}

```

```

    }
  }
]
}

```

Summary

In this tutorial, you learned how to import application fact sheets and update them using the `inboundFactSheet` inbound processor of the Integration API. You also learned how to set up value mappings in the processor configuration.

Calculating Scores from Numeric Fields and Updating Custom Fact Sheet Fields

Learn how to set up a custom calculation using the `inboundFactSheet` processor of the Integration API. The processor calculates risk scores from numeric fields and writes the calculated score and mapped risk tier to custom fact sheet fields.

Overview

You can use the Integration API to set up custom calculations for fact sheet fields. In this tutorial, you'll learn how to configure an `inboundFactSheet` processor to perform calculations and update custom fact sheet fields. The configuration includes three processors that run sequentially to:

1. Read values from two numeric fields and calculate the total risk score.
2. Write the total calculated score to a custom fact sheet field.
3. Map the total risk score to a user-friendly risk tier and write the tier to a custom fact sheet field.

Alternatively, you can set up calculations using trigger-based automations. In this method, a webhook triggers a calculation whenever a specific fact sheet field is updated. For tutorials on setting up trigger-based automations for calculations, please visit:

- [Calculating Risk Scores and Updating a Fact Sheet Based on Survey Responses](#)
- [Assigning Tags to Fact Sheets Based on a Custom Calculated Field](#)

Prerequisites

You need an admin role for your SAP LeanIX workspace.

This tutorial assumes you have a basic understanding of:

- [LeanIX Data Interchange Format \(LDIF\)](#) of the Integration API
- [Inbound processors](#) of the Integration API, including the `inboundFactSheet` processor
- [Fact sheets](#) in SAP LeanIX

Step 1: Create Custom Fact Sheet Fields

The processor in this tutorial is configured for application fact sheets. In the meta model configuration, create custom fields on the application fact sheet for individual risk scores, total risk score, and risk tier. To learn more about fact sheet configuration and fields, refer to:

- [Meta Model Configuration](#)
- [Fact Sheet Fields](#)

To create custom fields, follow these steps:

1. In the administration area, navigate to the **Meta Model Configuration** section.
2. Select the application fact sheet.
3. Create a subsection for risk assessment by clicking **Add subsection**. Alternatively, you can add custom fields to an existing subsection.
4. To create custom fields within a subsection, click **Add field** and configure each field in the right-side panel. For details, refer to the following table. You can generate labels and help texts using AI-supported translation.
5. Save the changes.

Key	Field Type	Displayed As	Values and Labels
InfrastructureRiskScore	Integer	Number	N/a
ApplicationOperationalRiskScore	Integer	Number	N/a
TotalRiskScore	Integer	Number	N/a
RiskTier	Single Select	Status	• Green • Yellow • Orange • Red

The following image shows how custom fields appear on the application fact sheet.

←

Meta Model: **Application**

Fields

Quality Seal

Permissions

Conditional Attributes

Audit Log

Risk Assessment

Risk assessment scores and tier

Infrastructure Risk Score

The Infrastructure Risk Score indicates the level of risk associated with the infrastructure of the application.

Operational Risk Score

The Operational Risk Score indicates the level of operational risk associated with the application.

Total Risk Score

The overall risk score assigned to this application based on its risk assessment.

Risk Tier

Indicates the risk tier of the application.

+ Add field

+ Add subsection

+ Add relation

o

globe

↑↓

Edit subsection Risk Assessment

Key

riskAssessment

Template

Field

Visible

Delete

Custom Subsection "Risk Assessment" with Custom Fields on the Application Fact Sheet

Step 2: Map Risk Scores to Risk Tiers

Map the total risk score to a user-friendly risk tier. Here's an example mapping:

This is custom documentation. For more information, please visit [SAP Help Portal](#).

359

Risk Score	Risk Tier
< 10	Green
10–16	Yellow
17–23	Orange
> 23	Red

Include these mappings in the processor configuration, as described in the following step.

```
{
  "key": {
    "expr": "RiskTier"
  },
  "values": [
    {
      "expr": "${ lx.factsheet.TotalRiskScore < 10 ? 'Green' : lx.factsheet.TotalRiskScore < 17 ?
    }
  ]
}
```

Step 3: Configure a Connector

To configure and run a connector, follow these steps:

1. Go to the [Integration API](#) section in the administration area.
2. Click [New Processor Configuration](#), fill in the required details, and then click [Create](#).
3. On the connector configuration page, enter processors in the left-side section.
4. In the [Input](#) section on the right, enter the input data.
5. To start a test run, click [Test run](#). If successful, proceed to the next step.
6. To start a run, click [Run](#).
7. To save the connector, click [Save](#) in the lower-left corner.

→ Tip

You can configure the processor to run automatically every day, keeping your inventory data up to date. To learn how to configure daily runs, refer to [Configuring Automated Nightly Runs for Inbound Processors](#).

Processor Steps

The connector includes three processors that perform these tasks:

1. Read values from the `InfrastructureRiskScore` and `ApplicationOperationalRiskScore` numeric fields and store them in the `TotalRiskScore` variable.
2. Write the calculated total risk score to the `TotalRiskScore` field.
3. Map the total risk score to a user-friendly risk tier and write it to the `RiskTier` field.

→ Tip

When specifying multiple processors in the configuration, use the `run` attribute to set the execution order. Numeration starts at 0, so the processor with `"run": 0` runs first, the processor with `"run": 1` runs second, and so on.

Processor Configuration

Example processor configuration:

```
{
  "processors": [
    {
      "processorType": "inboundFactSheet",
      "processorName": "1. Gather scores and store in variables",
      "processorDescription": "Gather Scores and Store in Variables",
      "type": "Application",
      "identifier": {
        "search": {
          "scope": {
            "ids": [],
            "facetFilters": [
              {
                "keys": [
                  "Application"
                ],
                "facetKey": "FactSheetTypes",
                "operator": "OR"
              }
            ]
          }
        }
      },
      "run": 0,
      "enabled": true,
      "variables": [
        {
          "key": "${lx.factsheet.id.concat('_TotalRiskScore')}",
          "value": "${helper:toList(lx.factsheet.InfrastructureRiskScore,lx.factsheet.Application0)}"
        }
      ],
      "read": {
        "fields": [
          "InfrastructureRiskScore",
          "ApplicationOperationalRiskScore"
        ]
      }
    },
    {
      "processorType": "inboundFactSheet",
      "processorName": "2. Write total risk score",
      "processorDescription": "Gather Scores and Store in Variables",
      "type": "Application",
      "identifier": {
        "search": {
```

```

    "scope": {
      "ids": [],
      "facetFilters": [
        {
          "keys": [
            "Application"
          ],
          "facetKey": "FactSheetTypes",
          "operator": "OR"
        }
      ]
    }
  },
  "run": 1,
  "updates": [
    {
      "key": {
        "expr": "TotalRiskScore"
      },
      "values": [
        {
          "expr": "${variables[lx.factsheet.id.concat('_TotalRiskScore')].getNumbers().sum() !=
        }
      ]
    }
  ],
  "enabled": true
},
{
  "processorType": "inboundFactSheet",
  "processorName": "3. Write Risk Tier",
  "processorDescription": "Assign Risk Tier based on Total Risk Score",
  "type": "Application",
  "identifier": {
    "search": {
      "scope": {
        "ids": [],
        "facetFilters": [
          {
            "keys": [
              "Application"
            ],
            "facetKey": "FactSheetTypes",
            "operator": "OR"
          }
        ]
      }
    }
  },
  "run": 2,
  "updates": [
    {
      "key": {

```

```

      "expr": "RiskTier"
    },
    "values": [
      {
        "expr": "${ lx.factsheet.TotalRiskScore < 10 ? 'Green' : lx.factsheet.TotalRiskScore
      }
    ]
  }
],
"enabled": true,
"read": {
  "fields": [
    "TotalRiskScore"
  ]
}
}
],
"variables": {}
}

```

Input Data

Example input data:

```

{
  "connectorType": "DerivedScoring",
  "connectorId": "DerivedScoring",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "description": "",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
    {
      "type": "",
      "id": "",
      "data": {}
    }
  ]
}

```

Summary

In this tutorial, you learned how to configure an `inboundFactSheet` processor to perform calculations and update custom fact sheet fields.

Importing Resources to Fact Sheets

Import links as resources to application fact sheets using the Integration API.

Overview

To store additional information on a fact sheet, you can add the following resources to it: links, files, diagrams, and logos. To learn how to work with fact sheet resources in the application UI, see [Storing Resources on Fact Sheets](#).

In this tutorial, you'll learn how to import links as resources to application fact sheets in batch using the SAP LeanIX [Integration API](#).

By the end of this tutorial, you'll be able to:

- Programmatically convert a CSV file into LDIF format using a Python script
- Import links to application fact sheets using an inbound processor of the Integration API

Prerequisites

This tutorial assumes you have basic knowledge of:

- [Python](#) 
- [pandas library](#)  for Python
- [LeanIX Data Interchange Format \(LDIF\)](#) of the Integration API
- [Inbound processors](#) of the Integration API
- [Fact Sheets](#) in SAP LeanIX

Step 1: Prepare Data

The Integration API supports the LDIF format for processing data. To import data, you should first convert it into this format. In this tutorial, we convert a CSV file into LDIF format using a Python script.

Prepare a CSV file with links that you want to import to application fact sheets. The file should contain the following columns:

- **type**: The type of the fact sheet. In this tutorial, we only import application fact sheets.
- **id**: The ID of the fact sheet.
- **name**: The name of the link to be added to the fact sheet as a resource.
- **url**: The link URL to be added to the fact sheet as a resource.

You can retrieve application fact sheets and their IDs in the following ways:

- By using the GraphQL API. For more information, see [Filtering Fact Sheets by Type](#).
- By exporting data in XLSX format in the application UI. To learn more, see [Export Your Data via Excel](#). Once you've downloaded the file, convert it into CSV format.

Here's an example CSV file with the required columns.

type	id	name	url
Application	28fe4aa2-6e46-41a1-....-72afb3acf256	Technical documentation	https://docs.example.net/

type	id	name	url
Application	2efa37b5-18aa-48d8-....-1328c0d856d7	Vendor's website	https://www.example-vendor.net/

i Note

You can import multiple links to a single fact sheet.

Step 2: Convert a CSV File into LDIF Format

Once you've prepared a CSV file, convert it into LDIF format that the Integration API can process. The script in this tutorial uses the pandas library, a Python tool for data analysis and manipulation.

The following script converts and maps data into LDIF format and returns an output JSON file named `output.json`.

Example script:

```
import pandas as pd
import json

df = pd.read_csv("LeanIX_Resources_Import_template.csv")

excelData = []

for index, row in df.iterrows():
    excelData.append({
        "type": row['type'],
        "id": row['id'],
        "data": {
            'name': row['name'],
            'url': row['url']
        }
    })

ldif = {
    "connectorType": "leanix",
    "connectorId": "leanix-pandasCSV",
    "connectorVersion": "1.0.0",
    "processingDirection": "inbound",
    "processingMode": "partial",
    "lxVersion": "1.0.0",
    "description": "LDIF Generated by PANDAS CSV data",
    "content": excelData}

print ("Writing output to output.json")
f = open("output.json", "w")
f.write(json.dumps(ldif, indent=2))
f.close()
print ("Done")
```

Example output:

```
{
  "connectorType": "leanix",
  "connectorId": "leanix-pandasCSV",
  "connectorVersion": "1.0.0",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "lxVersion": "1.0.0",
  "description": "LDIF Generated by PANDAS CSV data",
```

```

"content": [
  {
    "type": "Application",
    "id": "28fe4aa2-6e46-41a1-....-72afb3acf256",
    "data": {
      "name": "Technical documentation",
      "url": "https://docs.example.net/"
    }
  },
  {
    "type": "Application",
    "id": "2efa37b5-18aa-48d8-....-1328c0d856d7",
    "data": {
      "name": "Vendor's website",
      "url": "https://www.example-vendor.net/"
    }
  }
]
}

```

Step 3: Create an Inbound Data Processor

Now that you have data to be imported in the required LDIF format, you can create a data processor for the Integration API and complete a run to import links to Applications in batch.

The `inboundDocument` processor is used to create, update, and delete resources linked to fact sheets. To learn more about the processor, see [Inbound Document](#).

To create a processor, follow these steps:

1. In the administration area of SAP LeanIX, select **Integration API**.
2. Click **New Processor Configuration**.
3. Enter parameters for the processor:
 - **Connector Type**: Enter a name for the connector type.
 - **Connector Id**: Enter an ID for the connector.
 - **Connector Version**: Enter **1.0.0**.
 - **Processing Direction**: Select **inbound**.
 - **Processing Mode**: Select **partial**.
 - **Integrated Tool**: Select **None**.
4. Click **Create**.

A processor is created.

Step 4: Complete a Processor Run to Import Data

Follow these steps:

1. On the page of the processor that you created, enter the following code in the input field on the left:

```
{
  "processors": [
    {
      "processorType": "inboundDocument",
      "processorName": "applicationResourceMapper",
      "processorDescription": "Processor to link applications to their relevant resources",
      "identifier": {
        "internal": "${content.id}"
      },
      "run": 0,
      "updates": [
        {
          "key": {
            "expr": "name"
          },
          "values": [
            {
              "expr": "${content.data.name}"
            }
          ]
        },
        {
          "key": {
            "expr": "url"
          },
          "values": [
            {
              "expr": "${content.data.url}"
            }
          ]
        }
      ],
      "enabled": true
    }
  ],
  "variables": {}
}
```

2. Under **Input**, enter the following code. In the **content** field, add an array of objects in JSON format that you retrieved in the previous step. Each object should contain the **type** and **id** of the fact sheet and the links to be added. In the example, we import two links to application fact sheets.

```
{
  "connectorType": "leanix",
  "connectorId": "leanix-pandasCSV",
  "connectorVersion": "1.0.0",
  "lxVersion": "1.0.0",
  "description": "LDIF Generated by PANDAS CSV data",
  "processingDirection": "inbound",
  "processingMode": "partial",
  "customFields": {},
  "content": [
```

```

    {
      "type": "Application",
      "id": "28fe4aa2-6e46-41a1-....-72afb3acf256",
      "data": {
        "name": "Technical documentation",
        "url": "https://docs.example.net/"
      }
    },
    {
      "type": "Application",
      "id": "2efa37b5-18aa-48d8-....-1328c0d856d7",
      "data": {
        "name": "Vendor's website",
        "url": "https://www.example-vendor.net/"
      }
    }
  ]
}

```

3. Click **Save**.

4. To test the processor, click **Test run**. Once the run is completed, check the output log for possible errors. If the run is successful, proceed to the next step.

5. To initiate the import process, click **Run**.

The links with associated names are added as resources to the specified application fact sheets.

To verify that the import was successful, navigate to the **Resources** tab of a specific fact sheet. You can also run a GraphQL query in the [GraphiQL Tool](#) to retrieve resources for a specific fact sheet.

Example query:

```

{
  factSheet(id: "{id}") {
    id
    name
    documents {
      edges {
        node {
          origin
          url
          name
        }
      }
    }
  }
}

```

Summary

In this tutorial, you learned how to:

- Programmatically convert a CSV file into LDIF format using a Python script
- Import links to application fact sheets using an inbound processor of the Integration API

Webhooks

Listen to events in SAP LeanIX using webhooks.

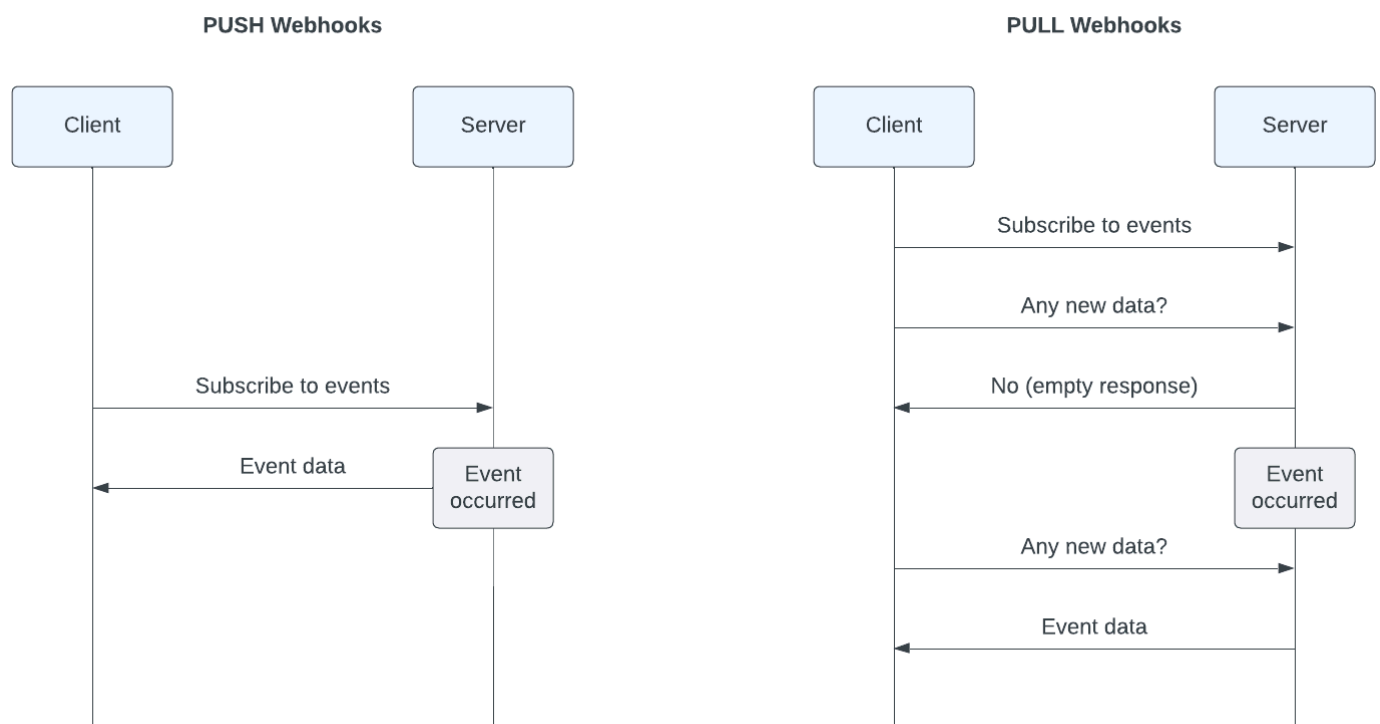
Overview

Webhooks let you receive updates about events as they happen in near real time in SAP LeanIX. With webhooks, you can set up event-driven triggers that initiate automated actions, which allows you to automate workflows for your organization and eliminate manual processes.

We support the following webhook types, based on the event delivery method:

- **PUSH webhooks:** As events occur in SAP LeanIX, they're sent through HTTP POST requests to the specified target URL. This is the recommended way to receive events from SAP LeanIX.
- **PULL webhooks:** A client requests events by polling a REST endpoint on the SAP LeanIX API. This method of retrieving events is useful if your environment doesn't allow exposing a public HTTP server to receive events, such as when your application operates behind a firewall. Events are retained on the server for 30 days, but it's recommended to retrieve them more regularly.

The following image illustrates how PUSH and PULL webhooks work.



Workflows for Data Retrieval Using PUSH and PULL Webhooks

i Note

Within the [Webhooks API](#) , webhooks are referred to as subscriptions. This term is also used in the application UI and this document.

Creating a Webhook

Follow these steps:

1. In the administration area, navigate to the **Webhooks** section.
2. Click **New Webhook**.
3. Optional: Enter a name and description for the webhook.
4. Configure the visibility of the webhook:
 - **Private**: Only you can access the webhook.
 - **Workspace**: Any admin in the workspace can view, modify, or delete the webhook.
5. Select the delivery type and configure relevant parameters:
 - **PUSH**: Enter a target URL for delivering events. Optionally, decide whether you want to skip events that can't be delivered.

After you save the webhook, you have the option to select an authorization method and enter a callback to manipulate the payload. For more information, see [PUSH Webhooks](#).
 - **PULL**: Optionally, adjust the maximum batch size that defines a soft upper limit for data in event batches. To learn how to fetch events, see [PULL Webhooks](#).

i Note

Once you save a delivery type for a webhook, you cannot change it.

6. The webhook is active by default. To deactivate the webhook, switch it off under **Status**.
7. In the **Triggering Events** section, click **Add new set** to add a triggering event for the webhook. For a full list of events, see [Webhook Events](#).
8. Click **Save**.

A webhook is created. You can update its configuration or delete it if needed.



Search by item name

Managed Code

Notifications Center

Optional Features and Early Access

Portals

Reports

To-Dos

Transformations

Workspace Views

DISCOVERY AND INTEGRATIONS

API Tokens

Developer Tools

Integrations

Integration API

Metrics

Reference Catalog

Self-Built Software Discovery

Sync Logging

Technical Users

Webhooks

[Webhooks](#) / Create Webhook

Create Webhook



Configuration

Authorization

Callback

General Settings

Name

Description

Visibility

Workspace ▾

Type

Push ▾

Status

☒ Active

Push Settings

Target URL *

Ignore errors

☐ Skip events that cannot be delivered to the target URL

Automatic Deactivation

Webhook will be automatically changed to 'Inactive' status after 10 days of unsuccessful deliveries. You can re-activate it anytime by coming back to this page.

Triggering Events

[+ Add new set](#)

Save

Cancel

Event Types and Payloads

For a full list of available event types, see [Webhook Events](#).

To view example payloads for selected event types, see [Webhook Payloads](#).

PUSH Webhooks

With PUSH webhooks, each time an event occurs in SAP LeanIX, it's sent to the specified target URL through an HTTP POST request. The request body contains an event payload in JSON format. To learn how to use PUSH webhooks, visit [PUSH Webhooks](#).

PULL Webhooks

With PULL webhooks, a client requests events by polling an endpoint on the SAP LeanIX Webhooks API. To learn how to use PUSH webhooks, visit [PULL Webhooks](#).

PUSH Webhooks

Receive events to a target URL as they occur in your workspace.

With PUSH webhooks, each time an event occurs in SAP LeanIX, it's sent to the specified target URL through an HTTP POST request. The request body contains an event payload in JSON format.

Your server should return a successful HTTP response code to confirm successful delivery. Delivery results for the past 30 days are listed on the webhook details page, below the configuration fields.

→ Tip

Before configuring automations with a PUSH webhook, verify the webhook configuration using a test target URL.

Authorization

In webhooks, we support the HTTPS protocol for the target URL, enhancing the security of your data transmission. Ensure to keep your target URL private to prevent unauthorized access.

We offer several authorization methods within the webhook configuration to further secure the delivery of webhook events to your target URLs. These include the Authorization header and OAuth 2.0. Additionally, you can append a token to the query string of the target URL for an extra layer of security. This ensures that only authorized entities can access your webhooks, further enhancing the security of your PUSH webhooks.

Authorization Header

This method uses an authorization header secret for security. To use it, generate a unique secret in your instance. For added protection, add a Basic authorization header by clicking [Add basic authorization header](#). The username and password you supply will be encoded for Basic authorization, ensuring their confidentiality during transmission.

OAuth 2.0

This method is recommended for its enhanced security. It involves authenticating with your identity provider (IdP) and using the Bearer token issued by your IdP to authorize with your target URL. To implement OAuth 2.0 authorization, configure your target URL instance to accept authorization through a Bearer token from your IdP. Once your JWT settings are correctly configured, specify the following details in the webhook configuration:

- **Token URL:** The endpoint in your IdP used to issue an OAuth 2.0 Bearer token.
- **Client ID:** The unique identifier for the client to which the Bearer token is issued.
- **Client Secret:** The client secret used to issue the Bearer token.
- **Scopes:** Scopes required for accessing the target URL endpoint. The Bearer token is issued with these scopes.
- **Credentials Location:** Select where to place credentials in the request:
 - **Authorization Header:** Credentials are sent as a Base64-encoded Basic authorization header.
 - **URL Encoded Body Request:** Credentials are sent in the request body as URL-encoded parameters.
- **Request Body Parameters:** A set of key/value pairs to include in the request body.
- **Request Query Parameters:** A set of key/value pairs to include as query parameters.

Error Handling

If event delivery fails, retries occur every 50 seconds until successful. Delivery may fail in the following cases:

- The target URL is unreachable or is improperly configured.
- The client returns a non-successful HTTP response code.

To skip events that cannot be delivered to the target URL, select the **ignore errors** option in the webhook configuration. With this option enabled, delivery retries do not occur.

Automatic Deactivation

Webhooks that have an unreachable target URL or respond with a non-successful HTTP status code for more than 10 days are automatically deactivated. An administrator needs to reactivate the webhook manually once the target URL becomes valid. To reactivate a webhook, in the webhook configuration, set the status to **Active**.

Callback

To manipulate the webhook payload before it's sent out to the target URL, you can define a callback using JavaScript code. You do this on the **Callback** tab of the webhook configuration page.

In the JavaScript code, the object `data` is available in the global scope. It includes the following properties, which you can modify directly in your code:

- `active`
- `payload`
- `targetMethod`
- `targetUrl`

You can view all custom code configurations used in webhooks with callbacks in the **Managed Code** section of the administration area. Here, you can monitor execution logs to track history and resolve issues. For more information, see [Managed Code](#).

Sample Code

```
export function main() {
  const payload = data.payload;

  data.targetMethod = 'POST';
  data.targetUrl = data.targetUrl += '?test=' + payload.factSheet.id;

  data.payload = {
    text : 'Hello World'
  };

  return data
}
```

PULL Webhooks

Request events by polling an API endpoint.

With PULL webhooks, a client requests events by polling an endpoint on the SAP LeanIX Webhooks API.

When configuring a PULL webhook, you can optionally adjust the maximum batch size that defines a soft upper limit for data in each event batch. The default `maxBatchSize` value is 512 KB. Usually, batches don't exceed the specified size limit. However, a single event exceeding this limit will still be delivered.

Intended Usage

PULL webhooks are intended to be used in a loop:

1. Make a request to retrieve some events.
2. Process events.
3. On success, make a request to advance the cursor or use the `autoCommit` option. Move the cursor forward only after you've processed the current batch of events. Once the cursor is moved, you can't fetch events from previous batches.
4. Repeat the above steps.

Concurrency

While we detect and reject processing of concurrent requests to the same subscription `id`, you should ensure that only one client uses the same webhook. Create one PULL webhook per client.

Event Availability

Events are retained on the server for 30 days. After this period, they are not available for retrieval.

Automatic Deactivation of PULL Webhooks

After 30 days of inactivity (no pulls or commits), a PULL webhook is automatically deactivated. To reactivate a webhook, in the webhook configuration, set the status to **Active**.

Retrieving Events

Before you start, ensure that you know how to authenticate to SAP LeanIX APIs. To learn more, see [Authentication to SAP LeanIX Services](#).

To retrieve events, make a GET request to the following endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/webhooks/v1/subscriptions/{id}/events
```

To get the subscription `id`, navigate to the webhook details page.

Example request:

```
curl --request GET \
  --url https://{SUBDOMAIN}.leanix.net/services/webhooks/v1/subscriptions/888eaaf3-c72f-411c-a78a-c
  --header 'Authorization: Bearer {ACCESS_TOKEN}'
```

Example response:

```
{
  "status": "OK",
```

```

"type": "PullResult",
"errors": [],
"total": 3,
"data": {
  "events": [
    {
      "cursor": {
        "offset": 178
      },
      "event": {...}
    },
    {
      "cursor": {
        "offset": 179
      },
      "event": {...}
    }
  ],
  "nextCursor": {
    "offset": 182
  }
}
}

```

The `event` object contains the webhook payload. Additionally, each event is annotated with the `cursor` attribute that can be used later to navigate to this event. The `nextCursor` attribute applies to the entire batch of events.

If there are no new events, an empty response is returned.

Moving the Cursor

Once you have successfully retrieved and processed a batch of events, you need to move the cursor forward to get the next batch. Otherwise, the next GET request would return the same events as in the previous response. This ensures that even if your application is temporarily malfunctioning, it will eventually receive all events.

Caution

When using PULL webhooks, move the cursor forward only after you've processed the current batch of events. Once the cursor is moved, you can't fetch events from previous batches.

To move the cursor to the next given `nextCursor`, make a PUT request to the following endpoint:

```
https://{SUBDOMAIN}.leanix.net/services/webhooks/v1/subscriptions/{id}/cursor
```

Example request:

```

curl --request PUT \
  --url https://{SUBDOMAIN}.leanix.net/services/webhooks/v1/subscriptions/888eaa3-c72f-411c-a78a-c \
  --header 'Authorization: Bearer {ACCESS_TOKEN}' \
  --header 'Content-Type: application/json' \
  --data '{"offset": 182}'

```

Alternatively, you can move the cursor forward automatically when retrieving events. To enable this behavior, add `autoCommit=true` as a query string parameter to the `/services/webhooks/v1/subscriptions/{id}/events`

endpoint. Enabling automated commits does not guarantee that all events will be successfully received and processed by your application due to possible network issues or other problems.

Example request:

```
curl --request GET \
  --url https://{SUBDOMAIN}.leanix.net/services/webhooks/v1/subscriptions/888eaaf3-c72f-411c-a78a-c
  --header 'Authorization: Bearer {ACCESS_TOKEN}'
```

Webhook Events

Available event types for webhook subscriptions.

Fact Sheets

i Note

For events related to fact sheets, you can optionally select a fact sheet type. This allows you to receive event notifications only for this fact sheet type without configuring a callback.

Event	Description
FACT_SHEET_CREATED	A new fact sheet is created.
FACT_SHEET_UPDATED	A fact sheet is updated. This also includes updates to fact sheet relations.
FACT_SHEET_VIEWED	A fact sheet is viewed by a workspace user.
FACT_SHEET_ARCHIVED	A fact sheet is archived.
FACT_SHEET_DELETED	A fact sheet is deleted. Deletion occurs when a fact sheet type is deleted from the meta model configuration or the retention period for an archived fact sheet ends.
FACT_SHEET_RECOVERED	A fact sheet is recovered.

Fact Sheet Relations

Event	Description
RELATION_CREATED	A new relation between fact sheets is created.
RELATION_UPDATED	A relation between fact sheets is updated.
RELATION_ARCHIVED	A relation between fact sheets is archived.
RELATION_DELETED	A relation between fact sheets is deleted. Deletion occurs when a fact sheet type is deleted from the meta model configuration or the retention period for an archived fact sheet ends.
RELATION_SWITCH	Only the target fact sheet within a relation changed.

Fact Sheet Tags

Event	Description
FACT_SHEET_TAG_ADDED	A tag is added to a fact sheet.
FACT_SHEET_TAG_REMOVED	A tag is removed from a fact sheet.

Fact Sheet Fields

Event	Description
FACT_SHEET_FIELD_UPDATED	A field on a fact sheet is updated.

Fact Sheet Resources

Event	Description
DOCUMENT_CREATED	A resource is added to a fact sheet.
DOCUMENT_UPDATED	A resource is updated on a fact sheet.
DOCUMENT_DELETED	A resource is deleted from a fact sheet.

Fact Sheet Subscriptions

Event	Description
SUBSCRIPTION_CREATED	A subscription is added to a fact sheet.
SUBSCRIPTION_UPDATED	A subscription is updated on a fact sheet.
SUBSCRIPTION_DELETED	A subscription is deleted from a fact sheet.

Fact Sheet Comments

Event	Description
COMMENT_CREATED	A comment is created on a fact sheet.
COMMENT_REPLY_ADDED	A reply is added to a comment on a fact sheet.
COMMENT_DELETED	A comment is deleted from a fact sheet.

Integration Runs

i Note

For events related to integrations, you can optionally select a specific integration. This allows you to receive event notifications only for this integration without configuring a callback.

Event	Description
INTEGRATION_RUN_STARTED	An integration run started.

Event	Description
INTEGRATION_RUN_FINISHED	An integration run finished.
INTEGRATION_RUN_ABORTED	An integration run is canceled.

Integration Configurations

Event	Description
CONFIGURATION_CREATED	A configuration for an integration is created.
CONFIGURATION_UPDATED	A configuration for an integration is updated.
CONFIGURATION_DELETED	A configuration for an integration is deleted.
CONFIGURATION_ACTIVATED	A configuration for an integration is activated.
CONFIGURATION_DEACTIVATED	A configuration for an integration is deactivated.

Automations

Event	Description
AUTOMATION_TRIGGERED	An automation is triggered. For more details, see Creating an Automation with a Webhook Action .

Surveys

Event	Description
POLL_RESULT_CREATED	A survey response is created.
POLL_RESULT_UPDATED	A survey response is updated.
POLL_RESULT_FINALIZED (within POLL_RESULT_UPDATED)	A survey response for a fact sheet is submitted.

To-Dos

Event	Description
TODO_CREATED	A to-do is created
TODO_UPDATED	A to-do is updated
TODO_DELETED	A to-do is deleted
TODO_ASSIGNEE_CREATED	An assignee is added to a to-do
TODO_ASSIGNEE_UPDATED	An assignee on a to-do is updated
TODO_ASSIGNEE_DELETED	An assignee is removed from a to-do

Webhook Payloads

Example payloads for selected webhook events.

i Note

This document provides example payloads for selected webhook events. To view a full list of event types, see [Webhook Events](#).

The data provided in a webhook payload is specific to each event type. In general, a payload contains the following information:

- Event details
- User who triggered the event
- Metadata, such as the fact sheet type, fields, and relations

The maximum supported payload size is 20 MB.

FACT_SHEET_CREATED

```
{
  "id": 42834,
  "type": "FactSheetCreatedEvent",
  "userId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
  "createdAt": "2024-02-16T13:12:27.032622Z",
  "factSheet": {
    "id": "95fbbeddd-abdf-4552-a82b-ce37c287735c",
    "rev": 0,
    "name": "AC Management",
    "type": "Application",
    "level": 1,
    "fields": [],
    "status": "ACTIVE",
    "lxState": "BROKEN_QUALITY_SEAL",
    "fullName": "AC Management",
    "naFields": [],
    "createdAt": "2024-02-16T13:12:26.995583770Z",
    "documents": [],
    "relations": [],
    "updatedAt": "2024-02-16T13:12:26.999252278Z",
    "completion": {
      "type": "SectionCompletion",
      "completion": 0.005952380952380952,
      "percentage": 1,
      "totalWeight": 7,
      "weightedSum": 0.041666666666666664,
      "subCompletions": {
        "section.projects": {
          "type": "SectionCompletion",
          "completion": 0,
          "percentage": 0,
          "totalWeight": 1,
          "weightedSum": 0,
          "subCompletions": {
```

```

        "subsection.factSheetsAffecting.label": {
            "type": "ConcreteCompletion",
            "completion": 0,
            "percentage": 0,
            "totalWeight": 1,
            "weightedSum": 0
        }
    },
    "section.sourcing": {...},
    "section.information": {...},
    "section.dependencies": {...},
    "section.dataManagement": {...},
    "section.businessSupport": {...},
    "section.lxPortfolioStrategy": {...}
},
"milestones": [],
"displayname": "AC Management",
"subscriptions": [],
"permittedReadACL": [],
"permittedWriteACL": [],
"lxExcludeFromQuota": false
},
"workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
"clonedFactSheet": false,
"transactionSequenceNumber": 75086
}

```

FACT_SHEET_UPDATED

```

{
    "id": 42844,
    "type": "FactSheetUpdatedEvent",
    "userId": "3et65c21-98f2-5376-aaaa-fchf654b9c22",
    "createdAt": "2024-02-16T13:15:23.702272Z",
    "factSheet": {
        "id": "95fbbeddd-abdf-4552-a82b-ce37c287735c",
        "rev": 3,
        "name": "AC Management",
        "tags": [...],
        "type": "Application",
        "level": 1,
        "fields": [],
        "status": "ACTIVE",
        "lxState": "BROKEN_QUALITY_SEAL",
        "fullName": "AC Management",
        "naFields": [],
        "createdAt": "2024-02-16T13:12:26.995584Z",
        "documents": [],
        "relations": [],
        "updatedAt": "2024-02-16T13:15:23.678693644Z",
        "completion": {...},
    }
}

```

```

    "milestones": [],
    "description": "AC Management app",
    "displayName": "AC Management",
    "subscriptions": [],
    "permittedReadACL": [],
    "permittedWriteACL": [],
    "lxExcludeFromQuota": false
  },
  "workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
  "transactionSequenceNumber": 75088
}

```

FACT_SHEET_VIEWED

```

{
  "id": 118540,
  "type": "FactSheetViewedEvent",
  "userId": "e84a8c18-a374-4ccd-b783-5a8050787614",
  "createdAt": "2024-07-05T11:13:11.834105Z",
  "factSheetId": "3921ec8c-a81a-470e-89de-b26b972bca59",
  "workspaceId": "95ed41a8-1856-4372-96b7-f3117fbc9675",
  "transactionSequenceNumber": null
}

```

FACT_SHEET_ARCHIVED

```

{
  "id": 42914,
  "type": "FactSheetArchivedEvent",
  "fsType": "Application",
  "userId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
  "comment": "Duplicate application",
  "createdAt": "2024-02-16T13:49:42.181834Z",
  "fsIdAndRev": {
    "id": "95fbeddd-abdf-4552-a82b-ce37c287735c",
    "rev": 11
  },
  "workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
  "affectedFactSheetIdAndRev": [
    {
      "id": "95fbeddd-abdf-4552-a82b-ce37c287735c",
      "rev": 11
    },
    {
      "id": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
      "rev": 27
    }
  ],
  "transactionSequenceNumber": 75126
}

```

RELATION_CREATED

```
{
  "id": 42846,
  "type": "RelationCreatedEvent",
  "userId": "3et65c21-98f2-5376-aaaa-fchf654b9c22",
  "relation": {
    "id": "675aacdd-cfdf-4702-8084-0646e74533b9",
    "toId": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
    "type": "relToChild",
    "fields": [],
    "fromId": "95fbeddd-abdf-4552-a82b-ce37c287735c",
    "status": "ACTIVE",
    "naFields": [],
    "typeToFS": "Application",
    "typeFromFS": "Application",
    "displayNameToFS": "AC Management / AC Management Cloud",
    "permittedReadACL": [],
    "constrainingRelations": []
  },
  "createdAt": "2024-02-16T13:17:20.965779Z",
  "toDetails": {
    "factSheetInfos": {
      "type": "Application",
      "idAndRev": {
        "id": "2efa37b5-18aa-48d8-9d70-1328c0d856d7",
        "rev": 25
      },
      "displayName": "AC Management / AC Management Cloud"
    },
    "effectiveRelationName": "relToParent"
  },
  "relationId": "675aacdd-cfdf-4702-8084-0646e74533b9",
  "fromDetails": {
    "factSheetInfos": {
      "type": "Application",
      "idAndRev": {
        "id": "95fbeddd-abdf-4552-a82b-ce37c287735c",
        "rev": 3
      },
      "displayName": "AC Management"
    },
    "effectiveRelationName": "relToChild"
  },
  "workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
  "transactionSequenceNumber": 75089
}
```

FACT_SHEET_TAG_ADDED

```
{
  "id": 42840,
  "type": "FactSheetTagAddedEvent",
```

```

    "tagId": "5ab3558a-580b-422d-8a95-12f7f075dfd7",
    "fsType": "Application",
    "userId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
    "tagName": "Public Cloud",
    "createdAt": "2024-02-16T13:14:08.732070Z",
    "fsIdAndRev": {
      "id": "95fbeddd-abdf-4552-a82b-ce37c287735c",
      "rev": 1
    },
    "workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
    "tagGroupName": "Cloud Transformation",
    "transactionSequenceNumber": 75087
  }

```

FACT_SHEET_FIELD_UPDATED

```

{
  "id": 42860,
  "path": "/externalId",
  "type": "FactSheetFieldUpdatedEvent",
  "fsType": "Application",
  "userId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
  "newValue": {
    "type": "ExternalId",
    "status": "ACTIVE",
    "comment": null,
    "externalId": "123456",
    "externalUrl": null
  },
  "oldValue": {
    "type": "ExternalId",
    "status": "ACTIVE",
    "comment": null,
    "externalId": "123456789",
    "externalUrl": null
  },
  "createdAt": "2024-02-16T13:23:03.697933Z",
  "fsIdAndRev": {
    "id": "95fbeddd-abdf-4552-a82b-ce37c287735c",
    "rev": 7
  },
  "workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
  "transactionSequenceNumber": 75093
}

```

DOCUMENT_CREATED

```

{
  "id": 42851,
  "type": "DocumentCreatedEvent",
  "refId": null,
  "fsType": "Application",

```

```

"userId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
"createdAt": "2024-02-16T13:19:09.116531Z",
"documentId": "d2421a7c-b4db-485f-873a-71a354eb7209",
"fsIdAndRev": {
  "id": "95fbeddd-abdf-4552-a82b-ce37c287735c",
  "rev": 4
},
"workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
"documentName": "Link to SAP LeanIX website",
"documentType": null,
"documentOrigin": "CUSTOM_LINK",
"transactionSequenceNumber": 75090
}

```

SUBSCRIPTION_CREATED

```

{
  "id": 42854,
  "type": "SubscriptionCreatedEvent",
  "fsType": "Application",
  "userId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
  "createdAt": "2024-02-16T13:20:23.769518Z",
  "roleInfos": [],
  "fsIdAndRev": {
    "id": "95fbeddd-abdf-4552-a82b-ce37c287735c",
    "rev": 6
  },
  "workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
  "subscriptionId": "39aae878-6ded-40de-aa01-9524ce3ee6d0",
  "subscribedUserId": "8393b7a8-8bfb-4989-a348-6cb31d56745f",
  "subscriptionType": "RESPONSIBLE",
  "transactionSequenceNumber": 75091
}

```

COMMENT_CREATED

```

{
  "id": 118533,
  "type": "CommentCreatedEvent",
  "fsType": "Application",
  "userId": "e84a8c18-a374-4ccd-b783-5a8050787614",
  "message": "This is a comment",
  "commentId": "e8182dd9-586b-4a5d-82f2-cfbec02cb118",
  "createdAt": "2024-07-05T11:02:00.129002Z",
  "fsIdAndRev": { "id": "3921ec8c-a81a-470e-89de-b26b972bca59", "rev": 35 },
  "workspaceId": "95ed41a8-1856-4372-96b7-f3117fbc9675",
  "transactionSequenceNumber": 106302
}

```

COMMENT_REPLY_ADDED


```
{
  "id": 118537,
  "type": "CommentReplyAddedEvent",
  "userId": "e84a8c18-a374-4ccd-b783-5a8050787614",
  "replyId": "847f58e0-8298-4f71-b541-71a38fccc0df",
  "commentId": "e8182dd9-586b-4a5d-82f2-cfbec02cb118",
  "createdAt": "2024-07-05T11:02:03.980262Z",
  "workspaceId": "95ed41a8-1856-4372-96b7-f3117fbc9675",
  "transactionSequenceNumber": 106303
}
```

COMMENT_DELETED

```
{
  "id": 118542,
  "type": "CommentDeletedEvent",
  "fsType": "Application",
  "userId": "e84a8c18-a374-4ccd-b783-5a8050787614",
  "comment": {
    "id": "e8182dd9-586b-4a5d-82f2-cfbec02cb118",
    "status": "ACTIVE",
    "userId": "e84a8c18-a374-4ccd-b783-5a8050787614",
    "message": "This is a comment",
    "replies": [
      {
        "id": "847f58e0-8298-4f71-b541-71a38fccc0df",
        "userId": "e84a8c18-a374-4ccd-b783-5a8050787614",
        "message": "This is a reply to a comment",
        "createdAt": "2024-07-05T11:02:03.939396098Z"
      }
    ],
    "createdAt": "2024-07-05T11:02:00.017209Z",
    "replaySequence": 0
  },
  "commentId": "e8182dd9-586b-4a5d-82f2-cfbec02cb118",
  "createdAt": "2024-07-05T11:15:06.301588Z",
  "fsIdAndRev": { "id": "3921ec8c-a81a-470e-89de-b26b972bca59", "rev": 35 },
  "workspaceId": "95ed41a8-1856-4372-96b7-f3117fbc9675",
  "transactionSequenceNumber": 106304
}
```

INTEGRATION_RUN_FINISHED (Integration API)

```
{
  "type": "INTEGRATION-API",
  "scope": "PARTIAL",
  "version": "1.0.0",
  "duration": 3.924405624,
  "progress": "FINISHED",
  "direction": "INBOUND",
  "errorCount": 0,
}
```

```

    "connectorId": "inbound-document-connector",
    "warningCount": 0,
    "connectorType": "inboundDocument",
    "integratedTool": "",
    "processedCount": 2,
    "synchronizationId": "865b71a1-e66f-49c0-89de-5bc5fb42401b"
  }

```

INTEGRATION_RUN_STARTED (ServiceNow Integration)

```

{
  "version": "?",
  "type": "SERVICENOW",
  "scope": "PARTIAL",
  "errorCount": 1,
  "warningCount": 2,
  "processedCount": 3,
  "duration": "PT0.123S",
  "progress": "RUNNING",
  "synchronizationId": "18886403-3edb-483d-a13f-cac5f4b34802"
}

```

INTEGRATION_RUN_FINISHED (ServiceNow Integration)

```

{
  "version": "?",
  "type": "SERVICENOW",
  "scope": "PARTIAL",
  "errorCount": 1,
  "warningCount": 2,
  "processedCount": 3,
  "duration": "PT0.123S",
  "progress": "FINISHED",
  "synchronizationId": "18886403-3edb-483d-a13f-cac5f4b34802"
}

```

INTEGRATION_RUN_ABORTED (ServiceNow Integration)

```

{
  "version": "?",
  "type": "SERVICENOW",
  "scope": "PARTIAL",
  "errorCount": 1,
  "warningCount": 2,
  "processedCount": 3,
  "duration": "PT0.123S",
  "progress": "ABORTED",
  "synchronizationId": "18886403-3edb-483d-a13f-cac5f4b34802"
}

```

CONFIGURATION_CREATED

```
{
  "type": "CONFIGURATION_CREATED",
  "userId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
  "connectorId": "inbound-document-connector",
  "workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
  "connectorType": "inboundDocument",
  "processingMode": "partial",
  "integrationType": "Integration API",
  "connectorVersion": "1.0.0",
  "processingDirection": "inbound"
}
```

AUTOMATION_TRIGGERED

```
{
  "tag": "AUTOMATION_WEBHOOK_TAG",
  "userId": "6ef78c21-98a2-4377-acaa-fc66ff2b78gf",
  "factSheet": {
    "id": "7a5bace3-4f7c-4682-b511-aecc9a6927d0",
    "type": "Application",
    "displayName": "Test Application Fact Sheet"
  },
  "automation": {
    "id": "3c72aa45-457a-4c10-9fac-40146121167c",
    "sourceUrl": "https://app.leanix.net/workspace-name/admin/automations/edit/3c72aa45-457a-4c10-9fac-40146121167c"
  },
  "workspaceId": "b7d655f1-4aa0-4132-92c6-a390a60cb8gh"
}
```

POLL_RESULT_CREATED

```
{
  "type": "POLL_RESULT_CREATED",
  "pollRun": {
    "id": "0b38d423-7b15-4ecd-ae3e-175c9fb8134a",
    "poll": {
      "id": "3754281a-f8f2-43b5-8ec5-b78465f48be6",
      "title": "Update Life Cycle Dates",
      "status": "ACTIVE",
      "language": "en",
      "creatorId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
      "timeFrame": 0,
      "userQuery": {
        "roles": [
          {
            "roleDetails": [],
            "subscriptionType": "RESPONSIBLE"
          }
        ]
      }
    }
  }
}
```

```

        "roleDetails": [],
        "subscriptionType": "OBSERVER"
    }
  ],
  "workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
  "creationDate": "2024-02-16T13:31:19.391993Z",
  "factSheetType": "Application",
  "questionnaire": {
    "id": "5af9a9b9-6da1-465a-bc95-8344b9ec2fb2",
    "questions": [
      {
        "id": "9de72fa1-d999-bff3-eabb-ba3315bb479b",
        "type": "factsheetelement",
        "label": "Update life cycle",
        "deleted": false,
        "disabled": false,
        "powerfeature": true,
        "factSheetElement": {
          "type": "FACT_SHEET_FIELD",
          "factSheetFieldName": "lifecycle",
          "factSheetFieldType": "LIFECYCLE"
        }
      }
    ]
  },
  "factSheetQuery": {
    "ids": ["95fbeddd-abdf-4552-a82b-ce37c287735c"],
    "filter": {
      "facetFilter": [
        {
          "keys": ["Application"],
          "facetKey": "FactSheetTypes",
          "operator": "OR"
        }
      ]
    }
  },
  "repeatIntervall": 0,
  "introductionText": "...",
  "introductionSubject": "...",
  "additionalFactSheetText": "...",
  "allowedPermissionStatus": "ACTIVE_ONLY",
  "sendChangeNotifications": true,
  "additionalFactSheetSubject": "A Fact Sheet has been added to a Survey you are participating",
  "additionalFactSheetCheckEnabled": true
},
"status": "PREPARING",
"language": "en",
"pollCopy": {
  "id": "3754281a-f8f2-43b5-8ec5-b78465f48be6",
  "title": "Update Life Cycle Dates",
  "status": "ACTIVE",
  "language": "en",

```

```

"creatorId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
"timeFrame": 0,
"userQuery": {
  "roles": [
    {
      "roleDetails": [],
      "subscriptionType": "RESPONSIBLE"
    },
    {
      "roleDetails": [],
      "subscriptionType": "OBSERVER"
    }
  ]
},
"workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
"creationDate": "2024-02-16T13:31:19.391993Z",
"factSheetType": "Application",
"questionnaire": {
  "id": "5af9a9b9-6da1-465a-bc95-8344b9ec2fb2",
  "questions": [
    {
      "id": "9de72fa1-d999-bff3-eabb-ba3315bb479b",
      "type": "factsheetelement",
      "label": "Update life cycle",
      "deleted": false,
      "disabled": false,
      "powerfeature": true,
      "factSheetElement": {
        "type": "FACT_SHEET_FIELD",
        "factSheetFieldName": "lifecycle",
        "factSheetFieldType": "LIFECYCLE"
      }
    }
  ]
},
"factSheetQuery": {
  "ids": ["95fbeddd-abdf-4552-a82b-ce37c287735c"],
  "filter": {
    "facetFilter": [
      {
        "keys": ["Application"],
        "facetKey": "FactSheetTypes",
        "operator": "OR"
      }
    ]
  }
},
"repeatIntervall": 0,
"introductionText": "...",
"introductionSubject": "...",
"additionalFactSheetText": "...",
"allowedPermissionStatus": "ACTIVE_ONLY",
"sendChangeNotifications": true,
"additionalFactSheetSubject": "A Fact Sheet has been added to a Survey you are participating

```

```

    "additionalFactSheetCheckEnabled": true
  },
  "senderId": "3et65c21-98f2-5376-acaa-fchf654b9c22",
  "startTime": "2024-02-16T13:40:56.246318Z",
  "resultView": "NEW",
  "pollRunType": "LIVE",
  "pollResultType": "PER_FACTSHEET",
  "factsheetsTotal": 1,
  "recipientsTotal": 2,
  "factsheetsProgress": 0,
  "factsheetsSelected": 1,
  "recipientsProgress": 0,
  "factsheetsCompleted": 0,
  "recipientsCompleted": 0,
  "sendChangeNotifications": true,
  "additionalFactSheetCheckEnabled": true
},
"createdAt": 1708090856819,
"pollResult": {
  "id": "1eda2425-2d33-4c5e-a3e3-7f767770b778",
  "users": [
    {
      "id": "8393b7a8-8bfb-4989-a348-6cb31d56745f",
      "email": "john.smith@organization.com",
      "fullName": "John Smith",
      "lastName": "Smith",
      "firstName": "John"
    },
    {
      "id": "3et65c21-98f2-5376-acaa-fchf654b9c22",
      "email": "john.doe@organization.com",
      "fullName": "John Doe",
      "lastName": "Doe",
      "firstName": "John"
    }
  ],
  "sender": {
    "id": "3et65c21-98f2-5376-acaa-fchf654b9c22",
    "email": "john.doe@organization.com",
    "fullName": "John Doe",
    "lastName": "Doe",
    "firstName": "John"
  },
  "status": "NEW",
  "answers": [
    {
      "questionId": "9de72fa1-d999-bff3-eabb-ba3315bb479b"
    }
  ],
  "factSheet": {
    "id": "95fbeddd-abdf-4552-a82b-ce37c287735c",
    "name": "AC Management",
    "type": "Application",
    "displayName": "AC Management App"
  }
}

```

```

    },
    "pollRunId": "0b38d423-7b15-4ecd-ae3e-175c9fb8134a"
  },
  "workspaceId": "ae6756f1-4aag-4356-92c6-a390a60cb8fg",
  "pollResultEventType": "POLL_RESULT_CREATED"
}

```

TODO_CREATED

```

{
  "createdAt": "2026-03-26T14:00:00+01:00",
  "type": "TODO_CREATED",
  "sourceName": "todo",
  "sourceId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "sourceUrl": "https://app.leanix.net/services/todo/v1/to-do?todoIds=a1b2c3d4-e5f6-7890-abcd-ef1234!",
  "workspaceId": "11111111-2222-3333-4444-555555555555",
  "userId": "aaaa1111-bbbb-2222-cccc-333333333333",
  "tags": ["TODO", "TODO_CREATED"],
  "payload": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "title": "Review architecture decision",
    "description": "Review the ADR for the new auth middleware",
    "factSheetId": "ff000000-1111-2222-3333-444444444444",
    "category": "ACTION_ITEM",
    "state": "OPEN",
    "dueDate": "2026-04-15",
    "creatorId": "aaaa1111-bbbb-2222-cccc-333333333333",
    "creationTime": "2026-03-26T13:00:00Z"
  }
}

```

TODO_UPDATED

```

{
  "createdAt": "2026-03-26T14:05:00+01:00",
  "type": "TODO_UPDATED",
  "sourceName": "todo",
  "sourceId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "sourceUrl": "https://app.leanix.net/services/todo/v1/to-do?todoIds=a1b2c3d4-e5f6-7890-abcd-ef1234!",
  "workspaceId": "11111111-2222-3333-4444-555555555555",
  "userId": null,
  "tags": ["TODO", "TODO_UPDATED"],
  "payload": {
    "changes": {
      "title": "Review architecture decision (updated)",
      "state": "DONE"
    },
    "state": {
      "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "title": "Review architecture decision (updated)",
      "description": "Review the ADR for the new auth middleware",
      "factSheetId": "ff000000-1111-2222-3333-444444444444",
      "category": "ACTION_ITEM",
      "state": "DONE",
      "dueDate": "2026-04-15",
      "creatorId": "aaaa1111-bbbb-2222-cccc-333333333333",
      "creationTime": "2026-03-26T13:00:00Z"
    }
  }
}

```

TODO_DELETED

```
{
  "createdAt": "2026-03-26T14:10:00+01:00",
  "type": "TODO_DELETED",
  "sourceName": "todo",
  "sourceId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "sourceUrl": "https://app.leanix.net/services/todo/v1/to-do?todoIds=a1b2c3d4-e5f6-7890-abcd-ef1234!",
  "workspaceId": "11111111-2222-3333-4444-555555555555",
  "userId": "bbbb2222-cccc-3333-dddd-444444444444",
  "tags": ["TODO", "TODO_DELETED"],
  "payload": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
  }
}
```

TODO_ASSIGNEE_CREATED

```
{
  "createdAt": "2026-03-26T14:15:00+01:00",
  "type": "TODO_ASSIGNEE_CREATED",
  "sourceName": "todo",
  "sourceId": "d4e5f6a7-b8c9-3210-fedc-ba9876543210",
  "sourceUrl": "https://app.leanix.net/services/todo/v1/to-do?todoIds=a1b2c3d4-e5f6-7890-abcd-ef1234!",
  "workspaceId": "11111111-2222-3333-4444-555555555555",
  "userId": "bbbb2222-cccc-3333-dddd-444444444444",
  "tags": ["TODO", "TODO_ASSIGNEE_CREATED"],
  "payload": {
    "userId": "cccc3333-dddd-4444-eeee-555555555555",
    "todoId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "isOwner": true,
    "changedBy": "bbbb2222-cccc-3333-dddd-444444444444",
    "id": "d4e5f6a7-b8c9-3210-fedc-ba9876543210"
  }
}
```

TODO_ASSIGNEE_UPDATED

```
{
  "createdAt": "2026-03-26T14:20:00+01:00",
  "type": "TODO_ASSIGNEE_UPDATED",
  "sourceName": "todo",
  "sourceId": "d4e5f6a7-b8c9-3210-fedc-ba9876543210",
  "sourceUrl": "https://app.leanix.net/services/todo/v1/to-do",
  "workspaceId": "11111111-2222-3333-4444-555555555555",
  "userId": "bbbb2222-cccc-3333-dddd-444444444444",
  "tags": ["TODO", "TODO_ASSIGNEE_UPDATED"],
  "payload": {
    "isOwner": false,
    "changedBy": "bbbb2222-cccc-3333-dddd-444444444444"
  }
}
```

TODO_ASSIGNEE_DELETED

```
{
  "createdAt": "2026-03-26T14:25:00+01:00",
  "type": "TODO_ASSIGNEE_DELETED",
  "sourceName": "todo",
  "sourceId": "d4e5f6a7-b8c9-3210-fedc-ba9876543210",
  "sourceUrl": "https://app.leanix.net/services/todo/v1/to-do",
  "workspaceId": "11111111-2222-3333-4444-555555555555",
  "userId": "bbbb2222-cccc-3333-dddd-444444444444",
  "tags": ["TODO", "TODO_ASSIGNEE_DELETED"],
  "payload": {
    "changedBy": "bbbb2222-cccc-3333-dddd-444444444444"
  }
}
```


Rate Limiting

Learn how our rate limiting in SAP LeanIX APIs works.

Overview

Rate limiting restricts the number of requests that a user or system can make to a network interface within a specific timeframe. Rate limiting ensures the following:

- **Fair resource allocation:** All customer workspaces receive equitable access to API resources regardless of individual usage patterns.
- **System reliability:** Prevents potential downtimes and service degradation caused by excessive request volumes.
- **Performance optimization:** Maintains consistent response times for all users by balancing load across the infrastructure.

Consider our rate limits when implementing high-frequency API calls.

How Rate Limiting Works

If you reach a rate limit, you get the HTTP 429 Too Many Requests response status code.

The API response may contain a message showing the reason for reaching the rate limit. For example, if you reach a user-based rate limit, you get a response like the following: HTTP 429 Too Many Requests. USER_INTERVAL_REQUEST_LIMIT_EXCEEDED.

In the event of rate limiting or temporary unavailability, the API response may include a Retry-After header, indicating the recommended duration the client should wait before initiating a new request. If this header is not present, the client must determine an appropriate retry interval based on the specific rate limits established by the API.

Rate Limits for the Pathfinder API

The [Pathfinder API](#) is the core service that allows you to work with your fact sheets. The API powers data and capabilities behind the fact sheets functionality.

i Note

Rate limits apply to both your interactions with fact sheets in the application UI and your backend API calls.

Rate Limits for the Pathfinder API	
Type	Rate Limit
User-based rate limit	1800 requests per minute per user (including both human users and technical users)
Rate limit for internal requests	1200 requests per minute per workspace for requests from specific internal services listed below. These services automatically reschedule rate-limited requests for you. This applies to: • Import Export API

Type	Rate Limit
	• Integration API ➡ • Integration ServiceNow API ➡ • Integration Signavio API ➡

Rate Limits for the Self-Built Software Discovery API

The [Self-Built Software Discovery API](#) ➡ applies rate limits to the following endpoints:

Rate-Limited Endpoints

Endpoint	Description	Rate Limit
POST /factSheets/{factSheetId}/sboms	Upload an SBOM file for a specific fact sheet	300 requests per minute per workspace
PUT /manifests	Upload a manifest file	300 requests per minute per workspace
GET /applications/{factSheetId}/components	Retrieve discovered SBOM library components for a business application	300 requests per minute per workspace
GET /components/search	Search for SBOM library components by package URL	300 requests per minute per workspace

If your integration exceeds these limits, the API returns an HTTP 429 Too Many Requests response. To handle rate limit responses gracefully, build retry logic with exponential backoff into your integration.

≡, Sample Code

```
import time
from requests.exceptions import HTTPError
def request_with_retry(request_func, max_retries=3, base_delay=1):
    """Execute request with exponential backoff on rate limit."""
    for attempt in range(max_retries):
        response = request_func()
        if response.status_code == 429:
            delay = base_delay * (2 ** attempt)
            time.sleep(delay)
            continue
        response.raise_for_status()
        return response
    raise HTTPError("Max retries exceeded due to rate limiting")
```

Response Status Codes

Overview of HTTP response status codes in SAP LeanIX APIs.

Overview

SAP LeanIX uses HTTP response codes to communicate the status of API requests, enabling developers to effectively interpret the outcome of their interactions with SAP LeanIX services. These codes fall into three broad categories, reflecting the nature of the response:

- Success codes (2xx)
- Client-side error codes (4xx)
- Server-side error codes (5xx)

i Note

Response codes may change in future API updates. Consider the information in this guide as a general reference, taking into account the dynamic nature of APIs and the potential for modifications in response codes with future updates. To view detailed information about response codes for each API, use our [OpenAPI Explorer](#) .

Success Codes (2xx)

2xx status codes indicate that the request was successful and the desired action was completed. These codes provide positive feedback, allowing developers to proceed with further processing or updates. Success codes adhere to the guidelines specified in [Section 6.3 of RFC 7231](#) , the Internet Standards Track document issued by the Internet Engineering Task Force (IETF).

Code	Meaning
200 (OK)	The request was successful, and the resource was retrieved or updated.

Client-Side Error Codes (4xx)

4xx status codes indicate errors originating from the client side, such as invalid requests or missing parameters. They provide actionable information for identifying and rectifying the problem before retrying the request. These error codes adhere to the guidelines specified in [Section 6.5 of RFC 7231](#) , the Internet Standards Track document issued by IETF.

Code	Meaning
400 (Bad Request)	The request was malformed or invalid.
401 (Unauthorized)	The request requires authentication.
403 (Forbidden)	The request is not authorized to access the resource.
404 (Not Found)	The resource does not exist.
405 (Method Not Allowed)	The requested method is not supported for the specified resource.
409 (Conflict)	The request could not be completed due to a conflict with the current state of the target resource.
410 (Gone)	The requested resource is no longer available.
422 (Validation Error)	The request was syntactically valid but semantically invalid, meaning that the data provided does not meet the specified validation rules.
429 (Too Many Requests)	The request rate exceeds the allowed limit. For more information, see Rate Limiting .

Server-Side Error Codes (5xx)

5XX codes indicate errors originating from the SAP LeanIX servers, suggesting system-level issues or temporary disruptions. These error codes adhere to the guidelines specified in [Section 6.6 of RFC 7231](#) , the Internet Standards Track document issued by IETF.

Code	Meaning
500 (Internal Server Error)	An unexpected server-side error has occurred.
502 (Bad Gateway)	The SAP LeanIX servers received an invalid response from an upstream service.
503 (Service Unavailable)	The SAP LeanIX servers are temporarily unavailable.

API Updates

[Poll API Deprecation: Transition to the New Survey API \(March 2027\)](#)

[GraphQL API: Error Handling for Multiple Mutations Sent in a Single Request \(July 28, 2025\)](#)

[Self-Built Software Discovery API: Asynchronous Processing of SBOM File Ingestions \(April 7, 2025\)](#)

[GraphQL API Changes \(January 15, 2025\)](#)

Poll API Deprecation: Transition to the New Survey API (March 2027)

Learn what's changing and how to prepare for the transition to the new API for surveys.

Overview

SAP LeanIX is introducing the Survey API v1 (survey/v1) to replace the legacy Poll API v2 (poll/v2). The legacy Poll API v2 remains available during a transition period so that you can migrate your integrations at your own pace. For migration steps, see [What You Need to Do](#).

Base URL changes:

- **Legacy:** `https://{SUBDOMAIN}.leanix.net/services/poll/v2`
- **New:** `https://{SUBDOMAIN}.leanix.net/services/survey/v1`

Transition Timeline

- **January 20, 2026:** Deprecation headers added to all Poll API v2 responses. The Poll API v2 is officially deprecated.
- **February 2026 to March 2027:** Migration period. Update your integrations to use the Survey API v1. Both APIs remain fully functional.
- **End of March 2027:** The Poll API v2 will be removed. The Survey API v1 will become the only available API.

Why We're Making These Changes

- **Improved performance:** Response times are 30 to 50 percent faster across all endpoints.
- **Modern architecture:** Better support for concurrent operations and scalability.
- **Simplified API contracts:** Fewer required parameters and cleaner data structures.

Important: Poll API v2 is Now Deprecated

As of January 20, 2026, all responses from the legacy Poll API v2 include deprecation headers indicating that this API is no longer recommended for use. Check for these headers in your code to detect deprecated endpoints automatically.

Response headers added:

```
X-SAP-stateInfo: {"state": "deprecated"}
X-API-Warn: 'poll/v2' is a deprecated API. Please use 'survey/v1'.
```

What this means:

- The Poll API v2 is officially deprecated and will be discontinued at the end of March 2027.
- Use the Survey API v1 for all integrations.
- If you use existing integrations, plan to migrate them now.
- Deprecation headers appear on every response from Poll API v2 endpoints.

Example response headers:

```
HTTP/1.1 200 OK
X-SAP-stateInfo: {"state": "deprecated"}
X-API-Warn: 'poll/v2' is a deprecated API. Please use 'survey/v1'.
Content-Type: application/json

{...response body...}
```

What's Changing

For a detailed overview of changes, refer to:

- [Key Changes](#)
- [Breaking Changes](#)

Key Changes

Base URL Update

Use the new endpoint for all API calls.

Before:

```
curl -X GET "https://{SUBDOMAIN}.leanix.net/services/poll/v2/polls?workspaceId=a1b2c3d4"
```

After:

```
curl -X GET "https://{SUBDOMAIN}.leanix.net/services/survey/v1/polls"
```

Removed workspaceId Query Parameter

The workspace is now automatically determined from your authentication token.

Endpoint	Poll API v2	Survey API v1
Get surveys	GET /polls?workspaceId={id}	GET /polls
Create a survey	POST /polls?workspaceId={id}	POST /polls
Get a survey run	GET /pollRuns/{id}?workspaceId={id}	GET /pollRuns/{id}
Create a survey run	POST /pollRuns?workspaceId={id}	POST /pollRuns

Sorting Parameter Format Changed (Breaking Change)

The sorting format has been simplified for better usability.

Poll API v2:

```
GET /polls?sort=createdAt-asc
GET /polls?sort=createdAt-desc
```

Survey API v1:

```
GET /polls?sort=+createdAt
GET /polls?sort=-createdAt
```

Use Case	Poll API v2	Survey API v1
Sort by creation date (newest first)	sort=createdAt-desc	sort=-createdAt
Sort by start time (oldest first)	sort=startTime-asc	sort=+startTime
Sort by title (A-Z)	sort=title-asc	sort=+title
Multiple sorts	sort=status-asc,createdAt-desc	sort=+status,-createdAt

HTTP Response Codes Changed (Breaking Change)

Some DELETE endpoints now return different HTTP status codes.

Endpoint	Poll API v2	Survey API v1	Change
DELETE /polls/{pollId}	HTTP 200	HTTP 204 No Content	Changed
DELETE /pollRuns/{pollRunId}	HTTP 200	HTTP 204 No Content	Changed

What this means:

This is custom documentation. For more information, please visit [SAP Help Portal](#).

- DELETE operations now return HTTP 204 (No Content) instead of HTTP 200 (OK).
- HTTP 204 indicates successful deletion with no response body.
- Update your client code to handle 204 responses on delete operations.
- Treat both 2xx responses as successful.

Example:

```
// Before (Poll API v2)
DELETE /services/poll/v2/polls/{pollId}
// Returns: HTTP 200 with empty body

// After (Survey API v1)
DELETE /services/survey/v1/polls/{pollId}
// Returns: HTTP 204 No Content
```

Core Endpoints Available in Both APIs

- GET/POST /polls
- GET/PUT /polls/{pollId}
- DELETE /polls/{pollId}
- GET/POST /pollRuns
- GET/PUT /pollRuns/{pollRunId}
- DELETE /pollRuns/{pollRunId}
- GET /pollResults
- GET /pollRuns/{pollRunId}/poll_results
- GET /pollResults.xlsx
- GET /pollTemplates

Breaking Changes**Response Wrappers**

The GET /pollRuns response is now wrapped differently:

Poll API v2:

```
{
  "status": "OK",
  "type": "PollRun",
  "total": 5,
  "data": [ { }, { } ]
}
```

Survey API v1:

```
{
  "status": "OK",
  "type": "PollRun",
```

```

    "total": 5,
    "data": {
      "pollRuns": [ { }, { } ],
      "count": { "total": 5, "completed": 3, "open": 2 }
    }
  }
}

```

POST /pollRuns Request Simplified

Field	Poll API v2	Survey API v1	Status
workspaceId	Required	Not needed	Removed
pollId	N/A	Required	New
users	N/A	Optional (List<User>)	New
resultView	Optional	Optional (default: NEW)	Unchanged

GET /pollRuns/{id}/recipients Response Restructured

The recipients endpoint now returns data in nested relationship objects instead of flat lists.

Poll Result Response Enhanced

New fields in the Survey API v1:

- **sender:** The user who sent the survey.
- **factSheet:** Enhanced fact sheet information.
- **requestingUser:** Requesting user information (conditional).

What You Need to Do

Migration Steps

1. **Update the base URL:** Change all references from /services/poll/v2 to /services/survey/v1.
2. **Remove the workspace ID:** Delete workspaceId from all query parameters.
3. **Update sorting parameters:** Convert from property-asc/desc format to +property/-property format.
4. **Update DELETE handling:** Handle HTTP 204 No Content responses from DELETE operations.
5. **Update response parsing:** Adjust code that parses GET /pollRuns responses.
6. **Update request payloads:** Modify the POST /pollRuns request structure.
7. **Update user handling:** Use firstName and lastName instead of name.
8. **Update recipients parsing:** Handle the nested structure in the recipients endpoint.
9. **Test thoroughly:** Run parallel tests against both endpoints.

Backward Compatibility

Fully compatible:

- Authentication headers and token format
- Database schema and existing data
- Core business logic and workflows
- Error handling and most status codes

Breaking changes requiring updates:

- Endpoint base URL
- `workspaceId` parameter removal
- Sorting parameter format
- HTTP response codes for DELETE operations (204 instead of 200)
- GET `/pollRuns` response wrapper structure
- POST `/pollRuns` request payload
- GET `/pollRuns/{id}/recipients` response structure
- User field names

Examples

Example 1: Retrieving Surveys

Before (Poll API v2):

```
curl -X GET "https://{SUBDOMAIN}.leanix.net/services/poll/v2/polls?workspaceId=abc123&sort=created/"
```

After (Survey API v1):

```
curl -X GET "https://{SUBDOMAIN}.leanix.net/services/survey/v1/polls?sort=-createdAt"
```

Example 2: Creating a Survey Run

Before (Poll API v2):

```
POST /services/poll/v2/pollRuns?workspaceId=abc123
{
  "status": "STARTED",
  "factsheetIds": ["id1", "id2"],
  "recipientIds": ["user1", "user2"]
}
```

After (Survey API v1):

```
POST /services/survey/v1/pollRuns
{
  "pollId": "poll-uuid",
  "resultView": "NEW"
}
```

}

Example 3: Sorting with Multiple Fields

Before (Poll API v2):

```
GET /polls?sort=status-asc,createdAt-desc
```

After (Survey API v1):

```
GET /polls?sort=+status,-createdAt
```

Example 4: Deleting a Survey

Before (Poll API v2):

```
DELETE /services/poll/v2/polls/{pollId}
# Returns: HTTP 200 OK
```

After (Survey API v1):

```
DELETE /services/survey/v1/polls/{pollId}
# Returns: HTTP 204 No Content
```

Related Resources

Find complete API specifications and detailed endpoint documentation in:

- **Survey API v1 OpenAPI documentation:** <https://{SUBDOMAIN}.leanix.net/services/survey/v1/openapi>
- **Survey API v1 API Explorer:** <https://{SUBDOMAIN}.leanix.net/openapi-explorer?urls.primaryName=Survey>
- **Legacy Poll API v2 reference:** <https://{SUBDOMAIN}.leanix.net/services/poll/v2/api-docs/swagger.json>
- **Legacy Poll API v2 API Explorer:** <https://{SUBDOMAIN}.leanix.net/openapi-explorer?urls.primaryName=Poll>

Refer to these resources for comprehensive endpoint details, request and response schemas, sorting field names, and full parameter documentation. Replace `{SUBDOMAIN}` with your value (for example, `mycompany.leanix.net`).

For additional support or migration questions, contact the SAP LeanIX support team.

GraphQL API: Error Handling for Multiple Mutations Sent in a Single Request (July 28, 2025)

i Note

The release date for this update has changed from the originally planned date of July 14, 2025, to **July 28, 2025**.

We plan to improve the error handling behavior of the GraphQL API for multiple mutations sent in a single request. If a payload contains at least one invalid mutation, the API will provide a clearer response in the `data` and `errors` attributes. The response will explicitly indicate that no mutations have been applied, preventing assumptions of partial success. The core behavior of the GraphQL API remains unchanged in how it handles multiple mutations in a single request.

To learn more about how error handling in GraphQL works, visit [Error Handling in GraphQL](#).

Current Behavior

When you send multiple mutations in a single API request and at least one mutation is invalid (such as a mutation for updating a non-existent field), none of the mutations in that request are executed. However, the API response might be mistaken for a successful partial execution because valid mutations are listed in the `data` attribute of the response.

What's Changing

On July 28, 2025, at 9 AM CEST, we'll update the error handling behavior of the GraphQL API. When multiple mutations are sent in a single API request and at least one mutation is invalid, the following will happen:

- The `data` attribute will return `null` in the API response.
- The `errors` attribute will include an entry for each failed mutation, along with an error description.
- A `messageError in Request. Transaction is rolled back!` will be returned to indicate that any other valid mutations that did not cause an error weren't effectively applied.

What You Need to Do

Adjust your GraphQL API calls and integrations where needed to account for the new error response.

Example

In the request below, the mutation `m2` for updating a fact sheet includes an invalid field: `descriptio`.

Example request:

```
mutation {
  m1: updateFactSheet(
    id: "d9506c79-5d4e-4661-9297-b3b84bd17b79"
    patches: [{ op: replace, path: "/description", value: "first description" }]
  ) {
    factSheet {
      id
      description
    }
  }
  m2: updateFactSheet(
    id: "3b2f5b03-8cff-4aa1-a619-38aa7e90d456"
    patches: [
      { op: replace, path: "/descriptio", value: "invalid field description" }
    ]
  ) {
    factSheet {
      id
      description
    }
  }
}
```

```

m3: updateFactSheet(
  id: "351ee5ff-c851-48c5-92e6-3d373c6483ce"
  patches: [
    { op: replace, path: "/description", value: "third description" }
  ]
) {
  factSheet {
    id
    description
  }
}

```

In the response, the `data` attribute returns `null`. The `errors` attribute provides details about the failed mutation. An error message indicates that the entire request hasn't been executed and none of the mutations have been applied.

Example response:

```

{
  "data" : null,
  "errors" : [ {
    "message" : "The path '/descriptio' is invalid in FactSheet schema Application.",
    "path" : [ "m2" ],
    "extensions" : {
      "errorType" : "BUSINESS_LOGIC"
    }
  }, {
    "message" : "Error in Request. Transaction is rolled back!",
    "path" : [ ],
    "extensions" : {
      "errorType" : "BUSINESS_LOGIC"
    }
  } ]
}

```

Self-Built Software Discovery API: Asynchronous Processing of SBOM File Ingestions (April 7, 2025)

Learn what's changing in the Self-Built Software Discovery API and how to prepare for these changes to avoid any issues with your integrations.

i Note

The changes to the Self-Built Software Discovery API described here have been implemented on April 7, 2025. If you have questions or need additional support, please contact SAP LeanIX support.

Overview

We're introducing enhancements to the [Self-Built Software Discovery REST API](#)  to improve SBOM file ingestion and system performance for our customers. These changes will come into effect on **April 7, 2025, following a transition period**. During this period, you can test and familiarize yourself with the new behavior while continuing to use your current API implementation. However, after the transition period, only the new behavior will be supported.

- **Transition period:** March 10, 2025 – April 6, 2025
- **Switchover date:** April 7, 2025

i Note

If your current implementation of `/factSheets/{factSheetId}/sboms` does not rely on the HTTP **status code** or the **payload**, these changes will not affect you. Otherwise, you will need to update your integrations as detailed below.

During the transition period, you can try out the updated API to see how the behavior will change. For details, see [Test the New API Behavior During the Transition Period](#).

To learn more about self-built software discovery and SBOMs, refer to:

- [Self-Built Software Discovery](#)
- [Software Bill of Materials \(SBOM\)](#)

Background

These API updates are designed to significantly improve SBOM file ingestion and enhance system efficiency. The changes are driven by two primary factors:

- **Scalability and resilience:** While our systems can handle the current load, we anticipate increased SBOM traffic. To maintain optimal user experience and ensure our systems can manage this growth, we're transitioning to asynchronous processing of SBOM files. This shift will address potential hurdles and ensure continued high performance.
- **Extensibility:** Currently, our system processes SBOM files to show library reuse in the wider business context (for example, the log4j case) and intelligently match technology stacks. Switching to asynchronous processing will allow us to add additional post-processing mechanisms in the future, such as vulnerability checks. Though there's no established timeline for these enhancements, we aim to have the right technical foundation in place for future implementation.

What's Changing

The changes affect the following endpoint on the Self-Built Software Discovery API:

Method	Endpoint	Schema
POST	<code>/factSheets/{factSheetId}/sboms</code>	Reference ➡

We're also adding a new endpoint for tracking the processing job status.

Here's what's changing:

- [Response Status Code](#)
- [Enhanced Response Payload](#)
- [New Endpoint for Tracking the Job Status](#)

i Note

While the processing of SBOM files will be asynchronous, the validation of request payloads will remain unchanged. The API immediately validates payloads upon receiving requests.

Response Status Code

The HTTP response status code changes as follows:

- **Current code:** 201 (Created)

- **New code:** 202 (Accepted)

Enhanced Response Payload

The following fields are added to the `data` object in the response payload:

Field	Description
<code>jobId</code>	Identifier of the processing job.
<code>workspaceId</code>	Identifier of the workspace.
<code>status</code>	Current status of the job.
<code>errorMessage</code>	Error message provided when the job fails.
<code>created_at</code>	Timestamp indicating when the job was created.
<code>plannedExecutionAt</code>	Timestamp indicating the scheduled execution time.
<code>updatedAt</code>	Timestamp indicating when the job was last updated.

Current payload:

```
{
  "data": {
    "message": "SBOM file successfully processed"
  }
}
```

New payload:

```
{
  "data": {
    "jobId": "5e28492a-5d0e-4dfb-9935-503b0463d0c5",
    "workspaceId": "56f7d2ae-4b03-4c4d-bfbb-b6aae4aa1eb9",
    "status": "SCHEDULED",
    "errorMessage": null,
    "createdAt": "2025-01-28T07:53:25.358Z",
    "plannedExecutionAt": "2025-01-28T07:55:22.102Z",
    "updatedAt": "2025-01-28T07:53:25.358Z"
  }
}
```

New Endpoint for Tracking the Job Status

We’re introducing a new endpoint for tracking the processing job status using the `jobId` returned in the [response payload](#). Job statuses are maintained for 48 hours after submitting SBOM files.

Method	Endpoint	Schema
GET	<code>/sboms/jobs/{jobId}</code>	Reference ➡

Response payload:

```
{
  "data": {
    "id": "1a10138e-302b-4797-9038-058e14d1a0b6",
    "workspaceId": "56914519-c634-41bb-b163-866333b907ee",
    "status": "SUCCEEDED",
    "errorMessage": null,
    "createdAt": "2025-03-06T07:33:25.833318Z",
    "plannedExecutionAt": "2025-03-06T07:33:31.479195Z",
    "updatedAt": "2025-03-06T07:33:31.479195Z"
  }
}
```

What You Need to Do

If your current implementation of `/factSheets/{factSheetId}/sboms` does not rely on the HTTP status code or the payload, these changes will not affect you. Otherwise, you will need to update your integrations as detailed below.

Adjust Your Integrations for the New Response Code and Payload

Update your systems to recognize the new 202 HTTP response status code and parse additional response payload fields.

Pseudo code to handle the new status code:

```
function handleApiResponse(response):
  if response.statusCode == 202:
    print("Request accepted. Processing asynchronously.")
    jobId = response.payload.data.jobId
    jobStatus = response.payload.data.status
    jobCreationTime = response.payload.data.created_at
    print("Job ID:", jobId)
    print("Job Status:", jobStatus)
    print("Job Created At:", jobCreationTime)
    return jobId # Save Job ID for status checks
  elif response.statusCode == 201:
    print("Request processed successfully.")
  else:
    print("Error with request. Status Code:", response.statusCode)

# Example of using the function with a hypothetical API response
apiResponse = {
  "statusCode": 202,
  "payload": {
    "data": {
      "jobId": "5e28492a-5d0e-4dfb-9935-503b0463d0c5",
      "status": "In Progress",
      "created_at": "2021-07-01T12:00:00Z"
    }
  }
}
jobId = handleApiResponse(apiResponse)
```

Adapt Your Requests for Asynchronous Processing

Implement logic to periodically check the job status, accommodating the change from synchronous to asynchronous processing.

Pseudo code to handle the job status:

```
function checkJobStatus(jobId):
    statusEndpoint = "https://api.example.com/job/status/" + jobId
    response = makeHttpRequest(statusEndpoint)
    if response.statusCode == 200:
        jobStatus = response.payload.jobStatus
        print("Current Job Status:", jobStatus)
        return jobStatus
    else:
        print("Failed to retrieve job status. Status Code:", response.statusCode)

function periodicStatusCheck(jobId, interval):
    import time
    jobComplete = False
    while not jobComplete:
        jobStatus = checkJobStatus(jobId)
        if jobStatus == "SUCCEEDED" or jobStatus == "FAILED":
            jobComplete = True
            print("Final Job Status:", jobStatus)
        else:
            print("Continuing to monitor job status...")
            time.sleep(interval) # Delay next check by 'interval' seconds

# Example usage with a job ID and a check interval of 30 seconds
periodicStatusCheck(jobId, 30)
```

Test the New API Behavior During the Transition Period

During the transition period, which lasts **until April 7, 2025**, you can try out the new behavior. To do this, add the `x-leanix-dev` header set to `dev` to your current API requests.

Example request:

```
curl --request POST \
  --url https://{SUBDOMAIN}.leanix.net/services/technology-discovery/v1/factSheets/{factSheetId}/st
  --header 'authorization: Bearer {API_TOKEN}' \
  --header 'content-type: multipart/form-data' \
  --header 'x-leanix-dev: dev' \
  --form sbom=<sbom file location>
```

For authentication details, see [Authentication to SAP LeanIX Services](#).

GraphQL API Changes (January 15, 2025)

Learn what's changing in our GraphQL API and how to prepare for these changes to avoid any issues with your integrations.

i Note

The GraphQL API changes described here have been implemented on January 15, 2025. If you have questions or need additional support, please contact SAP LeanIX support.

To improve our API's overall robustness and consistency, we'll apply changes to our GraphQL API validation rules, which will take effect on **January 15, 2025 after a grace period**.

What's Changing

Previously, our GraphQL API validation allowed certain malformed queries to pass through. These queries represent edge cases and do not reflect typical usage patterns. We've identified and addressed these inconsistencies to ensure stricter validation. We

anticipate that this change will only impact a few customers who may have unintentionally leveraged these validation loopholes in their integrations.

Also, we're implementing stricter validation of the number of tokens allowed in a single GraphQL request. This measure is aimed at enhancing the overall stability and performance of the API.

We're providing a grace period until January 15, 2025 to allow ample time for testing and adjustments. After this date, stricter validation will be enforced, and any non-compliant queries will fail.

What You Need to Do

To prepare for the upcoming changes to the GraphQL API, do the following:

- **Review API responses:** Review API responses to ensure that they don't contain warning messages. If a warning message is returned, review the details in the response and adjust your queries as necessary.
- **Review your GraphQL API usage:** Carefully examine your interaction with our API, particularly if you've implemented custom integrations.
- **Test your integrations:** Before the effective date, thoroughly test your integrations to identify any potential issues caused by the stricter validation.
- **Update your integrations (if necessary):** If your integrations rely on the previously allowed malformed queries, you'll need to update them to comply with the new validation rules. Also, you should check your queries to ensure that they don't exceed the allowed token limit.

How to Validate Queries

To validate queries in your integrations, include the HTTP header `x-graphql-enable-extensions` in your requests and set its value to `true`. If a query is invalid, a warning message is returned in `extensions` in the response. Without this HTTP header, no warning message will be returned for invalid queries.

HTTP header:

```
"x-graphql-enable-extensions": "true"
```

In the GraphiQL interface in your workspace, a warning message is automatically returned for invalid queries.

Example Warning Message

For invalid requests, a warning message is returned in the response, as shown in the following example. The warning message and relevant details are returned in `extensions`, which is a new element introduced in responses. Review the details provided in `reasons` and adjust your queries as necessary.

```
{
  "data": { ... },
  "extensions": {
    "warnings": [
      {
        "message": "Your request contains elements that will be considered invalid after January 15, 2025.",
        "reasons": [
          "Invalid syntax encountered. There are extra tokens in the text that have not been consumed."
        ]
      }
    ]
  }
}
```

```

    }
  ]
}
}

```

Resources and Support

We'll be updating our API documentation to reflect the new validation rules. The updated documentation will be available before the grace period ends.

If you have any questions or require further assistance, please don't hesitate to contact [SAP LeanIX Support](#) . If you're an SAP customer, submit a request from the [SAP for Me](#)  portal.

Upcoming Changes

The following sections provide an overview of upcoming changes to the GraphQL API and what you need to do to prepare for them.

- [Stricter Validation of Unconsumed Tokens](#)
- [Stricter Validation of Invalid Tokens](#)
- [Stricter Validation of the GraphQL Parsing Token Limit](#)
- [Stricter Data Type Validation](#)

→ Tip

Our interactive GraphiQL interface provides a convenient way to test and validate your queries against the latest schema. Paste your query into the GraphiQL editor and check the query for syntax errors. Syntax errors are highlighted in red. If needed, modify the query to ensure that there are no syntax errors. Then, run the query to ensure that it doesn't result in a warning message.

To learn more about using GraphiQL, see [GraphiQL Tool](#).

Stricter Validation of Unconsumed Tokens

One of the upcoming API changes addresses the issue of unconsumed tokens in the request body. Previously, our validation might have allowed requests containing extra tokens (for example, `}`) at the end of the payload to pass through. Under the new validation rules, such requests will be flagged as malformed and will fail validation.

What You Need to Do

Review the payloads of your GraphQL API requests for any extraneous characters following the intended data. To ensure successful calls after the grace period, you should modify your code or tools to construct requests that strictly adhere to the documented syntax, avoiding any extraneous characters after defined parameters.

Example Query and Warning Message

Under the new validation rules, the following example request would fail.

Example query with unconsumed tokens:

```

{
  allFactSheets(first: 10, factSheetType: Application) {

```


Example warning message:

```
[
  {
    "message": "Your request contains elements that will be considered invalid after January 1",
    "reasons": [
      "Invalid syntax with ANTLR error 'token recognition error at: '0'' at line 2 column 3"
    ]
  }
]
```

Stricter Validation of the GraphQL Parsing Token Limit

We're implementing stricter validation of the number of tokens allowed in a single GraphQL request. This change is introduced to enhance the overall stability and performance of our GraphQL API.

What the Limit Means

In GraphQL, a request is broken down into smaller units called tokens during the parsing process. These tokens represent elements like keywords, variable names, field names, and string values.

An excessively large number of tokens in a single request can strain our parsing capabilities and potentially lead to performance issues. The limitation on the number of tokens ensures optimal API performance and a smooth experience for all users.

How This Affects Your Integrations

This change only affects customers who construct exceptionally complex GraphQL queries with a very high token count.

What You Need to Do

- **Review your GraphQL queries:** Analyze your current GraphQL queries to identify any queries that might fail because of the token limit.
- **Refactor complex queries (if necessary):** If you find queries exceeding the limit, explore ways to simplify or refactor them. This could involve breaking down the query into smaller, more manageable requests or optimizing the query structure.

Example Error Message

Under the new validation rules, queries exceeding the token limit will fail, and the following error message will be returned.

```
{
  "errors": [
    {
      "message": "The request contains too many characters. To prevent Denial Of Service attacks, r"
    }
  ]
}
```

Stricter Data Type Validation

We're implementing stricter data type validation in GraphQL queries to improve data integrity and query accuracy. This enhancement aims to ensure that the data types passed in GraphQL queries align with the expected types, thereby mitigating potential errors and enhancing the overall query performance.

Here are some examples of warning messages that will be returned for requests where the data types do not match the expected ones:

- Variable has an invalid value: Expected a String input, but it was a 'Integer':Returned when an integer value is passed where a string is expected.
- Variable has an invalid value: Expected a String input, but it was a 'Boolean':Returned when a boolean value is passed where a string is expected.
- Variable 'patches' has an invalid value: Expected a String input, but it was a 'Double':Returned when a double value is passed where a string is expected.

What You Need to Do

Validate your queries to ensure that the data types of passed values correspond to the expected types.

Example Mutation and Warning Message

The following example illustrates a mutation where the data type of the passed value in the variable does not match the expected type. In the `description` field, an integer value is provided where a string is expected, resulting in a warning message returned for this query.

Example mutation with a variable:

```
mutation($patches: [Patch]!) {
  updateFactSheet(
    id: "ed37c4fe-7c86-45cd-97de-02cabf05bc4c",
    patches: $patches
  ) {
    factSheet {
      id
      name
      description
    }
  }
}
```

Variable with an incorrect data type passed in `value`:

```
{
  "patches": {
    "op": "replace",
    "path": "/description",
    "value": 12345
  }
}
```

Example warning message:

```
"warnings": [
  {
    "message": "Your request contains elements that will be considered invalid after January 15,"
    "reasons": [
      "Variable 'patches' has an invalid value: Expected a String input, but it was a 'Integer'"
    ]
  }
]
```

Reporting Library and Custom Report Tools

Build custom reports using the SAP LeanIX Reporting Library and Custom Report Tools.

Overview

SAP LeanIX offers a wide range of standard out-of-the-box reports. However, standard reports may be insufficient for your organization. For more details on out-of-the-box reports, see [Reports](#)

As a developer, you can build custom reports using the reporting toolset, which consists of two components:

- [SAP LeanIX Reporting Library](#)  : The library provides a connection to the SAP LeanIX app, allowing you to create custom reports using workspace data and functionality.
- [SAP LeanIX Custom Report Tools](#)  : Create, develop, build, and upload custom reports to your workspaces and the SAP LeanIX Store.

Reporting Library

The SAP LeanIX Reporting Library is built into SAP LeanIX. It exposes the Lx API. Your custom report uses this API to connect to the workspace, query data, and integrate with the SAP LeanIX user interface.

The reporting library provides these features:

- Filtering of fact sheet information
- View selection
- Tabular view of report data
- Export to PDF and PNG files
- Choice of visualization options

Links to the reporting library:

- Library: [GitHub - leanix/leanix-reporting: Library to develop custom reports for LeanIX.](#) 
- API Documentation: [Reporting Library documentation](#) 

The documentation provides a comprehensive reference for enumerations, classes, interfaces, type aliases, variables, and more.

Custom Report Tools

The SAP LeanIX custom report tools scaffold a new project with a React/TypeScript template and a pre-configured Vite build setup. The Vite build setup handles the development server, production build, and workspace upload.

Report Tools Options

To see all available options, run:

```
npm create @sap/leanix-custom-report@latest -- --help
```

Create a New Report

Use the CLI to bootstrap a new project:

```
npm create
    @sap/leanix-custom-report@latest
```

Follow the interactive prompts to configure the report ID, title, author, and workspace credentials. The tool generates a ready-to-use template-report with React and TypeScript. To learn more, see [Creating Your Custom Report Project](#).

Develop, Build, and Upload

Once your project is created, the following npm scripts are available:

Command	Description
npm run build	Build the report for production
npm run dev	Start the local development server with hot module replacement. Code edits will be automatically reflected once saved.
npm run upload	Build and upload the report to your SAP LeanIX workspace

→ Tip

Before you start developing, initialize a git repository in your project directory.

```
Run:git init && git add . && git commit -m "Initial commit"
```

Use add and commit regularly to create a log of your changes. This gives you recovery points if you need to undo changes.

For more information, refer to the [Git documentation](#) ➦ .

Links to custom report tools:

- Package including documentation: [GitHub - SAP/leanix-custom-report-tools: SAP LeanIX custom report tools](#) ➦
- npm scaffolding tool: <https://www.npmjs.com/package/@sap/create-leanix-custom-report> ➦

Required Knowledge

To work with the Reporting Library and Custom Report Tools, you should have a working knowledge of the following:

- Command-line interface
- React/TypeScript

Depending on the use case, you may need knowledge and skills in other areas.

Tutorials

To learn how to build custom reports using the reporting toolset, see our tutorials in [Custom Reports](#).